CS106B《编程抽象》(Programming Abstractions)学习笔记：C++ 实现与算法图解

> **资料基础**：斯坦福 CS106B 官方课程网站（2026 夏季学期，8 周压缩学期，共 28 讲）全部**公开页面**——课程主页、Syllabus、讲座页（含各讲文字讲义与公开附件）、作业页等。完整调研记录见同目录 `00_research_inventory.md`（及 `lecture_data_records.json` 数据记录）。 **版权与用途声明**：官方材料 © Stanford University，受保护、不得擅自传播。本文档是**自学整理笔记**：将 28 讲公开内容按主题重组成 17 章，讲解均为原创中文表述，未逐字转载任何官方讲义；全部代码为本笔记撰写的教学示例（现代 C++17、仅用标准库），并非课程作业提交物。 **学习建议**：每章先读“概述”，再看“核心概念与算法原理”里的图示/步骤分解，然后**亲手把代码敲一遍并运行**，最后用“思考题”自测。链表、二叉树、堆、图这几章强烈建议边读边在纸上画内存/结构图。

课程概览

这门课是什么？

CS106B（Programming Abstractions，编程抽象）是斯坦福入门编程序列的第二门课。先修 CS106A 用 Python 建立了编程方法论与问题求解基础；CS106B 在此基础上做三件大事：

1. **学一门新语言**：C++（类型系统、函数、字符串、类、指针与动态内存）；
2. **掌握编程抽象**：认识“抽象数据类型”(ADT)——把“用什么”与“怎么实现”分开，先会用（客户端视角），再自己实现（实现者视角）；
3. **吃透经典数据结构与算法**：线性表、栈、队列、集合/映射、树、堆、哈希表、图，以及递归、回溯、分治、贪心（Dijkstra、霍夫曼）、复杂度分析（大 O）。

官方 syllabus 给的主题推进顺序（“近似顺序”）正是本文档 17 章的骨架：

> C++ 基础 → 数据抽象与经典 ADT → 递归与回溯 → 类与面向对象 → 指针与动态内存 → 链式数据结构 → 进阶算法

与 CS106A 的衔接

- CS106A 结业水平 = 能写多函数程序、会用循环/条件/列表/字典、理解基础测试。CS106B **不重复教编程入门**，而是以“你会写程序”为前提，快速补 C++ 语法差异后直接进入抽象与算法。
- Python 经验迁移要点：列表→`std::vector`、字典→`std::map`/`std::unordered_map`、集合→`std::set`/`std::unordered_set`、字符串 API 差异大（C++ 字符串可原地修改）、一切传参默认**按值拷贝**（要显式写 `&` 引用）。这些差异会在 Lecture 1–3 反复强调。

官方课程要点（摘自 syllabus，概括）

- **教学团队**：讲师 Sean（Szumlanski）、Head TA Butch、14 名 section leaders；小班讨论每周一次、占 5%。
- **节奏与考核**：8 周、28 讲（MTWTh 13:30–14:45，NVIDIA Auditorium）；作业 25% + 期中 27.5% + 期末 37.5% + 小班 5% + 讲座小测 4% + Quiz0 1%；期中 7/17、期末 8/14（纸笔）。
- **作业（8 个，约每周一个，10–20 小时/个）**：| # | 名称 | 覆盖主题（对应章节） ||---|---|---|| 0 | Welcome to CS106B! | 环境与工具 || 1 | Getting Your C++ Legs | C++ 基础、字符串（L1） || 2 | Fun with Collections | 栈/队列/集合/映射等 ADT（L2–L3） || 3 | Recursion Etudes | 递归（含分形）（L5） || 4 | Recursive Backtracking | 回溯（L6） || 5 | Tone Matrix | 类/数组/动态内存（L8–L9） || 6 | Listy Things | 链表与树（L11–L12） || 7 | Huffman Coding（选做🌱） | 霍夫曼编码（L13） |
- **工具**：Qt Creator（编辑器+编译器）+ Stanford C++ Library（课程专用容器库，文档公开：web.stanford.edu/dept/cs_edu/resources/cslib_docs/）。**本文档代码一律用标准库 `std::` 容器**（等价关系见文末速查表），保证你能在任何现代 C++ 环境编译运行。
- **教科书**：Eric Roberts, *Programming Abstractions in C++*（第 5 版），ISBN 978-0133454840。
- **学习目标（概括）**：乐于用编程解决现实问题；识别常见抽象；理解日常技术背后的程序化概念；能用递归/算法推理拆分复杂问题；能评估数据结构与算法的设计取舍。

学习路线：真实 28 讲 ↔ 本文 17 章

官方 2026 夏季学期 28 讲实际顺序如下。本文档按**官方教学顺序**把它重组成 17 章主题笔记（每章开头标注“对应真实讲座 Lxx–Lyy”），数字编号即学习顺序：

本文章	主题	对应官方讲座
Lecture 1	C++ 基础回顾与 STL 容器入门	L01 Welcome! · L02 C++ Fundamentals · L03 C++ Strings · L04 Testing, Vectors, and Grids
Lecture 2	栈与队列	L05 Stacks and Queues
Lecture 3	集合与映射（基于树的有序容器）	L06 Sets and Maps
Lecture 4	算法分析：大 O 记号	L07 Big-O and Algorithmic Analysis
Lecture 5	递归：原理与递归策略	L08 Introduction to Recursion · L09 More Recursion · L10 Recursive Problem Solving
Lecture 6	递归回溯与枚举	L11 Recursive Backtracking and Enumeration · L12 More Recursive Backtracking
Lecture 7	排序算法	L13 Sorting Algorithms
（L14 Problem Solving Day 为复习/答疑，并入相关章节练习）		

本文章	主题	对应官方讲座
Lecture 8	面向对象编程：类与封装	L15 Object-Oriented Programming
Lecture 9	指针、数组与动态内存管理	L16 Pointers and Arrays · L17 Dynamic Memory Management
Lecture 10	优先队列与二叉堆	L18 Priority Queues and Binary Heaps
Lecture 11	链表	L19 Introduction to Linked Lists · L20 More Linked Lists
Lecture 12	二叉树、二叉搜索树与遍历	L21 Binary Trees, BSTs, and Tree Traversals · L22 More on Binary Trees
Lecture 13	霍夫曼编码	L23 Huffman Coding
Lecture 14	散列与哈希表	L24 Hashing
Lecture 15	图：表示、遍历与拓扑排序	L25 Graphs
Lecture 16	最短路径：Dijkstra 与 A*	L26 Dijkstra and A* · L27 Graph Coding
Lecture 17	拓展专题：Trie 与并查集	延伸内容（官方未设独立讲座；L24 曾提及 trie 思路）

（L28 Wrap 为期末回顾，无新主题。）

笔记约定

- 每章统一结构：**概述 → 核心概念与算法原理 → 代码示例与实现详解（代码做什么 / 实现机制解说）→ 复杂度分析 → 关键要点 → 常见陷阱与注意事项 → 思考题（带答案）**。
- ASCII 图一律放在 `text` 代码块中；`cpp` 代码块可直接复制编译。
- 术语采用“中文（English）”或“English（中文）”双语标注，方便对照原版材料。

Lecture 1: C++ 基础回顾与 STL 容器入门（C++ Fundamentals & STL Containers：Syntax, Functions, std::string, Vector & Grid, Testing）（对应课程真实讲座 L01–L04）

概述

本讲是 CS106B 的“开机课”：把大家从 CS106A 的 Python 世界平稳地接进 C++，先讲清一门语言的两副面孔——语法与语义——再讲程序从源码到运行的全过程，然后系统复习变量、循环、分支、函数、字符串等基础构件，最后引入第一批“抽象数据类型（Abstract Data Type，ADT）”容器 Vector 与 Grid，并介绍函数分解与单元测试的思想。全讲为后续栈、队列、集合、映射等容器铺好语言地基。 对应官方 L01（6/22，Welcome!）、L02（6/23，C++ Fundamentals）、L03（6/24，C++ Strings）、L04（6/25，Testing, Vectors, and Grids）四讲。

核心概念与算法原理

1. 语法 vs 语义、编译与执行

问题定义：新手学 C++ 常把“编译器报不报错”和“程序对不对”混为一谈，需要先分清两个层面。 **直观解释**：语法（syntax）是“怎么说才合规矩”，语义（semantics）是“这句话到底什么意思”。就像中文句子“把书放上书架”语法正确，但语义上我们明白说的是“放上书架的那本书”，而不是“放一个书架上坐着的人”。官方在第 2 讲正是用这种自然语言的例子引入这对术语。 **步骤分解**：写代码时先满足语法（编译器才肯放行），再保证语义（程序才做对的事）。语法错误在编译期被揪出来；语义错误往往要等到运行期甚至悄悄潜伏。

问题定义：C++ 源码是如何变成可运行程序的？ **直观解释**：编译器（compiler）是一个“翻译程序”：把整份 C++ 源码一次性翻译成机器能执行的指令；这与 Python 的解释器（interpreter）逐行边翻译边执行的方式不同（官方第 2 讲对比过这一点）。

```
hello.cpp(源码,人写的)
   │  ① 预处理:展开 #include
   ▼
   │  ② 编译(compiler):逐行检查语法,翻译成机器指令
   ▼
可执行文件(机器指令,0 和 1)
   │  ③ 操作系统加载它,并从 main() 开始执行
   ▼
屏幕输出:Hello, world!
```

操作要点：编译期错误（compile-time error，多为语法/类型问题）在点“运行”之前就会被报告；运行期错误（runtime error，如越界崩溃）则发生在程序真正跑起来之后。

2. main、注释、#include、命名空间、cout/endl

- **问题定义**：每个 C++ 程序都需要一个统一的“入口”和一套“打招呼”的仪式。

- **直观解释**：`main()` 是唯一特殊的函数——程序一启动就自动调用它，无需任何人手动调用；一个没有 `main()` 的程序无法通过编译（官方第 2 讲强调“main is special”）。`main()` 最后 `return 0;` 表示“零错误”，这个值会交还给操作系统。
- **步骤分解**：`#include <库名>`（标准库用尖括号）把别人写好的库“搬”进来；`using namespace std;` 让我们不必在每个标准库名字前敲 `std::`；`cout << 内容 << endl;` 向屏幕输出，`endl` 负责换行。注释分两种：`//` 单行注释与 `/* ... */` 块注释，注释是写给人的，编译器完全忽略。
- 一句话注明：课程官方在 L01/L02 使用 `#include "console.h"` 弹出课程专用终端，其输出机制等价于标准库 `<iostream>` 的 `std::cout`。

3. 变量与数据类型、未初始化陷阱

问题定义：C++ 是强类型语言——每个变量必须先声明、带一个固定类型，之后不能改类型。 **直观解释**：变量像贴了标签的储物盒，标签（类型）决定盒子里能放什么形状的东西。常用类型：`int`（整数）、`double`（浮点数）、`char`（单字符，单引号）、`bool`（真假）、`string`（字符串，双引号）。 **步骤分解**：`int a = 5;` 声明并初始化；之后改值只需 `a = 7;` 而不能再写 `int a = 7;`（那是重复声明，报错）。 **关键陷阱（官方第 2 讲专门点名）**：基本类型变量若不初始化，里面装的是“垃圾值”（garbage）——C++ 不会自动帮你清零；唯一的例外是 `string`，默认自动初始化为空串 `""`。大多数编译器会放行未初始化代码，但结果是不可预测的，务必养成“声明即初始化”的习惯。

4. 循环：while / for / range-for

问题定义：如何反复执行同一段逻辑？ **步骤分解**：

- `while (条件) { ... }`：先判断再执行，条件为假就退出；记得在循环体里推进条件，否则死循环。
- `for (int i = 0; i < n; i++) { ... }`：把“初始化、条件、步进”收拢到一行，适合“按次数/按下标”遍历。`i++` 是 `i = i + 1` 的简写。
- range-based for（又称 for-each）：`for (char ch : s) { ... }` 直接“逐个掏出元素”，无需下标。适合 vector、string、set 等容器；代价是拿不到当前下标。

5. 分支与布尔逻辑、短路求值

问题定义：如何让程序“看情况行事”？ **步骤分解**：`if (条件) ... else if (条件) ... else ...`。比较运算符 `== != < <= > >=`；布尔运算符 `&&`（且）、`||`（或）、`!`（非）。 **机制要点（短路 short-circuiting）**：`&&` 左侧为假时右侧根本不会执行；`||` 左侧为真时右侧也不会执行。这个特性常被用来“先安检再动手”，例如先检查下标合法再访问数组：`if (i >= 0 && i < v.size() && v[i] > 0)`——若 i 越界，后面的 `v[i]` 压根不会被访问，从而避免崩溃。官方第 2 讲还提醒过常见笔误 `if (numCupcakes == 1 || 2)`，这里的 `2` 恒为真，条件永远成立。

6. 函数：原型、传值 vs 传引用

问题定义：把一段逻辑命名并复用，同时明确它“吃什么（参数）、吐什么（返回值）”。 **直观解释**：C++ 编译器从上往下逐行看代码，所以调用一个“还没见过”的函数会报错。两种解法（官方第 3 讲）：把函数定义挪到 `main()` 之前；或在其上方放一个**函数原型（prototype）**——即“函数签名 + 分号”，例如 `int square(int x);`，相当于提前告诉编译器“这个函数长什么样，具体实现稍后见”。 **关键区分（传值 vs 传引用）**：默认是

传值（pass-by-value）——调用时把实参**拷贝一份**给形参，函数里改的是副本，外面的变量毫发无损；若形参写作 `int& n`（传引用，pass-by-reference），形参就成了指向实参的“虫洞/传送门”，函数里动它等于直接动调用者的变量。传引用还有第二个动机：容器可能很大，逐元素拷贝既费时又费内存，引用只花几十比特建立“纽带”。官方第 3 讲用“倒空海盗宝藏”的比喻演示了两种传参的天壤之别（传值清空不了原宝藏，传引用可以）。

对比维度	传值 pass-by-value	传引用 pass-by-reference
形参写法	`void f(int n)`	`void f(int& n)`
是否拷贝数据	是，生成独立副本	否，形参是实参的别名
函数内修改	只改副本，实参不变	直接改实参
适用场景	小数据、不希望改动实参	要“带出”多个结果、大容器省拷贝
时空开销	拷贝 O(n)	建立引用 O(1)

内存示意（左侧传值、右侧传引用）：

```
传值:                          传引用:
main() 的 n [ 3 ]              main() 的 n [ 4 ]
foo() 的 n  [ 3→4 ](副本)      foo() 的 n ──虫洞──► main() 的 n
```

7. std::string：对象、成员函数与逐字符处理

问题定义：文本处理是编程日常，需要一套趁手的字符串工具。 **直观解释**：C++ 的 `string` 不是基本类型而是**对象**：它内部是一块连续存放字符的内存（本质是字符数组），同时“随身携带”一批现成函数，用点号 `.` 调用，称为**成员函数（member function）**。它与 Python/Java 的一大不同是**可修改（mutable）**：`s[0] = 'Y'` 能直接改掉第 0 个字符。 **常用成员函数**：`s.length()`（字符个数）、`s[i]`（按下标读写字符）、`s += "xx"` 或 `s + t`（拼接）、`s.substr(起点, 长度)`（截子串）、`s.find(子串)`（返回首次出现下标，找不到返回 `string::npos`）、`s.insert(位置, 文本)`、`s.erase(位置, 长度)`、`s.replace(位置, 长度, 新文本)`。

```
字符串 "hello" 的内存布局(数组本质):
+----+----+----+----+----+
|'h' |'e' |'l' |'l' |'o' |
+----+----+----+----+----+
  0    1    2    3    4     ← 下标从 0 到 length()-1
```

逐字符遍历：用普通 for 按下标 `s[i]` 访问（可改），或 range-for 取出每个 `char` 副本（只读遍历更简洁，但拿不到下标、也改不了原串）。

8. 字符处理：ASCII 与 cctype、类型转换

问题定义：字符在计算机里只是数字，需要一套“字符—数字”对照表和现成的判断函数。 **直观解释**：`char` 背后就是整数——`'A'` 是 65，`'a'` 是 97，`'0'` 是 48（这就是 ASCII 标准）。所以字符可以比较大小、做

算术，也可以用函数式类型转换 `int(ch)` 把它“现出原形”。 **步骤分解**：`<cctype>` 库提供一族的 `isXxx(ch)` 判断函数：`isalpha`（字母）、`isdigit`（数字）、`isupper`/`islower`（大小写）、`isspace`（空白）等，以及转换函数 `toupper(ch)`/`tolower(ch)`（注意它们是传值：返回新字符，不改原变量）。 **风格要点（官方第 3 讲）**：别把 `96`、`65` 这类“魔数（magic number）”直接写死在代码里——`int(ch) - ('a' - 1)` 比 `int(ch) - 96` 自解释得多。能用 `isalpha` 等库函数表达意图时，就不要再手写 `ch >= 'a' && ch <= 'z'` 这种比较。

9. Vector：顺序容器、扩容、add vs insert(0,·)

问题定义：需要一个能自动伸缩、按下标快速访问的“列表”。 **直观解释**：Vector 是“同质、有序、按下标 0..n-1 索引”的容器，底层是连续内存数组——可以类比浏览器的标签页：有先后顺序，能增能减。课程官方使用 Stanford 的 `Vector<T>`，与标准库 `std::vector<T>` 等价（官方第 4 讲特意提醒两者大小写不同；本笔记一律用 std 版本）。 **常用操作对照**：`v.push_back(x)`（官方 `add`，末尾追加）、`v.insert(v.begin()+i, x)`（官方 `insert(i,x)`，在 i 前插入并右移后续元素）、`v.erase(v.begin()+i)`（官方 `remove(i)`，删除并左移）、`v.size()`、`v.empty()`（官方 `isEmpty`）、`v[i]` 下标访问、`v.clear()`。

```
push_back(末尾追加,快):                insert(0,x)(头部插入,慢):
[15][20][18]      加 33              [15][20][18][33]     插入 90
[15][20][18][33]                     [ ][15][20][18][33] ← 先把 4 个元素整体右移
                                     [90][15][20][18][33] ← 再写新值
```

运行时对比（官方第 4 讲核心实验）：`add` 只是往末尾“放一个”，偶尔后台扩容一次；`insert(0, x)` 则每次都要把已有的**每一个**元素往右挪一格。官方在课上用计时工具实测：规模 5 万时 insert 版比 add 版慢约 17 倍，规模到 50 万时差距膨胀到三百多倍。规模每翻倍，insert 版的工作量也翻倍式增长——这正是下一讲“大 O 记号”要量化的现象。这里的“扩容”概念先记住：vector 满了会申请一块更大的内存并把所有元素拷过去（偶尔发生，均摊后 add 仍是 O(1)），细节留待第 4 讲与实现篇展开。

10. 二维容器：Grid（用 vector> 表示）

问题定义：表格、棋盘、像素图……都需要“行列”二维数据。 **直观解释**：官方 `Grid<T>(行, 列)` 就是矩形网格，元素按 `g[r][c]` 访问（**行在前的“row-major”**，记忆口诀：row 是“major（主要）”，所以行号永远先写）。用标准库表示就是“vector 的 vector”：`std::vector<std::vector<int>> g(rows, vector<int>(cols, 0))`——外层是行、内层是列。官方 `numRows()/numCols()` 对应 `g.size()` 与 `g[0].size()`。 **遍历方式**：双层 for（外层行、内层列）打印；range-for 会按 row-major 顺序把所有元素平铺输出（拿不到行列号）。适用场景：井字棋棋盘、图像像素、乘法表等。

11. 函数分解与测试理念；ADT 概念引入

问题定义：怎么让一段正确但“又长又乱”的代码变得可读、可测、可维护？ **直观解释（官方第 4 讲）**：把大任务拆成“各司其职”的小函数叫**函数分解（functional decomposition）**；函数名用动词短语，变量名有意义，注释只解释“为什么/怎么做”，而不是把代码逐行翻译成英文。好的副产品是：每个小函数都能被独立、严格地测试。 **测试理念**：官方在 L04 引入 SimpleTest 框架（`STUDENT_TEST` 写用例、`EXPECT_EQUAL` 断言、`runSimpleTests` 批量跑），并展示了为 `extractAlpha` 设计的“边界场景清单”：全是字母/没有字母/

字母在头中尾/空串/长度 1、2、3/超长串……本笔记不引入任何 Stanford 库，用自写的迷你断言函数（见示例 3 附近说明）演示同一思想：**对边界条件穷举、断言期望值、一次运行全量验证。** **ADT 是什么**：抽象数据类型 = “数据 + 允许的操作”打包成的契约。使用者只需知道“能 push 什么、pop 得到什么”，不必关心内部是数组还是链表——这正是官方第 4 讲“先从客户端视角使用、后几讲再亲手实现”的教学路线。

代码示例与实现详解

示例 1：一门 C++ 程序的“骨架课”——变量、三种循环、分支与短路

```cpp
// 文件: basics_demo.cpp
// 演示: 注释、变量类型、cout/endl、while/for/range-for、if 与短路求值
#include <iostream>   // 标准库用尖括号: 提供 cout/endl
#include <string>     // 提供 std::string(写全可不加,<iostream> 常间接引入,但显式更稳)
using namespace std;  // 免写 std:: 前缀(std = standard 命名空间)

int main()            // 程序的唯一入口,操作系统自动调用它
{
    // (1) 变量: 声明即初始化,类型一旦定下不可更改
    int    age   = 20;        // 整数
    double pi    = 3.14159;   // 浮点数
    char   grade = 'A';       // 单字符必须单引号
    string name  = "Ada";     // 字符串必须双引号
    bool   likesCS = true;    // 布尔

    // (2) cout 输出 << endl 换行(不写 endl 所有输出会黏在一起)
    cout << "Hello, " << name << "!" << endl;
    cout << "age=" << age << ", grade=" << grade << ", pi=" << pi << endl;

    // (3) while: 先判断再执行
    int i = 1;
    while (i <= 3) {
        cout << "count " << i << endl;
        i++;                  // 推进条件,否则死循环
    }

    // (4) for: 初始化/条件/步进三合一;循环变量 j 只在循环内有效
    int sum = 0;
    for (int j = 0; j < 5; j++) {
        sum += j;             // 0+1+2+3+4
    }
    cout << "0..4 之和 = " << sum << endl;

    // (5) range-based for: 逐字符掏出,拿不到下标
    for (char ch : name) {
        cout << "[" << ch << "]";
    }
    cout << endl;

    // (6) if + 短路: age>=18 为假时,&& 右边根本不会执行
    if (age >= 18 && likesCS) {
        cout << "成年,且热爱计算机科学。" << endl;
    } else {
        cout << "条件不满足。" << endl;
    }
    return 0;   // main 返回 0 = "零错误",交还操作系统
}
```

【代码做什么】: (1) 声明五种常见类型的变量并输出；(2) 用 while 数 1 到 3；(3) 用 for 求 0..4 的和；(4) 用 range-for 把名字逐字打上括号；(5) 用 `&&` 组合两个条件。整体把本讲的“骨架语法”在一份可运行程序里串了一遍。

【实现机制解说】：注意 `for (int j = 0; j < 5; j++)` 里 `j` 的作用域只在循环内——出了右花括号 `j` 就不存在，这是 C++ 的**块作用域**规则。`sum += j` 是 `sum = sum + j` 的简写。`age >= 18 && likesCS` 演示短路：若 `age >= 18` 为假，整个表达式立即为假，右侧 `likesCS` 不再求值——若右侧是 `v[i] > 0` 这类可能越界的访问，短路正好保护它不被执行。

示例 2：字符串、字符与函数——成员函数、原型、传值 vs 传引用

```cpp
// 文件: string_func_demo.cpp
// 演示: string 成员函数、逐字符处理、cctype、函数原型、传值与传引用
#include <cctype>    // isspace / toupper 等字符判断与转换
#include <iostream>
#include <string>
using namespace std;

// —— 函数原型区: 放在 main 之前,让编译器先“认识”这些函数 ——
string makeAcronym(const string& phrase); // 取每个单词首字母拼缩写(大写)
int    asciiSum(const string& s);         // 求字符串各字符 ASCII 码之和
void   toUpperInPlace(string& s);         // 引用参数: 就地改成大写

int main()
{
    // (1) string 成员函数: length()/[]/substr()/+= (字符串是可改对象)
    string phrase = "abstract data type";
    cout << "长度: " << phrase.length() << endl;            // 18
    cout << "第 0 个字符: " << phrase[0] << endl;           // 'a'
    cout << "substr(0,3): " << phrase.substr(0, 3) << endl; // "abs"

    // (2) 传 const 引用: 省拷贝且保证不被改动(想改也改不了)
    string acr = makeAcronym(phrase);
    cout << "缩写: " << acr << endl;                        // "ADT"
    cout << "ASCII 码和: " << asciiSum(acr) << endl;        // A=65 D=68 T=84 → 217

    // (3) 传引用: 函数内改动会“穿透”到 main 里的 acr
    toUpperInPlace(acr);   // acr 已是 "ADT",再转一次不变(留作观察点)
    string lower = "hello";
    toUpperInPlace(lower);
    cout << "toUpperInPlace 后: " << lower << endl;         // "HELLO"
    return 0;
}

// —— 函数定义区 ——
string makeAcronym(const string& phrase)
{
    string result;
    bool newWord = true;                // 下一个字符是否是单词首字母
    for (char ch : phrase) {
        if (isspace(ch)) {
            newWord = true;             // 遇到空白 → 下一个非空白是词首
        } else if (newWord) {
            result += toupper(ch);      // 首字母转大写并拼接(注意: 需 string 打底)
            newWord = false;
        }
    }
    return result;                      // 返回新串,phrase 本身未动
}

int asciiSum(const string& s)
{
    int total = 0;
    for (char ch : s) {
```

```
        total += int(ch);                // 类型转换: char 露出 int 真身
    }
    return total;
}

void toUpperInPlace(string& s)           // & 表示引用: 形参是实参的别名
{
    for (int i = 0; i < (int)s.length(); i++) {
        s[i] = toupper(s[i]);            // 逐个字符原地改写(字符串可修改)
    }
}
```

【代码做什么】：`makeAcronym` 把“abstract data type”缩成“ADT”（体会逐字符扫描 + 词首判断）；`asciiSum` 用 `int(ch)` 累加字符的 ASCII 码；`toUpperInPlace` 借引用形参原地把字符串改成大写。main 里依次展示 length/下标/substr、传值式库函数（`toupper` 返回新字符）与引用式自定义函数。

【实现机制解说】：`result += toupper(ch)` 这一行有两个细节：(a) `result` 是 `string`，`+=` 把右侧 `char` 拼到末尾，这就是“字符串可原地增长”；(b) 若写成 `result = result + toupper(ch)` 也等价，但若两边都是裸字符串字面量（如 `"abc" + "xyz"`）则编译不过——C++ 里双引号字面量是 C 风格字符串，必须至少一边是 C++ 的 `string`。再看传参：`makeAcronym(const string& phrase)` 的 `const &` 是“只读借用”，既省去整串拷贝（若 1GB 文本按值传就要再拷 1GB），又由编译器保证函数内改不了它；`toUpperInPlace(string& s)` 没有 const，函数内 `s[i] = ...` 会直接改到调用者的变量——这正是“传值 vs 传引用”的分界线：有 `&` 是虫洞，没 `&` 是复印件。另外 `s.length()` 返回无符号类型，与 int 比较时建议先 `(int)` 强转，避免有符号/无符号比较的隐形坑（官方第 3 讲练习也提醒这类细节）。

示例 3：std::vector 与二维 vector——“add vs insert(0,·)”计时 + Grid 表示

```cpp
// 文件: vector_grid_demo.cpp
// 演示: vector 常用操作、add vs insert(0,·) 运行时对比、二维容器表示 Grid
#include <chrono>    // 计时
#include <iostream>
#include <vector>
using namespace std;

double timeAdd(int n);        // 原型: n 次 push_back 耗时(毫秒)
double timeInsertHead(int n); // 原型: n 次 insert(begin()) 耗时(毫秒)

int main()
{
    // (1) 常用操作(std 版; 官方 Stanford Vector 的 add/insert/remove 同名不同拼)
    vector<int> v = {15, 20, 18};   // 初始化列表
    v.push_back(33);                // 对应 add(33)   → {15,20,18,33}
    v.insert(v.begin() + 2, 90);    // 对应 insert(2,90) → {15,20,90,18,33}
    v.erase(v.begin());             // 对应 remove(0) → {20,90,18,33}
    for (int i = 0; i < (int)v.size(); i++) cout << v[i] << " ";
    cout << endl;

    // (2) 计时对比: 规模翻倍时,add 版近似线性,insert 头部版近似“翻倍再翻倍”
    cout << "n        push_back    insert(begin)" << endl;
    for (int n : {5000, 10000, 20000, 40000}) {
        double tAdd = timeAdd(n);
        double tIns = timeInsertHead(n);
        cout << n << "    " << tAdd << " ms      " << tIns << " ms"
             << "   (慢 " << tIns / tAdd << " 倍)" << endl;
    }

    // (3) 二维 vector 模拟 Grid: 3 行 4 列,row-major(行先列后)
    const int ROWS = 3, COLS = 4;
    vector<vector<int>> g(ROWS, vector<int>(COLS, 0));  // 全部初始化为 0
    g[2][3] = 18;                                       // 第 2 行第 3 列
    for (int r = 0; r < (int)g.size(); r++) {           // 外层行
        for (int c = 0; c < (int)g[r].size(); c++) {    // 内层列
            cout << g[r][c];
            if (c + 1 < (int)g[r].size()) cout << ", ";
        }
        cout << endl;
    }
    return 0;
}

double timeAdd(int n)
{
    vector<int> v;
    auto t0 = chrono::steady_clock::now();
    for (int i = 0; i < n; i++) v.push_back(i);   // 每次只写末尾一个位置
    auto t1 = chrono::steady_clock::now();
    return chrono::duration<double, milli>(t1 - t0).count();
}

double timeInsertHead(int n)
```

```
{
    vector<int> v;
    auto t0 = chrono::steady_clock::now();
    for (int i = 0; i < n; i++) v.insert(v.begin(), i); // 每次都要把所有元素右移
    auto t1 = chrono::steady_clock::now();
    return chrono::duration<double, milli>(t1 - t0).count();
}
```

【代码做什么】：(1) 用 std 容器复刻官方 Vector 的 add/insert/remove 三连操作并打印；(2) 用 `<chrono>` 对“n 次末尾追加”和“n 次头部插入”分别计时，规模从 5000 翻倍到 40000，观察两者差距如何被拉大（官方 L04 用 SimpleTest 的 TIME_OPERATION 做过同款实验，规模到 50 万时差距达数百倍）；(3) 用 `vector<vector<int>>` 造一张 3×4 网格并 row-major 打印——这就是官方 `Grid` 类的标准库替身。

【实现机制解说】：`v.insert(v.begin(), i)` 每执行一次，要把当前所有元素整体右移一格腾出第 0 位，因此第 i 次插入要搬 i 个元素，n 次总共约搬 `1+2+…+n ≈ n²/2` 次——规模翻倍，搬移次数约翻 4 倍，计时结果会直观地“翻倍再翻倍”。`push_back` 则不同：末尾有空位时只写一个位置；只有当容量（capacity）耗尽，vector 才会“扩容”——申请一块更大的连续内存、把旧元素全部拷过去再释放旧的（这就是示例说明里提到的后台扩容），因为扩容不常发生，均摊下来 add 仍接近 O(1)。这正是官方 L04 强调“insert(0,·) 危险地慢、add 相当快”的底层原因。网格部分：`vector<vector<int>> g(ROWS, vector<int>(COLS, 0))` 先造好 3 个“行”，每行是一个长度为 4 的全 0 列向量；`g[2][3] = 18` 先取第 2 行的 vector（引用语义），再改其第 3 个元素——两层下标 `[][]` 与官方 `g[r][c]` 一一对应。

复杂度分析

操作	复杂度	简要原因
`v[i]` 按下标访问	O(1)	连续内存 + 首地址 + 偏移量直接定位
`push_back`（add）平均	O(1) 均摊	末尾写入；仅容量耗尽时偶尔整块拷贝扩容
`insert(begin(), x)`（insert(0,·)）	O(n)	必须把已有 n 个元素全部右移
`erase(begin())`（remove(0)）	O(n)	必须把后续元素全部左移填补空位
`s[i]`、`s.length()`	O(1)	字符串同数组本质，长度被缓存
逐字符遍历 string/vector	O(n)	每个元素恰好处理一次
按值传参给函数（大容器）	O(n)	需要整体拷贝一份

关键要点

- 先语法后语义：编译器只把关“合不合规矩”，程序“对不对”永远靠你自己想清楚，再靠测试验证。
- 声明即初始化：C++ 不会替你清零基本类型变量，未初始化的 `int` 里装的是垃圾值。
- 想改实参就用引用 `&`，只想省拷贝就用 `const &`；大容器一律不要按值传。
- 字符串与容器都从 0 开始编号、都可用 range-for 遍历；先想清要不要下标、要不要修改，再选 for 还是 for-each。

- `add`（末尾追加）均摊 O(1) 快到飞起，`insert(0,·)` 每回都要“全体右移”慢得吓人——以后写循环优先往末尾堆数据。

常见陷阱与注意事项

- **未初始化变量**：`int a; cout << a;` 打印垃圾值。规避：声明时立刻给初值。
- **越界访问**：`s[10]`（字符串长度 5）可能打印乱码甚至段错误崩溃，`v[v.size()]` 同理。规避：始终让下标落在 `[0, size)`，必要时先判断再访问。
- **引号用错**：字符用单引号 `'A'`，字符串用双引号 `"A"`，混用会导致编译错误或歧义。
- **类型不可变**：`int a = 5; string a = "hi";` 是重复声明；改值只写 `a = 7;`。
- **`==` 写成 `=`**：`if (a = 5)` 是赋值而非比较，条件恒真。规避：条件里坚持用 `==`。
- **短路误用/漏用**：`if (x || y)` 若 x 恒真则 y 永不执行；反之访问数组前不检查边界可能崩。规避：把“安检”放在 `&&` 左侧。
- **魔数**：`int(ch) - 96` 让人看不懂。规避：用 `('a' - 1)` 或 `isalpha`/`toupper` 这类自解释写法。
- **注释废话**：`// 打印 hello` 这种把代码翻译成中文的注释是噪音。规避：注释只写“为什么/整体在干嘛”，函数名取动词短语。
- **循环里改 size**：`for (int i = 0; i < v.size(); i++)` 若循环体里 push_back 会越跑越多。规避：先缓存原始大小或改用 while + 明确退出条件。

思考题（带答案）

问题 1：`string s = "hello"; for (char ch : s) { ch = toupper(ch); } cout << s;` 输出是什么？为什么？ **答案**：输出 `hello`。range-for 里的 `ch` 是每个字符的**拷贝**，改 `ch` 只改副本，原字符串不受影响；想真正修改要写成 `for (int i = 0; i < s.length(); i++) s[i] = toupper(s[i]);`（即示例 2 的 `toUpperInPlace` 思路）。

问题 2：声明 `void mystery(int& b, int c)` 后调用 `int x = 5; mystery(x, x);`，函数内 `b++`、`c++`，回到 main 后 x 是多少？为什么？ **答案**：x 变成 6。`b` 是引用，`b++` 直接改 main 里的 x；`c` 是传值，`c++` 只改函数内副本，与 x 无关。要点：同名实参传给引用和值两个形参时，只有引用那一路会“穿透”。

问题 3：手头有一个很大的 `vector<string>`，只想统计里面有多少个元素，函数签名写成 `int countAll(vector<string> v)` 有什么问题？怎么改最好？ **答案**：按值传参会把整个 vector 逐元素拷贝一遍，时间 O(n)、内存翻倍，纯属浪费。改成 `int countAll(const vector<string>& v)`：`const` 表明只读不改，`&` 避免拷贝，调用方与函数语义都不变，开销降为 O(1)。

Lecture 2: 栈与队列（Stacks & Queues：LIFO/FIFO 与 ADT 客户端视角）（对应课程真实讲座 L05）

概述

本讲介绍两种最简单的“有序容器”：栈（stack）与队列（queue）。它们不提供任意位置读写，只各开一个口子，却因此换来了极简、极快的操作，并能优雅地解决反转、配对、任务排队等一大批问题。本讲延续“客户端视角”：只关心“能做什么、语义是什么”，暂不深究内部实现；同时会用容器按引用传递、取模运算符 `%`、`break` 语句等配套语法，并完整实现一个经典应用——后缀表达式（postfix）求值。 对应官方 L05（6/29，Stacks and Queues）一讲；官方还随讲附赠 StackViz / QueueViz 两个可视化小程序帮助直观感受进出顺序。

核心概念与算法原理

1. ADT 的“客户端视角”（Client-Side Approach）

问题定义：学了 Vector/Grid 之后，如何又快又稳地搭建新工具，而不必先懂内部实现？ **直观解释**：课程采取“先当用户、后当制造者”的策略：先学会**使用** ADT（调用它的操作、相信它的语义），实现细节（数组？链表？）留到讲完指针与动态内存之后再揭开。官方在 L04 与 L05 反复强调这一点：作为客户端，我们享受的是“契约”——只要按文档调用，内部怎么折腾我们不必操心。 **步骤分解**：使用一个 ADT 的标准流程 = ① 决定需要什么语义（后进先出还是先进先出）→ ② 选对容器（Stack 或 Queue）→ ③ 只通过其公开操作读写（不越权访问内部）→ ④ 按文档假设复杂度与行为。官方用 StackViz/QueueViz 动画直观演示入栈出栈、入队出队的每一步，建议下载摆弄一遍。

2. Stack：LIFO，后进先出

问题定义：有些任务要求“最后放进去的最先被处理”——比如撤销、后退。 **直观解释**：栈像一摞盘子：你永远只能从**最上面**取放（官方 L05 称其为 LIFO：last-in, first-out）。它只开一个口（栈顶 top），开口少反而让它行为确定、几乎不可能误操作。 **操作/步骤分解**（官方 Stack 与 std::stack 对照）：

语义	Stanford Stack	std::stack	说明
压入	`push(value)`	`push(value)`	放到栈顶
弹出	`pop()`（**返回**元素）	`pop()`（**不返回**，返回 void）	移走栈顶
窥看	`peek()`	`top()`	看栈顶但不移走
判空/大小	`isEmpty()` / `size()`	`empty()` / `size()`	
清空	`clear()`	无(循环 pop 或换新栈)	

```
栈的抽象视图(只从顶部进出):
        ┌──────┐
        │  12  │ ← top: peek/pop 都在这
        ├──────┤
        │  15  │
        ├──────┤
        │  20  │
        ├──────┤
        │  10  │ ← 最先 push 的,沉在最底
        └──────┘
 push(7) → 7 落在 12 之上; pop() → 移走 12
```

注意：C++ 标准库的 `stack::pop()` 返回值是 `void`——必须先 `top()` 看一眼再 `pop()` 移走，两步完成“取出”。官方 Stanford 的 `pop()` 一步到位返回元素，这是两者最易踩的差异。 **现实应用（官方列举）**：① 反转任何序列（后进先出天然倒序）；② 程序本身的“调用栈 call stack”记录函数调用顺序与返回地址；③ 浏览器“后退”按钮按访问历史回退；④ 文本编辑器撤销（undo）操作栈。官方还预告：图论里的深度优先搜索（DFS）本质上也能用栈实现，课程后段会再见面。

3. Queue：FIFO，先进先出

问题定义：另一些任务讲究“先来先服务”的公平排队。 **直观解释**：队列像排队买票：新来的排到**队尾**，服务完的从**队首**离开——FIFO（first-in, first-out）。只开两个口：队首（front，出）与队尾（back，入）。 **操作/步骤分解**：

语义	Stanford Queue	std::queue	说明
入队	`enqueue(value)`	`push(value)`	加到队尾
出队	`dequeue()` （**返回**元素）	`pop()` （**不返回**）	移走队首
窥看	`peek()`	`front()`	看队首不移走
判空/大小	`isEmpty()` / `size()`	`empty()` / `size()`	

```
队列的抽象视图(队尾进,队首出):
  enqueue(7)                      dequeue()
      │                               │
      ▼                               ▼
[ 尾 ] [ 4 ][ 3 ][ 2 ][ 1 ] [ 首 ] → 移走 1,其余前移
```

现实应用：打印店的打印任务队列、演唱会购票排队、游戏登录排队、LaIR 答疑排队（官方 L05 全数点名）；图论中的广度优先搜索（BFS）用队列逐层扩散。**广度式处理**的通用模式是：先把“起点/初始任务”入队，然后循环“出队一个 → 处理 → 把它的后继任务入队”，直到队列空。 **清空/遍历的标准句式**：队列没有下标、不能用 range-for（std 的 stack/queue 都是“只露一头的适配器”），想遍历就只能一边出队一边处理：`while (!q.empty()) { 处理 q.front(); q.pop(); }`。官方 L05 特意演示了错误写法 `for (int i = 0; i < q.size(); i++)`——每出队一次 size 就变小，循环会提前结束，只处理掉一半任务。

4. 配套工具：容器按引用传递、`%`、`break`

问题定义：写操作这些容器的函数时，有什么约定俗成的规矩？ – **容器一律按引用传**：队列/栈可能装大量数据，按值传等于整份拷贝（时间 O(n)、内存翻倍）；写 `void f(queue<int>& q)` 只建立 O(1) 的“纽带”。即使函数不打算改动容器，也应写 `const queue<int>&` 以省拷贝（官方 L05 把它列为“top take-away”）。 – **取模 `%`**：返回整除的余数：`17 % 3 == 2`（17÷3=5 余 2），`5 % 2 == 1`。高频用途：奇偶判断（`x % 2`）、每 N 次做某事（`次数 % N == 0`）、环形下标前进 `(i + 1) % n`。 – **`break`**：立即跳出**当前所在的那一层**循环，跳到循环后第一行继续。常配合 `while (true)` 做“满足条件就收手”的中断。 – **range-for 与输出流**：std 的 stack/queue 不能 range-for、也不能直接 `cout << s`（官方 Stanford 版支持打印与 == 比较，这是课程库的贴心之处）；本笔记统一用“边出边处理”的方式展示内容。

5. 用 Vector 充当 Stack——抽象的力量

问题定义：栈和 vector 有什么关系？为什么有了 vector 还要 Stack？ **直观解释**：官方 L05 演示：`push/pop` 完全可以用 vector 的“末尾增删”模拟——`v.push_back(x)` 当 push、`v.pop_back()` 当 pop。但两者不可同日而语：写 stack 版本时语义一眼可见、无下标可算、几乎不可能出错；写 vector 版本要自己小心“该从哪头删”，一不留神就出界或删错端。这就是**抽象**的价值：把“只能从顶部操作”的约束内建进类型，错误在编译与设计层面就被挡掉了。std::vector 提供 `push_back/pop_back/back()` 恰好可作此用，而 std::stack 则把这一约束固化成了专用类型。

6. 经典应用：后缀表达式（postfix）求值

问题定义：人习惯中缀（infix）写法 `3 + 5 * 2`，但解析它要处理优先级与括号、多次扫描；如何让程序一次从左到右扫完就算出结果？ **直观解释**：后缀表达式（也叫逆波兰记法 RPN）把运算符放到两个操作数**之后**：`3 5 2 * +` 表示“3 与 (5×2) 相加”。它完全不需要括号和优先级规则，天然适合计算机从左到右单遍处理（官方 L05 补充说明里完整推导过）。**谁在帮我们算？** 一台只认得“数就压、符就取二合一”的栈机器。 **步骤分解（算法）**：准备一个空栈；从左到右读每个 token：

1. 读到**数字** → 压栈；
2. 读到**运算符**（+ – * /）→ 先 `pop` 出**右**操作数，再 `pop` 出**左**操作数（顺序关键！减法和除法左右颠倒结果就错），算完把结果压回栈；
3. 全部读完 → 栈顶唯一剩下的数字就是答案。

走查 `3 5 2 * + 12 2 3 * / -`（对应中缀 `3 + 5*2 - 12/(2*3)`）：

```
token   动作                 栈(自底向上)
3       数字,压栈            [3]
5       数字,压栈            [3,5]
2       数字,压栈            [3,5,2]
*       取 5*2=10,压栈       [3,10]
+       取 3+10=13,压栈      [13]
12      数字,压栈            [13,12]
2       数字,压栈            [13,12,2]
3       数字,压栈            [13,12,2,3]
*       取 2*3=6,压栈        [13,12,6]
/       取 12/6=2,压栈       [13,2]
-       取 13-2=11,压栈      [11]
结束    pop → 答案 11
```

健壮性检查：非法表达式会露出马脚——运算符出现时栈里不足 2 个数（操作数不够）、除数为 0、token 既非数字也非运算符、结束时栈里不是恰好 1 个数（说明多/少了操作数）。一个完整的 `processPostfix` 实现见下方示例 3，返回 bool 表示成功与否，失败时保持结果参数原值。

代码示例与实现详解

示例 1：用 std::stack 反转字符串 + 模拟浏览器“后退”（含 break）

```cpp
// 文件: stack_demo.cpp
// 演示: push/top/pop、清空栈的 while 句式、LIFO 反转、“后退”应用与 break
#include <iostream>
#include <stack>
#include <string>
using namespace std;

// 用栈反转字符串: 后进先出 = 天然倒序
string reverseViaStack(const string& text)
{
    stack<char> s;
    for (char ch : text) {
        s.push(ch);              // 逐个压栈
    }
    string out;
    while (!s.empty()) {         // 清空栈的标准句式
        out += s.top();          // 先 top() 看一眼栈顶
        s.pop();                 // 再 pop() 移走(标准库 pop 不返回值!)
    }
    return out;
}

int main()
{
    string word = "stressed";
    cout << "stressed 反转: " << reverseViaStack(word) << endl;  // desserts

    // 浏览器“后退”按钮: 历史记录就是一个栈,越新访问的越先被退回
    stack<string> history;
    history.push("首页");
    history.push("课程主页");
    history.push("L05 讲义页");

    cout << "开始点击后退…" << endl;
    while (!history.empty()) {
        string current = history.top();   // 当前停在哪一页
        history.pop();                    // 后退 = 弹出当前页
        cout << "离开: " << current << endl;
        if (current == "课程主页") {       // 回到课程主页就收手
            cout << "已回到课程主页,停止回退。" << endl;
            break;                        // break: 跳出最近的 while
        }
    }
    return 0;
}
```

【代码做什么】：`reverseViaStack` 把“stressed”每个字符压栈再全部弹出，得到“desserts”——LIFO 的反转威力一目了然；main 里用 `stack<string>` 存放访问历史，模拟浏览器后退：循环里“看栈顶 → pop 离开该页”，一旦弹出的是“课程主页”就 `break` 退出循环。

【实现机制解说】：栈的操作全在“顶”上发生，因此每个操作都是 O(1)。`s.pop()` 不返回被移除的元素是 C++ 标准库与官方 Stanford 版最大的差异：官方 `Stack::pop()` 直接返回栈顶，而 std 版必须 `top()` + `pop()` 两步走——若直接 `s.pop()` 后想用返回值，会拿到垃圾甚至编译错误。`break` 只作用于它所在的最近一层循环：本例它在 `while` 内部，因此触发后直接跳到循环右花括号之后。注意 `while (!s.empty())` 是“排空容器”的通用句式：任何循环里若一边遍历一边改变容器大小，千万别把 `s.empty()` 换成循环次数上限之类的固定值——那正是官方 L05 演示的队列遍历翻车点。

示例 2：用 std::queue 模拟打印任务队列（FIFO + `%` + 引用传递）

```cpp
// 文件: queue_demo.cpp
// 演示: push/front/pop、FIFO 打印队列、% 运算符、容器按引用传递
#include <iostream>
#include <queue>
#include <string>
using namespace std;

// 处理整个打印队列。参数必须是引用: 传值会把整支队伍拷贝一份(费时费内存)
void runPrinter(queue<int>& jobs)
{
    int done = 0;
    while (!jobs.empty()) {           // 边出队边处理,直到队列空
        int job = jobs.front();       // peek: 看队首(最早提交的任务)
        jobs.pop();                   // dequeue: 移走队首
        done++;
        cout << "打印完成: 任务 #" << job << endl;
        if (done % 3 == 0) {          // % 取模: 每完成 3 个汇报一次
            cout << "   —— 已累计完成 " << done << " 个任务" << endl;
        }
    }
    cout << "队列已空,打印机待机。" << endl;
}

int main()
{
    queue<int> printer;               // 打印任务按提交顺序排队(FIFO)
    for (int job = 1; job <= 6; job++) {
        printer.push(job);            // enqueue: 依次进队尾
    }
    cout << "队首(最先打印)任务: #" << printer.front() << endl;  // 1
    cout << "队尾任务: #" << printer.back() << endl;             // 6

    runPrinter(printer);              // 先进先出: 1,2,3,4,5,6

    // % 的另两个常见用法: 奇偶判断 与 环形下标
    for (int i = 1; i <= 5; i++) {
        if (i % 2 == 0) cout << i << " 是偶数, ";
    }
    cout << endl;
    cout << "环形下标: 6 个槽(编号 0..5)里, 槽 5 的下一个是槽 "
         << (5 + 1) % 6 << "  ← 公式 (i+1) % 总槽数" << endl;
    return 0;
}
```

【代码做什么】：任务 1..6 按序入队，`front()` / `back()` 分别展示队首队尾，随后 `runPrinter` 用“看队首 → pop”的句式按 FIFO 顺序打印全部任务（输出 1 到 6，证明先来先服务），并用 `done % 3 == 0` 每三个任务汇报一次进度；结尾顺带展示 `%` 的奇偶判断与环形下标两种常见用法。

【实现机制解说】：`runPrinter(queue<int>& jobs)` 用引用而非值——若写 `queue<int> jobs`，函数入口就要把整支队伍逐元素拷贝（O(n) 时间 + 双倍内存），函数里清空的也只是一份副本，调用方的队列原封不动。这是官方 L05 反复强调的规矩：**容器进函数一律走引用**；不打算改就加 `const`。打印顺序 1→6 恰好验

证 FIFO：`push` 全在队尾、`pop` 全在队首，新任务永远不可能插队。`%` 的实质是“整除的余数”，`done % 3 == 0` 在 done=3、6 时为真——“每 N 次触发一次”是它的招牌用法；环形下标 `(i+1) % n` 让下标在 0..n−1 间循环打转，是轮询调度（round-robin）的基础。

示例 3：后缀表达式求值器（完整健壮版）

```cpp
// 文件: postfix_demo.cpp
// 演示: 用 std::stack 单遍求值后缀表达式,含非法输入检测
#include <iostream>
#include <sstream>   // istringstream: 按空白切分字符串
#include <stack>
#include <string>
using namespace std;

// 求值后缀表达式 expr; 成功返回 true 并把答案写进 result;
// 失败(非法表达式/除零)返回 false 且 result 保持原值不变。
bool processPostfix(const string& expr, int& result)
{
    stack<int> s;
    istringstream iss(expr);
    string token;
    while (iss >> token) {                        // 逐个取 token(按空格切)
        if (token == "+" || token == "-" ||
            token == "*" || token == "/") {
            if (s.size() < 2) return false;       // 操作数不够 → 非法
            int rhs = s.top(); s.pop();           // 先弹出的是右操作数!
            int lhs = s.top(); s.pop();           // 再弹出的是左操作数
            if (token == "+")      s.push(lhs + rhs);
            else if (token == "-") s.push(lhs - rhs);
            else if (token == "*") s.push(lhs * rhs);
            else {                                // 除法: 防除零
                if (rhs == 0) return false;
                s.push(lhs / rhs);
            }
        } else {                                  // 既非四则符 → 尝试当整数
            try {
                s.push(stoi(token));
            } catch (...) {
                return false;                     // 既非运算符又非整数 → 非法
            }
        }
    }
    if (s.size() != 1) return false;              // 栈应恰好只剩最终答案
    result = s.top();
    return true;
}

int main()
{
    const string exprs[] = {
        "3 5 2 * + 12 2 3 * / -",   // 等价于 3+5*2-12/(2*3) = 11
        "5 10 +",                   // 15
        "10 12 + 5 -",              // 17
        "2 3 + 4 5 + *",            // 45
        "5 10 + +",                 // 非法: 第二个 + 缺操作数
        "10 + 20",                  // 非法: 开头两个数没到就先遇运算符
        "4 0 /",                    // 非法: 除零
        "3 x 2 +",                  // 非法: x 不是数字也不是运算符
        ""                          // 非法: 空串
```

```
    };
    for (const string& e : exprs) {
        int result = 0;                              // 每次重置,便于观察“失败不改值”
        if (processPostfix(e, result)) {
            cout << "\"" << e << "\" = " << result << endl;
        } else {
            cout << "\"" << e << "\" → 非法表达式(结果仍为 " << result << ")" << endl;
        }
    }
    return 0;
}
```

【代码做什么】：`processPostfix` 从左到右单遍扫描：数字压栈，运算符则弹两个操作数（先弹右、后弹左）计算后压回；结束时若栈里恰好剩一个数即为答案。main 用 9 个用例覆盖：4 个合法表达式（含官方 L05 的经典式 `3 5 2 * + 12 2 3 * / -` = 11）与 5 类非法输入（缺操作数、数字不足、除零、乱 token、空串）。

【实现机制解说】：算法成立的关键是**栈暂存子结果**：遇到运算符时，栈顶两个数正是它该吃的“最近两个数”，算完压回后它们以单个结果的身份继续参与外层运算——这恰好是表达式树的后序遍历求值，因此无需括号与优先级。弹栈顺序必须“先右后左”：`-` 与 `/` 不满足交换律，`10 12 + 5 -` 若先弹左会算出 7 而非 17。`istringstream >> token` 自动按任意空白切词，比手写 split 简洁。错误处理走“早退原则”：任一环节发现非法立即 `return false`，且绝不动 `result`——官方练习强调用“失败不改参数”来测试，防止调用者误以为失败会写入哨兵值。该函数与官方 L05 课后练习 `bool processPostfix(string expr, int& result)` 同款签名，这里用纯标准库实现。

复杂度分析

操作	Stack (std::stack)	Queue (std::queue)	说明
push / enqueue	O(1) 均摊	O(1) 均摊	底部容器扩容偶尔整体搬迁
pop / dequeue	O(1)	O(1)	只动一端
top() / front()（peek）	O(1)	O(1)	只看不移
empty() / size()	O(1)	O(1)	计数被缓存
查找某元素 / 按下标访问	不支持	不支持	只开一头的容器刻意不提供
遍历全部元素	O(n)	O(n)	只能边出边处理(会清空)
空间	O(n)	O(n)	存 n 个元素

要点：栈与队列把“能做的事”刻意收窄，换来的是**所有核心操作都是 O(1)**，且语义无歧义、几乎不可能误用。std 默认用 deque 兜底（stack 也可指定 vector），两种底层都不影响客户端看到的复杂度。

关键要点

- 栈是 LIFO：只能碰栈顶，push 进去、pop 出来，天然反转一切“后来居上”的序列。

- 队列是 FIFO：队尾进、队首出，“先来先服务”，是公平排队的代名词。
- 清空/遍历栈与队列只有一种标准句式：`while (!空) { 取顶/首; 弹出; }`，绝不要用固定循环次数。
- 容器进出函数一律传引用（不修改就 `const &`），按值传等于白拷一份 O(n) 的数据。
- 后缀表达式求值 = “数字压栈、运算符取二合一”的栈机器，单遍扫描即可，先弹右操作数再弹左操作数。

常见陷阱与注意事项

- **`std::stack::pop()` 不返回元素**：直接 `int x = s.pop();` 编译报错。规避：先 `top()` 取值再 `pop()`。
- **循环条件随出队变小**：`for (i = 0; i < q.size(); i++) { q.pop(); }` 只处理一半。规避：一律 `while (!q.empty())`。
- **空容器上操作**：对空栈 `top()`/`pop()`、空队列 `front()`/`pop()` 是未定义行为，可能崩溃。规避：操作前先判空。
- **弹栈顺序弄反**：后缀求值里 `-`、`/` 先弹右操作数，反了结果错。规避：把“先弹右、后弹左”写进注释并自测减法/除法用例。
- **按值传容器**：`void f(queue<int> q)` 隐式整队拷贝。规避：写成 `queue<int>&` 或 `const queue<int>&`。
- **误以为栈/队列可随机访问或可 range-for**：下标、迭代器一概没有。规避：想“翻看”全部内容就先想清楚是否需要排队/栈语义，或改用 vector。
- **`%` 与 `/` 混淆**：`17 % 3` 是余数 2，`17 / 3` 是商 5。规避：默念“% 是取余”。

思考题（带答案）

问题 1：依次 `push(1), push(2), pop(), push(3), pop(), pop()` 后栈为空。问每次 `pop()` 各返回什么？若换成队列（push 当 enqueue、pop 当 dequeue）结果又如何？ **答案**：栈是 LIFO，三次 pop 依次返回 2、3、1（每次 pop 的都是当时的栈顶）。队列是 FIFO，三次 pop 依次返回 1、2、3（每次 pop 的都是最早的队首）。同一组操作、两种容器、完全相反的输出顺序——这正是 LIFO 与 FIFO 语义差异的最佳记忆点。

问题 2：为什么官方 L05 说“用 vector 也能当栈用，但我们还是推荐 Stack”？举一个用 vector 模拟栈时容易犯的错。 **答案**：vector 的 `push_back`/`pop_back` 确实能模拟栈，但 vector 同时暴露了下标、`insert`、`erase` 等“多余能力”，使用者要时刻自律“只能动尾巴”，容易下标算错、从错误的端删除、甚至越界。Stack 把“只能从顶部进出”固化为类型约束，犯错空间被设计层面抹掉——这就是抽象的力量：用更受限的接口换更低的出错率。

问题 3：给出后缀式 `2 3 4 * + 5 +` 的求值过程（写出每一步后的栈）。 **答案**：2 压栈 `[2]`；3 压栈 `[2,3]`；4 压栈 `[2,3,4]`；`*` 取 3×4=12 压回 `[2,12]`；`+` 取 2+12=14 压回 `[14]`；5 压栈 `[14,5]`；`+` 取 14+5=19 压回 `[19]`；结束，答案 19。（可对照官方 L05 练习用例 `"2 3 4 * + 5 +" == 19`。）

Lecture 3: 集合与映射（Set & Map：去重、词频统计与基于树的有序容器）（对应课程真实讲座 L06）

概述

本讲解决两个高频需求：**去重**（同一批数据里只保留每种元素一份）与**关联查询**（按某个键快速找到对应的值）。为此引入两个新 ADT：集合 Set（元素的“成员关系”容器）与映射 Map（键→值的查表容器）。两者都建立在“有序结构”之上：元素/键始终按序排列，插入与查找都快得惊人（O(log n)），与课堂上之后才讲的哈希版本（HashSet/HashMap，平均 O(1)）形成对照。本讲用一个“去重挑战题”开场，并实现词频统计经典应用。 对应官方 L06（6/30，Sets and Maps）一讲；官方课上用《德古拉》全文做过词频统计演示。

核心概念与算法原理

1. 挑战题：去重（De-Dupe）

问题定义：写一个函数，输入一串字符串，把其中每种字符串**恰好打印一次**。例如 `{"unicorn", "starfish", "hummingbird", "starfish", "unicorn", "unicorn"}` 应打印 unicorn、starfish、hummingbird 各一次。 **直观解释**：官方 L06 开场先给“笨办法”：对每个元素回看它之前有没有出现过（双重循环），再进阶成“用一个辅助函数查区间”。两种做法都能对，但要么 $O(n^2)$ 慢、要么代码绕。直到把元素“扔进一个 Set”——重复项被容器自动吸收——一行核心逻辑解决问题，课堂的用意就是让大家先体会“没有好工具的痛”，再享受 ADT 的甜。 **朴素法步骤分解**：对第 i 个元素，向前扫描 0..i-1 检查是否重复，无重复才打印——最坏要两两比较 $n^2/2$ 次。**Set 法**：见示例 1。

2. Set：无重复的“成员判断机”

问题定义：需要一种容器回答“某元素在不在里面”，并且保证绝不存两份。 **直观解释**：数学集合的数据结构版。两个铁律（官方 L06 原话拆解）：① **不允许重复**——同一元素插 100 次也只留 1 份；② **不保存插入顺序、也没有下标**。所以 Set 的本质是“二元成员判断装置”：一个元素要么是成员，要么不是。 **操作/步骤分解**（官方 Set ↔ std::set）：

语义	Stanford Set	std::set	说明
加入	`add(value)`	`insert(value)`	已存在则静默忽略
判断在否	`contains(value)`	`count(value)`（0/1）或 `find`	二元判断,不是计数
删除	`remove(value)`	`erase(value)`	不在则无事发生
判空/大小	`isEmpty()` / `size()`	`empty()` / `size()`	
运算符	`s1+s2` 并、`s1*s2` 交、`s1-s2` 差	无内建运算符,用 `set_union` 等算法或手写	课程库的贴心重载

语义	Stanford Set	std::set	说明
遍历	range-for（有序）	range-for（有序）	无下标、无随机访问

“有序性”与内部机制：打印或遍历 Set 时元素总是按序出现（官方 L06 点名：按 **ASCII 序**排列——大写字母排在小写前，因为 `'A'=65 < 'a'=97`）。这不是巧合：官方 Stanford Set 由**平衡二叉搜索树（balanced BST）**支撑，元素按键有序存放，从而插入、查找、删除都只要 O(log n)。C++ 标准库的 `std::set` 同样是平衡 BST（红黑树），行为完全一致——本笔记一律用 std 版，与课程的 Set 一一对应。

```
std::set<string> 内部是二叉搜索树(红黑树), 插入 "starfish" 已存在 → 直接忽略:
           hummingbird
          /            \
      dragon          starfish
                          \
                        unicorn
  遍历(中序)= dragon, hummingbird, starfish, unicorn  ← 天然有序
```

步骤分解（插入）：从根开始，与当前节点比较大小：小走左、大走右、相等说明重复直接返回——一路下行到空位即挂上新节点。树高约 $\log_2 n$，故每次比较 O(log n)。**遍历**：中序遍历（左—根—右）即得升序序列。

3. Set 的应用：去重与查重

- **全量去重**：把 vector 元素全部 `insert` 进 set，重复被自动吸收；要回 vector 就遍历 set 倒回去（代价：排序后的顺序与原顺序无关）。
- **保留首次出现顺序的去重**：set 只当“查重台账”——遍历原 vector，若 `count(w)==0` 就打印并登记。
- **找重复项**：登记“已见过”，第二次见到就报“重复”；再套一个 set 可保证每个重复项只报一次（官方 L06 有整套变体练习）。
- **“Set 很快”**：官方 L06 强调 Set 操作远比“反复扫 vector 查重”快——后者最坏 $O(n^2)$，Set 版每个元素只花 O(log n) 的树查找，总耗时 O(n log n)。

4. Map：键 → 值 的“活字典”

问题定义：要按“键”快速检索“值”：学号→姓名、ISBN→书名、单词→出现次数。 **直观解释**：Map 是关联式（associative）结构：每个**键唯一**、恰好映射**一个值**。官方 L06 的直观例子：把全班的社保号映射到姓名；喂进一个社保号，吐出一个名字。键集合本身就是一个 Set（键互不相同、按序排列），值是它“名下”挂的东西。 **操作/步骤分解**：

语义	Stanford Map	std::map	说明
建映射	`m[key] = value` / `put`	`m[key] = value` / `insert`	键已存在则**覆盖**旧值
取值	`m[key]` 或 `get(key)`	`m[key]` 或 `at(key)`	细节见下
探键	`containsKey(key)`	`count(key)` / `find(key)`	判断键在否

语义	Stanford Map	std::map	说明
删键	`remove(key)`	`erase(key)`	
键集合/值集合	`keys()` / `values()`	无现成(遍历取 first/second)	
遍历	range-for 得到键	range-for 得到 `pair<key,value>`	按键升序

“探键”行为——官方 L06 的招牌知识点：

1. 查询不存在的键时返回**该值类型的默认值**（int 得 0、string 得空串）；
2. 用 `m[key]` 语法查询不存在的键时，map **会把该键加进去**并配上默认值——副作用！官方演示：`map["Sonia"]` 查完，map 里多出一个 “Sonia: 空串”。`get(key)` 则只返回默认值、**不插入**。

C++ 标准库忠实复刻了这一对行为：`std::map` 的 `operator[]` 找不到键就**插入**默认值；`at(key)` 找不到则抛异常（相当于“无副作用查询”，但要小心异常）；`count(key)` 只回答“在不在”。惯用法：先 `count` 探键、再 `[]` 取值，避免误插与异常。 **键不可变语义**：给已有键赋新值 = 覆盖旧映射（官方：`m["Julie"] = "Zelenski"` 之后再 `m["Julie"] = "Stanford"`，旧值被覆盖，size 不变）。 **多值关联（键 → 容器）**：一个键只能映射一个值，但那个值可以是整个容器！`map<string, vector<string>>` 让“同名（键）→ 多个姓氏（值容器）”成为可能。官方 L06 强调：取出容器后必须用**引用**接收（`vector<string>& v = m["Julie"]`），否则拿到的是拷贝，往里 `add` 等于白干（练习 2 专门考这一点）。

5. 词频统计与“出现最多的词”

问题定义：给一段文本，统计每个词出现几次；进一步找出出现次数最多的词。 **直观解释**：官方 L06 用《德古拉》全文演示：`map<string,int>` 键为单词、值为次数，一行 `counts[word]++` 完成“没见过就记 1、见过就加 1”的完整逻辑——因为 `[]` 对不存在的键先补 0 再自增。要按频率找最热词，遍历键值对维护最大值即可（示例 2）。 **命名约定（官方 L06 提及）**：map 变量名建议写成“键To值”式，如 `wordToFrequency`、`isbnToTitle`，把键值关系直接写进名字；词频表常见命名 `counts`。

6. 有序 vs 无序：Set/Map 与 HashSet/HashMap 的取舍

问题定义：既然要“超快查找”，为什么课程先教“有序版”而非哈希版？ **直观解释**：官方 L06 预告：课程库里同时存在 HashSet/HashMap（无序、基于哈希表，平均 O(1)）与 Set/Map（有序、基于平衡 BST，O(log n)）。四个容器的查找都快到在日常数据上几乎无感；差别在于：**有序版保证遍历有序、支持“找前驱/后继、区间”等操作，代价是每次 O(log n)；无序版更快但顺序随机**。官方明确“本期只需知道概念差异，哈希实现细节到第 24 讲”。 **对照表**：

维度	std::set / std::map（课程 Set/Map）	std::unordered_set / unordered_map（课程 HashSet/HashMap）
内部结构	平衡 BST（红黑树）	哈希表（桶 + 哈希函数）
插入/查找/删除	O(log n)	平均 O(1)、最坏 O(n)
遍历顺序	按键升序（可预期）	无意义随机序

维度	std::set / std::map（课程 Set/Map）	std::unordered_set / unordered_map（课程 HashSet/HashMap）
需要	元素可比较（`<`）	元素可哈希（`hash`）
适用	要有序输出、范围查询	只要速度、不在乎顺序

```
同样插入 {"starfish","unicorn","hummingbird","dragon"}:
  std::set 遍历:    dragon → hummingbird → starfish → unicorn   (有序)
  std::unordered_set 遍历:  顺序随机,每次运行都可能不同
```

代码示例与实现详解

示例 1：Set 去重——挑战题的标准解法与“保序”变体

```cpp
// 文件: set_dedupe_demo.cpp
// 演示: std::set 去重(有序输出) 与 保留首次出现顺序的去重
#include <iostream>
#include <set>
#include <string>
#include <vector>
using namespace std;

// 解法 A: 把元素全扔进 set,重复项自动被吸收; 遍历即得有序去重结果
void printUniqueSorted(const vector<string>& words)
{
    set<string> uniqueSet;                  // 空集合
    for (const string& w : words) {
        uniqueSet.insert(w);                // 重复插入被静默忽略
    }
    for (const string& w : uniqueSet) {     // 按键升序遍历
        cout << w << endl;
    }
}

// 解法 B: 用 set 当“查重台账”,保持元素在原 vector 里的首次出现顺序
void printUniqueInOrder(const vector<string>& words)
{
    set<string> seen;                       // 记录“已见过的词”
    for (const string& w : words) {
        if (seen.count(w) == 0) {           // 首次出现才打印 (count 返回 0 或 1)
            cout << w << endl;
            seen.insert(w);                 // 登记,防止下次再打印
        }
    }
}

int main()
{
    vector<string> creatures = {"unicorn", "starfish", "hummingbird",
                                "starfish", "unicorn", "unicorn"};
    cout << "== 解法 A: 有序去重输出 ==" << endl;
    printUniqueSorted(creatures);           // hummingbird / starfish / unicorn
    cout << "== 解法 B: 保留首次出现顺序 ==" << endl;
    printUniqueInOrder(creatures);          // unicorn / starfish / hummingbird
    return 0;
}
```

【代码做什么】：解法 A 一字排开地 `insert`，由 set 自己吞掉重复，遍历输出即升序去重结果；解法 B 用 `seen.count(w) == 0` 判断“从没见过”，头一回见到才打印并登记。main 用官方 L06 的独角兽/海星/蜂鸟数据验证两种输出的顺序差异。

【实现机制解说】：`set::insert` 返回 `pair<迭代器, bool>`，`bool` 位告知“这次是否真的插入了”（已存在时为 false）——官方练习里“只打印重复项一次”等变体可借它实现。`count(w)` 对 set 只会返回 0 或 1，因为集合语义禁止重复，用它判断成员关系最直白。**复杂度对比**：若用 vector 的双重循环去重，最坏 $O(n^2)$

次比较；这里每个词一次树查找 O(log n)，总 O(n log n)。遍历 set 得到的是**升序**而非插入序，所以解法 A 与解法 B 输出顺序不同——想保序就自己维护“首次出现”逻辑，想让容器代劳排序就接受字典序。

示例 2：Map 词频统计——找“出现最多的词”

```
// 文件: word_freq_demo.cpp
// 演示: std::map 词频统计、按键序遍历、查找最高频词(官方用《德古拉》全文,
//       这里用内嵌小文本代替文件输入,逻辑完全一致)
#include <iostream>
#include <map>
#include <sstream>   // istringstream: 自动按空白切词
#include <string>
using namespace std;

int main()
{
    // 一段模拟课文(可想象成官方课上打开的 poem.txt / dracula.txt)
    const string text =
        "roses are red butterflies are beautiful "
        "red roses are lovely beautiful butterflies";

    map<string, int> wordToFreq;        // 命名约定: 键To值
    istringstream iss(text);
    string word;
    while (iss >> word) {
        wordToFreq[word]++;             // 探键: 新词自动补 0,再自增
    }

    // (1) 遍历: std::map 的 range-for 每次给出 pair<const 键, 值>
    cout << "== 词频表(按键升序) ==" << endl;
    for (const auto& kv : wordToFreq) {
        cout << kv.first << ": " << kv.second << endl;
    }

    // (2) 找出现次数最多的词(遍历一遍,维护当前冠军)
    string topWord;
    int topFreq = -1;
    for (const auto& kv : wordToFreq) {
        if (kv.second > topFreq) {      // 严格大于: 平手时保留先遇到的
            topWord = kv.first;
            topFreq = kv.second;
        }
    }
    cout << "出现最多的词: \"" << topWord << "\", 共 " << topFreq << " 次" << endl;

    // (3) 探键行为演示: operator[] 会“顺手”插入默认值
    cout << "查不在表里的词 zzz 的次数: " << wordToFreq["zzz"] << endl;
    cout << "副作用? zzz 被插进表了: " << wordToFreq.count("zzz") << " (1=是)" << endl;
    cout << "用 count 探键(无副作用): " << wordToFreq.count("qix") << endl;
    return 0;
}
```

【代码做什么】：istringstream 按空白把课文切成单词，`wordToFreq[word]++` 一行完成“首次出现记 1、再次出现加 1”；随后按键升序打印词频表，再单遍扫描找出最高频词；最后演示 `operator[]` 探键会插入默认值、`count()` 无副作用。

【实现机制解说】：词频统计的“魔法”全在 `wordToFreq[word]++` 的求值顺序：`operator[]` 找不到键时先构造 `{word, 0}` 插入并返回其引用，`++` 再把它加到 1；找得到则直接对旧值自增——两行 if/else 压缩成一行。这与官方 Stanford Map 的 `m[word]++` 行为一致（官方特别注明 `get(word)++` 会编译失败，因为 `get` 返回的是临时值不可自增）。`const auto& kv` 中 `kv.first` 是键、`kv.second` 是值；`auto&` 引用遍历避免复制整个 pair。找最高频词是线性扫描：map 已按键排好序，但“最大频率”与键序无关，必须逐对比较——复杂度 O(n)。最后一个知识点是**副作用**：查 “zzz” 之后表里真的多了 `zzz:0`；需要“只查不改”就用 `count` / `find`（或 `at`，但要注意它找不到会抛异常）。官方课上跑通《德古拉》全文后还加了“打印出现超过 100 次的词”的阈值筛选，道理与这里完全相同。

示例 3：Map 的多值关联——键 → 容器，以及引用陷阱

```cpp
// 文件: multivalue_map_demo.cpp
// 演示: map<string, vector<string>> 让一个键关联多个值(值本身是个容器)
#include <iostream>
#include <map>
#include <string>
#include <vector>
using namespace std;

int main()
{
    // 课程 → 选课学生名单 (键唯一, 值是一个可增长的 vector)
    map<string, vector<string>> courseToStudents;

    // 关键: 用【引用】接收返回值! 否则拿到拷贝, push_back 改的是副本
    vector<string>& cs106b = courseToStudents["CS106B"];
    cs106b.push_back("Ada");
    cs106b.push_back("Alan");

    // 也可以不建中间变量, 直接链式操作(每次返回的都是同一份引用)
    courseToStudents["CS103"].push_back("Grace");
    courseToStudents["CS103"].push_back("Edsger");

    // 遍历: 键升序; 每个键名下再遍历其 vector
    cout << "== 选课名单(按键升序) ==" << endl;
    for (const auto& kv : courseToStudents) {
        cout << kv.first << ": ";
        for (const string& name : kv.second) {
            cout << name << " ";
        }
        cout << endl;
    }

    // 对照: 若忘记引用, 会发生什么?
    vector<string> copy = courseToStudents["CS106B"];   // 整份拷贝!
    copy.push_back("Linus");                            // 只改了副本
    cout << "CS106B 名单末尾是否多了 Linus? "
         << (courseToStudents["CS106B"].back() == "Linus" ? "是" : "否(拷贝被丢弃)")
         << endl;
    return 0;
}
```

【代码做什么】：以“课程→学生名单”演示键到容器的映射：用引用接收 `operator[]` 的返回值后 push_back，真正把学生加进 map 内的 vector；遍历打印时外层按键升序、内层逐个输出名字；最后故意演示“忘写引用”的后果——改动只落在副本上，被悄悄丢弃。

【实现机制解说】：`map` 的值类型是 `vector<string>` 时，`m[key]` 的返回类型是 `vector<string>&`（引用）。写成 `vector<string> v = m["CS106B"]` 会触发**拷贝构造**——复制整条名单；对 `v` 的任何修改都与 map 无关，函数结束副本销毁，改动蒸发。官方 L06 练习 2 正是这个坑：没写 `&` 时打印出的名单全是空的。这也是“值语义”的体现：C++ 里普通赋值默认拷贝，想“拿到并操作原件”必须显式用引用或指针。此外注意 `kv.second` 遍历 vector 时用 `const string& name` 只读引用，避免每轮复制一个字符串。

复杂度分析

操作	std::set / std::map	std::unordered_set / unordered_map	原因
insert / operator[]（新键）	O(log n)	平均 O(1)，最坏 O(n)	平衡树沿路径下钻 vs 哈希桶定位
count / find（查找）	O(log n)	平均 O(1)，最坏 O(n)	同上
erase / remove	O(log n)	平均 O(1)，最坏 O(n)	同上
size / empty	O(1)	O(1)	计数缓存
遍历全部元素	O(n)（且有序）	O(n)（无序）	树中序 / 桶扫描
用 set 去重 n 个元素	O(n log n)	平均 O(n)	n 次插入 × 单次代价

要点：有序容器把“保持有序”内建进每次操作，换来可预期的升序遍历；无序容器放弃顺序换平均常数时间。两者都远快于“在 vector 里线性扫描查重”的 $O(n^2)$。n 不大时差距无所谓，选型口诀：**要顺序输出用 set/map，只求快不求序用 unordered 版**（官方 L06 亦如此建议）。

关键要点

- Set 是“二元成员机”：同一元素永远只存一份，回答只有“在/不在”，遍历天然升序。
- 有序的代价与回报对称：set/map 每次 O(log n)，换来排序遍历；由平衡 BST（红黑树）支撑。
- `map[key]` 会“探键即插入”：查不存在的键会顺手塞进一个默认值；只查不改用 `count` / `find`。
- 键要“挂”多个值：让值本身成为容器（如 `map<string, vector<string>>`），且取出时务必用引用接收。
- 词频统计一行流：`counts[word]++` = “没见过记 1、见过加 1”；变量命名用“键To值”，如 `wordToFreq`。

常见陷阱与注意事项

- **误以为 set 保插入序**：set 遍历永远是升序。规避：要保序就去重时自己维护“首次出现”判断。
- **`map[key]` 查询的副作用**：读一下不存在的键就污染了表。规避：先 `count(key)` 再决定是否 `[]` 取值。
- **拿 map 的容器值时不写引用**：`vector<string> v = m[k]` 是拷贝，改动无效。规避：写 `auto& v = m[k]`。
- **对 set 用下标**：set 没有下标、没有 `[]`。规避：用 `count` / `find` 判断成员、range-for 遍历。
- **值覆盖的误判**：`m[k] = v2` 不会新增一对，而是覆盖旧值。规避：先想清楚“覆盖 vs 新增”的语义，必要时先 `count`。
- **给元素排序的依据想当然**：字符串按 ASCII 序排，大写全在小写之前（`"Apple" < "banana"`）。规避：要“正常字典序”先统一小写再入 set/map。

- **遍历时修改容器**：range-for 遍历 set/map 时插入或删除元素会使迭代器失效。规避：先收集要删的键，遍历结束后再删。

思考题（带答案）

问题 1：`vector<string> v = {"a","b","a","c","b"}`，想把 v 变成无重复版本（顺序随意），代码怎么写？复杂度多少？ **答案**：`set<string> s(v.begin(), v.end()); v.assign(s.begin(), s.end());`——构造 set 自动去重并排序，再倒回 vector。复杂度 O(n log n)（n 次插入 × log n）。要保留原顺序则改为：遍历 v，用 `set<string> seen` 判重，首次见到的元素 push_back 进新 vector。

问题 2：为什么官方 L06 说 Set/Map 的插入查找“很快”，比“反复扫 vector”快？请给出数量级对比。 **答案**：vector 里线性查重 n 个元素最坏 $O(n^2)$ 次比较；set 每次插入沿树下行 O(log n)，共 O(n log n)。对 n=100 万，前者约 10^{12} 次比较（不可行），后者约 2×10^7 次（瞬间完成）。树高 $\log_2 n$ 意味着“规模翻倍只多 1 层比较”——这就是 O(log n) 的威力（下一讲用大 O 记号正式刻画）。

问题 3：写出用 map 统计字符频次的代码片段：给定 `string s`，统计每个字母出现次数并输出次数最多的字母。 **答案**：`map<char,int> freq; for (char ch : s) freq[ch]++;` 然后单遍扫描找最大：`char best; int mx=-1; for (auto& kv : freq) if (kv.second > mx) { mx = kv.second; best = kv.first; }`。若要忽略大小写/只统计字母，可先 `tolower` 并用 `isalpha` 过滤（把示例 2 的“词”换成“字符”即可）。

Lecture 4: 算法分析：大 O 记号与运行时间估算（Big-O & Algorithmic Analysis）（对应课程真实讲座 L07）

概述

本讲回答一个贯穿全课程的问题：“这个算法到底快不快？”先论证两种直觉方案都不可靠——用秒表计时受机器与负载干扰，逐条数指令又繁琐且无意义；随后引入**大 O 记号（Big-O）**：用“输入变大时工作量如何增长”来刻画算法的本质快慢。接着建立常见增长函数族的直觉（常数、对数、线性、n log n、二次、指数……），推导求和恒等式 1+2+…+n = n(n+1)/2，并据此解释为何 `add` 均摊很快而 `insert(0,·)` 慢到爆炸，最后学会根据增长阶数估算可处理的问题规模。 对应官方 L07（7/1，Big-O and Algorithmic Analysis）一讲；官方随讲给出 Prezi 与“对数运行时的推导”补充视频。

核心概念与算法原理

1. 为什么“墙钟计时”与“数指令”都不靠谱

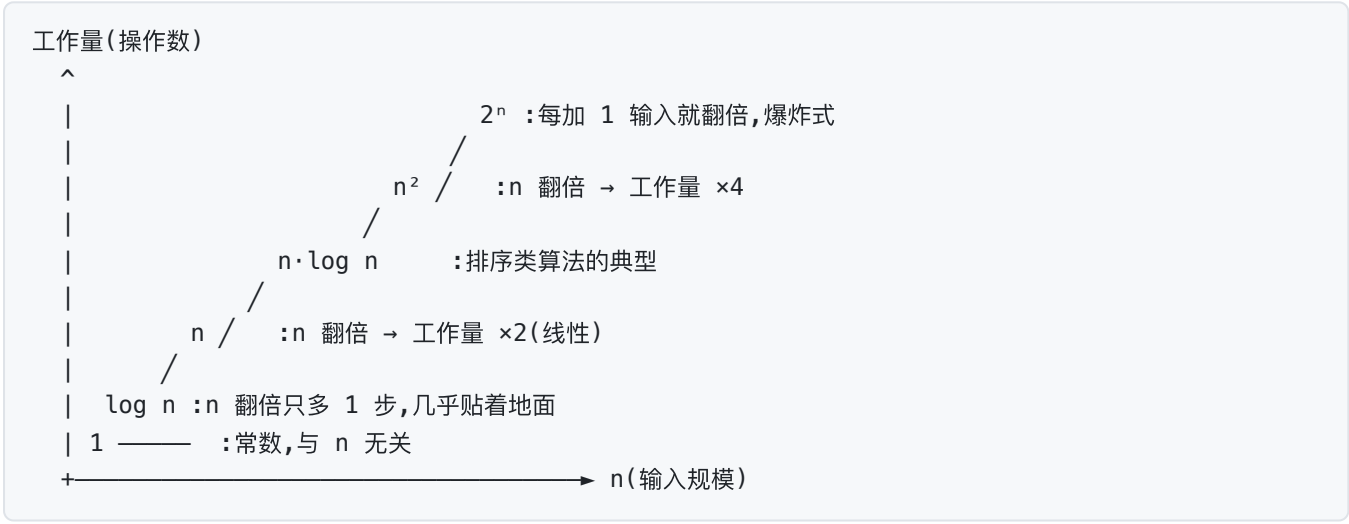
问题定义：如何公平地比较两个解决同一问题的算法谁快？ **直观解释**：官方 L07 先泼两盆冷水。**方案一：直接掐秒表（墙钟时间）**——同一程序在不同机器、不同负载下时间天差地别；后台开个浏览器都能干扰结果；测试用例大小不公平，结论就失真。**方案二：数代码执行了多少条“操作”**（赋值、比较、算术、函数调用……）——听起来更“公平”，但很容易数漏（`i++` 其实含一次算术+一次赋值，`v[i]` 背后藏着一乘一加的内存寻址……），而且不同指令在 CPU 上的时钟周期不同，编译器还会优化改写你的代码——数出来的

4n+4 这种数字基本没有意义（官方原话：这是“傻瓜的差事”）。 **结论**：我们真正该关心的是**增长率**——当输入规模 n 增大时，工作量以什么“形态”增长：是翻倍就翻倍（线性），还是翻倍就变四倍（二次）？ 大 O 记号就是描述这种形态的语言。

2. Big-O 的直觉与（非正式）定义

问题定义：给运行时间函数 T(n) 找一个最简“成长形状”来描述它。 **直观解释**：设输入规模为 n（数组长度、字符串长度、某个整数等）。Big-O 回答：“当 n 变得**足够大**时，T(n) 大致像什么函数在长？”它只关心“最高阶项”，忽略常数系数与低阶项——因为这些在 n 巨大时才是决定命运的。 **严谨定义（了解即可）**：称 $T(n) = O(f(n))$，若存在常数 c 与 n_0，使得对所有 $n \geqslant n_0$ 都有 $T(n) \leqslant c \cdot f(n)$。意思是：从某个足够大的 n 开始，T(n) 的曲线总被 $c \cdot f(n)$ 压住。 **化简法则（官方 L07 三步流程）**： 1. 假设输入任意大； 2. 找到执行次数最多的语句，统计它跑了多少次（机器无关的近似）； 3. 扔掉常数系数、只留最高阶项 → 大 O 结果。 例如 $T(n) = 4n + 4 \to O(n)$；$T(n) = (1/6)n^2 + 1000n \to O(n^2)$；$T(n) = 6n + 2^n \to O(2^n)$（指数项吞掉一切多项式）。

3. 常见增长函数族与直觉



记号	读法	中文	直觉样例	n=100 时的量级
O(1)	Big-O of one	常数	数组按下标取元素	1
O(log n)	Big-O of log n	对数	反复把输入减半（二分查找）	≈7（$\log_2 100$）
O(n)	Big-O of n	线性	单遍扫描求最大值	100
O(n log n)	Big-O of n log n	线性对数	归并排序（后续讲）	≈700
$O(n^2)$	Big-O of n squared	二次	双重循环、`insert(0,·)`×n	10,000
$O(2^n)$	Big-O of 2 to the n	指数	枚举硬币 n 次的全部正反序列	$\approx 10^{30}$
O(n!)	Big-O of n factorial	阶乘	枚举 n 个物品的全排列（补充）	$\approx 10^{158}$

增长顺序铁律（n 巨大时）：$1 < \log n < n < n \log n < n^2 < 2^n < n!$。前两名几乎贴地、后两名都是“灾难级”，中间的差距也动辄千万倍。官方 L07 特别提醒一个**常见误解**：不要把 O(log n) 看成“O(1) 与 O(n) 之间的一半”——对数增长极其接近常数：输入从 1 涨到 10 亿，$\log_2$ 只从 0 涨到 30。

4. 求和恒等式：1+2+…+n = n(n+1)/2

问题定义：`insert(0,·)` 连续 n 次总共搬移 1+2+…+n 个元素，这串和到底等于什么？ **推导（官方 L07 同款手法）**：设 S = 1 + 2 + … + (n−1) + n，把它倒过来再写一遍，两式相加：

```
  S =   1   +   2   + … + (n-1) +   n
+ S =   n   + (n-1) + … +   2   +   1
-------------------------------------
 2S = (n+1) + (n+1) + … + (n+1) + (n+1)      ← 共 n 个 (n+1)
 2S = n(n+1)
  S = n(n+1)/2
```

关键结论（官方点名要记住）：1+2+…+n = n(n+1)/2，它的数量级是 **O(n²)**（因为 (n²+n)/2 的最高阶是 n²）。凡是在循环里看到“第 i 次做 i 件事”，总工作量多半就是这条恒等式。

5. 幕后：v.add 的“均摊 O(1)” vs v.insert(0,·) 的 O(n)

问题定义：官方文档说 `add` 是 O(1)、`insert(0,·)` 可达 O(n)，两者差在哪？ **add 的均摊机制**：vector 底层是一块连续数组。末尾有空位时，`add` 只写一个新元素（O(1)）；当容量（capacity）恰好用尽，vector 会**扩容**：申请一块更大的内存（常见策略是翻倍）、把 n 个旧元素全部拷过去、释放旧块——单看这一次是 O(n)。但因为容量翻倍，这次 O(n) 之后要再等 n 次 O(1) 的 add 才会再次扩容：

```
容量=4,已满:   [1][2][3][4]        ← push_back(5): 放不下!
扩容到 8:      [1][2][3][4][ ][ ][ ][ ]   ← O(n)=4 次拷贝,腾出 4 个空位
再写 5:        [1][2][3][4][5][ ][ ][ ]
   ── 接下来 3 次 add 都只需写 1 个位置,把刚才的 O(n) 摊薄
```

n 次 add 的总成本 ≈ n 次 O(1) + 少数几次 O(n) 扩容 ≈ O(n)，**平均每次 O(1)**——这就是“均摊（amortized）O(1)”的含义：单次可能贵，长期平均便宜。 **insert(0,·) 为什么是 O(n)**：在开头插一个元素，必须把现有的每一个元素都右移一格腾位置；第 i 次插入要搬 i 个元素，n 次共搬 1+2+…+n = n(n+1)/2 ≈ O(n²)。官方在 L04/L07 的课上实测（TIME_OPERATION，规模 5 万→50 万）：add 版耗时从毫秒级缓慢爬升，insert 头部版则爆炸式增长，两版差距从十几倍扩大到数百倍。中间位置插入同理要搬一半元素（O(n)），只有紧贴末尾插入才接近 O(1)（写的元素数 = size − index + 1）。

6. 由增长阶数估算运行时间（规模估算）

问题定义：已知某 O(f(n)) 函数在 n_0 时耗时 t_0，如何预测 n_1 时的耗时？ **方法**：运行时间按“输入规模比值的 f 次方”放大。设 $k = n_1/n_0$： – **线性 O(n)**：耗时 ×k。例：n=50 用 100ms → n=100 用 200ms。 – **二次 O(n²)**：耗时 $\times k^2$。例：n=50 用 100ms → n=100 用 400ms；若 n 冲到 100 万，放大 $(10^6/50)^2 = 4\times10^8$ 倍 → 100ms × 4×10^8 = 4×10^{10} ms ≈ **463 天**！ – **指数 O(2ⁿ)**：耗时 ×2^(n_1-n_0)。例：n=5 用 100ms → n=30，放大 2^{25} ≈ 3355 万倍 → 约 **38.8 天**。官方提醒：n 只加了 25，就从 0.1 秒变一个多月——输入每 +1，运行时间翻倍，这是识别 O(2ⁿ) 的“指纹”。 – **对数 O(log n)**：n 翻倍只多 1 步。n=10 亿时 $\log_2 n \approx 30$ ——**只需约 30 步**就能处理 10 亿规模（因为 $2^{30} \approx 10$ 亿），这正是二分查找等对数算法的“可怕之处”（官方 L07 称之为 logarithmic runtimes 的震撼）。

判断口诀：n 翻倍 → 时间翻倍 = 线性；变 4 倍 = 二次；只加常数 = 对数；直接翻倍翻倍再翻倍 = 指数。$O(n^2)$ 函数不一定“慢”，O(n) 函数也不一定“快”——大 O 只描述**增长趋势**，不承诺具体秒数（官方 L07 特别指出这一点，并举例自己实测过 n=1 万仅 14ms 的 $O(n^2)$ 函数）。

代码示例与实现详解

示例 1：亲手测量 O(n) 与 $O(n^2)$——翻倍实验

```
// 文件: growth_demo.cpp
// 演示: 规模翻倍时, push_back 总耗时近似翻倍(线性), 头部插入总耗时近似 ×4(二次)
#include <chrono>
#include <iostream>
#include <vector>
using namespace std;

double timeAppend(int n) {                // n 次在末尾追加(push_back)
    vector<int> v;
    auto t0 = chrono::steady_clock::now();
    for (int i = 0; i < n; i++) v.push_back(i);
    auto t1 = chrono::steady_clock::now();
    return chrono::duration<double, milli>(t1 - t0).count();
}

double timePrepend(int n) {               // n 次在头部插入(insert(begin()))
    vector<int> v;
    auto t0 = chrono::steady_clock::now();
    for (int i = 0; i < n; i++) v.insert(v.begin(), i);
    auto t1 = chrono::steady_clock::now();
    return chrono::duration<double, milli>(t1 - t0).count();
}

int main()
{
    cout << "n       push_back(ms)   insert(begin)(ms)   头部/末尾" << endl;
    double prevP = -1;
    for (int n = 2000; n <= 32000; n *= 2) {     // 规模每次翻倍
        double tA = timeAppend(n);
        double tP = timePrepend(n);
        cout << n << "    " << tA << "    " << tP << "     " << tP / tA;
        if (prevP > 0) {                         // 与上一档头部耗时比
            cout << "   (头部较上一档 ×" << tP / prevP << ")";
        }
        cout << endl;
        prevP = tP;
    }
    return 0;
}
```

【代码做什么】：对 n = 2000、4000、…、32000 逐档测量“n 次末尾追加”与“n 次头部插入”的总耗时，打印两列及比值。预期：push_back 档间近似 ×2（线性）；insert(begin) 档间近似 ×4（二次），两列差距越拉

越大。

【实现机制解说】：为什么头部插入档间是 ×4？第 i 次 `insert(begin())` 要搬 i 个元素，n 次共搬 1+2+…+n ≈ $n^2/2$ 次——n 翻倍 → 总搬移 ×4，这就是求和恒等式的活教材（$O(n^2)$ 的来源）。push_back 则每次几乎只写一个位置，偶尔触发一次 O(n) 扩容，均摊后总 O(n)，所以翻倍 → 总时间约 ×2。注意两次测量都在同一进程内进行、用 `steady_clock` 取墙钟差，量级上足以看清增长形态；若机器抖动较大，可把档位起点调大或每档重复几次取平均——但**不要**用绝对毫秒数跨机器比较，这正是本讲开头“墙钟不可靠”的教训。

示例 2：用“操作计数器”实证三种增长形态 + 验证求和恒等式

```cpp
// 文件: opcount_demo.cpp
// 演示: 用计数器亲眼看到 线性 / 二次 / 对数 的执行次数; 验证 1+…+n = n(n+1)/2
#include <iostream>
using namespace std;

long countLinear(int n) {                  // 线性: 循环 n 次
    long ops = 0;
    for (int i = 0; i < n; i++) ops++;
    return ops;                             // 恒等于 n
}

long countNested(int n) {                  // 二次: 双层循环, 恰好执行 n×n 次
    long ops = 0;
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++) ops++;
    return ops;
}

long countHalving(int n) {                 // 对数: 反复减半, 次数 ≈ log2(n)+1
    long ops = 0;
    while (n > 0) { n /= 2; ops++; }
    return ops;
}

int main()
{
    cout << "== 表 A: 线性 vs 减半(对数) ==" << endl;
    cout << "  n          线性次数    减半次数" << endl;
    for (int n : {1000, 1000000, 100000000}) {
        cout << "  " << n << "    " << countLinear(n) << "    "
             << countHalving(n) << endl;
    }
    cout << "  (若 n=10 亿: 线性要 10 亿步, 减半只需 30 步, 因为 2^30≈10^9)" << endl;

    cout << "== 表 B: 双层循环实际次数 == n² ==" << endl;
    cout << "  n      实际次数       n×n" << endl;
    for (int n : {500, 1000, 2000, 4000}) {
        cout << "  " << n << "    " << countNested(n) << "    " << (long)n * n << endl;
    }

    // 求和恒等式: 循环累加 vs 闭式公式, n=1..100000 全量核对
    long long check = 0;
    bool ok = true;
    for (long long n = 1; n <= 100000; n++) {
        check += n;
        long long formula = n * (n + 1) / 2;
        if (check != formula) { ok = false; break; }
    }
    cout << (ok ? "恒等式成立: 1+2+…+100000 = " : "恒等式出错!")
         << check << " = 100000×100001/2" << endl;
    return 0;
}
```

【代码做什么】：用真实的计数器展示三种增长：线性计数器等于 n；减半计数器在 n=1000/10^6/10^8 时只从 10 涨到 27；双层循环计数器恰等于 n^2（表 B 三列一致，眼见 $O(n^2)$ 的来源）；最后把 1 累加到 10 万，与 n(n+1)/2 公式逐项核对，验证恒等式。

【实现机制解说】：三个计数器函数就是三种复杂度的“活体标本”。`countHalving` 揭示 O(log n) 的模式——**每轮把输入除以 2，且每轮只做 O(1) 工作**，则总轮数 ≈ $\log_2 n$：n=10^8 只跑 27 轮，n=10^9 也才 30 轮（2^{30} ≈ 10 亿，官方 L07 引用的震撼数字）。`countNested` 内层循环独立执行 n 次、外层又套 n 次，总执行恰好 n^2 次——这是 $O(n^2)$ 最直白的来源。恒等式验证采用“暴力求和 vs 闭式公式”对拍：C++ 里 `n * (n + 1)` 若 n 是 int 可能溢出，故这里用 `long long` 并先乘后除——顺带复习了类型宽度与运算顺序的坑（除以 2 放最后才能保证 (n+1) 为奇数时也整除）。

示例 3：规模估算器——把“n 翻倍会怎样”变成数字

```
// 文件: estimate_demo.cpp
// 演示: 由基准 (n0, t0) 预测 n1 的耗时(线性/二次/指数三种增长)
#include <iostream>
using namespace std;

// 多项式增长: power=1 → 线性(×k), power=2 → 二次(×k²)
double predictMs(int n0, double t0ms, int n1, int power)
{
    double k = (double)n1 / n0;          // 输入规模放大倍数
    double factor = 1.0;
    for (int i = 0; i < power; i++) factor *= k;
    return t0ms * factor;
}

// 指数增长: 每多 1 输入翻一倍 → 放大 2^(n1-n0) 倍
double predictExpMs(int n0, double t0ms, int n1)
{
    double factor = 1.0;
    for (int k = n0; k < n1; k++) factor *= 2;
    return t0ms * factor;
}

int main()
{
    cout << "官方同款例题(基准: n=50 用 100ms):" << endl;
    cout << "  线性 n=100:   " << predictMs(50, 100, 100, 1) << " ms" << endl;   // 200
    cout << "  二次 n=100:   " << predictMs(50, 100, 100, 2) << " ms" << endl;   // 400
    cout << "  二次 n=1e6:   " << predictMs(50, 100, 1000000, 2)
         << " ms = " << predictMs(50, 100, 1000000, 2) / 1000.0 / 3600 / 24
         << " 天" << endl;                                                       // ≈463 天
    cout << "  指数 n=5→30:  " << predictExpMs(5, 100, 30)
         << " ms = " << predictExpMs(5, 100, 30) / 1000.0 / 3600 / 24
         << " 天" << endl;                                                       // ≈38.8 天
    return 0;
}
```

【代码做什么】：把“规模估算”做成函数：给定基准点 (n_0, t_0) 与目标 n_1，线性增长按 $\times(n_1/n_0)$ 放大、二次按平方放大、指数按 2 的差次方放大；main 直接复算官方 L07 的三道例题并换算成“天”，让数字自己说话。

【实现机制解说】：估算的本质是**比例推理**——我们从不预测绝对秒数，只预测“相对基准放大了多少倍”，因此与机器、语言无关，这正是大 O 的价值。指数分支里 `for (k = n0; k < n1; k++) factor *= 2` 累乘 n_1-n_0 次，避开浮点溢出地算出 $2^{(n_1-n_0)}$；二次分支的 for 循环等价于 `factor = k * k`，写循环是为了与“幂次可扩展”的教学语义一致。结果对照官方数字：二次在 $n=10^6$ 时约 463 天、指数在 n=30 时约 38.8 天——注意这些预测都**锚定在 n=50 用 100ms 这个假设基准**上；官方 L07 强调，换一个基准点（例如实测 $n=10^4$ 只要 14ms 的 $O(n^2)$ 函数）绝对数字会完全不同，但**增长形态不变**。这也解释了为何不要拿别人的秒数吓自己：该警惕的是增长阶数，不是某个绝对值。

复杂度分析

形态	记号	n 翻倍时	处理 10^9 规模的大致成本	典型来源
常数	O(1)	不变	1 步	下标访问、栈顶操作
对数	O(log n)	+1 步	≈30 步	反复减半、二分查找
线性	O(n)	×2	10^9 步	单遍扫描、push_back×n（均摊）
线性对数	O(n log n)	略大于 ×2	$\approx 3\times10^{10}$ 步	高效排序（后续讲）
二次	$O(n^2)$	×4	10^{18} 步（不可行）	双层循环、insert(0,·)×n
指数	$O(2^n)$	平方级爆炸	n=30 就已 10^9 步	枚举全部子集/硬币序列

附：空间复杂度同规则——只关心“随 n 增长的内存形态”，例如 vector 扩容临时占 O(n) 额外空间、树与哈希表存 n 个元素占 O(n)。

关键要点

- 别用秒表比算法：机器、负载、用例都会污染结论；大 O 只谈增长形态。
- 化简法则：扔常数、留最高阶——$4n+4 \to O(n)$，$(1/6)n^2+1000n \to O(n^2)$。
- 记住增长链条 $1 < \log n < n < n\log n < n^2 < 2^n$，以及“对数≈贴着常数、指数≈灾难”。
- 求和恒等式 $1+2+\ldots+n = n(n+1)/2 = O(n^2)$：见到“第 i 次做 i 件事”就想起它。
- 判断指纹：输入翻倍，时间翻倍=线性、×4=二次、只加常数=对数、整体再爆炸=指数。

常见陷阱与注意事项

- **用墙钟秒数跨机器比较**：同一程序两台机器差 10 倍很正常。规避：只比较同一环境下的相对增长，或干脆用操作计数/大 O 表述。
- **拿“单个 O(n) 函数慢”下结论**：大 O 不承诺常数大小——O(n) 里藏着 1000n 也可能比 $O(n^2)$ 的 $0.001n^2$ 在中小规模更慢。规避：大 O 用于讨论 n 很大时的趋势。

- **把 O(log n) 当“介于常数与线性之间的一半”**：对 10 亿输入 log 只需 30 步，它贴着常数跑。规避：背下“$2^{30}\approx10^9$”这个数感锚点。
- **忘记均摊**：说“push_back 是 O(n)”不完全错但会误导——单次可能 O(n)，长期平均 O(1)。规避：描述为“均摊 O(1)”。
- **溢出破坏估算/求和**：`n*(n+1)/2` 中 int 相乘可能溢出；`2^n` 也别真算。规避：用 long long，指数估算用对数或累乘。
- **把双层循环一律当 $O(n^2)$**：内层若与 n 无关（如固定 100 次）则整体仍是 O(n)。规避：数清内外层各自的迭代次数再相乘。
- **规模估算忘了基准**：所有“预测天数”都锚定在某个假设基准上，基准不同绝对数字全变。规避：先明确“什么规模、多少时间”的基准再外推。

思考题（带答案）

问题 1：函数 A 对 n 个元素做两遍单层扫描，函数 B 对同样输入做一层嵌套循环。分别求大 O，并说明“A 一定比 B 快吗”。 **答案**：A 是 2n 次操作 → O(n)；B 是 n^2 次 → $O(n^2)$。大 O 上 A 优于 B，但不保证 A 在**每个具体 n** 都快——若 A 的单步代价很高而 B 极简，小规模时可能反超；不过 n 足够大后 O(n) 必然碾压 $O(n^2)$。这正是大 O 只谈趋势、不谈常数的含义。

问题 2：某算法在输入 n=10 时耗时 1ms，在 n=20 时耗时 4ms，在 n=40 时耗时 16ms。它大致是什么复杂度？n=160 时预计多久？ **答案**：n 翻倍 → 时间 ×4，符合二次 $O(n^2)$。从 40 到 160 是两轮翻倍（40→80→160），时间 ×16：16ms × 16 = 256ms。（也可直接按比例 $(160/40)^2 = 16$ 计算。）

问题 3：为什么说“在 vector 开头反复 insert 的总成本是 $O(n^2)$”，用求和恒等式解释，并给出 $n=10^5$ 时的搬移次数量级。 **答案**：第 i 次在开头插入要右移 i 个元素，n 次总搬移 = $1+2+\dots+n = n(n+1)/2 \approx n^2/2$，最高阶 $n^2 \to O(n^2)$。$n=10^5$ 时约搬 5×10^9 个元素，即便每秒搬 10^9 个也要约 5 秒；而同样的 n 次 `push_back` 均摊总成本只有 O(n)。这就是为什么“优先往末尾堆数据”是铁律。

Lecture 5: 递归：原理、策略与递归式思维（Recursion: Principles & Strategies）（对应课程真实讲座 L08–L10）

概述

本讲正式引入"递归（recursion）"：让函数调用自己，把一个大问题不断拆成同构（形状相同、规模更小）的子问题，直到某个最小问题能直接给出答案。我们会从阶乘、回文这类"热身"例子中吃透递归三要素与调用栈机制，再实现递归二分查找、字符串逆序打印和全排列枚举——这些技巧是后续回溯、分治（归并/快排）与树遍历的共同地基。 （对应官方 2026 夏季 CS106B：L08 Monday, July 6 "Introduction to Recursion"；L09 Tuesday, July 7 "More Recursion"；L10 Wednesday, July 8 "Recursive Problem Solving"。官方在第 8 讲开篇就让大家 DON'T PANIC——递归第一次出现在代码里觉得"玄、晕、难"完全正常，多练习就会变成第二天性。）

核心概念与算法原理

1. 什么是递归：函数调用自己 – 问题定义：要解决问题 P(输入 x)，先把 x 拆成一个或多个"更小的 x'"，然后调用**正在写的这个函数自己**去解决 P(x')，最后把子问题的答案拼成 P(x) 的答案。 – 直观解释：俄罗斯套娃——打开最大的娃娃，里面是一个小一号的娃娃，再打开又是一个更小的；大自然里 Koch 雪花、罗马花椰菜都呈现这种"整体里嵌套同形局部"的自相似（self-similar）结构。 – 官方在第 8 讲给出递归函数的两大关键组件，也就是常说的**三要素**： 1. **基准情形（base case）**：对某些"显然能答"的规范输入直接返回结果，不再调用自己——这是递归的终止条件； 2. **递归步骤（recursive case）**：把当前输入分解成子问题，其中至少一个交给函数自身处理； 3. **向基准靠拢（progress toward base case）**：每次递归调用传入的输入必须比当前更"小"，保证迟早撞上基准情形——而不是朝反方向无限增长。

2. 调用栈：先"递"下去，再"归"回来 程序运行时，每调用一次函数，系统就在内存的"程序栈（call stack）"上压入一个**栈帧（stack frame）**，记录参数、局部变量和返回地址；函数返回时该帧弹出。递归 = 不断压栈（"递"的过程），撞到基准后逐层弹出（"归"的过程），每层带着返回值回到上一层继续未完成的计算。以阶乘为例（本讲第一个正式程序），`factorial(5)` 的栈是这样长高又变矮的：

```
factorial(5) 的调用栈（框越往上越"新"；栈顶是最深的那一层调用）

“递”的途中：一帧一帧压栈

┌──────────────────────────────────┐
│ factorial(0)   基准情形 return 1 │ ← 栈顶（最深）
├──────────────────────────────────┤
│ factorial(1)   等待 1 × ____     │
├──────────────────────────────────┤
│ factorial(2)   等待 2 × ____     │
├──────────────────────────────────┤
│ factorial(3)   等待 3 × ____     │
├──────────────────────────────────┤
│ factorial(4)   等待 4 × ____     │
├──────────────────────────────────┤
│ factorial(5)   等待 5 × ____     │
├──────────────────────────────────┤
│ main()                           │
└──────────────────────────────────┘

“归”的途中：帧从栈顶依次弹出，返回值逐层回传、就地完成乘法：
factorial(1) = 1 × 1   = 1
factorial(2) = 2 × 1   = 2
factorial(3) = 3 × 2   = 6
factorial(4) = 4 × 6   = 24
factorial(5) = 5 × 24  = 120   ← 最终答案交回给 main()
```

3. 无限递归与栈溢出（stack overflow） 如果函数没有基准情形、或递归调用不向基准靠拢，栈帧会无穷无尽地堆积，最终占满栈空间导致程序崩溃（官方在第 8 讲用 `foo(){ foo(); }` 演示，崩溃前能压入约二十六万个帧）。规避方法只有一个：动笔写递归体之前，先问自己"最小输入是什么？答案是什么？参数怎么变小？"

4. 递归 vs 迭代 官方在第 8 讲直言：今天大部分例子用 for 循环也能写，我们偏要用递归，是为了用温和的题目建立递归直觉。两者的取舍大致是：

维度	递归	迭代（循环）
可读性	贴合"问题天然分层/分形"的表述，代码极短	需要手工维护状态变量，逻辑可能绕
开销	每次调用压栈/弹栈，有额外时间与栈空间	无函数调用开销，通常更快更省
适用	二分、分治、回溯、树/图遍历、枚举	简单线性任务（求和、计数）
风险	忘基准 → 栈溢出	条件写错 → 死循环（但占内存更少）

5. 信任飞跃（leap of faith） 写递归最常见的心理障碍是想把每一层调用都追踪一遍。官方反复鼓励的姿势恰恰相反：**假设"对更小输入的递归调用已经正确返回了"，你只需负责当前这一层如何利用子结果、以及基准情形是否正确**。好比接力赛中你相信前一棒会把棒交到你手里，你只操心自己这一段怎么跑。

6. 包装函数（wrapper function）与函数重载（overloading） 递归函数往往需要额外参数（如二分查找的 lo、hi），但调用者不想关心这些。于是写一个"门面"包装函数：只接收用户想传的参数，内部负责初始化并调用真正的递归函数；若两函数同名（仅参数不同），就叫**函数重载**，C++ 会根据实参个数与类型自动选择调用哪一个——这比 `xxxHelper` 式的命名更清爽（官方在第 9 讲的二分查找里就用了两个同名 `binarySearch`）。

7. 语句顺序决定输出顺序：打印 vs 递归 递归调用前后各放一句代码，执行时机完全不同：调用**之前**的语句在"递"的途中执行（自顶向下）；调用**之后**的语句要等子调用全部返回才执行，即"归"的途中执行（自底向上）。正序打印字符串，就是先打印再递归；逆序打印只需把两句对调——栈天然替我们把顺序反转了：

```
#include <iostream>
#include <string>
using namespace std;

// 先打印、后递归：顺着"递"的路径输出 → 正序
void printForward(const string& s, int k) {
    if (k == (int)s.length()) { cout << endl; return; }
    cout << s[k];
    printForward(s, k + 1);
}
// 先递归、后打印：顺着"归"的路径输出 → 逆序
void printBackward(const string& s, int k) {
    if (k == (int)s.length()) { return; }
    printBackward(s, k + 1);
    cout << s[k];
}
```

注意逆序版里换行若放在基准情形，会打印在整串**之前**（基准最先执行），正确做法是把换行交给包装函数收尾——这就是"包装函数负责 setup/tear-down"的典型用途（官方第 8 讲的教训）。

8. 从线性查找到递归二分查找（binary search） – **线性查找（linear search）**：从下标 0 逐个比到末尾，找到即停。它不要求数据有序，但最坏要扫 n 个元素。 – **二分查找**：前提是容器**已按非降序排好**。每次取当前

搜索区间 `[lo, hi]` 的中点 `mid` 与 key 比较：key 小则 key 只可能落在左半 `[lo, mid-1]`，key 大则只可能在右半 `[mid+1, hi]`——无论哪种，都一次性丢掉一半搜索空间。官方在第 9 讲强调：每步 O(1) 的比较把空间减半，正是第 7 讲讲的"对数运行时"：1 亿（约 2^{30}）个元素也只需约 30 次比较。 – **中点公式的溢出陷阱（官方第 9 讲补充）**：int 最大值约 21.47 亿。若 lo、hi 都很大，`(lo + hi) / 2` 的加法本身可能溢出成负数，得到非法下标导致程序崩溃（如 lo = 1,000,000,001、hi = 2,000,000,001，`lo + hi` 溢出为 −1,294,967,294）。改用代数等价但**永不溢出**的 `lo + (hi - lo) / 2`——中间量最大只到 hi。这个细节常出现在技术面试里。

9. 递归枚举与分形的"预览" 硬币序列（每次抛 H/T，n 次共 2^n 种）、骰子序列（6^n 种）、全排列（n! 种）都属于**递归枚举**：每层递归在一个"决策点"上尝试所有候选并深入。官方在第 9–10 讲用递归树/Prezi 演示：coinFlip 只是两处递归调用，dice 把两次调用改成 for 循环六次，permutation 则在 for 里逐一挑"下一个字符"。这类问题迭代写非常痛苦、递归写却只有几行，是下一讲"回溯"的直接前奏。分形（Koch 雪花、谢尔宾斯基三角形）则是递归的图形化表达：官方动画特别提醒，代码里四个子段是按**深度优先**依次画完的（先把最左分支一路画到底再回头），并非并行绘制——递归调用的书写顺序决定图形呈现顺序。

代码示例与实现详解

下面 4 个示例全部只依赖 C++17 标准库、可独立编译运行。

示例 1：递归阶乘——三要素与"递/归"全景

```
#include <iostream>
using namespace std;

// n! = n × (n-1)!, 0! = 1
int factorial(int n) {
    if (n == 0) {           // ① 基准情形：0! 直接给答案
        return 1;
    }
    return n * factorial(n - 1);   // ② 递归步骤 ③ 参数 n-1 朝 0 靠拢
}

int main() {
    for (int n = 0; n <= 10; ++n) {
        cout << n << "! = " << factorial(n) << endl;
    }
    return 0;
}
```

【代码做什么】：`factorial` 把 `n!` 的定义直接翻译成代码：`n! = n × (n-1)!`。主函数打印 0! 到 10! 验证结果。若把基准情形注释掉或参数写成 `n + 1`，程序将因栈溢出而崩溃——这是理解递归的第一块试金石。

【实现机制解说】：调用 `factorial(5)` 时，第 2 节的栈帧图精确描述了全过程：**递**——五个 `factorial` 帧层层压栈，每帧都"冻结"在自己那行 `return n * factorial(n-1)` 上，等待子调用返回值填空；最深处的 `factorial(0)` 命中基准，直接返回 1，不再压栈；**归**——从 `factorial(1)` 起每帧依次弹出并完成自己

的乘法，答案如多米诺骨牌般回传。理解"乘法发生在归途、而非递途"是看懂一切递归计算的关键：递途只负责把问题拆到最小，真正的计算在回卷时完成。

示例 2：回文判断——剥皮法递归

```
#include <iostream>
#include <string>
using namespace std;

// 回文：正着读反着读都一样，如 racecar
bool isPalindrome(const string& s) {
    if (s.length() <= 1) {          // 基准：空串与单字符都是回文
        return true;
    }
    if (s.front() != s.back()) {    // 首尾不等 → 直接判否
        return false;
    }
    // 首尾相同 → 剥掉它们，检查剩下的子串是否回文
    return isPalindrome(s.substr(1, s.length() - 2));
}

int main() {
    for (string t : {"racecar", "kayak", "hello", "a", "", "step on no pets"}) {
        cout << "\"" << t << "\" -> " << (isPalindrome(t) ? "是回文" : "非回文") << endl;
    }
    return 0;
}
```

【代码做什么】：`isPalindrome("racecar")` 先比较首 'r' 与尾 'r' 相同，于是递归检查剥皮后的 `"aceca"`；依次剥到长度 ⩽ 1 时返回 true。对 `"hello"`，首 'h' ≠ 尾 'o'，第一层就直接返回 false，不再深入。

【实现机制解说】：这里有两个关键设计。其一，**递归调用必须发生在"首尾相等"这一检查之后**——官方在第 8 讲的 Common Pitfall #3 指出：若忘了比较首尾就无脑递归，函数会对一切输入返回 true，而且如果你只写了"期望 true"的测试用例，这种坏函数会全部通过，让人毫无察觉。其二，用 `substr` 切片每次会复制约一半长度的新串（n 层累计约 $O(n^2)$ 时间）；若追求效率，可改用"传整个字符串的引用 + 两个下标 lo、hi 向中间靠拢"的写法，每层只做 O(1) 工作（这是官方面试风格的改进题，见思考题）。

示例 3：递归二分查找（含包装重载与防溢出中点）

```cpp
#include <iostream>
#include <vector>
using namespace std;

// 在有序数组 v 的闭区间 [lo, hi] 内查找 key；找不到返回 -1
int binarySearch(const vector<int>& v, int key, int lo, int hi) {
    if (lo > hi) {                  // 基准：区间为空 → 不存在
        return -1;
    }
    int mid = lo + (hi - lo) / 2;   // 防溢出中点公式（勿写成 (lo+hi)/2）
    if (key < v[mid]) {
        return binarySearch(v, key, lo, mid - 1);   // 只搜左半
    }
    if (key > v[mid]) {
        return binarySearch(v, key, mid + 1, hi);   // 只搜右半
    }
    return mid;                     // key == v[mid]，命中
}

// 包装函数：与上面构成函数重载，调用者只需传数组和 key
int binarySearch(const vector<int>& v, int key) {
    return binarySearch(v, key, 0, (int)v.size() - 1);  // 空数组时 hi=-1 直接命中基准
}

int main() {
    vector<int> v = {2, 5, 8, 12, 16, 23, 38, 56, 72, 91};
    for (int key : {2, 38, 91, 99}) {
        cout << "在有序数组中查找 " << key << " -> 下标 "
             << binarySearch(v, key) << endl;
    }
    return 0;
}
```

【代码做什么】：每次比较 `key` 与 `v[mid]`，据此把搜索区间收缩为左半或右半并递归，直至区间为空（返回 −1）或命中（返回下标）。主函数对四个键演示查找：2 命中下标 0，38 命中下标 6，91 命中下标 9，99 不存在返回 −1。

【实现机制解说】：注意三点。第一，**为什么能安全丢掉一半**：数组有序，若 `key < v[mid]`，则 key 不可能出现在 mid 右侧任何位置，右半可以整体放弃——剪枝的正确性完全建立在有序性上，这也是本讲唯一"必须传引用而不传值"也成立的场景（传引用省去每次 O(n) 拷贝，官方在第 9 讲明确传值会变 O(n) 操作）。第二，**基准情形** `lo > hi`：区间收缩到 lo 越过 hi 说明搜索空间已空，这正是"用递归把迭代终止条件表达出来"的典范；空数组经包装函数进入后 hi = −1，同样安全返回。第三，**两个同名函数靠参数个数区分**（3 参数版 + 2 参数版），这就是函数重载：编译器看到 `binarySearch(v, key)` 自动选 2 参数版，看到四参调用选 3 参数版。每层递归栈深 O(log n)，约 $\log_2(10) \approx 4$ 层即可查完 10 个元素。

示例 4：递归生成全排列——枚举的雏形

```cpp
#include <iostream>
#include <string>
using namespace std;

// soFar：已经确定的前缀；rest：还没安放位置的剩余字符
void permute(const string& soFar, const string& rest) {
    if (rest.empty()) {                 // 基准：没有剩余字符 → 得到一个完整排列
        cout << soFar << endl;
        return;
    }
    for (int i = 0; i < (int)rest.length(); ++i) {
        // 把 rest[i] 安到下一个位置，剩余字符 = 去掉 rest[i] 后的串
        string newRest = rest.substr(0, i) + rest.substr(i + 1);
        permute(soFar + rest[i], newRest);   // explore 每一个候选
    }
}

// 包装函数：从空前缀、完整字符串开始
void permute(const string& s) {
    permute("", s);
}

int main() {
    permute("cat");   // 应输出 6 个排列：cat cta act atc tca tac
    return 0;
}
```

【代码做什么】：`permute("", "cat")` 在每一层用 for 循环遍历 `rest` 里的每个字符，将其追加到 `soFar` 并递归处理剩余字符。当 `rest` 为空，说明所有字符都已就位，输出一个排列。共输出 3! = 6 行。

【实现机制解说】：这是"决策树"式递归的第一次正式登场：第 1 层有 3 个分支（c/a/t 谁打头），第 2 层每个分支又有 2 个选择，第 3 层 1 个，叶子总数 3×2×1 = 6。因为 `soFar`、`rest` 都是**按值传递**的新拷贝，父调用自身状态从未被改动，返回上一层时"天然撤销"了本层的选择——这个特性在下一讲的回溯里会与"共享状态需要显式撤销"形成鲜明对比。若把本函数改成硬币序列：for 循环换成两次固定递归 `coinFlip(soFar+'H', n-1)` 与 `coinFlip(soFar+'T', n-1)`，就得到 2^n 个 H/T 序列（官方第 9 讲的 coinFlip 正是如此）——排列与序列共享同一套"递归枚举"骨架。注意按值传串有拷贝开销（总代价量级 O(n·n!)），面试改进版是传引用 + swap（官方第 10 讲展示过 swap 版排列，需在递归后把字符换回来，即"撤销"）。

复杂度分析

算法/操作	时间（最好）	时间（最坏）	额外空间	原因简述
`factorial(n)`	O(n)	O(n)	O(n)（栈深）	递 n 层、每层一次乘法；栈帧数与 n 成正比
线性查找	O(1)	O(n)	O(1)	key 在首位立即停；找不到要扫完整数组

算法/操作	时间（最好）	时间（最坏）	额外空间	原因简述
递归二分查找	O(1)	O(log n)	O(log n)（栈深）	每次比较砍掉一半区间；命中中点时一步即返回
字符串打印（引用+下标版）	O(n)	O(n)	O(n)（栈深）	每层 O(1) 工作，共 n 层
硬币序列 / 全排列	—	结果数 2^n / n!，输出规模本身爆炸	O(n)（栈深）	每个结果都要被产出；按值传串另有拷贝开销

注意："最好/最坏"永远指**输入规模任意大**时的差异（官方在第 9 讲强调：不能说"输入只有一个元素时最快"，那样任何函数都成 O(1) 了）；另外回文的 `substr` 切片版每层复制剩余串，最坏 $O(n^2)$，用下标版可回到 O(n)。

关键要点

- 三要素缺一不可：先有能直接回答的**基准情形**，再写**把输入缩小并调用自己**的递归步骤——否则要么栈溢出，要么根本没在递归。
- 用**信任飞跃**写递归：先假设"更小输入的递归调用是对的"，只检查当前层如何拼装子结果；不要逐层手工追踪。
- 语句的位置决定时机：递归调用**之前**的代码在"递"时执行，**之后**的代码在"归"时执行——逆序打印的秘密就在这一行顺序里。
- 用**包装函数 + 重载**把"用户接口"和"递归细节"分开，让调用者只传最少的参数。
- 递归是"分而治之/自相似"的思维工具：二分把 O(n) 降到 O(log n)，枚举天然指数/阶乘级——能用递归几行说清的问题，往往迭代写起来又长又绕。

常见陷阱与注意事项

1. **忘基准情形或基准漏输入**：如 `factorial` 只判 `n == 1`，调用 `factorial(0)` 会无限调用 `factorial(-1)`… 直至栈溢出（官方 Common Pitfall #2）。规避：把 0、空串这类"边界但合法"的输入都列出来测试；若参数改 `unsigned`，负数会下溢成巨大正数，同样死循环——类型替代不了基准设计。
2. **int 函数漏 return**：写下 `n * factorial(n - 1);` 却没写 `return`（官方 Common Pitfall #1），结果未定义。规避：编译加 `-Wall`，并把返回值直接写在 `return` 表达式里。
3. **回文忘了比较首尾**：函数变成恒真，且全 true 的测试还发现不了（官方 Pitfall #3）。规避：测试集必须包含期望 false 的用例。
4. **递归调用不向基准靠拢**（如 `foo(n+1)` 却以 `n==0` 为基准）。规避：写前先确认每层参数严格"变小"。
5. **改名后忘改函数体内的递归调用**（官方 Pitfall #4，常见于复制粘贴）。规避：改名后全文搜索旧函数名。
6. **中点公式写成 `(lo+hi)/2`** 导致大区间溢出崩溃。规避：一律 `lo + (hi - lo) / 2`。
7. **把换行/收尾语句放进逆序打印的基准情形**：换行会出现在整串之前。规避：收尾工作交给包装函数。

8. **每层用 `substr` 造大拷贝**：递归深、串长时浪费严重。规避：传引用 + 下标参数（如 printStringHelper 风格）。

思考题（带答案）

问题 1：把 `factorial` 的基准情形从 `n == 0` 改成 `n == 1`，然后调用 `factorial(0)` 会发生什么？为什么？ **答案**：`factorial(0)` 不会命中基准（0 ≠ 1），于是调用 `factorial(-1)`、`factorial(-2)`……参数不但没向 1 靠拢反而越来越远，栈帧无限堆积，最终栈溢出崩溃。基准情形必须覆盖全部合法输入的边界，且递归方向必须朝向基准。 **问题 2**：用"信任飞跃"写递归二分查找时，你具体信任了什么？哪些细节可以完全不管？ **答案**：信任 `binarySearch(v, key, lo, mid-1)` 与 `binarySearch(v, key, mid+1, hi)` 这两个"更小区间"的调用会各自返回正确下标或 −1——不必手工模拟这两次调用内部几十帧的执行。你只需要确保：区间确实在收缩（mid 两侧都严格小于原区间）、基准 `lo > hi` 正确、以及比较逻辑（小于走左、大于走右、相等命中）在当前层是对的。若这三件事成立，整体必然正确。 **问题 3**：为什么逆序打印字符串的 `endl` 不能放在基准情形里？正确的放置位置在哪？ **答案**：基准情形是整条递归链**最早执行**的代码（它在"归"的起点），放在那里会让换行先于任何字符打印出来（输出变成先空一行再是反转串）。正确做法是把换行放在**包装函数**里：包装函数调用完递归主体后再补 `endl`，于是换行恰好出现在所有字符之后（官方第 8 讲 printStringReverse 的解法）。

Lecture 6: 递归回溯与枚举（Recursive Backtracking & Enumeration）（对应课程真实讲座 L11–L12）

概述

本讲在"枚举"（序列、排列、子集）的基础上引入**递归回溯（recursive backtracking）**：沿着决策点一步步尝试，撞上死胡同就退回上一个决策点换一条路，并把这一范式归纳为三字诀 **choose–explore–unchoose（选择–探索–撤销）**。我们会用生成子集、集合平分 isPartitionable、0–1 背包三个经典问题吃透回溯骨架与指数复杂度，为后面迷宫、数独等"迭代很难写"的问题储备通用武器。 （对应官方 2026 夏季 CS106B：L11 Thursday, July 9 "Recursive Backtracking and Enumeration"；L12 Monday, July 13 "More Recursive Backtracking"。官方用迷宫小人动画演示回溯：一路 Search…，碰壁就 Backtrack… 回到上一个岔路口换方向。）

核心概念与算法原理

1. 序列、排列、子集：三类枚举问题一家亲 官方在第 11 讲先做了小结：我们已能用递归生成三类结果——**序列**（抛 n 次硬币 2^n 种、掷 n 次骰子 6^n 种）、**排列**（"cat" 的 6 种重排）、**子集**（{a, c, t} 的 8 个子集，含空集）。它们都长在同一棵"决策树"上，区别只是每层的分支数（2、n、6…）与何时停止。子集问题里**顺序无关**，{b, a} 与 {a, b} 算同一个子集，所以决策树只需考虑"每个元素要 or 不要"，共 2^n 个叶子。

2. 回溯是什么：碰壁就回头 – 问题定义：在多个连续决策构成的状态空间中搜索满足条件的解；当前路径走到头仍无解（死胡同 dead end）时，**回到最近一个还有未试选项的决策点**，改走另一条路，而不是从头重来。 – 直观解释：走迷宫——每个岔路口是一个决策点；走到死路就退回上一个岔路口选另一条走廊；再死

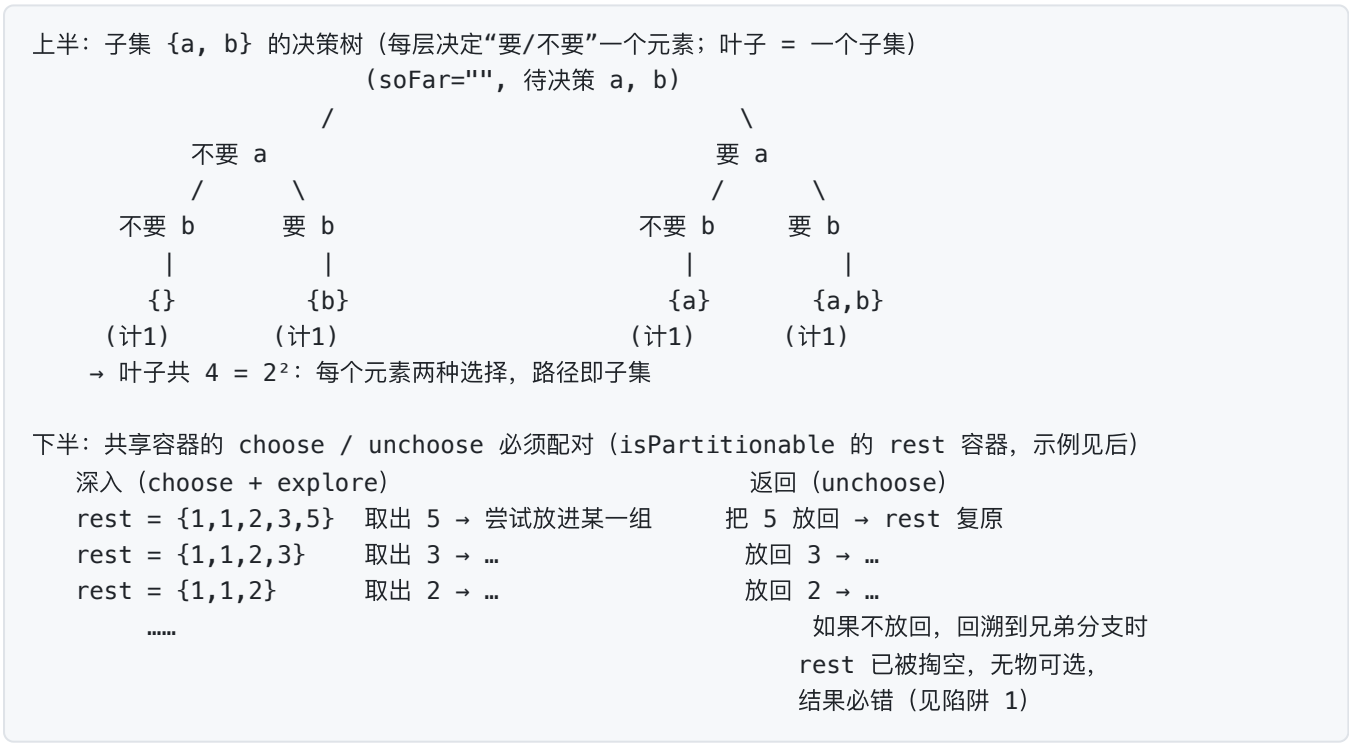
路再退。官方动画里的小人正是这样"Searching… / Backtracking…"反复进退。若迷宫可绕圈，还需"撒面包屑"标记走过的地方，防止在两个格子间无限往返（即回溯骨架中的"查重状态"环节）。

3. 回溯的标准骨架：choose—explore—unchoose 把回溯算法解剖开，官方在第 11 讲给出如下通用结构（每步是否必需因问题而异）：

```
1. 基准情形（base case）：到达叶子。若当前状态是解 → 打印/计数/返回 true；
   否则返回"此路不通"的哨兵值（false / 0）。
2. （可选）查重：若进入过相同状态（如迷宫已撒过面包屑的格子）→ 直接返回。
3. 生成候选：用循环枚举本决策点的全部合法选择；非法选择跳过（剪枝）。
   对每个候选：
   a. 改变状态（choose）       —— 把选择付诸实施（放入容器/移动棋子/计入和）
   b. 递归深入（explore）      —— 带着新状态调用自己
   c. 处理返回值（可选）       —— 命中即停（return true）或累加计数
   d. 撤销状态（unchoose）     —— 把 a 的改动还原，供下一个候选使用
```

关键认识：什么情况下必须显式撤销？ 官方特别指出：若问题状态（容器/网格/字符串）是**按值拷贝**传下去的，父调用的拷贝从未被改动，函数返回即天然回到旧状态，"撤销"就自动完成了（上一讲的子集字符串版正是如此）；若状态是**按引用共享**的（vector、数组、网格），子调用对它的修改会永久残留，返回后必须手动撤销，否则兄弟分支看到的是一副被掏空/改坏的状态。本节三个例子里，子集用"按值"免撤销，平分与背包用"共享容器 + 显式撤销"。

4. 子集决策树与回溯撤销示意图



5. 把"打印"改成"计数/返回 bool"：三种返回风格 同一副骨架，只要改基准情形的返回值与返回语句的汇总方式，就能切换用途（官方在第 11 讲把 printSubsets 改成返回子集个数，并强调这种变形必须掌握）：叶子处 **return 1（或解的值）**、内部用 **+** 汇总 → 计数；叶子处 **return 布尔条件**、内部用 **||** 汇总 → 判断"是否存在解"。bool 版本还白得一个福利——**短路求值（short-circuiting）**：`a || b` 中 a 为 true 时 C++ 根本不会

执行 b，于是"找到一个解就整棵树提前停止"不用写任何额外代码（官方第 11 讲用 isPartitionable 的两个版本对比了短路带来的巨大差异）。bool 函数收尾也建议直接 `return sum1 == sum2;` 而不是 if-else 绕弯。

6. isPartitionable：集合能否平分成两组 – 问题定义：能否把向量 V 的元素**全部、恰好一次**地分进两组 V_1、V_2，使两组元素和相等？例：{1,1,2,3,5} 可分（{1,5} 与 {1,2,3} 各为 6）；{1,4,5,6} 不可分。 – 思路：对每个元素做二选一（进组 1 / 进组 2），全部放完时检查两桶和是否相等。这是"子集"骨架的 bool 版本，且官方实现用了"从剩余容器取一个元素 + 递归后放回"的显式撤销写法。

7. 0–1 背包：struct 承载物品 + 剪枝 – 问题定义（官方第 12 讲）：背包容量 c，物品 i 有重量 w_i 与价值 v_i，每个物品只能整件拿或不拿（"0–1"即二选一），求不超过容量前提下的最大总价值。贪心（先拿最贵的，或按 v/w 比值拿）都不保证最优——官方给了反例：容量 10，物品 (w=8,v=160) 与三个 (3,58) 加一个 (1,2)：按比值贪心拿 8 号只能再拿 1 号得 162，而拿后四个共重 10 得 176。 – **剪枝（pruning）**：某物品比剩余容量还重时，它**不可能被拿**，只能走"不拿"单分支——这比盲目二分少了半棵子树。官方原话是：背包问题里并非每个物品都面对二选一，"拿不动就被迫放弃"。 – C++ 配套知识——**struct（结构体）**：把重量与价值打包成一个新类型，避免两个平行 vector 错位（官方第 12 讲演示了 weights/values 分开存的风险）。定义写在函数外、右花括号后**必须有分号**；成员用 `.` 访问：`Item i; i.weight = 4;`。

8. 官方三种背包递归写法对照：为什么有的要撤销、有的不用？ 官方在第 12 讲一口气展示了三种等价写法，目的是让人看到"同题多解、各有权衡"，并强调考试不要求复刻某种写法、只要求正确可读。三者的本质差别是**状态如何管理**，而"要不要 unchoose"完全由此决定：

写法	状态管理方式	需要 unchoose 吗	特点
方案 1：从容器取末尾 + 放回	共享 vector（按引用）逐个取出物品	必须：返回前把物品放回	取末尾是 O(1)；超重物品强制走"不拿"单分支
方案 2：下标 k 推进	vector 只读，另传整数 k	不需要：从未改动共享状态	k 表示"前 k 件已决策"；k == size() 即基准；代码最省心
方案 3：valueSoFar 累计	共享 vector + 累计价值参数	必须；另加 capacity < 0 基准	不提前用 if 拦超重（即不做"arm's length recursion"），而是让基准情形拒绝装超重的非法路径

本节示例 3 采用"方案 2 + 剪枝"：既保有按引用传容器、避免按值拷贝整表的效率，又因为只用下标游走而完全免去 unchoose 簿记——是初学者最容易写对的一种。

代码示例与实现详解

下面 3 个示例只依赖 C++17 标准库，可独立编译运行。

示例 1：生成子集（打印版 + 计数版 + bool 判断版）

```cpp
#include <iostream>
#include <string>
#include <vector>
using namespace std;

// 打印版：soFar=已选元素组成的子集，rest=尚未决策的元素
void printSubsets(const string& soFar, const string& rest) {
    if (rest.empty()) {                     // 全部元素决策完毕 → 打印一个子集
        cout << "{" << soFar << "}" << endl;
        return;
    }
    string newRest = rest.substr(1);        // 剥出当前元素 rest[0] 后的剩余
    printSubsets(soFar, newRest);           // 不要 rest[0]（exclude）
    printSubsets(soFar + rest[0], newRest); // 要 rest[0]（include）
}

// 计数版：把打印换成“每片叶子贡献 1”，内部用 + 汇总
int countSubsets(const string& soFar, const string& rest) {
    if (rest.empty()) {
        return 1;                           // 到达一个完整子集 → 计 1
    }
    string newRest = rest.substr(1);
    return countSubsets(soFar, newRest) +   // 不要
           countSubsets(soFar + rest[0], newRest);   // 要
}

// bool 版：是否存在某个子集，其元素和恰好等于 target?
// nums 只读，用下标 k 游走（无需撤销）；叶子直接给出“这条路成不成”
bool hasSubsetSum(const vector<int>& nums, int k, int sumSoFar, int target) {
    if (k == (int)nums.size()) {
        return sumSoFar == target;
    }
    // 不选 nums[k] 或 选 nums[k]；|| 短路 → 一旦找到 true 立即整树停止
    return hasSubsetSum(nums, k + 1, sumSoFar, target) ||
           hasSubsetSum(nums, k + 1, sumSoFar + nums[k], target);
}

int main() {
    printSubsets("", "ab");                 // 输出 {} {b} {a} {a,b}（顺序因先 exclude）
    cout << "子集总数: " << countSubsets("", "abc") << endl;   // 8 = 2³
    cout << "存在和为6的子集: " << hasSubsetSum({1, 2, 3, 7}, 0, 0, 6) << endl; // 1（1+2+3）
    return 0;
}
```

【代码做什么】：`printSubsets("", "ab")` 沿决策树下行：每个元素先试"不要"再试"要"，到叶子（rest 空）打印 `soFar`，共 4 行；`countSubsets` 结构完全相同，仅把叶子行为从打印改成 `return 1`、把两条递归路径用 `+` 相连，于是根调用返回 $2^3 = 8$；`hasSubsetSum` 是第三个"bool 风格"函数：对 nums 里每个元素同样做"不选/选"的二选一，叶子用 `sumSoFar == target` 判定成败，主函数里 {1,2,3,7} 存在子集 {1,2,3} 和为 6，输出 1。

【实现机制解说】：`soFar` / `rest` 都是按值参数，每层调用拿到的是父状态的拷贝，**因此不需要任何显式撤销**——这就是第 3 节"按值天然撤销"的实例。三个函数演示了"打印 → 计数 → 判断"的返回风格切换：叶子处 return 1 并用 `+` 汇总得计数；叶子处返回布尔条件并用 `||` 汇总得判断，而 `||` 的短路让"找到解就整棵停止"**零成本**发生（官方在第 11 讲强调这种变形必须熟练掌握）。代价也要看清：递归调用数 2^n 是指数级，若 n 大到 30+，即使每层 O(1) 也无法承受——官方提醒，对指数算法连"每层多一次 O(n) 的字符串拷贝"都会被放大成明显变慢，面试时往往要改成传引用 + 下标的高效版。

示例 2：isPartitionable——回溯 + 显式撤销 + 短路早停

```
#include <iostream>
#include <vector>
using namespace std;

// rest: 还没分配的元素（共享容器，按引用）；sum1/sum2: 两组当前总和
bool isPartitionable(vector<int>& rest, int sum1, int sum2) {
    if (rest.empty()) {                 // 基准：元素全部分完 → 两桶和相等即成功
        return sum1 == sum2;
    }
    int item = rest.back();             // 从末尾取（O(1)，比从 0 号取快）
    rest.pop_back();                    // ← choose：item 离开“待分配”容器
    // explore：两条路——给组1 或 给组2。|| 短路 → 组1 成功就不再探索组2
    bool ok = isPartitionable(rest, sum1 + item, sum2) ||
              isPartitionable(rest, sum1, sum2 + item);
    rest.push_back(item);               // ← unchoose：放回，供兄弟分支继续用
    return ok;
}

// 包装函数：两组初始和都是 0
bool isPartitionable(vector<int>& v) {
    return isPartitionable(v, 0, 0);
}

int main() {
    vector<int> a = {1, 1, 2, 3, 5};
    vector<int> b = {1, 4, 5, 6};
    cout << "可分: {1,1,2,3,5} -> " << isPartitionable(a) << endl;   // 1 (true)
    cout << "可分: {1,4,5,6}   -> " << isPartitionable(b) << endl;   // 0 (false)
    return 0;
}
```

【代码做什么】：`isPartitionable` 每次从 `rest` 末尾取一个元素，递归尝试把它放进组 1 或组 2；`rest` 空了就检查两组和。主函数验证两个官方用例：{1,1,2,3,5} 可分（如 {1,5} 与 {1,2,3}），{1,4,5,6} 不可分。

【实现机制解说】：这是"共享状态 + 显式撤销"的标准样板，请对照第 3 节骨架逐行读：`pop_back` 是 **choose**；两处递归是 **explore**；`push_back` 是 **unchoose**，它必须与 choose 一一配对，位置在返回值算完之后、`return` 之前。若删掉 `push_back`，第一分支把 `rest` 掏空后不还原，回溯到上层换走另一条分支时容器是空的，函数会在半途的某个 sum 组合上误判成功——官方演示过：删掉该行后 {1,4,5,6} 会错误地返回 true。另外两处细节：**短路**——若组 1 的分支已返回 true，`||` 右侧的组 2 分支根本不执行，整棵搜索树

立即收工；**取末尾而非取开头**——`pop_back` / `push_back` 都是 O(1)，而从下标 0 移除/插入是 O(n)，在指数递归里每一层省下的 O(n) 会放大成巨大差异（官方第 11 讲的 Exam Prep 专门点了这一点）。

示例 3：0–1 背包回溯版（struct Item + 剪枝）

```
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

struct Item {           // 注意：struct 定义在函数外；右花括号后要有分号！
    int weight;
    int value;
};

// 从下标 k 起的物品中挑，剩余容量 capacity，返回能获得的最大价值
int knapsack(const vector<Item>& items, int capacity, int k) {
    if (k == (int)items.size()) {       // 基准：没有物品可选了
        return 0;
    }
    const Item& it = items[k];
    int best = knapsack(items, capacity, k + 1);   // 不拿：作为基线
    if (it.weight <= capacity) {        // 剪枝：拿不动只能放弃（少走半棵子树）
        int take = it.value + knapsack(items, capacity - it.weight, k + 1);
        best = max(best, take);
    }
    return best;                        // 拿与不拿中取更优
}

// 包装函数：从第 0 号物品、满容量开始
int knapsack(const vector<Item>& items, int capacity) {
    return knapsack(items, capacity, 0);
}

int main() {
    vector<Item> items1 = {{4,6}, {2,4}, {3,5}, {1,3}, {6,9}, {4,7}};  // {w, v}
    vector<Item> items2 = {{8,160}, {3,58}, {3,58}, {3,58}, {1,2}};
    cout << "容量10 最优价值: " << knapsack(items1, 10) << endl;   // 19
    cout << "容量10 最优价值: " << knapsack(items2, 10) << endl;   // 176（贪心只有162）
    return 0;
}
```

【代码做什么】：对每个物品做"拿/不拿"决策。拿得动才尝试拿（剪枝），拿与不拿两个候选取 `max`；基准是没有物品可选时价值为 0。主函数复现官方两个测试：前者最优 19（拿 2、3、5 号与 (4,7) 号：重 10 值 19），后者最优 176——顺便戳穿贪心。

【实现机制解说】：这个版本采用官方的"下标 k 推进"写法：k 表示"前 k 件已决策、从第 k 件起待决策"，因此**不需要修改任何共享容器，也就没有 unchoose 环节**——撤销的缺失正是按引用传递但只用下标游走的红利（官方第 12 讲的方案 2/3 都属此类，方案 1 则需要显式把取出的物品放回）。再看复杂度与剪枝：最坏情形（每件都装得下）每个节点分裂成 2，是 $O(2^n)$；最好情形（每件都超重）只剩"不拿"单分支，退化 O(n)。

官方在第 12 讲坦率指出：这套朴素回溯**不是**最高效解法——同一条递归树里有大量重复子问题（如"还剩容量 3、还剩物品 3..n"会反复求解），后续课程会学 memoization（记忆化）与动态规划把它优化到多项式级，本章只要求掌握回溯骨架与指数本质。

复杂度分析

算法	时间（最好）	时间（最坏）	额外空间	原因简述
打印/计数子集	O(2ⁿ)	O(2ⁿ)	O(n)（栈深）	必须访问 2ⁿ 片叶子；字符串拷贝再多付 O(n) 因子
isPartitionable	远小于 O(2ⁿ)（短路早停）	O(2ⁿ)	O(n)（栈深）	每个元素两个去向；命中即停时通常远快于最坏
0–1 背包回溯	O(n)	O(2ⁿ)	O(n)（栈深）	每件都超重→只有"不拿"一支；每件都装得下→二叉树 2ⁿ
（对比）线性任务	O(n)	O(n)	O(1)	无分支，仅顺序处理

说明：三个问题的共同点是"决策树叶子数随 n 指数增长"，n 每加 1 工作量翻倍——这就是官方反复强调"指数运行时不可扩展"的原因：n = 30 的 $2^{30} \approx 10$ 亿次调用已接近极限，n = 60 则彻底不可行。剪枝与短路只能砍掉明显无望的分支，不能改变指数本性。

关键要点

- 把回溯背成三字诀：**choose（改状态）→ explore（递归）→ unchoose（还原）**，三步缺一不可、顺序不乱。
- 状态**按值拷贝**则返回即撤销；状态**按引用共享**则必须显式撤销，且撤销与选择严格配对——放回位置在递归返回之后、return 之前。
- 同骨架三种返回风格：叶子打印（void）、叶子 `return 1` + 内部 `+` 汇总（计数）、叶子返回布尔条件 + 内部 `||` 汇总（判断）；bool 版用短路免费获得"找到即停"。
- 剪枝要趁早：在递归调用**之前**用约束检查砍掉不可能的分支（超重强制不拿），一行 if 常常省下半棵子树。
- 回溯解是"正确性优先"的暴力搜索：指数复杂度注定了它只适合小规模 n；看到大规模输入要立刻想到 memo/DP 这类优化方向。

常见陷阱与注意事项

1. **忘了 unchoose**：共享容器（vector/数组）被上一层分支掏空或改坏，兄弟分支得到错误状态，结果可能"假阳性"（官方反例：删掉放回行后 {1,4,5,6} 误报可分）。规避：凡按引用修改状态，写完后立刻补还原语句，并用"两个分支都要正确"的用例测试。

2. **把 unchoose 写在 `return` 之后**：那条语句永远不执行，等于没写。规避：还原必须发生在返回之前；若提前 return，先把还原语句复制到每个出口前面。
3. **短路被误用/漏用**：`a || b` 只在 a 为 false 时才执行 b——若两个分支都有必须执行的"收尾动作"（如各放回一个元素），把它们都包在递归调用内部或先算出两个 bool 再合并。官方指出：先 `bool r1 = f(); bool r2 = g();` 再 `r1 || r2` 会失去短路带来的早停。
4. **修改了按引用传入的调用方数据且不还原**：即使算法正确，调用者的 vector 也被毁了。规避：要么只读 + 用下标游走，要么严格 choose/unchoose 配对。
5. **基准情形漏掉空输入/空容器**：空集可分吗（两空桶和为 0 相等，答案是 true）？空物品的背包价值是 0？先想清楚边界再写代码。
6. **误用贪心代替搜索**：0–1 背包按价值或 v/w 贪心都可能次优（官方两个反例）。规避：只要题目说"最大/最优 + 组合爆炸"，先按回溯枚举想，别默认贪心成立。
7. **struct 定义细节**：右花括号忘分号、定义在函数内部、成员名拼错。规避：struct 放函数外、`};` 结尾、用 `.` 访问成员。
8. **在指数递归里埋 O(n) 操作**（在开头删元素、每层复制整串）：n 稍大就慢到不可接受。规避：优先从容器末尾操作或改用下标参数。
9. **把"顺序无关"的子集当成"顺序有关"的排列去枚举**：对 {a, b} 会同时生成 {a, b} 与 {b, a}，既重复又使规模从 2^n 膨胀成 n! 量级。规避：子集决策树里元素顺序固定，每个元素只决策一次"要/不要"；只有真正讲究顺序的问题才用排列骨架。
10. **计数版基准情形 `return 0` 而不是 `return 1`**：空子集也是合法子集，叶子应计 1；return 0 会让 `countSubsets("")` 输出 0（正确应为 1），并在更大集合上少算。规避：先在空输入上验证边界：`countSubsets("")` 应为 1、空集合的"子集存在和 target"问题要单独想清语义。

思考题（带答案）

问题 1：把示例 1 的 `countSubsets` 改成"bool 风格"：判断集合中是否存在某个子集，其元素之和等于给定目标值 target。只需要改动哪几处？ **答案**：签名变为 `bool subsetSumExists(const vector<int>& nums, int k, int sumSoFar, int target)`：基准改为 `if (k == nums.size()) return sumSoFar == target;`；中间不再需要 `+` 计数，而是 `return subsetSumExists(..., k+1, sumSoFar, target) || subsetSumExists(..., k+1, sumSoFar + nums[k], target);`（分别对应不选/选当前元素）。`||` 的短路保证一旦某分支凑出 target，其余分支立即停止——这是"打印→计数→判断"三种返回风格切换的标准示范。
问题 2：官方演示过：把示例 2 中 `rest.push_back(item)`（unchoose）那一行删掉，{1,4,5,6} 这个本应返回 false 的输入会错误地返回 true。请解释为什么。 **答案**：没有放回，第一分支（把元素逐个试放进组 1/组 2）递归到底把 `rest` 掏空后就再没还原过。返回上层尝试"另一个组"时，`rest` 仍是空的，函数立刻命中基准 `rest.empty()`，拿当时半途的 sum1、sum2 直接比较——某个中间状态下两桶和恰好相等，于是误报 true。可见 unchoose 不是风格问题而是正确性要求：它保证每个分支看到的 `rest` 都是"自己该决策的那批元素"。 **问题 3**：0–1 背包回溯的最坏复杂度 $O(2^n)$ 何时出现？"超重只能不拿"的剪枝为什么能让最好情形降到 O(n)？ **答案**：最坏出现在每个物品都轻于或等于剩余容量、每个节点都分裂出"拿/不拿"两个递归调用，形成满二叉树 2^n 片叶子；最好情形是每件物品都超重，每个节点只有"不拿"一条分支，递归变成一条直线

O(n)。剪枝的本质是：装不下的选择**不可能成为最优解的一部分**，提前砍掉它既不影响正确性，又避免探索一整棵注定无解的子树。

Lecture 7: 排序算法：选择、插入、归并与快速排序（Sorting: Selection, Insertion, Merge & Quicksort）（对应课程真实讲座 L13）

概述

本讲系统学习四类排序算法：先讲两个 $O(n^2)$ 的朴素算法——选择排序与插入排序，通过它们体会"比较多 vs 交换多"的工程权衡；再讲分治思想的代表归并排序 O(n log n)，最后简介快速排序。排序是无数上层算法（二分查找、去重、中位数、统计）的基石，官方称其在计算机科学中应用极广；学会从"最好/平均/最坏 + 空间"四个维度比较算法，是本讲真正的目的。 （对应官方 2026 夏季 CS106B：L13 Tuesday, July 14 "Sorting Algorithms"。官方配套发布了 sorting-stuff.zip 幻灯片与代码，并附有选择/插入/归并在不同规模随机向量上的实测耗时对比。）

核心概念与算法原理

先约定：本课程说"已排序（sorted）"默认指**非降序**（从小到大，允许相等），这比"严格递增"更精确（官方第 9 讲的定义）。下面所有算法都在原容器内就地排序，只依赖元素间的 `<` 比较，属于**比较排序**。

1. 选择排序（selection sort）——"多看少动"的等待型算法 – 问题定义：把数组从小到大排好序。 – 直观解释：像每次从一堆牌里挑出最小的一张放到最前面，再在剩下的牌里挑最小的放第二位……直到挑完。每轮只做**一次交换**，但为了确定"谁最小"要比较很多次——官方戏称它是"wait and see"算法，花大量时间比较，才决定把哪个元素换到目标位。 – 步骤分解： 1. 维护前缀 `[0, start)` 已排好，`start` 从 0 开始； 2. 在 `[start, n)` 里线性扫描找出最小值下标 `minIdx`（第一轮 n 次比较，第二轮 n−1 次……）； 3. 把 `v[start]` 与 `v[minIdx]` 交换（每轮至多 1 次）； 4. `start++`，重复直到只剩一个元素。 – 逐轮走查（竖线左边是已排序前缀）：

```
数组 [3 1 4 2]
第1轮 start=0：扫 [3 1 4 2] 最小=1 → 交换 → [1 | 3 4 2]
第2轮 start=1：扫 [3 4 2]   最小=2 → 交换 → [1 2 | 4 3]
第3轮 start=2：扫 [4 3]     最小=3 → 交换 → [1 2 3 | 4]
第4轮 只剩 4，结束。比较总次数 = 3+2+1 = n(n−1)/2，即 O(n²)
```

2. 插入排序（insertion sort）——"边走边停"的理牌算法 – 直观解释：像打扑克理牌：左手牌已排好，右手每次摸一张新牌，把它**往左拖**，一路与左边的牌比较，直到遇到比它小的牌就停，落位。官方戏称它是 "let's gooooo!" 算法——每经过一个位置就交换/挪动一次，但**一旦遇到更小的牌立即停止**，因此比较次数可能远少于选择排序。 – 步骤分解： 1. 前缀 `[0, start)` 已有序，`start` 从 1 开始； 2. 取 `key = v[start]`（新摸的"牌"），在已排序前缀里从右向左把比 key 大的元素逐个右移一格（腾出空位）； 3. 遇到 ⩽ key 的元素或到达开头，把 key 填入空位； 4. `start++` 重复。 – 逐轮走查：

```
数组 [3 1 4 2]
第1轮 取1：3>1 → 3右移 [3 3 4 2]，1落位 → [1 3 4 2]
第2轮 取4：3<4 立即停，原位不动        → [1 3 4 2]
第3轮 取2：4>2右移、3>2右移、1<2停     → [1 2 3 4]
对“已有序输入”，每轮只比 1 次就停 → 总比较 n−1 次，O(n)！
```

3. 比较多 vs 交换多：工程权衡（官方第 13 讲重点） 选择排序**比较多、交换少**（$O(n^2)$ 次比较但最多 n−1 次交换）；插入排序**交换/挪动可能多、比较可以少**（遇到小牌提前停）。官方给的场景化例子很形象：

场景	贵的是谁	应选
仓库给一批"冰箱"排序	搬动（交换）昂贵，比价便宜	选择排序（交换次数最少）
赛跑排定名次	一次"比较"（比赛）又累又贵	插入排序（可比到一半提前停）
生物信息学排序基因序列	比较=昂贵模拟；交换只是换数字 ID	选择排序（少交换）
学生记录按学号排	比较便宜；记录数据大、写回贵	选择排序（少写入）

选择/插入的其他取舍：插入排序对"几乎有序"数据表现惊艳（官方建议：若应用里输入经常接近有序，选插入排序吃它的最好情形 O(n)）；而排序算法越简单越容易写对、容易调试——快速原型 vs 极致性能之间永远要权衡。

4. 归并排序（merge sort）——"先分后合"的分治算法 – 问题定义与思路：把数组从中间一分为二，递归把两半各自排好，再写一个"合并（merge）"过程把两个有序子数组合成一个有序数组。官方概括为"**易分难合（easy split, hard join）**"：拆是两行递归的事，难点全在合并。 – 步骤分解： 1. 基准：区间只剩 0 或 1 个元素，天然有序，直接返回； 2. 分：`mid = lo + (hi − lo) / 2`（防溢出公式，官方再次强调 `(lo+hi)/2` 会溢出），递归排 `[lo, mid]` 与 `[mid+1, hi]`； 3. 合：用两个指针 i、j 分别指向左右两半开头，每次把较小者放入辅助数组 aux；某半耗尽后把另半剩余整体倒入；最后把 aux 拷回原区间。 – 递归树与复杂度图解（每层总工作量 O(n)，共约 $\log_2 n$ 层）：

```
                [38 27 43 3 9 82 10]
               /                    \
      [38 27 43 3]                [9 82 10]
       /        \                  /      \
   [38 27]    [43 3]           [9 82]    [10]
   /    \     /    \           /    \
 [38]  [27] [43]  [3]        [9]   [82]      ← 拆到单元素（分）
     ↑ 每层“合”的工作量加起来都是 O(n)，层数 O(log n) → 总 O(n log n)
   [27 38]  [3 43]          [9 82]    [10]
      \        /              \        /
   [3 27 38 43]              [9 10 82]
         \                      /
      [3 9 10 27 38 43 82]              ← 最终有序（合）
```

- 合并过程示例（左右各一指针，比较后取小者放入辅助区）：

```
左半 [3 27 38 43]   右半 [9 10 82]
i→3  vs  9 → 取3            aux: [3]
i→27 vs  9 → 取9            aux: [3 9]
i→27 vs 10 → 取10           aux: [3 9 10]
i→27 vs 82 → 取27           aux: [3 9 10 27]
……直至右半耗尽，左半剩余 [38 43] 整体倒入 → aux: [3 9 10 27 38 43 82]
```

- **缺点（官方明说）**：① 合并需要 O(n) 辅助空间，内存吃紧时不合适；② 递归调用有压栈开销，对很小的数组（如 ≤100 个元素）选择/插入反而更快——官方实测：n=10 时三者都在毫秒级且插排略胜；n=100 时归并才开始反超；n=50,000 时差距拉大到 2.4 秒（选择）vs 1.4 秒（插入）vs 约 7 毫秒（归并）。

5. 快速排序（quicksort）——"难分易合"，实践中最快之一 – 官方把它与归并对比：归并是"易分难合"，快排是"**难分易合（hard split, easy join）**"——难点在 O(n) 的**分区（partition）**，合并不需要（递归返回时数组已经就位，官方原话：join 是递归的自然结果）。 – 步骤分解： 1. 选一个基准 pivot（示例取区间末尾元素）； 2. 分区：一趟把数组重排为"≤ pivot 的元素都在左，pivot 居中，> pivot 都在右"，返回 pivot 最终下标 p——一趟 O(n)； 3. 递归排序 pivot 左右两个子区间（都严格小于原区间）； 4. 基准：区间 ≤ 1 个元素。 – 快排性质：平均 O(n log n)、最坏 O(n^2)（见"最坏情形"：已有序输入 + 固定取尾做 pivot 时，每趟只切掉一个元素）；实践中因为**原地交换、缓存友好**，往往跑得比归并还快——这也是它名字的由来。官方说明快排会在后续作业与 CS161 中正式登场，本讲只要求理解分区思想。

代码示例与实现详解

下面 3 个示例只依赖 C++17 标准库、自包含可编译。

示例 1：选择排序与插入排序

```cpp
#include <iostream>
#include <utility>      // std::swap
#include <vector>
using namespace std;

// 每轮找未排序区间的最小值，换到区间开头
void selectionSort(vector<int>& v) {
    for (int start = 0; start < (int)v.size() - 1; ++start) {
        int minIdx = start;                     // 假设当前位已是最小
        for (int j = start + 1; j < (int)v.size(); ++j) {
            if (v[j] < v[minIdx]) minIdx = j;   // 只更新下标，不急着交换
        }
        swap(v[start], v[minIdx]);              // 每轮至多一次交换
    }
}

// 每轮取未排序区第一个元素，往左拖到合适位置（比它大的整体右移）
void insertionSort(vector<int>& v) {
    for (int start = 1; start < (int)v.size(); ++start) {
        int key = v[start];                     // 新摸的“牌”
        int gap = start;                        // 空位从 start 开始向左找
        while (gap > 0 && key < v[gap - 1]) {   // 左邻更大 → 右移腾位
            v[gap] = v[gap - 1];
            --gap;
        }
        v[gap] = key;                           // key 落位
    }
}

void show(const vector<int>& v) {
    for (int x : v) cout << x << ' ';
    cout << endl;
}

int main() {
    vector<int> a = {3, 1, 4, 2};
    vector<int> b = a;
    selectionSort(a);   cout << "选择排序: "; show(a);
    insertionSort(b);   cout << "插入排序: "; show(b);
    return 0;
}
```

【代码做什么】：两个排序函数各就各位地对同一个输入演示。`selectionSort` 双层循环：外层固定"待安放位置"，内层找最小、外层结束后才交换一次；`insertionSort` 把新元素存进 `key`，用 `gap` 指针把比它大的元素一个个右移，最后把 `key` 写回空位（这就是"拖拽"的实现，官方代码里管这张牌叫 peach）。

【实现机制解说】：选择排序的交换**必须放在内层循环之外**——内层只是不断刷新 `minIdx`，找到最终目标才动手，否则每发现一个更小值就换一次，交换次数会从 O(n) 退化到 O(n^2)（这正是"比较多 vs 交换多"取舍的代码体现）。插入排序的移动是**整体平移而非两两交换**：`v[gap] = v[gap-1]` 把大牌往右推一格，`key` 始终攥在手里，最后一次性落位，一趟的写次数等于"它左边比它大的元素个数"；而 `while` 的提前退出条件 `key`

`< v[gap-1]` 一旦不成立（左邻 ⩽ key）立即停止——对已排序输入，每轮只比较 1 次，整趟 O(n)，这就是插入排序最好情形的来源。两函数都就地修改、空间 O(1)。

示例 2：归并排序（递归 + 辅助 vector 归并）

```
#include <iostream>
#include <vector>
using namespace std;

// 归并排序 v[lo..hi]（闭区间）
void mergeSort(vector<int>& v, int lo, int hi) {
    if (lo >= hi) return;                    // 基准：0 或 1 个元素，天然有序
    int mid = lo + (hi - lo) / 2;            // 防溢出中点（勿写 (lo+hi)/2）
    mergeSort(v, lo, mid);                   // 分：左半
    mergeSort(v, mid + 1, hi);               // 分：右半
    // 合：双指针归并两个有序子数组
    vector<int> aux;                         // O(n) 辅助空间（本算法最大“缺点”）
    int i = lo, j = mid + 1;
    while (i <= mid && j <= hi)              // 两边都还有货：取较小者
        aux.push_back(v[i] <= v[j] ? v[i++] : v[j++]);
    while (i <= mid) aux.push_back(v[i++]); // 左半剩余整体倒入
    while (j <= hi)  aux.push_back(v[j++]); // 右半剩余整体倒入
    for (int k = 0; k < (int)aux.size(); ++k)   // 拷回原区间
        v[lo + k] = aux[k];
}

void mergeSort(vector<int>& v) {             // 包装函数（重载）
    if (!v.empty()) mergeSort(v, 0, (int)v.size() - 1);
}

int main() {
    vector<int> v = {38, 27, 43, 3, 9, 82, 10};
    mergeSort(v);
    for (int x : v) cout << x << ' ';
    cout << endl;                            // 3 9 10 27 38 43 82
    return 0;
}
```

【代码做什么】：`mergeSort(v, lo, hi)` 先递归排左右两半，再用辅助 vector `aux` 完成"合"：两个游标 i、j 分别走在左右半，谁小谁进 aux；某半耗尽就整体倒入另一半剩余；最后把 aux 的 lo..hi 段拷回原位。包装函数负责对外只暴露 `mergeSort(v)` 一个参数。

【实现机制解说】：结合第 4 节的递归树理解执行流：`mergeSort(v,0,6)` 先递归到最深——`[38]`、`[27]` 等单元素区间逐一返回（基准 `lo >= hi`），然后**归途上自底向上**合并：`[27 38]`、`[3 43]`、`[9 82]`，再合并成 `[3 27 38 43]` 与 `[9 10 82]`，最后合成整段有序——合并总是发生在左右都已有序之后，所以双指针比较取小必然正确。归并用 `<=` 取左半元素，相等元素保持左先右后的相对顺序，因此归并是**稳定**的。三个 while 一个都不能少：主循环结束后必有一半先耗尽，剩余元素必须整体续尾，否则会丢元素。中点公式在此与二分查找同样防溢出。空间上每层合成都新建一个 aux，栈深度 O(log n) 层，同层最多共存约 n 大小的辅助区，故总辅助空间 O(n)。

示例 3：快速排序（完整小实现，Lomuto 分区）

```
#include <iostream>
#include <utility>      // std::swap
#include <vector>
using namespace std;

// 一趟分区：以末尾元素为 pivot，返回 pivot 最终位置；
// 结束后 pivot 左侧都 ≤ pivot，右侧都 > pivot
int partition(vector<int>& v, int lo, int hi) {
    int pivot = v[hi];
    int i = lo;                             // i 左边都是已确认 ≤ pivot 的
    for (int j = lo; j < hi; ++j) {
        if (v[j] < pivot) {                 // 发现该去左边的元素
            swap(v[i], v[j]);               // 换到“小元素区”末尾
            ++i;
        }
    }
    swap(v[i], v[hi]);                      // pivot 归位
    return i;
}

void quickSort(vector<int>& v, int lo, int hi) {
    if (lo >= hi) return;                   // 基准：0/1 个元素
    int p = partition(v, lo, hi);           // “难”的一步：O(n) 分区
    quickSort(v, lo, p - 1);                // 左边递归（比 pivot 小的）
    quickSort(v, p + 1, hi);                // 右边递归（比 pivot 大的）
}

void quickSort(vector<int>& v) {
    if (!v.empty()) quickSort(v, 0, (int)v.size() - 1);
}

int main() {
    vector<int> v = {7, 2, 9, 3, 6, 1, 8, 5, 4};
    quickSort(v);
    for (int x : v) cout << x << ' ';
    cout << endl;                           // 1 2 3 4 5 6 7 8 9
    return 0;
}
```

【代码做什么】：`partition` 选末元素为 pivot，用快慢两个下标 i、j 扫一遍：j 负责考察每个元素，凡小于 pivot 的都与 i 所指位置交换，使 i 左侧始终是"已确认 ≤ pivot"的区域；扫描结束把 pivot 换到 i 处并返回 i。`quickSort` 递归排 pivot 左右两侧。主函数对 9 个乱序整数排序验证。

【实现机制解说】：体会"难分易合"：`partition` 一趟 O(n) 完成重排后，pivot 已在其最终位置，左右两侧只需各自递归——**没有合并步骤**，因为"合"被分区天然完成了。为什么平均 O(n log n) 而最坏 $O(n^2)$？若 pivot 每次都能把区间切成大致两半，递归树与归并一样深 O(log n)、每层 O(n)，总计 O(n log n)；但若输入已有序且 pivot 固定取末尾，每趟分区只把 pivot 挪走一个位置（一侧为空），递归树退化成一条长链，深度 O(n) → 总 $O(n^2)$。实践中快排快的原因：完全原地（无辅助数组）、缓存局部性好；工程上还会用"随机选

pivot 或三数取中"来避免有序输入触发最坏情形。注意本实现用 `<` 而非 `<=` 比较，重复元素会偏向 pivot 右侧，功能正确但可进一步优化。

复杂度分析

算法	时间（最好）	时间（平均）	时间（最坏）	空间（额外）	备注
选择排序	$O(n^2)$	$O(n^2)$	$O(n^2)$	O(1)	比较固定 n(n−1)/2；交换仅 O(n) 次
插入排序	O(n)	$O(n^2)$	$O(n^2)$	O(1)	已有序输入每轮 1 次比较即停；稳定
归并排序	O(n log n)	O(n log n)	O(n log n)	O(n)（辅助区 + 递归栈）	稳定；小数组时递归开销反而吃亏
快速排序	O(n log n)	O(n log n)	$O(n^2)$	O(log n)（平均栈深）	原地；有序输入+固定 pivot 触发最坏

官方实测参考（随机向量多次取平均）：n=10 时三者耗时都在 1 ms 上下（插排 0.000793 ms 甚至略快于归并 0.001027 ms）；n=1,000 时归并（约 0.10 ms）反超插排（0.56 ms）；n=50,000 时选择约 2.43 s、插入约 1.38 s、归并约 7.4 ms——印证两点：递归/辅助开销让小 n 时简单算法更优，而 n 一大 O(n log n) 对 $O(n^2)$ 就是数量级碾压。稳定性上：插入、归并（用 `<=` 取左）稳定；选择、快排的常见就地实现不稳定——若"相等元素保持原相对次序"重要，选稳定算法。

关键要点

- 选择排序"多看少动"：$O(n^2)$ 次比较但只有 O(n) 次交换；每轮**只交换一次**（交换放内层循环外）。
- 插入排序"边走边停"：遇到不比它大的牌立即停，因此**几乎有序的输入接近 O(n)**——判断数据形态后优先考虑它。
- 归并排序"先分后合"：O(n log n) 最坏也稳，代价是 O(n) 辅助空间与小数组时的递归开销——空间紧张或 n 很小就别用它。
- 快排"难分易合"：分区一趟 O(n)，平均 O(n log n)、原地且缓存友好；pivot 选得差（有序输入）会退化 $O(n^2)$。
- 没有"万能最好"的排序：选算法先问四个问题——输入规模多大？是否接近有序？比较/交换谁更贵？内存够不够？

常见陷阱与注意事项

1. **选择排序把交换写进内层循环**：每发现更小值就交换一次，交换次数退化为 $O(n^2)$，丢掉"少交换"的优点。规避：内层只更新 `minIdx`，内层结束后交换一次。
2. **插入排序的循环边界写错**：写成 `gap >= 0 && key < v[gap]` 会越界或漏比较。规避：用"空位在 gap、比较左邻 v[gap−1]"的写法：`while (gap > 0 && key < v[gap-1])`，落位写 `v[gap] = key`。

3. **归并基准写成 `lo > hi`（或忘处理 lo==hi)**：单元素区间还会继续分裂，陷入不必要的递归甚至死循环。规避：基准用 `lo >= hi`。
4. **合并的三个 while 少写一个**：主循环结束后必有一半还有剩余，不续尾就丢元素。规避：两个"整体倒入剩余"的 while 一个都不能删。
5. **中点公式 `(lo+hi)/2` 溢出**：与二分查找同款陷阱（官方第 13 讲再次强调)。规避：`lo + (hi-lo)/2`。
6. **忘记把辅助数组拷回原数组**：排完序却仍是无序原数组。规避：合并收尾用 `v[lo+k] = aux[k]` 回写。
7. **快排的有序输入最坏情形**：固定取尾做 pivot + 输入已有序 → $O(n^2)$ 且可能栈深爆掉。规避：随机选 pivot、三数取中，或对小分区切换插入排序。
8. **混淆"比较次数"与"运行时间"**：复杂度分析针对的是比较/交换这些基本操作的数量级，别用墙钟秒数直接下结论（机器、输入分布都影响实测)。

思考题（带答案）

问题 1：给你一个"基本有序"的 10 万元素数组（只有少量位置错乱)，你会选插入排序还是归并排序？两者复杂度分别如何？ **答案**：优先插入排序。它的最好情形 O(n) 恰好在"近乎有序"时出现：每个元素往左拖不了几步就停，总比较接近 O(n)；归并虽然最坏保证 O(n log n)，但对这种输入没有利用有序性的优势，还要付出 O(n) 辅助空间。这正体现"选算法要看输入形态"（官方讨论过：若应用里常见某种触发最好情形的输入，就该选那个算法)；工程上也常用"归并/快排 + 小分区切插入排序"的混合策略。 **问题 2**：归并排序的 O(n) 辅助空间具体花在哪？能否完全省掉？ **答案**：花在"合"：每一层合并都要一个能容纳整个待合并区间的辅助数组 aux（两个有序子数组无法在不覆盖对方的情况下原地两两归并)。递归栈本身只占 O(log n)。完全原地归并在理论上可行但实现复杂、常数巨大，实践中几乎不用——所以内存敏感场景往往选择原地排序（如快排、堆排序，后者会在后续章节出现)。 **问题 3**：什么输入会让示例 3 的快排退化到 $O(n^2)$？怎样缓解？ **答案**：输入已经有序（升序或降序）且 pivot 固定取末尾时，每趟分区只会把 pivot 移到一端，另一侧为空，递归深度变成 n，总时间 $O(n^2)$。缓解手段：①随机选 pivot（把最坏输入变成概率极低事件)；②三数取中（取 lo、mid、hi 三者的中位数做 pivot)；③遇到小区间切换插入排序并限制递归深度。

Lecture 8: 面向对象编程：类、对象与封装（OOP: Classes, Objects & Encapsulation)（对应课程真实讲座 L15）

概述

本讲是课程的重要转折：此前我们一直以"客户端"视角使用别人写好的 ADT（Vector、Stack 等)，从这一讲开始第一次亲手"造"一个类，从使用者变成建造者。核心问题是：如何用类把"数据 + 操作数据的函数"打包成一个整体，并通过接口（.h）与实现（.cpp）分离、私有成员与 getter/setter 等手段划清抽象边界，让对象从生到死都处于合法状态。(官方对应：2026 夏季学期 L15，Monday, July 20 — Object-Oriented Programming，课上用一只只可爱的小袋鼠 Quokka 从零搭起一个类，本讲也是作业 5 Tone Matrix 的知识铺垫。)

核心概念与算法原理

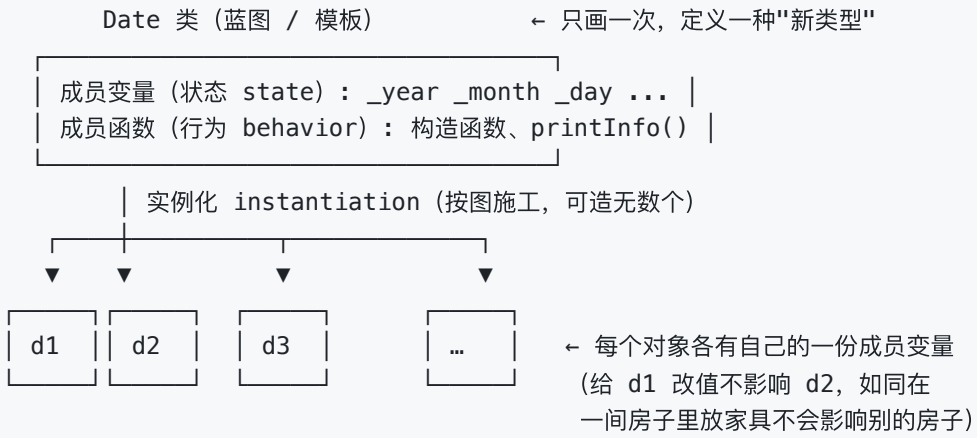
1. 范式转变：从"操作数据"到"对象收发消息"

问题定义：在 C 这类过程式语言里，数据结构与操作它的函数是分离的，写出来往往是 `goToFloor(elevator, 5)` 这种"把对象塞给外部函数"的样子。OOP 要回答的问题是：能不能让"能力"长在"数据"身上，让电梯自己会"去 5 楼"？

直观解释：坐电梯时你不会想"把电梯交给一个外部函数处理"，而是直接按电梯里的按钮——按钮和电梯是一体的。OOP 把这个直觉搬进代码：我们对对象"发消息"（调用它的成员函数），对象自己决定如何改变自己的内部状态。官方称这是一次根本性的范式转变（paradigm shift）。

2. 类 = 蓝图/类型，对象 = 实例

它是什么：类是"图纸"，也等价于声明一种**新的数据类型**；对象是按图纸造出来的具体"房子"（变量）。官方特别指出：这一季用过的 Vector、Stack、Set、Map 全都是类——`Vector<int> v;` 中 Vector 是类型，v 就是 Vector 类的一个对象（实例，instance）。同一张蓝图可以造出无数座结构相同、内部各异的房子。

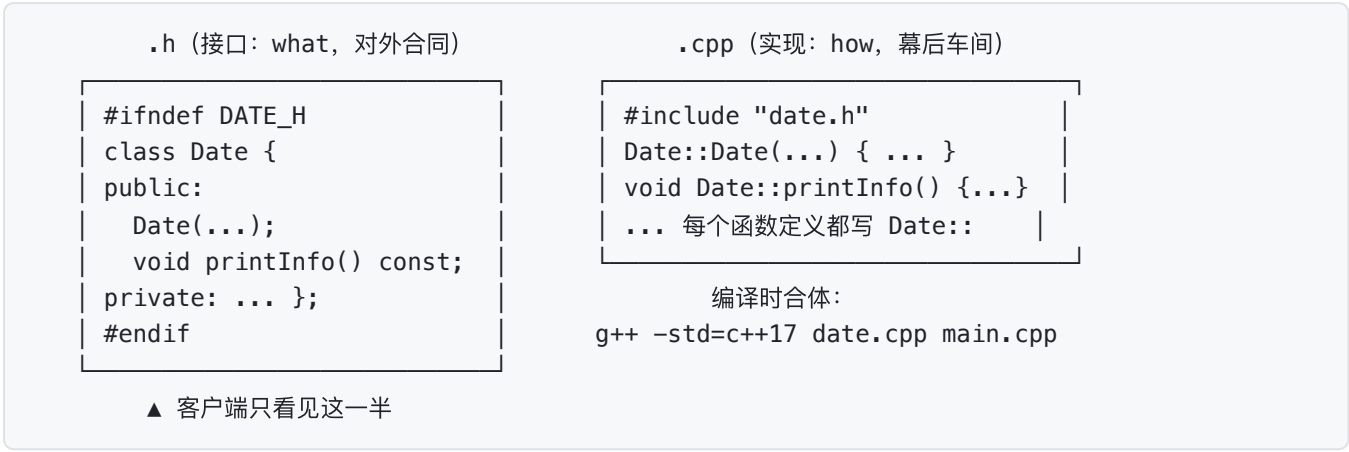


为什么要自己造类：官方给的理由是"扩充抽象词汇表"。把常用概念（如"日期"）封装成类型后，调用方不再需要盯着 `v.remove(v.size()-1)` 这类细节去猜意图，直接说"pop 一个"即可——抽象让代码更好读、更好交流、更难写错。

3. 接口（.h）与实现（.cpp）分离

问题定义：一个类有两类信息——外面的人需要知道"能调用哪些函数"（what）；而"具体怎么实现"（how）不必、也不应全部暴露。

它是什么：**接口文件**（.h，header）里放类声明：成员变量、成员函数**原型**，客户端 `#include` 它就够用了；**实现文件**（.cpp）里放每个成员函数的**定义**。官方把接口比作"从外面看到的类长什么样"，把实现比作"这些行为背后真正的代码"。



头文件顶部的三行守卫 `#ifndef DATE_H / #define DATE_H / … / #endif` 叫 include guard，防止同一头文件被间接包含多次造成重复声明错误（官方在 L15 指出 Qt Creator 新建类时会自动生成）。

4. 构造函数与析构函数：对象的一生

它是什么：与类**同名**的函数叫构造函数，任何时刻创建对象都会**自动**调用它，最适合做初始化；参数不同可以写多个构造函数，这叫构造函数重载。名字为 `~类名` 的函数叫析构函数，对象**离开作用域死亡时自动调用**。

对象的一生：局部对象在声明处出生（栈帧里分配空间 + 调构造函数），在函数返回或到达代码块末尾时死亡（自动调析构）。官方在 L15 用"会打印 R.I.P. 的析构函数"做了证明：`createQuokka()` 一返回，销毁信息立刻出现——局部变量确实随函数退出而消亡。这个"死亡"事实是下一讲（动态内存）的出发点。

5. 封装：private 成员与 getter/setter

问题定义：如果客户端能直接改内部数据，就可能把对象弄进"破损状态"。官方举例：假如 Vector 的 size 是公开变量，客户端写 `v.size = 1;`，底层明明还有 3 个元素却对外声称 1 个，遍历和删除全部错乱。

它是什么：把成员变量声明为 private（私有），只留受控的公共出入口：**getter** 只读返回某个私有成员的值；**setter** 是修改私有成员的唯一通道，可在里面加合法性校验。私有化不等于"锁死"，而是"所有改变都必须经过把门人"。官方归纳的理由：防止对象进入破损状态、能对写入值设限制、能保证对象一生状态可预期。

6. 匿名临时对象

有时我们不想给对象起名字（造出来立刻塞进容器或传给函数），可以直接调用构造函数，例如 `Quokka("Muffinface", 5, "muffinface.jpg")`——这种没有变量名的对象叫匿名临时对象，它只活在当前这一条语句里，语句结束随即析构（官方在 L15 演示了把 7 只匿名 Quokka 直接 add 进 Vector 的写法）。

代码示例与实现详解

示例 1：接口与实现分离的 Date 类（date.h + date.cpp + main.cpp 三文件项目）

这是官方推崇的工程组织方式：三个文件一起编译（`g++ -std=c++17 date.cpp main.cpp -o date_demo`）。注意头文件里全程使用 `std::` 前缀而**不写** `using namespace std;`——头文件会被任意客户端包含，把整个 std 命名空间"泄漏"给别人是很糟糕的做法。

```
// ================= date.h：接口（what）=================
#ifndef DATE_H
#define DATE_H
#include <string>

class Date {
public:
    Date();                                              // 默认构造函数
    Date(int year, int month, int day,
         const std::string& label);                      // 重载构造函数（4 参数版）
    ~Date();                                             // 析构函数：死亡时自动调用

    int year() const;        // getter；const 承诺"不改状态"
    int month() const;
    int day() const;
    std::string label() const;

    void setYear(int year);  // setter：带范围校验
    void setMonth(int month);
    void setDay(int day);

    void printInfo() const;

private:
    int _year;               // 私有成员：客户端无法直接读写
    int _month;
    int _day;
    std::string _label;
};
#endif // DATE_H
```

```cpp
// ================= date.cpp：实现（how）=================
#include <iostream>
#include "date.h"

Date::Date() {                          // 每个成员函数定义都要写 Date::
    _year = 2026; _month = 1; _day = 1; _label = "(未命名)";
}
Date::Date(int year, int month, int day, const std::string& label) {
    _year = year; _month = month; _day = day; _label = label;
}
Date::~Date() {
    std::cout << "【销毁】" << _label << " 离开这个世界" << std::endl;
}
int Date::year() const { return _year; }
int Date::month() const { return _month; }
int Date::day() const { return _day; }
std::string Date::label() const { return _label; }

void Date::setYear(int year) {
    if (year < 1900 || year > 2100) {
        std::cout << "年份 " << year << " 非法，已忽略" << std::endl;
        return;                         // 拒绝非法值，对象保持原状态
    }
    _year = year;
}
void Date::setMonth(int month) {
    if (month < 1 || month > 12) {
        std::cout << "月份 " << month << " 非法，已忽略" << std::endl;
        return;
    }
    _month = month;
}
void Date::setDay(int day) {
    if (day < 1 || day > 31) {
        std::cout << "日期 " << day << " 非法，已忽略" << std::endl;
        return;
    }
    _day = day;
}
void Date::printInfo() const {
    std::cout << _label << ": " << _year << "-" << _month << "-" << _day << std::endl;
}
```

```
// ================= main.cpp：客户端视角 =================
#include <iostream>
#include "date.h"           // 只用 #include 接口即可
using namespace std;

int main() {
    Date d1;                                  // 默认构造
    Date d2(2026, 7, 23, "考试日");        // 重载构造
    d2.setMonth(12);                          // setter 正常更新
    d1.printInfo();
    d2.printInfo();
    return 0;   // d2、d1 逆序析构（先各打印一行销毁信息）
}
```

【代码做什么】 date.h 只含类声明：public 区列出接口原型，private 区列出 4 个成员变量；date.cpp 用 `Date::` 前缀逐个给出定义。默认构造把日期初始化为 2026-01-01（避免未初始化垃圾值），4 参数构造按参数填充；析构打印"告别语"；三个 setter 先校验范围，非法输入直接拒绝。

【实现机制解说】 – 成员函数定义必须写成 `返回类型 类名::函数名(...)`，`Date::` 告诉编译器"这个函数属于 Date 类"；漏掉它会被当成与类无关的全局函数而报错。 – `printInfo() const`、getter 的 const 放在函数名后，等于向编译器承诺"本函数不修改任何成员变量"，一旦违反编译直接失败——这是防止未来误改的好习惯。 – 构造与析构成对触发：执行到 `Date d2(...)` 这行时，先为 d2 在栈上分配空间，再调用 4 参数构造函数；main 结束时 d2、d1 按**声明逆序**（后声明先销毁）自动析构并打印。生命周期由作用域决定，程序员不用手动"清场"。

示例 2：单文件等价可编译版（.h + .cpp + main 合并）

把示例 1 的三个文件合并成一个文件就是下面的样子——内容完全等价，适合初学者先把注意力放在语法上，之后再练习拆分。main() 额外演示 setter 校验、局部对象生命周期、匿名临时对象与按值拷贝现象。

```cpp
#include <iostream>
#include <string>
using namespace std;

class Date {
public:
    Date() { _year = 2026; _month = 1; _day = 1; _label = "(未命名)"; }
    Date(int y, int m, int d, const string& label) {
        _year = y; _month = m; _day = d; _label = label;
    }
    ~Date() { cout << "【销毁】" << _label << " 离开这个世界" << endl; }

    int year() const { return _year; }
    int month() const { return _month; }
    int day() const { return _day; }
    string label() const { return _label; }

    void setYear(int y) {
        if (y < 1900 || y > 2100) { cout << "年份 " << y << " 非法，已忽略" << endl; return; }
        _year = y;
    }
    void setMonth(int m) {
        if (m < 1 || m > 12) { cout << "月份 " << m << " 非法，已忽略" << endl; return; }
        _month = m;
    }
    void setDay(int d) {
        if (d < 1 || d > 31) { cout << "日期 " << d << " 非法，已忽略" << endl; return; }
        _day = d;
    }
    void printInfo() const {
        cout << _label << ": " << _year << "-" << _month << "-" << _day << endl;
    }
private:
    int _year, _month, _day;
    string _label;
};

// 生命周期演示：局部对象在函数返回那一刻死亡
void demoLifecycle() {
    cout << "--- 进入 demoLifecycle() ---" << endl;
    Date local(2026, 7, 20, "本地聚会");
    local.printInfo();
    cout << "--- 即将 return，local 将在此死亡 ---" << endl;
}   // ← local 在这里被析构（打印销毁信息）

// 参数用 const 引用：不产生拷贝
void announce(const Date& d) {
    cout << "公告：" << d.label() << " 的活动在 "
         << d.year() << " 年 " << d.month() << " 月 " << d.day() << " 日举行" << endl;
}

int main() {
    cout << "=== main 开始 ===" << endl;
```

```
    Date d1;                                  // 默认构造
    Date d2(2026, 7, 23, "考试日");           // 重载构造
    d1.setYear(9999);                         // 非法! setter 拒绝并保持原状态
    d1.setDay(30);                            // 合法，写入成功
    d1.printInfo();
    d2.printInfo();

    demoLifecycle();                          // 观察 local 随函数返回而销毁

    announce(Date(2026, 8, 3, "临时讲座")); // 匿名临时对象：本语句结束即析构

    Date d3 = d2;   // 按值"拷贝构造"，d3 是 d2 的独立副本
    cout << "d3 标签 = " << d3.label() << endl;

    cout << "=== main 即将结束（d3、d2、d1 将逆序销毁）===" << endl;
    return 0;
}
```

【代码做什么】 main() 依次验证：默认构造与重载构造都能用；非法年份 9999 被 setter 拦下（d1 保持原值，随后 setDay(30) 正常生效）；`demoLifecycle()` 内的局部对象在 return 那一刻析构，早于 main 继续执行；`announce(Date(...))` 用匿名临时对象当参数，临时对象在该语句结束时析构；`Date d3 = d2;` 触发默认拷贝构造，产生一个内容相同但各自独立的副本（因此结束时"考试日"会打印两次销毁信息：一次是 d3、一次是 d2）。

【实现机制解说】 – 构造函数重载的调度规则与函数重载一致：编译器按实参个数与类型挑选匹配版本，`Date()` 与 `Date(int,int,int,const string&)` 互不干扰。 – 输出顺序是观察对象生命周期的利器：先看到 printInfo 的结果；接着 demoLifecycle 内部"构造 → 打印 → 析构"一气呵成；匿名对象在 announce 语句结束后立刻析构；最后 d3、d2、d1 按"后进先出"的栈式顺序销毁。 – 按值传参会拷贝：若把 `announce` 的参数类型改成 `Date d`，每次调用都会多一次拷贝构造和一次析构。当对象内部持有堆内存时，默认拷贝只是"浅拷贝"，会让两个对象共享同一块内存、析构时双重释放——这正是指针与动态内存那一讲的核心动机，届时会引入"拷贝规则"（Rule of Three）来根治。 – 封装的收益在此可见：客户端永远无法把 Date 弄到"月份 = 13"的破损状态，因为唯一入口 setter 把守着合法性。

复杂度分析

操作	时间复杂度	说明
默认 / 重载构造	O(1)	仅对少量成员赋值（含 string 时取决于串长）
getter / setter	O(1)	读一个成员 / 一次范围比较后写入
printInfo 等输出成员函数	O(1)	输出固定数量的字段
析构	O(1)	成员逐个消亡（普通数据类型）
按值拷贝（默认拷贝构造）	O(成员个数)	逐成员复制；拷贝含 string 的成员要复制其内容

要点：类操作的开销几乎都来自"拷贝"；封装本身不增加任何复杂度，它买来的是正确性与可维护性。

关键要点

- 类是蓝图/新类型，对象是按蓝图造出的实例：每个对象各有自己的一份成员变量，互不干扰。
- 接口（.h）回答 what、实现（.cpp）回答 how；头文件不放 `using namespace std;`，并记得 include guard。
- 构造函数与类同名、创建对象时自动执行、可重载；析构函数形如 `~类名`，对象离开作用域自动调用——生命周期由作用域决定。
- 成员变量一律 private，通过 getter/setter 受控访问；setter 内做校验，保证对象永不进入破损状态。
- 匿名临时对象"造完即用、用完全毁"，适合一次性塞进容器或传给函数。

常见陷阱与注意事项

- **类定义结尾忘写分号**：`};` 必不可少，漏掉会引发一连串莫名其妙的编译错误。
- **.cpp 忘 `#include` 自己的 .h**：实现文件必须先包含头文件才能引用类声明。
- **定义成员函数忘写 `Date::`**：会被当成全局函数，编译器立即报"未定义"或签名不匹配。
- **在 .h 里写 `using namespace std;`**：会污染所有包含该头文件的代码；头文件里请写全称 `std::string`。
- **构造函数漏初始化某些成员**：成员会留下垃圾值；默认构造应给出安全初值（如 2026-01-01）。
- **把 setter 校验写成"悄悄纠错"而非"拒绝"**：比如月份 13 想当然地取模；正确做法是拒绝非法值并保持原状态（或报错），让调用方知情。
- **析构函数名漏 `~`**：`Date()` 与 `~Date()` 长得像但完全不同，前者是构造、后者是析构。
- **以为 getter 必须叫 `getXxx`**：官方约定只读属性可省略 get（如 `size()`、`year()`），需要写回时才配套 `setXxx`。

思考题（带答案）

问题 1：`void f() { Date d(2026, 7, 1, "内部"); }` 中，d 的构造函数与析构函数分别在什么时刻被调用？若 f 被调用 100 次，构造与析构各执行几次？ **答案**：构造在声明 d 的那一行执行（f 的栈帧里创建对象）；析构在 f 返回、d 离开作用域那一刻自动执行。调用 100 次则构造、析构各 100 次，永远成对——这正是"对象生命周期由作用域管理"的含义，也是本季第 16–17 讲讨论动态内存时的前提。

问题 2：为什么把 `_year` 设为 private 后，客户端反而"更安全"？请仿照官方"直接改 size"的例子说明。 **答案**：private 意味着内部数据只有类自己的成员函数能碰。若 size 是公开变量，客户端写 `v.size = 1;` 会让对象陷入"实际有 3 个元素、对外声称 1 个"的自相矛盾，遍历与删除全部出错；而 setter 是唯一入口，可以加校验（如年份必须在 1900~2100），从而保证对象从构造到析构始终合法、可预期。

问题 3：`vector<Date> v; v.push_back(Date(2026, 8, 1, "会议"));` 中那个匿名 Date 何时析构？为什么运行时常会看到不止一次析构打印？ **答案**：匿名临时对象在本条语句结束时析构（打印一次）；但 push_back 会把它按值拷贝进容器，之后容器扩容重排时还会对已有元素做拷贝构造/析构。官方在 L15 演示 Vector 装 Quokka 时就观察到每只 Quokka 被析构多次——那些"额外"的析构都源于按值拷贝与扩容搬移。等到掌握拷贝规则（Rule of Three）后，就能精确预测每一次构造/析构的来龙去脉。

Lecture 9: 指针、数组与动态内存管理（Pointers, Arrays & Dynamic Memory）（对应课程真实讲座 L16–L17）

概述

本讲解决一个关键痛点：局部变量随函数返回而"死亡"，导致返回大容器只能靠昂贵的整份拷贝；我们需要的是一块"比函数活得久"的内存。为此先用两天搭建地基——数组与指针（L16），再引入 `new` / `delete` 在堆上手动管理内存（L17），最后把它们与面向对象结合，亲手实现一个会自动扩容的数组版栈（ArrayBasedStack），为今后实现各种 ADT 备好全部工具。（官方对应：2026 夏季学期 L16, Tuesday, July 21 — Pointers and Arrays；L17, Wednesday, July 22 — Dynamic Memory Management。）

核心概念与算法原理

1. 动机：让对象"长生"

问题定义：`vector<int> createRandoVector(int n)` 在函数里造好一个 vector 再返回，返回的是**拷贝**而不是原物——因为原物在 return 时已死。官方在 L17 开头用两种方法证明：打印地址能看到两个 vector 地址不同；给 Quokka 类加打印析构函数，能看到函数一返回对象就"R.I.P."。返回拷贝既慢（要把每个元素复制一遍），又没法"把大对象本身带出去"。

解决方案预览：用 `new` 在堆上申请内存，对象就活过了函数的寿命；函数只返回一个**指针**（8 字节地址），飞快。而指针正是 L16 的主角。

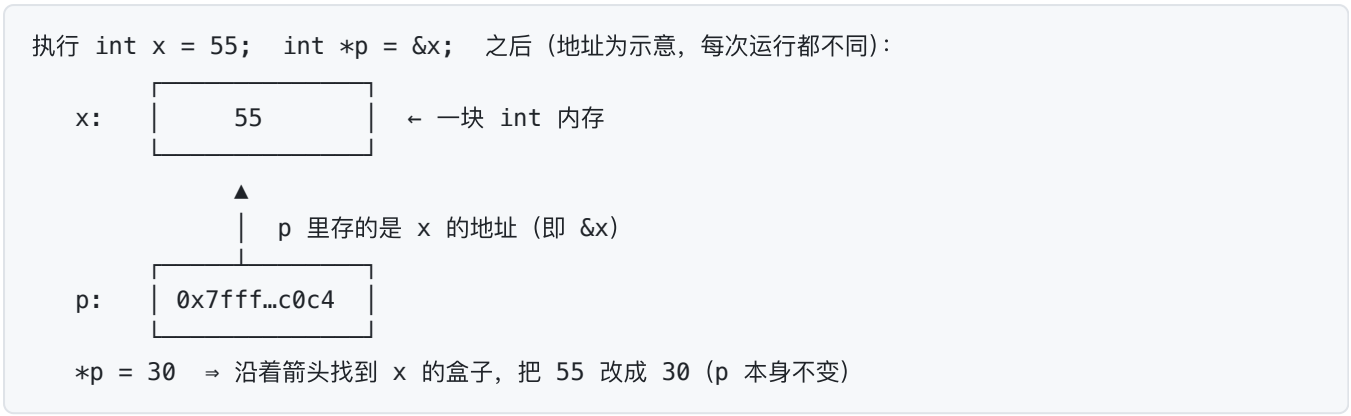
2. 数组：连续内存与它的危险

它是什么：数组是能装多个同类型值的变量，格子（cell）从 0 编号，长度 n 则有格子 0..n−1。元素存储在**一块连续的内存**里，`arr[i]` 就是"从首地址向前跳 i 格"，因此按下标访问是 O(1)。与 vector 相比：数组大小定死、没有 size()/add() 等成员函数、是 C++ 语言内建类型（vector 内部往往就藏着一个数组）。

危险：不初始化数组 → 垃圾值；**越界不检查** → 可能覆盖无关内存、程序崩溃且报错信息毫无帮助。vector 每次访问都会查界并给出明确错误，而裸数组把这层保护撤掉了——责任回到了程序员身上。官方把越界造成的崩溃称作分段错误（segmentation fault）。

3. 内存地址与指针

它是什么：每个变量在内存里都有一个地址（通常用十六进制表示）。指针就是"存地址的变量"——像一个只存一个号码的通讯录。声明指针要先说明它准备存"哪种东西的地址"：`int *p;` 表示 p 能存一个 int 的地址。



& 的上下文双义（超级重点）：在**声明**里 `int &r = x;` 中的 & 制造的是**引用**（别名）；对**已存在的变量** `&x` 中的 & 是**取地址**。二者语法相同、语义完全不同。

- 的上下文双义：*在声明里 `int *p;` 中的 * 声明 p 是指针；在表达式里 `*p = 30` 是解引用*——"去 p 里那个地址看看，操作那里的变量"。多个指针可同时指向同一变量（通讯录副本），任一解引用都能改到同一个 x。声明风格提醒：`int* p, q;` 里只有 p 是指针、q 是普通 int，所以更稳妥的写法是 `int *p;` 一个变量一行或每行都带 `*`。

4. 指针与数组的关系

它是什么：裸数组名（不带方括号）就是首元素的地址：`arr` 等价于 `&arr[0]`。方括号对指针同样生效：让 `int *p = arr;` 之后，`p[i]` 就是 `*(p + i)`，于是可以完全用指针遍历数组。区别在于：指针可以被重新赋值指向别处，而数组名被"焊死"在自己的数组上，不能再指向他处。

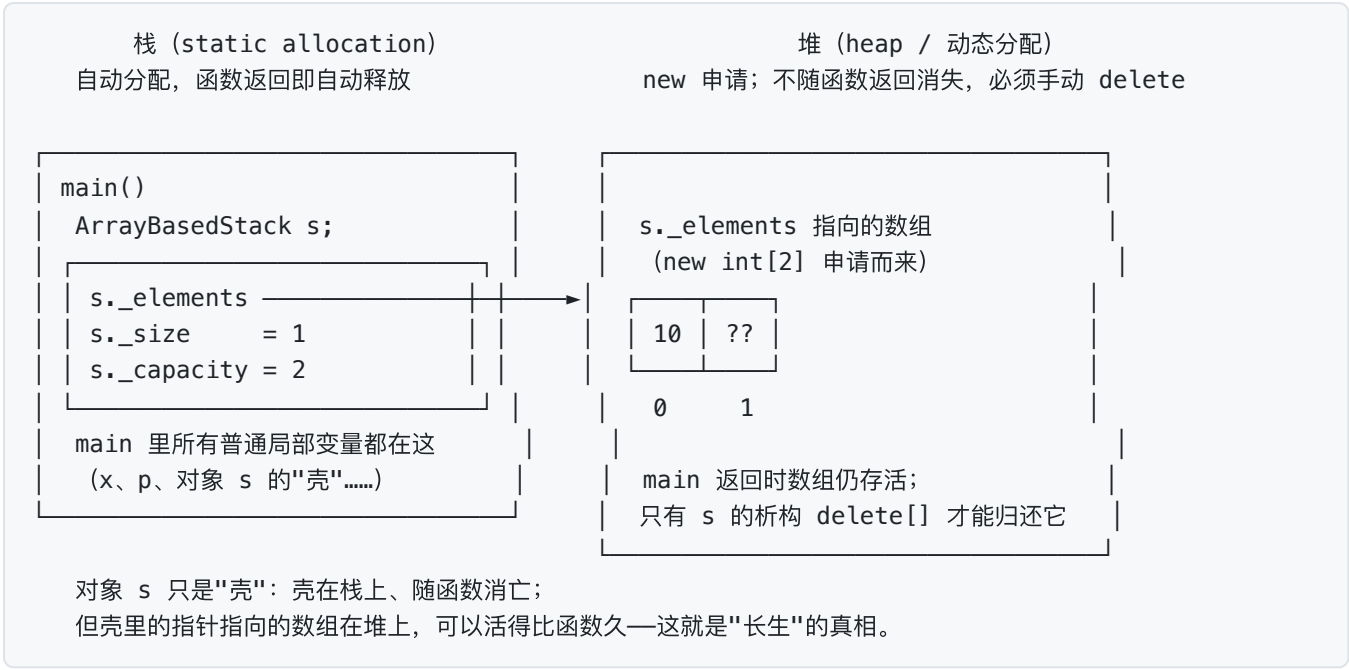
5. nullptr 与悬垂指针

它是什么：暂时不指向任何有用东西的指针应赋 `nullptr`（空指针）。对 nullptr 解引用会段错误，所以使用指针前先判空是好习惯。另一种危险是**悬垂指针（dangling）**：返回"局部变量的地址"，函数一结束那块栈内存已被回收，拿着地址再去访问就是访问死人的遗物，可能崩溃或得到垃圾——这正是官方强调"局部变量随函数消亡"的原因。

6. 动态内存：new / delete 与栈 vs 堆

问题定义：栈空间（static allocation）随函数调用自动分配、自动回收，无法产生"比函数长寿"的对象。C++ 提供 `new`：`new DataType` 在**堆空间**（heap）申请一块内存并返回其地址；`new int[5]` 申请数组；数组长度可以是运行时才知道的变量。堆上对象不随函数返回消失，但**必须由我们手动归还**。

操作步骤：① `int *p = new int[5];` 申请；② 使用；③ 用完 `delete[] p;` 归还。单对象用 `delete p;`，数组必须用 `delete[] p;`（带方括号），二者不可混用。



内存泄漏：堆内存不会自己回家。若丢了最后指向它的指针（没 return、被覆盖）或忘了 delete，这块内存就成了孤儿，程序一直占着它。官方警告：在循环里调用一个"new 了却不归还、也不返回地址"的函数一亿次，内存会被悄悄吃光。规矩：**每个 new 必须配一个 delete**，且要在丢失最后一个指针之前执行。

delete 后的危险：`delete p` 只是归还内存、并没有删除 p 这个变量，p 里仍留着那块地址。此时再 `*p` 解引用就像去翻垃圾桶喝隔夜咖啡——可能还有味儿，也可能已经中毒。官方原话的精神是：**绝不要对已 delete 的地址再解引用**，稳妥做法是随后把 p 置为 nullptr。

指针访问成员：对结构/类对象用 `.` 取成员；对**指向它的指针**用箭头 `->`（即 `(*p).成员` 的简写，后者又丑又长，别用）。

7. const 成员函数

读函数（如 `peek()`、`size()`、`isEmpty()`）不会修改对象状态，应在声明与定义处写成 `int peek() const;`——const 让编译器替我们保证该函数改不了任何成员变量，改就编译失败。

8. 浅拷贝灾难与 Rule of Three（预告）

当一个类的成员是指向堆内存的指针（如 ArrayBasedStack 的 `int *_elements`），默认拷贝构造/赋值做的是**浅拷贝**：只复制指针值，于是两个对象的 `_elements` 指向**同一块**堆数组——任一方修改影响另一方，更糟的是双方析构都会 `delete[]` 同一块内存，造成**双重释放（double free）**崩溃。要安全支持拷贝，必须自己实现三件套（Rule of Three）：**拷贝构造、拷贝赋值、析构**，让每个对象各自 new 自己的数组并逐元素**深拷贝**。本讲先把"为什么必须"讲透，链表那一周的课程会完整实现。

代码示例与实现详解

示例 1：指针基本功综合演示（取址/解引用/指针传参/数组与指针/箭头操作符）

```cpp
#include <iostream>
using namespace std;

// 用指针交换两个变量的值：拿到地址，函数内解引用即可改回 main 里的变量
void swapByPointer(int *a, int *b) {
    int temp = *a;    // *：表达式里 = 解引用，去 a 指向的地址取值
    *a = *b;
    *b = temp;
}

struct Point {        // 小型结构：用来演示 -> 箭头操作符
    int x;
    int y;
};

int main() {
    int x = 55;
    int *p = &x;              // &x = 取地址；声明里的 * = p 是指针
    cout << "x = " << x << ", &x = " << &x << endl;
    cout << "p = " << p << ", *p = " << *p << endl;
    *p = 30;                  // 表达式里的 * = 解引用：把 30 放进 x
    cout << "修改后 x = " << x << endl;        // 30

    int a = 10, b = 20;
    swapByPointer(&a, &b);    // 传地址；不传地址则函数内改不到原变量
    cout << "交换后 a = " << a << ", b = " << b << endl;

    int arr[5] = {7, 11, 13, 17, 19};
    int *q = arr;             // 裸数组名 = 首元素地址（arr == &arr[0]）
    for (int i = 0; i < 5; i++)
        cout << "arr[" << i << "] = " << q[i]
             << " (地址 " << (q + i) << ")" << endl;   // q[i] ≡ *(q + i)

    Point pt{3, 4};
    Point *pp = &pt;
    pp->x = 99;               // -> 等价于 (*pp).x，但简洁得多
    cout << "pt = (" << pt.x << ", " << pt.y << ")" << endl;

    // int *bad = nullptr;  *bad = 5;   ← 解引用空指针 = 段错误！注释掉，别运行
    return 0;
}
```

【代码做什么】 建立 x 与 p 后打印值、地址、指针内容，用 `*p = 30` 间接改写 x；`swapByPointer` 通过两个 int 指针交换 main 里 a、b 的值（对比"按值传参改不到原变量"）；把数组名交给指针 q，用 `q[i]` 与 `q + i` 两种视角遍历数组；再用 `pp->x` 修改结构成员。

【实现机制解说】 – `int *p = &x;` 一行同时出现两种运算符语义：声明上下文里的 `*`（p 是指针）与表达式上下文里的 `&`（取 x 的地址）。`*p = 30` 则是表达式里的 `*`（解引用）。官方特意强调：& 与 * 在不同上下文含义不同，这是初学最绕的地方——判断标准是看它出现在**声明**里还是**对已存在变量**的操作里。 – 数组访问 `q[i]` 本质上就是 `*(q + i)`：方括号先让指针"跳 i 个格子"，再解引用。这解释了为何数组元素地

址连续、按下标访问 O(1)。 – `swapByPointer(&a, &b)` 若忘写两个 &，函数收到的是 a、b 的**拷贝值**，交换只发生在函数内部——传地址是"能改到外面变量"的前提。

示例 2：ArrayBasedStack——用动态数组实现的自动扩容栈（new[]/delete[] 完整配套）

把指针 + 动态内存 + 类三样工具合体：对象 s 是栈上的"壳"，壳里的 `_elements` 指向堆上数组；push 满了就"买新房子、搬东西、拆旧房"。这是官方 L17 的重点示例，也是作业 5 的前奏。

```
#include <iostream>
using namespace std;

class ArrayBasedStack {
public:
    ArrayBasedStack();            // 构造：new 一块初始数组
    ~ArrayBasedStack();           // 析构：delete[] 释放堆数组，防泄漏
    void push(int value);         // 入栈；满了先扩容（容量 ×2 + 1）
    int pop();                    // 出栈并返回栈顶
    int peek() const;             // 只看栈顶（const：保证不改状态）
    int size() const;
    bool isEmpty() const;

private:
    int *_elements;   // 指向堆上数组首元素
    int _size;        // 当前元素个数
    int _capacity;    // 数组容量
};

ArrayBasedStack::ArrayBasedStack() {
    _capacity = 2;                    // 故意给个小容量，便于观察扩容
    _elements = new int[_capacity];
    _size = 0;
    cout << "构造：容量 " << _capacity << endl;
}

ArrayBasedStack::~ArrayBasedStack() {
    delete[] _elements;               // 铁律：每个 new[] 配一个 delete[]
    cout << "析构：堆数组已归还系统" << endl;
}

void ArrayBasedStack::push(int value) {
    if (_size >= _capacity) {                    // 满了？先扩容
        int *newArray = new int[_capacity * 2 + 1];   // 新家（×2+1：容量为 0 时也能变大）
        for (int i = 0; i < _size; i++)
            newArray[i] = _elements[i];          // 逐个搬运旧元素
        delete[] _elements;                      // 拆旧房——必须先释放再换指针，否则旧地址丢失 = 泄
        _elements = newArray;                    // 壳里的指针指向新家
        _capacity = _capacity * 2 + 1;
        cout << "扩容：容量 " << _capacity << endl;
    }
    _elements[_size] = value;        // 数组下标当栈顶用
    _size++;
}

int ArrayBasedStack::pop() {
    if (isEmpty()) { cout << "错误：空栈不能 pop" << endl; return -1; }
    int result = _elements[_size - 1];   // 栈顶 = 下标 _size-1
    _size--;                             // 逻辑删除即可，不必清掉旧值
    return result;
}

int ArrayBasedStack::peek() const {
```

```
    if (isEmpty()) { cout << "错误：空栈不能 peek" << endl; return -1; }
    return _elements[_size - 1];
}
int ArrayBasedStack::size() const { return _size; }
bool ArrayBasedStack::isEmpty() const { return _size == 0; }

int main() {
    ArrayBasedStack s;                // 对象壳在栈上，数组在堆上
    for (int i = 1; i <= 10; i++)
        s.push(i * 10);               // 观察容量：2 → 5 → 11
    while (!s.isEmpty())
        cout << s.pop() << " ";
    cout << endl;
    return 0;                         // s 离开作用域 → 析构自动 delete[]
}
```

运行输出：`构造：容量 2`、两次 `扩容：容量 5 / 容量 11`、倒序弹出 100 到 10、`析构：堆数组已归还系统`。

【代码做什么】 构造时用 `new int[_capacity]` 在堆上申请数组并清零计数；push 先检查 `_size >= _capacity`，满了就申请一块 `×2+1` 的新数组、逐元素搬运、`delete[]` 旧数组、再把 `_elements` 指过去；pop/peek 都只操作下标 `_size-1`；析构函数 `delete[] _elements` 完成"善后"。

【实现机制解说】 – **new/delete 配对铁律**：本例中每块 `new int[...]` 都对应一个 `delete[]`——构造申请、析构归还、扩容时先释放旧数组。顺序也讲究：扩容时必须**先 delete[] 旧数组再改 `_elements`**，若先改指针，旧地址从此丢失，那块内存就泄漏了。 – **扩容公式为什么是 ×2+1**：官方给出的理由——若某结构初始容量恰为 0，×2 永远是 0，永远扩不了容；+1 保证任何初始容量都能增长。翻倍增长还保证了均摊 O(1)（见复杂度表）。 – **`const` 成员函数**：peek/size/isEmpty 声明为 const，编译器强制它们不得修改 `_size`、`_elements` 等成员，是"读操作"的身份证。 – **浅拷贝的双释放灾难**：若有人写 `ArrayBasedStack s2 = s1;`，默认拷贝构造把 s2 的 `_elements` 也指向 s1 那块数组——两个壳共享一份堆内存；main 结束时 s2、s1 各自析构，对同一块内存 `delete[]` 两次，程序直接崩溃。这正是"有堆指针成员的类必须实现拷贝三件套"的原因（示例 2 为聚焦指针主题刻意未实现，见下补充）。

补充——深拷贝三件套（Rule of Three）的修法思路：

```
// 拷贝构造：自己 new 一块新数组，逐元素深拷贝
ArrayBasedStack::ArrayBasedStack(const ArrayBasedStack& other) {
    _capacity = other._capacity; _size = other._size;
    _elements = new int[_capacity];
    for (int i = 0; i < _size; i++) _elements[i] = other._elements[i];
}
// 拷贝赋值：先释放自己的旧资源，再深拷贝对方
ArrayBasedStack& ArrayBasedStack::operator=(const ArrayBasedStack& other) {
    if (this == &other) return *this;        // 防自赋值
    delete[] _elements;                       // 归还旧数组
    _capacity = other._capacity; _size = other._size;
    _elements = new int[_capacity];
    for (int i = 0; i < _size; i++) _elements[i] = other._elements[i];
    return *this;
}
// 再配合已有的析构 delete[]，三件套齐了：s2 = s1 后各有一块数组，互不影响、各删各的。
```

复杂度分析

操作	均摊/平均	最坏	原因
按下标访问数组元素 arr[i]	O(1)	O(1)	基址 + 偏移直接定位，无需查找
取地址 &x / 解引用 *p	O(1)	O(1)	拷贝一个地址 / 一次间接寻址
push（动态数组栈）	O(1)	O(n)	平时直接写栈顶 O(1)；扩容那次要搬运全部旧元素
pop / peek / size / isEmpty	O(1)	O(1)	只碰栈顶位置与两个计数器
空间	O(n)	O(n)	容量按 2 倍+1 增长，与元素个数同阶
new / delete 单次申请	O(1)（均摊）	视分配器	内存管理由运行库负责，调用本身是常数级操作

要点：扩容虽偶发 O(n)，但容量翻倍使"搬运成本"被摊薄到每次 push 上，故 push 整体均摊 O(1)——这正是 vector 内部 add 高效的原因。

关键要点

- 指针 = 存地址的变量；& 与 * 在"声明"与"表达式"里含义不同：声明里 & 造引用、 *造指针，表达式里 & 取地址*、 解引用。
- 数组是连续内存、下标从 0 开始、越界不检查；裸数组名就是首元素地址，`arr[i]` 即 `*(arr + i)`。
- 栈自动管理、随函数消亡；new 在堆上申请的内存活得比函数久，但必须手动归还——**每个 new 配一个 delete（数组用 delete[]）**。
- delete 只归还内存不删除指针，绝不要对已 delete 的地址再解引用，之后顺手置 nullptr；也不要解引用 nullptr。
- 类持有堆指针成员时，默认浅拷贝会导致双释放；需要拷贝构造 + 拷贝赋值 + 析构三件套（Rule of Three）做深拷贝。

常见陷阱与注意事项

- **未初始化数组直接读**：格子是垃圾值；要么声明时初始化 `int arr[5] = {0};`，要么先赋值再读。
- **数组越界**：C++ 不查界，越界可能悄悄改写别的内存或段错误；自己盯紧 0..size−1 边界。
- **忘 delete / 丢指针**：泄漏；在丢失最后一个指针前完成 delete，或用析构函数统一善后。
- **delete 后仍解引用**：那块内存可能已被别人占用，行为未定义；delete 后置 nullptr 再判空使用。
- **单对象与数组混淆**：`new int` 配 `delete`，`new int[n]` 必须配 `delete[]`，混用是未定义行为。
- **声明 `int* p, q;` 的错觉**：只有 p 是指针；每行写清 `int *p; int *q;` 更不易错。
- **函数想改原变量却忘传地址/引用**：按值传参只改副本；要么传引用 `int &a`，要么传地址 `int *a`。
- **对 nullptr 解引用 / 对悬垂指针解引用**：前者段错误，后者是"访问死人的遗物"；返回局部变量的地址前先想想它是否已死。

- **浅拷贝双释放**：把含堆指针的类按值拷贝（传参、赋值、塞进容器）前，先确认该类实现了拷贝三件套或禁止拷贝。
- **扩容顺序写反**：先换指针后释放旧数组 = 旧地址丢失 = 泄漏；务必"先 delete[] 旧的，再让 _elements 指向新的"。

思考题（带答案）

问题 1：写一个函数 `bool samePlace(int *p, int *q)` 判断两个指针是否指向同一块内存，再写 `bool sameValue(int *p, int *q)` 判断指向的值是否相等（参考官方练习题）——两者差别在哪？ **答案**：`samePlace` 直接比较指针值：`return p == q;`（比地址）。`sameValue` 必须先解引用再比：`return *p == *q;`。两个不同地址里可以存相同的值（如 a=11、b=11），所以 `samePlace(&a,&b)` 为 false 而 `sameValue(&a,&b)` 为 true——比较"在哪里"与比较"是什么"是两回事。

问题 2：`int *p = new int[100];` 之后若直接执行 `p = new int[50];`（忘了先释放），会发生什么？若改成先 `delete[] p;` 再赋新值呢？ **答案**：第一种写法把旧数组唯一的地址覆盖了，100 个 int 的堆内存永远无法归还——内存泄漏（官方把这种"完全失去指针记录"的块叫 orphaned memory）。第二种写法先 `delete[] p;` 把旧数组还给系统，再申请新数组，新旧交替毫无泄漏——这就是"每个 new 都要配 delete，且在丢失指针之前"的含义。

问题 3：ArrayBasedStack 的扩容为什么用 `new int[_capacity * 2 + 1]` 而不是 `* 2`？如果把 `int *_elements` 换成 `std::vector<int>`，哪些问题会自动消失？ **答案**：若初始容量为 0，`* 2` 永远得 0，永远无法扩容，+1 保证容量严格增长（官方在练习里给出的理由）。换用 vector 后，扩容、拷贝、释放都由 vector 自己管理，双释放与泄漏风险消失——但这正是我们看不见"幕后发生了什么"的原因，学本讲就是要掀开 vector 的引擎盖看个明白。

Lecture 10: 优先队列与二叉堆（Priority Queues & Binary Heaps）（对应课程真实讲座 L18）

概述

本讲引入第一棵树形结构——二叉最小堆（minheap），并借此实现"优先队列"这个 ADT：元素按**优先级**出队，而非先进先出。核心问题是如何让"插入"与"取出最小"两个操作都快；答案是利用一棵"完全二叉树 + 父不大于子"的堆，藏在数组里，用上滤/下滤两招在 O(log n) 内完成维护。堆还是堆排序与后续 Dijkstra、Huffman 等算法的心脏。（官方对应：2026 夏季学期 L18，Thursday, July 23 — Priority Queues and Binary Heaps，配有一份手写讲义 minheaps-written-notes.pdf；其中 heapify 与 maxheap 部分官方标注为选学补充。）

核心概念与算法原理

1. 树术语热身（为二叉树铺路）

它是什么：树由**节点**与连接节点的**边**组成；每个节点最多一个**父**、可有多个**子**；没有父的是**根**，没有子的是**叶**；父与子的关系构成**子树**；从根到最深叶的边数叫**高度**。若每个节点最多两个子，叫**二叉树**（区分左孩子/

右孩子）。若除最后一层外每层都填满，且最后一层节点全部**靠左无空洞**，叫**完全二叉树（complete binary tree）**——它是堆的地基。

2. 二叉最小堆：两条性质

问题定义：想要一种结构同时支持"插入任意值"与"取出最小值"都很快。普通有序数组插入慢，无序数组找最小慢，链表两头不讨好——堆用**部分有序**换来了两全。

它是什么：最小堆 = 完全二叉树 + 两条性质：①**结构性质**：完全（逐层从左到右填满）；②**堆序性质**：任意节点的值 ⩽ 其两个孩子（于是也 ⩽ 整棵子树）。推论：**根永远是全局最小**，任何子树的根也是该子树的最小。最大堆（maxheap）只是把 ⩽ 换成 ⩾，根是最大（官方把 maxheap 列为补充选学，std::priority_queue 默认就是最大堆）。

3. 数组表示：把树"压平"

为什么能压：因为完全二叉树"从左到右无空洞"，按层序遍历放进数组恰好占满下标 0..n−1，没有浪费的洞（官方 L18 特别表扬了这一点）。

```
下标:    0     1     2     3     4     5     6
       ┌─────┬─────┬─────┬─────┬─────┬─────┬─────┐
数组   │ 3   │ 5   │ 8   │ 12  │ 7   │ 20  │ 15  │     ← 同一份数据折成树看:
       └─────┴─────┴─────┴─────┴─────┴─────┴─────┘

                  3 (0)              ← 根 = 最小值，永远在下标 0
                /       \
           5 (1)           8 (2)     ← 下标 1、2
          /    \          /    \
      12 (3)   7 (4)   20 (5)  15 (6) ← 叶子/内部节点（括号里是数组下标）

下标公式（数组从 0 开始）:
  节点 i 的左孩子 = 2i + 1          节点 i 的右孩子 = 2i + 2
  节点 i 的父节点 = (i − 1) / 2      （整数除法）
例: i=1（值 5）的孩子是 3（12）与 4（7）; i=4（值 7）的父是 (4−1)/2 = 1（值 5）
```

4. insert：放到末尾，再上滤 percolateUp

操作步骤：① 把新值 push 到数组末尾（树的最左下空位，保持"完全"）；② 与父比较：若比父小就交换上移一层，重复直到不小于父或到达根。官方也叫它 sift up / bubble up（上冒）。

```
insert(2) 进 [3,5,8,12,7,20,15]:
 ① 末尾追加 → [3, 5, 8, 12, 7, 20, 15, 2]   新值下标 7，父 (7−1)/2=3 → 值 12
 ② 2 < 父12 → 交换 → [3, 5, 8, 2, 7, 20, 15, 12]  新下标 3，父 = 1 → 值 5
 ③ 2 < 父5  → 交换 → [3, 2, 8, 5, 7, 20, 15, 12]  新下标 1，父 = 0 → 值 3
 ④ 2 < 父3  → 交换 → [2, 3, 8, 5, 7, 20, 15, 12]  到达根，停
 上滤路径: 12 → 5 → 3，即沿"父链"向上冒；最多走整棵树的高度层。
```

5. extractMin：取根，末元素补位，再下滤 percolateDown

操作步骤：① 记下根的值（就是最小）；② 把**最后一个元素**挪到根（保持"完全"）；③ 删掉末尾；④ 与两个孩子中**较小者**比较，若比它大就换下去，重复到不大于任何孩子或成为叶。必须与"较小的孩子"换：若与较大的换，另一个更小的孩子会违反堆序。

```
extractMin()（接上例，堆为 [2,3,8,5,7,20,15,12]）：
 ① 返回值 = 2；末元素 12 补到根：→ [12, 3, 8, 5, 7, 20, 15]
 ② 12 与孩子 3、8 比：换较小的 3 → [3, 12, 8, 5, 7, 20, 15]
 ③ 12 与孩子 5、7 比：换较小的 5 → [3, 5, 8, 12, 7, 20, 15]
 ④ 12 的下标 3 已无孩子 → 停。返回的 2 即被删除的最小值。
```

运行时间：最坏 O(log n)（一路沉到底）；**最好 O(1)**——补位元素到位即停（如所有值相等，或它已经不大于两个孩子；官方强调堆再大也可能 O(1)）。注意：堆**不支持删除任意值**——找任意值最坏 O(n)，且删除会破坏"完全"结构难以恢复；需要任意删除时应换别的数据结构。

6. 优先队列（priority queue）扩展

它是什么：把"优先级 + 数据"捆绑成一个节点（如打印任务 = 页数 + 内容），按优先级排堆，数据跟着优先级走。官方举了共享打印机的例子：以页数为优先级，小任务先打印，没人需要等一个要打 500 页的大任务。操作命名上官方给出三组同义词：插入 = enqueue/insert/add；取最小 = dequeue/delete/deleteMin；只看最小 = peek/findMin/getMin。**peek 恒为 O(1)**：直接看根。

7. 应用：堆排序（heapsort）

思路：① 把 n 个元素逐个 insert 进最小堆：O(n log n)；② 反复 extractMin 并把结果依序放入数组：O(n log n)。第 ② 步出来的值天然升序，总时间 O(n log n)——与归并排序同级，且无需额外大数组做合并（本实现用 vector 存储即 O(n) 空间）。官方把 n 次插入的总代价写成逐项求和：第 k 次插入最坏 O(log k)，总和 log1+log2+…+logn = log(n!) = O(n log n)（斯特林近似）——这个"小心逐项求和"的习惯很重要，因为有些算法（如下面 heapify）按直觉估会错。

8. 补充：heapify 一次建堆为什么是 O(n)（官方列为选学，但值得懂）

把任意数组就地变成堆，做法是从**最后一个非叶节点**（下标 n/2−1）开始，自底向上逐个 percolateDown，直到根。粗看"n/2 次 × O(log n)"似乎该是 O(n log n)，但树底部的节点下沉距离很短：高度 1 的节点最多沉 1 步、高度 2 的沉 2 步……把每层工作量加起来是几何级数，收敛为 O(n)。教训：对"结构大小在变化"的操作做估算时，逐项求和往往比"单次最坏 × 次数"更准确（官方 L18 专门花篇幅讲了这一点）。

9. 高度推导

完全二叉树第 k 层满员时有 2^k 个节点；含 n 个节点的堆满足 2^h ⩽ n < 2^(h+1)，故高度 h = $\lfloor \log_2 n \rfloor$——这就是所有堆操作最坏 O(log n) 的来源。

代码示例与实现详解

示例 1：手写 MinHeap 类（std::vector 存储 + insert 上滤 + extractMin 下滤 + heapify 建堆）

```cpp
#include <iostream>
#include <vector>
#include <algorithm>   // std::swap
#include <stdexcept>   // std::runtime_error
using namespace std;

class MinHeap {
public:
    MinHeap() = default;                            // 空堆
    explicit MinHeap(const vector<int>& values);  // 由任意数组一次建堆（heapify）
    void insert(int value);                         // 插入：末尾 + 上滤
    int extractMin();                               // 取出并删除最小值
    int peek() const { return _data[0]; }         // 只看根（最小）
    int size() const { return static_cast<int>(_data.size()); }
    bool isEmpty() const { return _data.empty(); }
    const vector<int>& data() const { return _data; }  // 教学用：观察内部数组

private:
    vector<int> _data;
    void percolateUp(int index);    // 上滤（bubble up）
    void percolateDown(int index);  // 下滤（bubble down）
};

// heapify：从最后一个非叶节点（n/2-1）开始自底向下滤，总代价 O(n)
MinHeap::MinHeap(const vector<int>& values) : _data(values) {
    for (int i = static_cast<int>(_data.size()) / 2 - 1; i >= 0; i--)
        percolateDown(i);
}

void MinHeap::insert(int value) {
    _data.push_back(value);                           // ① 放到末尾，保持"完全"
    percolateUp(static_cast<int>(_data.size()) - 1); // ② 上滤恢复堆序
}

void MinHeap::percolateUp(int index) {
    while (index > 0) {
        int parent = (index - 1) / 2;                // 父节点公式
        if (_data[index] >= _data[parent]) break;    // 不比父小 → 到位
        swap(_data[index], _data[parent]);           // 比父小 → 与父交换
        index = parent;                              // 上移一层继续
    }
}

int MinHeap::extractMin() {
    if (isEmpty()) throw runtime_error("空堆不可 extractMin");
    int minValue = _data[0];        // ① 最小就是根
    _data[0] = _data.back();        // ② 末元素补到根，保持"完全"
    _data.pop_back();               // ③ 删掉末尾
    if (!_data.empty()) percolateDown(0);  // ④ 下滤恢复堆序
    return minValue;
}

void MinHeap::percolateDown(int index) {
```

```
    int n = static_cast<int>(_data.size());
    while (true) {
        int left = 2 * index + 1, right = 2 * index + 2;
        int smallest = index;                       // 在"我、左、右"中找最小
        if (left < n && _data[left] < _data[smallest])  smallest = left;
        if (right < n && _data[right] < _data[smallest]) smallest = right;
        if (smallest == index) break;               // 我已是三者最小 → 到位
        swap(_data[index], _data[smallest]);        // 与较小孩子交换
        index = smallest;
    }
}

int main() {
    MinHeap h;
    for (int v : {42, 17, 33, 5, 90, 1, 55, 8, 21, 3})
        h.insert(v);
    cout << "peek = " << h.peek() << endl;          // 1

    cout << "extractMin 依次取出（天然升序 = 堆排序的雏形）: " << endl;
    while (!h.isEmpty()) cout << h.extractMin() << " ";
    cout << endl;

    MinHeap built(vector<int>{9, 4, 7, 1, 3, 6});   // heapify 一次建堆
    cout << "heapify 后内部数组: ";
    for (int v : built.data()) cout << v << " ";    // 1 3 6 4 9 7
    cout << endl;
    cout << "再全部取出: ";
    while (!built.isEmpty()) cout << built.extractMin() << " ";  // 1 3 4 6 7 9
    cout << endl;
    return 0;
}
```

【代码做什么】 main 先逐个 insert 十个数，peek 确认根最小，然后反复 extractMin 得到升序序列——这正是堆排序的两步曲。随后用 heapify 构造器把任意数组 {9,4,7,1,3,6} 一次变成合法最小堆：先打印内部数组可见数组形态变成 {1,3,6,4,9,7}（读者可对照下滤过程自证），再依次取出验证升序。

【实现机制解说】 – **insert 的上滤过程**：push_back 保证"完全"（数组末尾即最左下空位）；percolateUp 每轮用 (index−1)/2 找父，只与父比——因为堆序性质只需保证"父 ≤ 子"，新值一路上行即可让整条路径恢复有序。终止条件写成 `>=` 即相等时也停（等值堆一切操作都 O(1)）。 – **extractMin 的下滤过程**：直接删根会在树顶留"洞"破坏完全性，所以先把末尾元素搬到根再下滤。percolateDown 每轮在 index、left、right 三者中选**最小**者，只有当"我不是最小"才交换——这保证换完后 `父 ≤ 两个孩子` 同时成立；若误与较大孩子交换，另一个孩子会变成"父比子大"而再次违规。 – **边界检查是生命线**：`left < n && right < n` 缺一不可，越界读数组是未定义行为（对照上一讲"数组越界不检查"的教训，这里必须自己守边界）。 – **heapify 的正确姿势**：只能自底向上 percolateDown（从 n/2−1 到 0）；若自顶向下逐个 insert 则是 O(n log n)。原因是"把子树弄成堆"要先保证子树已经是堆，底部的小堆先成立，上层才能一次下沉到位。

示例 2：堆排序函数与 std::priority_queue 用法小示例

官方堆排序配方就是"全部 insert + 依次 extractMin"；示例 1 的弹出循环已是其雏形。下面给出标准库版（自包含可编译）：用 `std::priority_queue` 的最小堆形态排序，并演示默认最大堆与最小堆的用法差异。

```cpp
#include <iostream>
#include <vector>
#include <queue>
#include <functional>   // std::greater
using namespace std;

// 标准库最小堆版堆排序：全部 push，再依次 top+pop，出来即升序
vector<int> heapSort(const vector<int>& values) {
    priority_queue<int, vector<int>, greater<int>> pq;  // 最小堆
    for (int v : values) pq.push(v);
    vector<int> sorted;
    while (!pq.empty()) {
        sorted.push_back(pq.top());   // 当前最小
        pq.pop();
    }
    return sorted;
}

int main() {
    vector<int> a = {42, 17, 33, 5, 90, 1, 55, 8};
    vector<int> sorted = heapSort(a);
    cout << "堆排序结果：";
    for (int v : sorted) cout << v << " ";   // 1 5 8 17 33 42 55 90
    cout << endl;

    // 用法对比：priority_queue 默认是大根堆（top 最大）；
    // 模板参数 <类型，底层容器，比较器> 换成 greater<int> 即最小堆
    priority_queue<int> maxq;
    priority_queue<int, vector<int>, greater<int>> minq;
    for (int v : a) { maxq.push(v); minq.push(v); }
    cout << "大根堆 top = " << maxq.top() << endl;   // 90
    cout << "最小堆 top = " << minq.top() << endl;   // 1
    maxq.pop();
    cout << "大根堆 pop 后 top = " << maxq.top() << endl;  // 55
    return 0;
}
```

【代码做什么】 heapSort 把元素全部 push 进最小堆，再循环 top + pop 收集成升序数组。main 演示 priority_queue 的默认形态（最大堆）与换比较器后的最小堆：同样数据，两个堆的 top 分别给出最大 90 与最小 1。

【实现机制解说】 – priority_queue 只暴露 top/push/pop 三个口，**没有迭代器、不支持任意删除**——这就是"ADT 边界"的体现：底层是堆还是别的结构对调用方透明，接口只承诺"能拿到最大/最小"。 – 比较器方向容易绕晕：`greater<int>` 让 top 返回**最小**值。原因：priority_queue 约定"比较器返回 true 表示 a 排在 b 后面/优先级更低"，于是 greater 语义下最小值排最前。官方用最小堆讲原理，而标准库默认给你最大堆——写代码前先想清楚自己到底要最大还是最小。 – 手写 MinHeap 与标准库本质同一算法，差异只在：vector 是"存储 + 堆化"自己管，priority_queue 把"谁是最小/最大"的比较规则参数化了。

复杂度分析

操作	最好	最坏	原因简述
insert（enqueue/上滤）	O(1)	O(log n)	新值够大一步不升 / 从叶一路升到根
extractMin（dequeue/下滤）	O(1)	O(log n)	补位值到位即停（如全等）/ 一路沉到底
peek（findMin/getMin）	O(1)	O(1)	直接返回根 _data[0]
heapify / buildHeap	O(n)	O(n)	从最后非叶节点自底向下滤，各层工作量几何收敛
连续 n 次 insert	O(n log n)	O(n log n)	逐项求和 log1+…+logn = log(n!) = O(n log n)
heapsort	O(n log n)	O(n log n)	建堆 + 取 n 次最小，各 O(n log n)
空间	—	O(n)	数组/vector 连续存储（堆排序可原地，本实现 O(n)）

要点：堆的高度 $h = \lfloor \log_2 n \rfloor$ 决定一切最坏上界；"重复操作的总代价"要用逐项求和，别用"单次最坏 × 次数"拍脑袋（heapify 就是反例）。

关键要点

- 最小堆 = 完全二叉树 + 堆序（父 ≤ 子），根恒为最小；完全性保证了紧凑的数组表示与三条下标公式（左 2i+1、右 2i+2、父 (i−1)/2）。
- insert 先放末尾再上滤；extractMin 先取根、末元素补位再下滤——两个"滤"都沿树高走，最坏 O(log n)。
- 下滤必须与**较小**孩子交换；数组访问前务必检查左右孩子下标是否越界。
- 优先队列 = 最小堆 + 优先级数据捆绑：peek O(1)，插入/取最小 O(log n)，不支持任意值删除。
- 堆排序 = n 次 insert + n 次 extractMin，O(n log n)；heapify 自底向上一次建堆只需 O(n)。

常见陷阱与注意事项

- **下标公式记混**：从 0 开始时父是 (i−1)/2，孩子是 2i+1/2i+2；把父写成 i/2 或把根当下标 1 都会错位（若坚持下标 1 起，公式会变，务必全程序统一）。
- **下滤时与较大的孩子交换**：会让另一个孩子违反堆序；永远选两个孩子中较小者。
- **越界读孩子**：percolateDown 里不判 `left < n` / `right < n` 就会读数组尾巴外面——行为未定义（对照数组越界的教训）。
- **extractMin 忘了"末元素补根"**：直接删根留下空洞，完全性被破坏，之后下标公式全部失效。
- **建堆用错方向**：逐个 insert 是 O(n log n)；想 O(n) 必须自底向上对非叶节点 percolateDown。
- **手写时忘维护 size**：push_back 与 pop_back 之外若自管计数器，加一减一都要与 vector 同步。
- **对空堆 peek/extractMin**：先 isEmpty() 再动手；标准库则是先判 empty 再 top/pop。
- **priority_queue 默认是大根堆**：想要最小堆必须显式传 `greater<int>`；别想当然以为 top 是最小。
- **把"最好 O(1)"当"总是 O(1)"**：插入最坏仍是 O(log n)，均摊分析要逐项求和而不是乘法估算。

思考题（带答案）

问题 1：往一个高度为 4 的最小堆插入 99，什么情况下是最好情形？什么情况下 99 会一路升到根（最坏情形）？根位置的值最小，为什么 99 还能"升"到根？ **答案**：若 99 的父节点及祖先都 ⩽ 99（例如整棵堆的数值都比 99 小），它一步不升，O(1)——这就是最好情形。注意：堆序只约束"父 ⩽ 子"，**并不要求树从上到下整体有序**，所以完全可能某条"父链"上的值（如 100、150、200）都比 99 大，99 就会一路与父交换升到根，触发最坏 O(log n)。把 99 放进"父链数值很大"的堆即最坏用例。

问题 2：用数组表示堆时，为什么 insert 的新元素总是放"末尾"，extractMin 也总是从"末尾"取补位元素？这两个位置的下标分别怎么算？ **答案**：都是为了保住"完全性"——树只能从左到右逐层填满。新元素永远放在最左下第一个空位，即数组当前末尾下标 n（追加后为 n）；补位元素取当前最后一个元素下标 n−1（pop 前）。于是任何时刻数组 0..n−1 都紧凑无洞，下标公式恒成立。

问题 3：heapify 对数组 {9,4,7,1,3,6} 建堆为何是 O(n) 而不是 O(n log n)？请给出你的估算直觉。 **答案**：自底向上从最后一个非叶节点（n/2−1=2，值 7）开始下滤，最终得到 {1,3,6,4,9,7}。耗时关键在：接近叶子的节点（数量多）下沉距离短（1 步、2 步），只有靠近根的少数节点才可能沉满 log n 步；把"节点数 × 各自下沉距离"按层求和是一个收敛的几何级数，总量 O(n)。若反过来自顶向下逐个 insert，每个新节点都可能上滤满高度，才是 O(n log n)——"单次最坏 × 次数"的直觉在这里恰好失效。

Lecture 11: 链表（单向/双向、尾指针）（Linked Lists）（对应课程真实讲座 L19–L20）

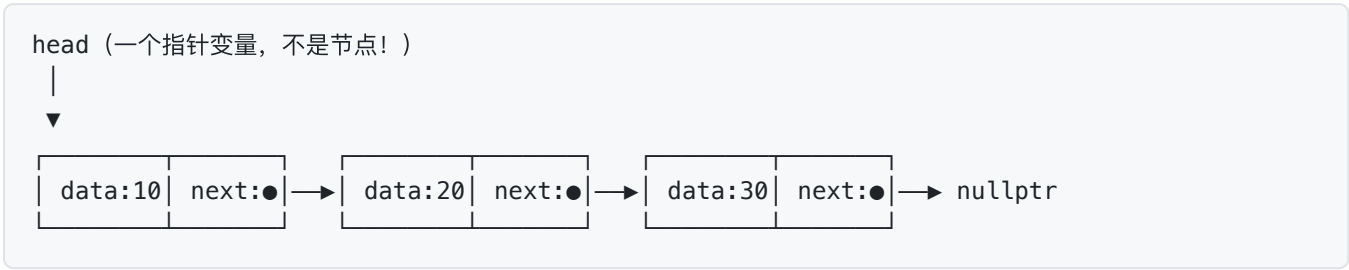
概述

本讲引入课程中第一种"基于节点的链式结构"——链表：元素不再挤在一整块连续内存里，而是散落在堆上，彼此用指针（存着下一个节点的地址）串成一条"链"。核心问题是数组在任意位置插入/删除时的 O(n) 搬移代价与"预分配容量"的浪费；链表用指针换来 O(1) 的头尾插入与按需增长，代价是失去 O(1) 的随机访问。我们还会讨论维护"尾指针"带来的运行时收益与维护成本、双向链表（每个节点再多一个 prev 指针），以及如何用链表实现栈与队列。 对应官方讲座：L19（Monday, July 27 — Introduction to Linked Lists）与 L20（Tuesday, July 28 — More Linked Lists）；官方配套作业为 Assignment 6（Listy Things）。官方在第 19 讲开场就用"前方有龙（Here Be Dragons）"警告：链表是本季最需要指针操控与动态内存功底、也最容易让人受挫的主题之一，官方给的学习建议是——尽早开工、多画内存图、多去答疑。

核心概念与算法原理

1. 数组的两块短板：为什么需要链表。 数组的单元（cell）在内存里连续摆放，所以 `arr[3]` 可以 O(1) 直达——C++ 只做一次"基地址 + 3 × 单元大小"的算术。但连续摆放带来两个麻烦：其一，容量要事先估计，估计大了浪费空间，估计小了就得扩容（新建更大的数组、把旧元素整体拷过去、释放旧数组，是昂贵的 O(n) 操作，vector 扩容就是这套流程）；其二，在数组头部或中部插入/删除，需要把一串元素逐个"挪窝"，同样是 O(n)。链表的思路是：不再承诺连续摆放，每个节点自带"下一个节点在哪"的地址，节点需要几个就 new 几个，头插永远只是两步指针操作。

2. 链表的基本解剖。 把"数据 + 指向下一个节点的指针"打包成一个 struct，就得到一个节点（node）；一串节点由 next 指针互相咬合就是链表：

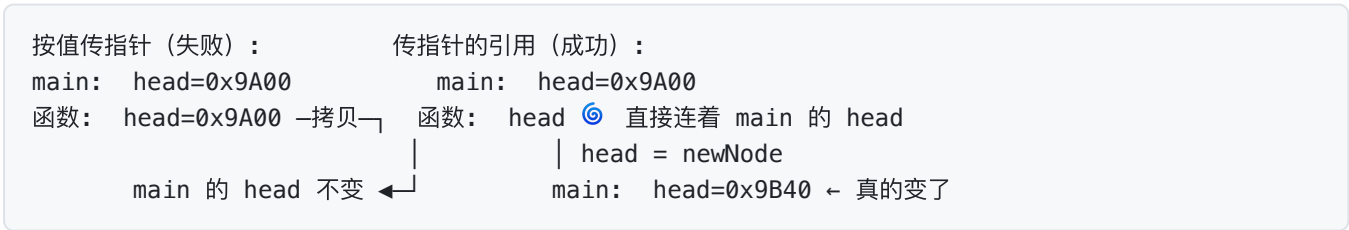


图中每个 ● 里存的是"下一个节点的内存地址"。注意两点：节点在内存里其实散落各处（上面只是画得整齐），地址才是把链条粘起来的胶水；`head` 是一个独立存在的指针变量，它只保存第一个节点的地址，并不是节点本身。遍历链表就像"顺着面包屑一路找下去"（官方原话的比喻是跟着记忆里的面包屑走）。

3. 头指针、nullptr 与箭头运算符。 链表的唯一（通常也是主要）入口是头指针 head。`nullptr` 专门表示"此指针当前不指向任何有用的东西"，我们用它标记链尾，也用它给"还没准备好指向哪里"的指针做初始化——在解引用一个指针之前先判空，叫"防御性编程"。解引用空指针会触发段错误（segmentation fault）直接崩溃。若有一个指向 struct 的指针 p，访问其字段用箭头 `p->data`；它等价于 `(*p).data`，但后者既啰嗦又不规范，课程明确要求别那么写。用 `->` 跟在 `head->next->next` 这种"长链"后面访问深层字段，是链表代码的日常。

4. 遍历：跟着 current 走。 打印链表的经典写法是拿一个临时指针 current 从 head 出发，循环体内打印 `current->data`，然后执行 `current = current->next`（把指针"往前走一步"），直到 current 变成 nullptr。注意两种循环条件的差别：`while (current != nullptr)` 是在 current 自己"掉下链尾"后停止，适合"访问每个节点"；而尾插时我们想停在*最后一个节点*上（它的 next 还是 nullptr），所以条件要写成 `while (current->next != nullptr)`。printList 用值传递 `Node *head` 就够了——因为函数只是让局部拷贝往前走，并没有改动链表本身；改 head 里保存的地址与改节点内容完全是两回事。

5. 修改头指针必须传引用（Node *&head）。 这是官方在笔记里用 🤯 强调的最关键预备知识。如果函数要"改变调用方那个指针变量里存的值"（例如头插要 head 指向新节点），按值传指针是不行的：形参只是主调方指针的一份拷贝，函数里 `head = newNode` 改的是拷贝，回到 main() 后原指针纹丝不动，插入等于"丢失"。要修改原指针，必须传"指针的引用"，语法是 `Node *&head`——可以把它想成一个直通主调方变量的"传送门（🌀）"：函数里对这个引用赋值，就是直接改 main() 里那个 head 变量。



反之，如果函数只是"读"链表（打印、数长度、查找），传普通 `Node *head` 即可；如果函数要"造一个新节点并把它交还调用方"，则用返回值 `Node *createNode(...)`。判断标准就一条：**这个函数会不会改主调方那个指针变量的值？会 → 传引用。**

6. 头插 headInsert：先连新节点、后改头指针。 头插只需三步，顺序至关重要：

```
初始:    head ─► [20]─►[30]─►nullptr
① 造新节点 n（new 出来的 10 号节点，next 暂为乱值→先置 nullptr）
② n->next = head          先让新节点指向旧头 —— 链条先接上，旧链表没丢!
      n=[10]─►[20]─►[30]─►nullptr
③ head = n                最后才把 head 拨到新节点上
      head─►[10]─►[20]─►[30]─►nullptr   ✔ 头插完成，O(1)
```

若把 ②③ 颠倒成 `head = n; n->next = head;`，第二步 `n->next` 指向的其实是 n 自己——旧链表 `[20] [30]` 从此没有任何指针可达，白白泄漏在堆里。所以口诀是：**先让新节点指到旧头，再把头指针拨到新节点**（先连后改）。

7. 头删 removeFront：先移头、再删旧头。 删除节点必须回收它占的堆内存（delete），但顺序同样讲究：

```
head─►[10]─►[20]─►[30]─►nullptr
① 判空：空表直接报错，绝不解引用 nullptr
② retval = head->data             先保存要返回的值
③ victim = head                   记住旧头的地址（待会儿要 delete 它）
④ head = head->next               先把 head 拨到下一个节点
⑤ delete victim                   最后才释放旧头节点
   head─►[20]─►[30]─►nullptr ✔
```

最经典的反面教材是 `delete head; head = head->next;`：先 delete 再解引用 `head->next`，是在访问"程序已不再拥有"的内存，属于未定义行为，可能崩溃也可能悄悄出错。铁律：**delete 之后再也不要碰那个地址**，所以要把需要的指针先保存下来。

8. 尾插：O(n) 走路 vs O(1) 尾指针。 没有额外信息时，尾插只能从 head 出发一路走到最后一个节点再挂新节点，O(n)。优化思路是再维护一个 tail 指针，让它始终指向最后一个节点：

```
tailInsert(head, 25) 无尾指针:                tailInsert(head, tail, 25) 有尾指针:
head─►[10]─►[20]─►nullptr                     head─►[10]─►[20]─►nullptr
      cur=10→cur=20→停! cur->next=新节点            tail 已在 20: tail->next=新节点
      O(n)                                        tail = tail->next   O(1) ✨
```

代价只有两样：一个额外的 8 字节指针变量，以及"每个可能改变链头/链尾的操作都要顺手维护 tail"的编码复杂度。空表是特殊情形：tail 为 nullptr 时，新节点同时成为 head 和 tail，必须两个指针一起更新。

9. 尾指针让头插/头删也变复杂了。 维护尾指针之后，头插/头删多出两个必须处理的边界：往空表头插，新节点既是头又是尾，要 `head = tail = newNode`；头删把唯一一个节点删光后链表空了，要 `head = tail = nullptr`（如果忘了把 tail 置空，tail 就成了指向已释放节点的悬垂指针）。官方还点破一个"聪明方案"为何不成立：有人提议再维护一个"倒数第二个"指针来实现 O(1) 尾删——可删完尾之后倒数第二个指针自己也得后退一格，除非给每个位置都准备一个指针，否则退不回去。**尾删在单向链表上永远是 O(n)**，无论有没有尾指针，因为单向的 next 没法"倒车"：

```
          插入          删除
头部      O(1)          O(1)
尾部      O(1)（有尾指针）        O(1)（尾指针 + 双向链表）
          否则 O(n)               否则 O(n)
```

10. 双向链表：每节点多一个 prev，多两步操作。 给每个节点补一个指向前驱的 prev 指针，就能 O(1) 后退一格，尾删随之变成 O(1)：

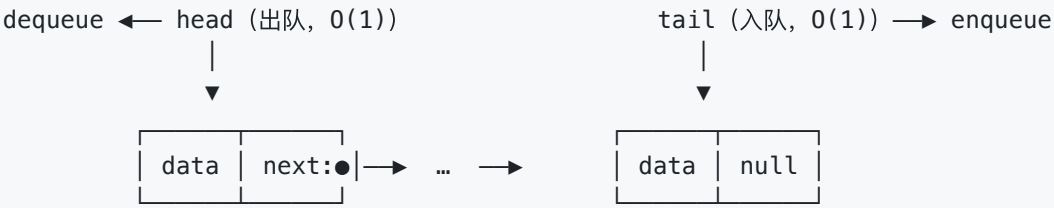
```
head                                                          tail
 |                                                             |
 ▼                                                             ▼
nullptr ◀── [prev| 87 |next] ⇄ [prev| 93 |next] ⇄ [prev| 12 |next] ──▶ nullptr
```

在节点 cur 之后插入新节点 n，需要四步（单向只要两步）：

```
① n->prev = cur            ② n->next = cur->next
③ if (cur->next) cur->next->prev = n     ← 原后继回指 n（若 cur 是最后一个则跳过）
④ cur->next = n
```

删除节点 cur 则是两步"绕开"：`cur->prev->next = cur->next;` 与 `cur->next->prev = cur->prev;`（删头/删尾时还要同步 head/tail），然后 delete。双向链表的代价：每个节点从 12 字节涨到 20 字节（int 4B + 两个指针各 8B），空间多约 67%；且一切插入/删除都要维护两条方向的指针，代码明显更绕。这是计算机科学里最经典的一类交易：**多花一点内存，换大幅提升的运行时间。**

11. 用链表实现栈与队列。 栈要求 push/pop 在同一端，全部放在头部即可（O(1)）。队列要求入队、出队在两端：入队放队尾（借助 tail 指针 O(1)），出队放队头（O(1)）——绝不能在尾部出队，否则撞上"尾删 O(n)"。这就是 LLQueue 的布局：



与动态数组版队列相比：数组版绝大多数入队 O(1)，但偶发一次 O(n) 扩容（最坏延迟不可控）；链表版每次入队都是稳定 O(1)（代价是每次都要 new 节点、设置多个字段，常数因子更慢）。如果软件对"任何单次操作都不许超时"有硬性要求，链表版的确定性反而是优点。

12. 权衡总表：数组 vs 链表。

维度	数组	链表
内存布局	一整块连续内存	节点散落堆中，指针相连（不怕内存碎片化）
随机访问第 k 个	O(1)，直接算地址	O(k)，必须从 head 一路走过来
头部插入/删除	O(n)，整体挪窝	O(1)，两步指针操作
尾部插入	O(1) 均摊（可能扩容）	O(1)（维护尾指针）否则 O(n)
尾部删除	O(1)	O(n)（单向）；O(1)（双向 + 尾指针）
空间开销	每元素 4B（int），无冗余	每节点 12B（4+8，单向）/ 20B（双向）；空间 ≈ 3~5 倍

维度	数组	链表
容量	固定/需扩容（可能浪费或 O(n) 扩容）	按需生长，用多少 new 多少
二分查找	有序数组可 O(log n)	不行（无随机访问），查找最坏 O(n)
适用场景	频繁按下标访问、数据量稳定	频繁在两端增删、数据量动态、内存碎片环境

代码示例与实现详解

示例 1：完整单向链表类 LinkedList（struct Node + head，含全套基础操作与析构）

```cpp
#include <iostream>
#include <stdexcept>
using namespace std;

// 链表节点：一个数据字段 + 一个指向下一个节点的指针
struct Node {
    int data;      // 节点存放的值
    Node *next;    // 指向"下一个节点"的指针；链表末尾为 nullptr
    Node(int d) : data(d), next(nullptr) {}
};

// 单向链表类：只维护一个头指针作为整个链表的入口
class LinkedList {
public:
    LinkedList() : head(nullptr), _size(0) {}
    ~LinkedList() { clear(); }

    // 析构需要释放堆内存，因此禁用浅拷贝，避免同一块内存被 delete 两次
    LinkedList(const LinkedList &) = delete;
    LinkedList &operator=(const LinkedList &) = delete;

    void insertFront(int val);   // 头插 O(1)
    int removeFront();           // 头删 O(1)
    void insertBack(int val);    // 尾插 O(n)
    int removeBack();            // 尾删 O(n)
    bool contains(int val) const;
    void print() const;
    int size() const { return _size; }

private:
    void clear();                // 逐个 delete 所有节点
    Node *head;
    int _size;
};

void LinkedList::insertFront(int val) {
    Node *newNode = new Node(val); // ① 在堆上造一个新节点
    newNode->next = head;          // ② 先让新节点指向旧头（先连！）
    head = newNode;                // ③ 再让头指针指向新节点（后改！）
    ++_size;
}

int LinkedList::removeFront() {
    if (head == nullptr)
        throw runtime_error("removeFront() on empty list!");
    Node *victim = head;        // 先记住要删除的旧头节点
    int retval = victim->data;  // 保存要返回的值
    head = head->next;          // 头指针先往前走一步
    delete victim;              // 最后才释放旧头节点
    --_size;
    return retval;
}
```

```
void LinkedList::insertBack(int val) {
    Node *newNode = new Node(val);
    if (head == nullptr) {        // 空链表：新节点同时就是头
        head = newNode;
    } else {
        Node *cur = head;
        while (cur->next != nullptr)  // 一路走到最后一个节点
            cur = cur->next;
        cur->next = newNode;          // 把新节点挂在末尾
    }
    ++_size;
}

int LinkedList::removeBack() {
    if (head == nullptr)
        throw runtime_error("removeBack() on empty list!");
    if (head->next == nullptr) {   // 只剩一个节点：删完链表就空了
        int retval = head->data;
        delete head;
        head = nullptr;
        --_size;
        return retval;
    }
    Node *cur = head;
    while (cur->next->next != nullptr) // 停在"倒数第二个"节点
        cur = cur->next;
    int retval = cur->next->data;
    delete cur->next;
    cur->next = nullptr;
    --_size;
    return retval;
}

bool LinkedList::contains(int val) const {
    for (Node *cur = head; cur != nullptr; cur = cur->next)
        if (cur->data == val)
            return true;
    return false;
}

void LinkedList::print() const {
    cout << "head";
    for (Node *cur = head; cur != nullptr; cur = cur->next)
        cout << " -> " << cur->data;
    cout << " -> nullptr" << endl;
}

void LinkedList::clear() {
    while (head != nullptr) {
        Node *tmp = head->next;  // 先保存下一个节点的地址
        delete head;             // 再删除当前节点
        head = tmp;              // 指针前移（tmp 为 nullptr 时循环结束）
    }
```

```
    _size = 0;
}

int main() {
    LinkedList list;
    list.insertFront(30);
    list.insertFront(20);
    list.insertFront(10);       // 链表现在为: 10 -> 20 -> 30
    list.insertBack(99);        // 链表现在为: 10 -> 20 -> 30 -> 99
    list.print();

    cout << "contains(20)? " << list.contains(20) << endl;
    cout << "size = " << list.size() << endl;

    cout << "removeFront() -> " << list.removeFront() << endl; // 10
    cout << "removeBack()  -> " << list.removeBack() << endl;  // 99
    list.print();               // 20 -> 30

    list.insertBack(7);
    list.print();               // 20 -> 30 -> 7
    return 0;                   // 离开作用域，析构函数自动释放剩余节点
}
```

【代码做什么】 main() 演示了全套操作：三次头插得到 `10->20->30`，一次尾插得到 `10->20->30->99`；打印、查找 20、查询 size；removeFront 删掉 10、removeBack 删掉 99，链表回到 `20->30`；再尾插 7 得到 `20->30->7`。程序结束时 list 对象离开作用域，析构函数 ~LinkedList 调用 clear() 把仍存活的两个节点全部 delete，无内存泄漏。insertFront 忠实落实"先连新节点、后改头指针"：新节点先指向旧头（`newNode->next = head`），再更新头指针；removeFront 忠实落实"先移头、再删旧头"：先用 victim 保存旧头地址，head 前移之后才 `delete victim`。两个删除函数都对空表抛出 std::runtime_error，避免解引用 nullptr。

【实现机制解说】 类的全部状态就是 `head` 指针与计数器 `_size`，head 是唯一入口，所以每个成员函数都从 head 出发。insertBack/removeBack 没有尾指针，只能靠 `while (cur->next != nullptr)` / `while (cur->next->next != nullptr)` 先定位到"最后一个/倒数第二个"节点——前者停住时 cur 的 next 为 nullptr 可挂新节点，后者停住时 `cur->next` 就是要删的尾节点，删完要把 `cur->next = nullptr` 让新尾封口。removeBack 还特判了"只剩一个节点"：此时 `head->next == nullptr`，直接删 head 并置空，否则通用循环里 `cur->next->next` 会解引用空指针。析构/clear 用"先存 tmp 再 delete 再前移"的三步循环，正是为了避免第 7 节那个 `delete head; head = head->next;` 的悬垂错误；循环结束后 head 自然为 nullptr。`_size` 在每次插入/删除时同步增减，使 size() 达到 O(1)。因为类管理着堆内存，浅拷贝会让两个对象共享同一串节点、析构时 double free，所以示例用 `= delete` 显式禁用拷贝（若业务需要拷贝，应按"Rule of Three"补拷贝构造与拷贝赋值——本讲重点是链表的指针机制，故不展开）。

示例 2：用链表实现队列 LLQueue（front + back，两端 O(1)）

```
#include <iostream>
#include <stdexcept>
using namespace std;

struct Node {
    int data;
    Node *next;
    Node(int d) : data(d), next(nullptr) {}
};

// 用单向链表实现队列：队尾入队(enqueue)、队头出队(dequeue)。
// 同时维护 front(头) 与 back(尾) 两个指针，使两端操作都是 O(1)。
class LLQueue {
public:
    LLQueue() : front(nullptr), back(nullptr), _size(0) {}
    ~LLQueue() { clear(); }

    LLQueue(const LLQueue &) = delete;
    LLQueue &operator=(const LLQueue &) = delete;

    void enqueue(int val); // 在队尾（链表尾）加入：O(1)
    int dequeue();         // 从队头（链表头）取出：O(1)
    int peek() const;      // 只看队头不取走
    int size() const { return _size; }
    bool isEmpty() const { return front == nullptr; }

private:
    void clear();
    Node *front;  // 队头 = 链表头
    Node *back;   // 队尾 = 链表尾
    int _size;
};

void LLQueue::enqueue(int val) {
    if (back == nullptr) {            // 空队列：新节点既是头也是尾
        front = back = new Node(val);
    } else {
        back->next = new Node(val);   // 挂到当前队尾之后
        back = back->next;            // 尾指针向后移动
    }
    ++_size;
}

int LLQueue::dequeue() {
    if (front == nullptr)
        throw runtime_error("dequeue() on empty queue!");
    Node *victim = front;
    int retval = victim->data;
    front = front->next;              // 头指针前移
    if (front == nullptr)             // 队列变空：尾指针也必须归零！
        back = nullptr;
    delete victim;
    --_size;
```

```
    return retval;
}

int LLQueue::peek() const {
    if (front == nullptr)
        throw runtime_error("peek() on empty queue!");
    return front->data;
}

void LLQueue::clear() {
    while (front != nullptr) {
        Node *tmp = front->next;
        delete front;
        front = tmp;
    }
    back = nullptr;   // 防止留下指向已释放节点的悬垂尾指针
    _size = 0;
}

int main() {
    LLQueue q;
    q.enqueue(10);
    q.enqueue(20);
    q.enqueue(30);
    cout << "front = " << q.peek() << ", size = " << q.size() << endl;

    while (!q.isEmpty())
        cout << q.dequeue() << " ";
    cout << endl;

    q.enqueue(99);                      // 清空后再入队：考验尾指针的维护
    cout << "again: " << q.dequeue() << endl;
    return 0;
}
```

【代码做什么】 main() 先入队 10、20、30，打印队头 10 与大小 3；随后循环出队打印 `10 20 30`，队列被清空；此时再入队 99 并立刻出队——这一步专门考验"队列清空后 front/back 指针是否仍被正确维护"。enqueue 在队尾加节点：空队时新节点同时成为 front 与 back，非空时挂到 back 之后并把 back 后移；dequeue 在队头取：保存旧头地址与返回值，front 前移，若队列因此变空则把 back 一并置空（否则 back 将指向已 delete 的节点），最后释放旧头。peek/size/isEmpty 都只读不写，声明为 const 成员函数——编译器会保证它们不会误改成员变量。

【实现机制解说】 队列是 FIFO，入队、出队发生在链表两端：入队走尾（back 让我们 O(1) 直达链尾，不必像示例 1 的 insertBack 那样从头走一遍），出队走头（front 就是链头，O(1)）。维护 back 的核心是两条"同步规则"：入队时空表特判（新节点 = 头 = 尾）；出队后若链表变空，必须把 back 置 nullptr。dequeue 里 `front = front->next` 之后再判空，而不是先判 `front->next` 再移动——若队列只有一个节点，front->next 本来就是 nullptr，移动后判空恰好命中，逻辑统一。clear() 循环释放所有节点后把 back 归零，是因为循环结束时 back 仍指着最后一个被删除的节点，不归零就是悬垂指针。这个结构再次印证：**"用空间（多**

维护一个指针）换时间（尾插 O(n)→O(1)）"，同时所有可能改变链首/链尾的操作都必须意识到 back 的存在。

示例 3：双向链表——pushFront / popBack / 中间 insertAfter（演示多出的 prev 步骤）

```
#include <iostream>
#include <stdexcept>
using namespace std;

// 双向链表节点：比单向多一个 prev 指针，可向前走
struct DNode {
    int data;
    DNode *prev;   // 指向前一个节点
    DNode *next;   // 指向后一个节点
    DNode(int d) : data(d), prev(nullptr), next(nullptr) {}
};

// 头插：除单向链表的两个步骤外，还要让旧头指回新节点（多一步）
void pushFront(DNode *&head, DNode *&tail, int val) {
    DNode *n = new DNode(val);
    n->next = head;              // ① 新节点指向旧头
    if (head != nullptr)
        head->prev = n;          // ② 旧头回指新节点（双向特有的步骤）
    else
        tail = n;                // 空表：新节点同时成为尾
    head = n;                    // ③ 头指针指向新节点
}

// 头删：返回被删值；只剩一个节点时 head/tail 都要置空
int popFront(DNode *&head, DNode *&tail) {
    if (head == nullptr)
        throw runtime_error("popFront() on empty list!");
    DNode *victim = head;
    int retval = victim->data;
    head = head->next;
    if (head != nullptr)
        head->prev = nullptr;    // 双向特有：新头没有前驱了
    else
        tail = nullptr;          // 链表空了，尾指针同步归零
    delete victim;
    return retval;
}

// 尾删：双向链表 O(1)（单向链表即使有尾指针也要 O(n) 找倒数第二）
int popBack(DNode *&head, DNode *&tail) {
    if (tail == nullptr)
        throw runtime_error("popBack() on empty list!");
    DNode *victim = tail;
    int retval = victim->data;
    tail = tail->prev;
    if (tail != nullptr)
        tail->next = nullptr;    // 双向特有：新尾的后继清空
    else
        head = nullptr;          // 链表空了，头指针同步归零
    delete victim;
    return retval;
}
```

```
// 在节点 node 之后插入新节点（演示"中间插入"的四步指针重连）
void insertAfter(DNode *node, int val) {
    DNode *n = new DNode(val);
    n->prev = node;               // ① 新节点指回 node
    n->next = node->next;         // ② 新节点指向 node 原来的后继
    if (node->next != nullptr)
        node->next->prev = n;     // ③ 原后继回指新节点（若存在）
    node->next = n;               // ④ node 的 next 指向新节点
}

void printForward(DNode *head) {
    for (DNode *p = head; p != nullptr; p = p->next)
        cout << p->data << " ";
    cout << endl;
}

void printBackward(DNode *tail) {
    for (DNode *p = tail; p != nullptr; p = p->prev)
        cout << p->data << " ";
    cout << endl;
}

void destroy(DNode *&head) {
    while (head != nullptr) {
        DNode *tmp = head->next;
        delete head;
        head = tmp;
    }
}

int main() {
    DNode *head = nullptr, *tail = nullptr;
    pushFront(head, tail, 30);
    pushFront(head, tail, 20);
    pushFront(head, tail, 10);   // 10 <-> 20 <-> 30

    cout << "forward : ";
    printForward(head);
    cout << "backward: ";
    printBackward(tail);

    insertAfter(head->next, 99); // 在 20 之后插入 99: 10 <-> 20 <-> 99 <-> 30
    cout << "after insertAfter(20, 99): ";
    printForward(head);

    cout << "popBack -> " << popBack(head, tail) << endl;  // 30
    cout << "popFront -> " << popFront(head, tail) << endl; // 10
    cout << "remaining: ";
    printForward(head);

    destroy(head);   // 释放所有节点（head 变 nullptr）
    return 0;
}
```

【代码做什么】 main() 用三次头插建成 `10 ⇄ 20 ⇄ 30`，正序打印 `10 20 30`、用 prev 逆序打印 `30 20 10`（证明双向行走有效）；insertAfter 在 20 之后插 99，链表变成 `10 20 99 30`；popBack 删掉 30、popFront 删掉 10，剩 `20 99`；最后 destroy 释放全部节点。

【实现机制解说】 双向链表的每个操作都比单向多"一到两步对称动作"：pushFront 多一句 `head->prev = n`（旧头回指新节点），popFront 多一句 `head->prev = nullptr`（新头斩断前驱），popBack 之所以 O(1)，正是因为它能借 `tail->prev` 直接后退一格找到新尾，再 `tail->next = nullptr` 把新尾封口——单向链表做不到这一步，所以尾删必须 O(n)。两个边界情形贯穿始终：空表时头插的节点同时是尾（`tail = n`）；删空链表时头、尾必须双双归零（popFront 删最后一个时 `tail = nullptr`，popBack 删最后一个时 `head = nullptr`）。insertAfter 的四步顺序是经过设计的：第③步要访问 `node->next->prev`，因此必须先于第④步执行（第④步一旦改写 node->next，原后继就找不到了），并且要先判 `node->next != nullptr`——若 node 是尾节点，原后继不存在，跳过第③步即可。对比示例 1 会发现：双向链表多出来的所有代码，本质上都在维护"对称的第二条链"。

复杂度分析

操作	数组	单向链表（无尾指针）	单向链表（维护尾指针）	双向链表（头+尾指针）
访问第 k 个元素	O(1)	最好 O(1)（k 小），最坏 O(k)=O(n)	同左	同左
头插 / 头删	O(n)（全体挪窝）	O(1)	O(1)	O(1)
尾插	O(1) 均摊（偶发扩容 O(n)）	O(n)（走到尾）	O(1)	O(1)
尾删	O(1)	O(n)（走到倒数第二）	O(n)（单向无法后退）	O(1)（经 prev 后退）
按值查找	O(n)（有序可二分 O(log n)）	O(n)（无序可遍历）	O(n)	O(n)
空间（每元素）	4 字节 + 可能浪费/扩容	12 字节（4 数据 + 8 指针）	+8 字节尾指针	20 字节（4 数据 + 16 指针）

最好/平均/最坏说明。 链表的形态不随插入历史变化，头尾操作基本是"恒定 O(1) 或恒定 O(n)"，没有明显的最好/最坏之分；真正分化的只有"访问/查找第 k 个"（头附近快、越深越慢，最坏到链尾 O(n)）以及数组版队列的"入队"（平常 O(1)、扩容那一次 O(n)）。因此官方给出的精确说法是"取决于实现"：问"尾插多快"要答"若维护尾指针则 O(1)，否则 O(n)"。空间上，链表节点比数组单元贵得多：官方以 int 4 字节、指针 8 字节（64 位系统）为例，单向链表每节点 12 字节约为数组的 3 倍，双向链表每节点 20 字节约为 5 倍——这是"把地址作为胶水"的固有开销。

关键要点

- 头指针是链表的唯一入口，nullptr 是链尾哨兵；画内存图（连地址一起画）是理解与调试链表的头号武器。
- 头插永远"先让新节点指向旧头，再移动头指针"（先连后改）；头删永远"先保存/移动指针，再 delete"——delete 之后绝不访问那个地址。
- 只要函数要改变主调方指针变量的值，就用 `Node *&head` 传引用；只读遍历用普通 `Node *head`，造节点用返回值。
- 尾指针把尾插从 O(n) 降到 O(1)，但空表、只剩一个节点这两类边界必须在头插/头删/出队时同步维护 head 与 tail。
- 双向链表用"每个节点多一个 prev"换取 O(1) 尾删与逆序遍历，代价是空间 +67% 与两套指针维护——这是"空间换时间"的经典示范。

常见陷阱与注意事项

- **丢失头指针**：`head = head->next` 前忘了先保存旧头地址，旧节点无法释放（泄漏），链表入口也丢了。规避：删除前先 `Node *tmp = ...` 保存，或先移动再 delete。
- **头插顺序写反**：`head = n; n->next = head;` 会让 n 指向自己，旧链表整体丢失。规避：牢记"先连后改"，写完立即画图验证。
- **delete 后解引用**：`delete head; head = head->next;` 是未定义行为。规避：先保存 `head->next`，再 delete，再赋值。
- **解引用 nullptr / 对空表操作**：空表上 removeFront 会段错误。规避：进入函数先判 `head == nullptr` 并抛出异常或提前返回。
- **忘记初始化头指针**：`Node *head;` 不初始化就 tailInsert，head 里是垃圾地址，函数会"顺着垃圾地址走"然后崩溃。规避：一律 `Node *head = nullptr;`。
- **维护尾指针时漏掉边界**：头插进空表忘更新 tail、删光唯一节点忘把 tail 置空，都会留下悬垂的尾指针。规避：把"空表 / 只剩一个节点"作为 checklist 逐函数过一遍。
- **内存泄漏**：每个 `new Node` 都必须有对应的 delete（析构/clear 循环释放全部节点）。规避：写完链表类先测空表、单节点、多节点三种情况，确认析构无泄漏。
- **把"改指针"误当"改节点"**：`current = current->next` 只是让局部指针前进，不会破坏链表；而 `*current = ...` 才是改节点内容——两者别混淆，printList 里想明白这一点就能放心用局部拷贝遍历。
- **忘了 const**：peek()/size()/isEmpty() 这类只读成员函数应在声明末尾加 const，防止误改成员，也让常量对象可调用。
- **不会用 -> 的等价物**：`(*head).data` 虽合法但官方明令避免，统一写 `head->data`。

思考题（带答案）

问题 1：为什么 `void insertFront(Node *head, int val)`（按值传指针）无法真正完成头插？如何修改？**答案**：形参 head 是 main() 里 head 的一份拷贝，函数内 `head = newNode` 只改写拷贝，返回后 main() 的头指针不变，新节点脱离链表（泄漏且"插入失败"）。修改方法：参数改为 `Node *&head`（指针的引用），让函数直通调用方的变量；或者用返回值 `Node *insertFront(Node *head, int val)` 由调用方接住新头。

问题 2：单链表删除尾节点时，为什么必须停在"倒数第二个"节点而不是停在尾节点？尾删在维护了尾指针的单向链表上为何仍是 O(n)？ **答案**：停在尾节点时我们手里只有尾节点自己，无法把它从链表上"摘下来"——摘除需要改写它前驱的 next 字段，而单向链表没有 prev 指针，找不到它的前驱，所以循环条件要写成 `cur->next->next != nullptr`，停在倒数第二，改写 `cur->next`。同理，即便维护了尾指针，删除尾节点后 tail 要后退一格指向新尾，单向链表无法从旧尾一步退到前驱，只能从头重新走 O(n) 找到倒数第二；要 O(1) 尾删必须引入 prev 指针（双向链表）。

问题 3：LLQueue 用"front + back"实现，为什么入队走队尾、出队走队头，而不能反过来？ **答案**：出队（删除）若发生在队尾，就是"尾删"：单向链表上无论如何都是 O(n)，即使有 back 指针也退不回去；而入队（插入）发生在队尾时，back 指针能让我们 O(1) 直达链尾完成插入。反过来配置（入队走头、出队走尾）会把 O(n) 的尾删强加给每次出队，队列两端就无法同时高效了。栈则不同，push/pop 都发生在同一端，放在头即可全部 O(1)。

Lecture 12: 二叉树、二叉搜索树与树遍历（Binary Trees, BSTs & Traversals）（对应课程真实讲座 L21–L22）

概述

本讲把"节点 + 指针"的思路从一条链升级成一棵树：每个节点至多有两个孩子指针 left/right，形成二叉树。核心数据结构是二叉搜索树（BST）——它给二叉树加上"左小右大"的排序约束，让查找/插入/删除在平衡时只需 O(log n)；本讲完整给出递归的插入、查找与三分支的删除算法，以及前序/中序/后序/层序四种遍历。树也是文件压缩（霍夫曼编码）与图论等领域中反复出现的组织形态。 对应官方讲座：L21（Wednesday, July 29 — Binary Trees, Binary Search Trees, and Tree Traversals）与 L22（Thursday, July 30 — More on Binary Trees）；官方课件另附 tree-notes.pdf（手写讲义）、traversal-puzzle.pdf（遍历谜题）与 bst-code.zip（课上代码）可自行研读。

核心概念与算法原理

1. 从链表到树：术语与 TreeNode。 链表每个节点只有一个 next，是一条"退化"的树；树的每个节点可以有多个孩子。二叉树（binary tree）约定每个节点至多两个孩子，分别叫左孩子 left、右孩子 right。沿用树形结构的通用术语：最上面的节点叫根（root），没有孩子的节点叫叶（leaf），一个节点连同它下面的所有后代叫一棵子树（subtree），从根到最深叶的"边数"叫树的高度。树的节点结构与链表节点如出一辙，只是把 next 换成两个指针：

```
struct TreeNode {
    int data;
    TreeNode *left;   // 左孩子（没有则为 nullptr）
    TreeNode *right;  // 右孩子（没有则为 nullptr）
};
```

与 head 类似，一个 root 指针保存根节点的地址，是整棵树的唯一入口。二叉堆用数组表示之所以可行，是因为堆是"完全二叉树"（逐层从左到右填满、无空洞）；而普通二叉树形状任意，必须用节点 + 指针才能真正表达。

2. 二叉搜索树（BST）的性质。 BST = 二叉树 + 一条排序规则：**对任意节点，其左子树里所有值都小于它，右子树里所有值都大于它**（课程通常假定键不重复；若允许重复，可约定相等的放左边或直接忽略）。这条规则是"自找的"：它让查找变成"每走一步就排除掉半棵树"。以树根 50 为例：

```
          50            ← 左子树 {20,30,40} < 50 < 右子树 {60,70,80}
         /  \
       30    70
       / \   / \
     20  40 60  80        ← 每一层都同样满足"左小右大"
```

注意规则是对"整棵子树"而言，不只是对直接孩子：例如 40 虽然大于 30，但它处在 30 的右子树，同时仍小于根 50，这完全合法。

3. 查找：顺着大小方向走。 查找 40：与根 50 比，40 < 50 → 进左子树；与 30 比，40 > 30 → 进右子树；与 40 比，命中。每比较一次就放弃一侧子树，走的高度是多少步数就是多少。递归版的三行骨架：空节点返回 false（没找到）；相等返回 true；否则把问题缩小到左或右子树继续。因为树的高度决定步数，所以"树长得越高，操作越慢"——这正是本讲复杂度讨论的核心。

4. 插入：找到空位挂上去。 插入新值的过程就是一次"失败的查找"：按大小一路下探，直到撞上一个 nullptr，就把新节点挂在那里。递归写法妙在传引用：`void insertRec(TreeNode *&node, int v)`——当递归到达空位时，`node` 引用直通"父节点的 left/right 字段（或 root）"，`node = new TreeNode(v)` 便自动把新节点焊回树上，不需要返回指针再手动接线。插入 21 到以 44 为根的子树，等价于把 21 插进"44 的右子树"…… 如此递归下去，直到某个节点的空孩子成为新节点的家。

5. 删除：三情形逐一击破。 删除是 BST 最精细的操作，先递归找到目标节点（值与目标相等的那一个），再按它的孩子数分三种情形：

```
情形1：没有孩子（叶子）——直接摘除，父指针置空
      50                50
     /  \              /  \
    30   70   删20→   30   70
   / \   / \             / \
  20 40 60 80          40 60 80

情形2：只有一个孩子——孩子"顶上来"接替父位
      50                50
     /  \              /  \
    30   70   删30→   40   70
     \   / \             / \
     40 60 80          60 80

情形3：两个孩子——用"中序后继"的值覆盖自己，再去右子树删掉那个后继
      50                60
     /  \              /  \
    40   70   删50→   40   70
        / \               / \
       60 80             60? 80   ← 实际是把 60 的"值"搬来，节点 60 从原位移除
```

情形 3 的思路：目标节点两个孩子都在，直接删它会把两棵子树都弄丢。于是用右子树里的最小值（中序后继 in-order successor——中序遍历中排在它后面的那个元素）来"顶班"：先把后继的值抄进目标节点，再递归地到右子树把那个后继节点删掉。后继是右子树的最左节点，它必然没有左孩子，所以对它的删除一定落回情形 1 或 2，不会无限递归。官方课程出于考试批改的一致性，习惯采用对称方案"用左子树的最大值（中序前驱）顶班"，两种方案都产生合法的 BST，本笔记按需求采用中序后继（右子树最小）。

6. 四种遍历。 遍历 = 按某种顺序访问每个节点恰好一次。三种递归遍历只是"访问自己（根）"相对两个孩子的位置不同；层序遍历则按"从上到下、每层从左到右"逐层推进：

```
          50              前序(根左右): 50 30 20 40 70 60 80
         /  \             中序(左根右): 20 30 40 50 60 70 80  ← 对 BST 恒为升序!
       30    70           后序(左右根): 20 40 30 60 80 70 50
       / \   / \          层序(BFS):   50 30 70 20 40 60 80
      20  40 60 80
```

递归遍历的代码极短：前序是"先打印自己，再递归左，再递归右"；中序把打印挪到两次递归之间；后序把打印放到最后。层序不用递归，改用队列：根入队；每次出队一个节点就访问它，并把它的左、右孩子依次入队——队列天然保证"先来先访问"，于是同一层从左到右、层与层自上而下。中序遍历对 BST 输出升序序列这一性质，是"为什么要有 BST"的最直观回报。

7. 遍历的应用。 中序：把 BST 里的元素按序输出（如需有序遍历容器）。前序：先处理祖先再深入，适合"找到目标就停"的搜索（官方举例浏览器 DOM 的按 id 找元素）以及树的复制/序列化。后序：先孩子后自己，是释放整棵树的唯一安全顺序——删节点之前它的两个孩子子树必须已经全部释放完毕（官方把释放函数戏称为 forestFire，先烧光两片子树再烧根）；"先删自己再递归孩子"的版本会在 delete 之后解引用已释放节点的 left/right，是未定义行为。

8. 好树、坏树与复杂度三兄弟。 把 1~10000 顺序插入 BST，每个新值都比前一个大、永远走最右分支，树会退化成一条"右斜链"，高度 9999，查找退化为 O(n)——和链表一样慢。若按随机顺序插入，树大致平衡，高度约 O(log n)，查找飞快。官方给出的结论是：最好 O(1)（值恰在根附近）、平均 O(log n)（随机/平衡树）、最坏 O(n)（树退化成链）。严谨的说法可以写成 O(h)，h 为树高；但课程按惯例用 n 表达上表。**结论：BST 的性能完全取决于插入顺序带来的树形。**

9. 自平衡 BST：把最坏情形也按下去。 现实数据常常天然有序（如字典文件按字母序存放），直接插入必退化成链。自平衡 BST 在插入/删除后自动"整形"——通过旋转（rotation）等操作保证任何节点的左右子树高度差不超过约定限度，使整棵树永远保持 O(log n) 高度，于是查找/插入/删除的最坏情形也是 O(log n)。代价只是插入/删除多了少量平衡维护的开销，仍在 O(log n) 内。常见家族：AVL 树（每个节点记录平衡因子，严格限制高度差 ≤1）与红黑树（给节点染色、用颜色约束维持近似平衡）。标准库的 `std::set` 与 `std::map` 正是用红黑树实现的，所以它们的有序性、O(log n) 操作与"键不可重复"全都由此而来；官方指出 Stanford 的 Set/Map 同样由自平衡 BST 驱动。

10. 为什么用 BST 存数据？ 相比有序数组/vector：插入新元素到头部要整体挪窝 O(n)，BST 平衡时只需 O(log n)；数组还要面对扩容 O(n) 与"固定容量浪费/不够"的两难，BST 按需 new 节点。相比链表：链表即使有序也只能线性查找 O(n)，BST 平衡时 O(log n)。代价：每个节点 20 字节（int 4B + 两个指针 16B），约为等量 int 数组的 5 倍；且指针解引用有轻微额外开销（对本课程规模可忽略）。一句话：BST 用空间和一点指针开销，换来"插入/删除/查找三样都对数级"的均衡能力。

代码示例与实现详解

示例 1：完整的 BST 类——递归 insert/search/remove、四种遍历、析构（后序释放）

```
#include <iostream>
#include <queue>
using namespace std;

struct TreeNode {
    int data;
    TreeNode *left;
    TreeNode *right;
    TreeNode(int d) : data(d), left(nullptr), right(nullptr) {}
};

// 二叉搜索树（BST）：左子树所有值 < 根 < 右子树所有值；约定键不重复
class BST {
public:
    BST() : root(nullptr) {}
    ~BST() { clear(root); }

    void insert(int v)   { insertRec(root, v); }
    bool search(int v) const { return searchRec(root, v); }
    void remove(int v)   { removeRec(root, v); }

    void preorder()  const { preorderRec(root);  cout << endl; }
    void inorder()   const { inorderRec(root);   cout << endl; }
    void postorder() const { postorderRec(root); cout << endl; }
    void levelorder() const;   // 层序：需要队列，单独实现

private:
    TreeNode *root;
    void insertRec(TreeNode *&node, int v);
    bool searchRec(TreeNode *node, int v) const;
    void removeRec(TreeNode *&node, int v);
    TreeNode *findMin(TreeNode *node) const;  // 右子树最小 = 中序后继
    void preorderRec(TreeNode *node) const;
    void inorderRec(TreeNode *node) const;
    void postorderRec(TreeNode *node) const;
    void clear(TreeNode *&node);
};

void BST::insertRec(TreeNode *&node, int v) {
    if (node == nullptr) { node = new TreeNode(v); return; } // 空位！在此挂上新节点
    if (v < node->data)
        insertRec(node->left, v);    // 比根小 → 去左子树
    else if (v > node->data)
        insertRec(node->right, v);   // 比根大 → 去右子树
    // 相等：约定不存重复值，直接忽略
}

bool BST::searchRec(TreeNode *node, int v) const {
    if (node == nullptr) return false;        // 走到空：没找到
    if (v == node->data) return true;         // 命中
    return (v < node->data) ? searchRec(node->left, v)
                            : searchRec(node->right, v);
}
```

```
void BST::removeRec(TreeNode *&node, int v) {
    if (node == nullptr) return;              // 树里没有这个值
    if (v < node->data) { removeRec(node->left, v); return; }
    if (v > node->data) { removeRec(node->right, v); return; }

    // 找到了要删的节点 node：
    if (node->left == nullptr && node->right == nullptr) {
        delete node;                          // 情形1：叶子 → 直接摘除
        node = nullptr;                       // 让父指针（或 root）归零
    } else if (node->left == nullptr) {
        TreeNode *tmp = node->right;          // 情形2a：只有右孩子
        delete node;
        node = tmp;                           // 右孩子顶上来
    } else if (node->right == nullptr) {
        TreeNode *tmp = node->left;           // 情形2b：只有左孩子
        delete node;
        node = tmp;                           // 左孩子顶上来
    } else {
        // 情形3：两个孩子 → 用"中序后继"（右子树最小）的值覆盖本节点，
        //         再去右子树里把那个最小节点删掉（它必然没有左孩子）。
        TreeNode *succ = findMin(node->right);
        node->data = succ->data;
        removeRec(node->right, succ->data);
    }
}

TreeNode *BST::findMin(TreeNode *node) const {
    while (node->left != nullptr) node = node->left; // 一路向左到底
    return node;
}

void BST::preorderRec(TreeNode *node) const {
    if (node == nullptr) return;
    cout << node->data << " ";   // ① 先处理自己（根）
    preorderRec(node->left);     // ② 再左子树
    preorderRec(node->right);    // ③ 最后右子树
}

void BST::inorderRec(TreeNode *node) const {
    if (node == nullptr) return;
    inorderRec(node->left);      // ① 先左子树
    cout << node->data << " ";   // ② 再自己（对 BST 而言即有序输出）
    inorderRec(node->right);     // ③ 最后右子树
}

void BST::postorderRec(TreeNode *node) const {
    if (node == nullptr) return;
    postorderRec(node->left);    // ① 先左子树
    postorderRec(node->right);   // ② 再右子树
    cout << node->data << " ";   // ③ 最后自己
}
```

```
void BST::levelorder() const {
    if (root == nullptr) return;
    queue<TreeNode *> q;          // 用 std::queue 按层推进
    q.push(root);
    while (!q.empty()) {
        TreeNode *cur = q.front();
        q.pop();
        cout << cur->data << " ";
        if (cur->left)  q.push(cur->left);   // 下一层的左孩子排队
        if (cur->right) q.push(cur->right);  // 下一层的右孩子排队
    }
    cout << endl;
}

void BST::clear(TreeNode *&node) {
    if (node == nullptr) return;
    clear(node->left);     // 先释放左子树（后序！）
    clear(node->right);    // 再释放右子树
    delete node;           // 最后才删除自己
    node = nullptr;
}

int main() {
    BST t;
    for (int v : {50, 30, 70, 20, 40, 60, 80})
        t.insert(v);

    cout << "preorder : ";  t.preorder();   // 50 30 20 40 70 60 80
    cout << "inorder  : ";  t.inorder();    // 20 30 40 50 60 70 80（有序！）
    cout << "postorder: ";  t.postorder();  // 20 40 30 60 80 70 50
    cout << "level    : ";  t.levelorder(); // 50 30 70 20 40 60 80

    cout << "search(40)? " << t.search(40) << ", search(55)? "
         << t.search(55) << endl;

    t.remove(20);  // 叶子：直接摘除
    cout << "after remove(20) : ";  t.inorder(); // 30 40 50 60 70 80

    t.remove(30);  // 一个孩子（右孩子 40 顶上来）
    cout << "after remove(30) : ";  t.inorder(); // 40 50 60 70 80

    t.remove(50);  // 两个孩子：中序后继 60 顶替
    cout << "after remove(50) : ";  t.inorder(); // 40 60 70 80
    return 0;
}
```

【代码做什么】 main() 先按 50、30、70、20、40、60、80 的顺序建出一棵完全平衡的小 BST，四种遍历分别打印（结果见注释，中序恰好升序）；search 分别测试命中 40 与未命中 55。随后连续演示删除三情形：remove(20) 是叶子直接摘除；remove(30) 时 30 只剩右孩子 40，由 40 顶上来；remove(50) 时根 50 有两个孩子，右子树最小值 60（中序后继）的值被搬进根节点，60 原节点被递归删除，中序输出 `40 60 70 80` 验证树仍是合法 BST。程序结束时析构函数 clear(root) 以后序顺序释放全部节点。

【实现机制解说】 递归函数全部接受 `TreeNode *&`（指针引用）或 const 值拷贝，分工不同：insertRec 与 removeRec 会改写"指针变量里存的地址"（空位挂新节点、删空后把父指针置空、孩子顶班），所以必须传引用，否则改动留在栈帧副本上、树根本没变；searchRec 与遍历只读，传普通指针即可。递归的栈变化值得细想：insertRec 一路下探时，每一层栈帧都持有一个对"父节点某个孩子字段"的引用；最深一层撞上 nullptr，`node = new TreeNode(v)` 写穿引用链，直接改到树上——返回过程不需要任何额外动作，这是"引用 + 递归"组合的优雅之处。removeRec 的情形 2 用 tmp 先保存唯一孩子，再 delete node、再 `node = tmp`，顺序不可颠倒（delete 后不能再访问 node 的孩子）；情形 3 不重接指针，只抄值再递归删除后继，逻辑上最省心。clear 的后序顺序是安全释放的前提：先递归释放左右子树，回来时 node 的孩子字段已经不再指向有效内存，随后 delete node 正好收官，最后 `node = nullptr` 防止调用方持有悬垂地址（与链表 destroy 同理）。levelorder 用 std::queue 而非递归：每次出队即访问，左右孩子入队，队列的 FIFO 属性保证严格逐层从左到右。

示例 2：插入顺序决定树形——按序插入退化成链 vs 乱序插入近似平衡

```cpp
#include <iostream>
#include <vector>
#include <algorithm>
#include <random>
using namespace std;

// 极简 BST：只保留插入、求高、销毁三个函数，用于演示"插入顺序决定树形"
struct Node {
    int data;
    Node *left, *right;
    Node(int d) : data(d), left(nullptr), right(nullptr) {}
};

void insert(Node *&root, int v) {
    if (root == nullptr) { root = new Node(v); return; }
    if (v < root->data) insert(root->left, v);
    else                insert(root->right, v);
}

// 树高：约定空树为 -1，单节点树为 0
int height(Node *root) {
    if (root == nullptr) return -1;
    return 1 + max(height(root->left), height(root->right));
}

void destroy(Node *&root) {
    if (root == nullptr) return;
    destroy(root->left);
    destroy(root->right);
    delete root;
    root = nullptr;
}

int main() {
    const int N = 10000;
    vector<int> keys(N);
    for (int i = 0; i < N; ++i) keys[i] = i + 1; // 1..10000 已有序

    // ① 按升序插入 → 每次新值都比根大，一路向右，树退化成"右斜链"
    Node *r1 = nullptr;
    for (int v : keys) insert(r1, v);
    cout << "升序插入 10000 个键的树高 = " << height(r1) << endl; // N-1

    // ② 打乱顺序再插入 → 近似平衡的"灌木状"树
    mt19937 g(20260727);
    shuffle(keys.begin(), keys.end(), g);
    Node *r2 = nullptr;
    for (int v : keys) insert(r2, v);
    cout << "乱序插入 10000 个键的树高 = " << height(r2) << endl; // ~O(log n)

    destroy(r1);
    destroy(r2);
```

```
    return 0;
}
```

【代码做什么】 生成 1 到 10000 的键。第一棵 BST 按升序逐个插入：每个新键都大于当前所有键，递归永远走向右分支，树长成一条 9999 层高的右斜链。第二棵先把同样的键用 mt19937（固定种子 20260727）洗牌，再逐个插入，树形接近随机平衡。程序打印两棵树的高度，直观对比"同样 10000 个键、同样的插入代码"，仅仅插入顺序不同，树高就从 9999 掉到几十层。

【实现机制解说】 这个示例剥掉类的包装，只剩裸函数，凸显树形只由插入顺序决定这一事实。升序情形中每次 insert 都在树的最右端触底，新增节点永远成为最右叶，于是链长 = 键数 − 1，查找/插入退化为与链表相同的 O(n)。乱序情形下新键落在左右两侧的概率大致均等，树向"灌木状"生长，其高度约为对数级别乘一个常数（本机固定种子实测约 27 层，因标准库实现略有差异，但都在几十层的量级，与 9999 判若云泥）。height() 递归返回 `1 + max(左高, 右高)`，空树返回 −1 使单节点树高度恰为 0，这是后序式递归（先孩子后自己）的又一应用。若对真实系统接入有序数据（如按字母序的词典），升序情形就是必然发生的灾难——这正是下一节自平衡 BST 存在的理由。

复杂度分析

操作	最好	平均	最坏	原因
search 查找	O(1)	O(log n)	O(n)	最好：值恰在根/浅层；最坏：树退化成链，深度 n；平均：随机建树高度 O(log n)
insert 插入	O(1)	O(log n)	O(n)	先查找再挂叶子，步数 = 树高；最坏对应链状树
remove 删除	O(1)	O(log n)	O(n)	查找 O(h) + 情形3 的子树递归也 O(h)；最坏仍对应链状树
前/中/后序遍历	O(n)	O(n)	O(n)	每个节点恰好访问一次（与树形无关）
层序遍历	O(n)	O(n)	O(n)	每个节点入队出队各一次；额外空间 O(w)，w 为最大层宽（最坏 O(n)）
空间（存储）	—	—	—	每节点 20 字节（4B int + 2×8B 指针），约为等量数组的 5 倍

说明。 精确写法是 O(h)、h 为树高：对任意 BST 都成立，但没有传达"h 的上限"。课程采用上表的传统写法：只要树是平衡的（高度 O(log n)），三种核心操作就都是 O(log n)；树退化成链则全部 O(n)。这也是为什么生产环境一律使用自平衡 BST——`std::map` / `std::set` 内部的红黑树保证最坏情形也是 O(log n)。

关键要点

- BST 的核心是"左小右大"这条递归规则：对任意节点，左子树全小、右子树全大，查找即"每步排除半棵树"。
- 插入 = 失败的查找：递归下探到空位，靠 `TreeNode *&` 引用把新节点焊回父指针，返回时无需额外接线。

- 删除三分支：无子直接摘、单子孩子顶班、双子用中序后继（右子树最小，或课程惯例的左子树最大）抄值再递归删后继；删任何节点后 BST 性质必须保持。
- 中序遍历 BST = 升序输出；释放整棵树必须用后序（先孩子后自己），否则 delete 后解引用是未定义行为。
- 树形决定命运：按序插入退化成链（O(n)），随机/自平衡才是 O(log n)——自平衡 BST（AVL、红黑，如 std::map/set）把最坏情形也压到对数。

常见陷阱与注意事项

- **把"整棵子树都小"误写成"只跟直接孩子比"**：插入 40 时只检查根 50 的孩子 30 会漏掉 BST 性质。规避：始终递归比较并整棵下探。
- **插入/删除函数忘了传引用**：`insertRec(TreeNode *node, ...)` 内部改的是拷贝，树毫无变化。规避：凡可能改写指针变量的函数一律 `TreeNode *&`。
- **删除两子情形只删值不删后继**：把后继值抄进来却不递归删除原后继节点，会留下重复值。规避：抄值后必须 `removeRec(node->right, succ->data)`。
- **删除单子情形顺序颠倒**：`delete node; node = node->left;` 在 delete 后解引用。规避：先用 tmp 保存唯一孩子，再 delete，再顶班。
- **树退化成链还不自知**：把有序数据直接灌进普通 BST。规避：理解最坏 O(n) 的来源；需要稳定性能就用自平衡结构（std::map/set）。
- **遍历顺序混淆**：前=根左右、中=左根右、后=左右根。规避：背口诀"看根的位置"，并用"中序必升序"自检 BST 遍历实现。
- **前序/中序释放树**：`delete node; 再递归删孩子` 是未定义行为。规避：释放只能用后序（forestFire 模式）。
- **层序用递归实现**：递归天然是深度优先，实现不了"逐层"语义。规避：层序用 std::queue 迭代；递归留给前/中/后序。
- **对空树解引用**：findMin、height 等函数若假定 root 非空，传入空树会段错误。规避：约定并注明前置条件，或先判空。
- **节点内存泄漏**：只 new 不 delete，或析构用错遍历顺序。规避：析构统一后序 clear，空树与单节点都测一遍。

思考题（带答案）

问题 1：为什么 `insertRec` 必须传 `TreeNode *&`，而 `searchRec` 传普通 `TreeNode *` 即可？如果 insertRec 改成传值，会发生什么？ **答案**：insertRec 需要在找到空位时把新节点挂到"父节点的孩子字段（或 root）"上——这是改写指针变量的值，必须用引用才能写穿到树里；searchRec 只沿着指针读值、从不改写任何指针，传值（一份拷贝）足够，还能防止误改。若 insertRec 传值，递归最深一层 `node = new TreeNode(v)` 只改了栈帧里的拷贝，返回后新节点失去所有引用（泄漏），而树纹丝不动——查找时永远找不到新插入的值。

问题 2：删除"有两个孩子"的节点时，为什么方案是"抄后继的值 + 递归删后继"，而不是把后继节点整个搬上来重接指针？为什么后继一定没有左孩子？ **答案**：直接重接指针需要同时处理目标节点与后继节点的左右子树共四个连接，极易出错；抄值方案不动树的骨架，只改一个 int 再递归删一个"结构简单"的节点，正确性容易论证。后继 = 右子树的最左节点，按 BST 性质"最左"意味着它没有左孩子（若有左孩子，那个更小的节点才是后继），所以对它的递归删除必然落入情形 1（叶子）或情形 2（只有右孩子），递归必然终止。

问题 3：给定一棵 BST 的中序序列 `[20, 30, 40, 50, 60, 70, 80]`，能否唯一还原这棵树？为什么课程说"中序对 BST 恒为升序"是重要性质？ **答案**：不能唯一还原——同一中序序列可对应多棵不同形状的 BST（例如以 50 为根的平衡树和以 20 为根的右斜链，中序都是升序）。仅当附加前序或后序（或树的形状信息）时才能唯一重建。中序恒升序的价值在于：它是 BST 正确性的免费自检（遍历结果不是升序，树一定被破坏了），也让"有序遍历容器"成为 BST 的天然卖点。

Lecture 13: 霍夫曼编码（Huffman Coding）（对应课程真实讲座 L23）

概述

本讲把二叉树用于一个经典应用——数据压缩。ASCII 用固定 8 位表示每个字符，但"常用字符与生僻字符一样长"其实是浪费；霍夫曼编码按字符出现频率分配码字：高频字符码字短、低频字符码字长，并保证任何字符的码都不是另一个码的前缀（prefix-free），从而可以唯一解码。构造最优码表的方法是贪心合并：把字符按频率放进最小堆，反复取出两棵最小的树合并，直到只剩一棵——这就是 1952 年 David Huffman 在读研时发明的著名算法。 对应官方讲座：L23（Monday, August 3 — Huffman Coding）；本讲官方页面内容与课程最终作业 A7 的霍夫曼讲义同源（由 Julie Zelenski 执笔，Kenneth Huffman 与 Keith Schwarz 增补），课堂另有 Sean 制作的 Prezi。官方在本讲以 13 字符的示例文本演示：ASCII 需 104 位，自制定长 3 位编码需 39 位，霍夫曼变长编码只需 34 位。

核心概念与算法原理

1. 编码的动机与三种方案。 计算机里一切信息归根结底是比特（bit），字符到比特串的映射就是编码。ASCII 用 8 位编一个字符，能表示 2^8=256 种；Unicode 用 16/32 位支持更多语言文字。定长编码简单（每 8 位一组，解码无需额外信息），但"所有字符一律平等"会浪费空间：英文里 e 到处都是、Z 难得一见，给 e 和 Z 一样长的码显然不划算。早在电报时代，莫尔斯码就懂得给常用字母 e 分配 1 个点、给生僻字母 q 分配 4 划——但它不够系统，有些字母的码长分配并不最优。压缩的本质就是利用这种"不均衡"：给频繁出现的符号短码，给罕见的符号长码。

2. 变长编码的难题与无前缀性质。 变长编码立刻带来一个新问题：比特流没有固定边界，解码时怎么知道一个字符在哪结束？假设 e→"0"、t→"01"，收到比特串 "01" 时既可以读成 e+t（0,1），也可能被误解为 t（01）。解决办法是**前缀性质（prefix-free）**：任何字符的码字都不得是另一个字符码字的前缀。满足该性质时，从比特流开头逐个累积比特，一旦某个累积串正好等于某个字符的码字，它必然是唯一的切分点，解码绝不产生歧义。在"编码树"的视角下，前缀性质等价于一句话：**所有字符都只出现在叶子节点**——若某字符出现在内部节点，它的码就是沿路径到该节点的 0/1 串，恰为更深处字符码的前缀。

3. 编码树：码字即路径。 把编码画成一棵二叉树：从根到每个叶子的路径就是该叶子字符的码字，约定左走记 0、右走记 1。例如：

```
          (根)
         /    \
      0/       \1
     [e]       (内部)
              /    \
           0/       \1
          [t]       [q]
    e 的码 = "0"; t 的码 = "10"; q 的码 = "11"
```

读码表时从根出发：比特 0 向左、1 向右；**走到叶子就输出叶子上的字符，然后回到根继续读下一个字符。**字符只占叶子，路径长短不一正好对应码字长短不一。树的"歪"在这里是优点而不是缺点：高频字符占短路径，低频字符被推入长路径，总比特数反而最小——这与 BST 追求平衡恰好相反（官方特别点出：编码树的不平衡是好事，等频字符才会得到平衡树，而等频意味着没有可压缩的空间）。

4. 解码算法。 拿到一棵编码树与一段比特流，解码是一趟"沿树下行"的旅程：指针 cur 从根开始；读一个比特，是 0 就 cur = cur->left，是 1 就 cur = cur->right；一旦 cur 是叶子，输出其字符并把 cur 重置回根。以下面例子（见第 6 节）的树解码比特串开头的 "0 110 111"：

```
比特 0   → 从左走到 a 叶 → 输出 'a'，回根
比特 110 → 右→右→左 → 输出 'b'，回根
比特 111 → 右→右→右 → 输出 'r'，回根
```

5. 构造最优树（贪心，Huffman 1952）。 给定一段文本，构造"使总编码长度最短"的树： ① 统计每个字符的出现次数（频率），每个字符做成一棵只有单个叶子的树，全体构成"森林"； ② 反复执行：在森林里找权重最小的两棵树，把它们合并成一棵新树——新树是这两棵树的父节点，权重等于二者权重之和； ③ 把新树放回森林，重复 ②，直到森林只剩一棵树，即最终编码树。 "每次只合并当前最小的两棵"正是贪心策略：低频率字符先被合并、被埋得更深（码字更长），高频率字符尽量晚合并、留在浅层（码字更短）。合并必须高效地反复找最小，于是最小堆（优先队列）是天然工具：每轮两次出堆、一次入堆。权重相同的树选哪两棵合并、左右怎么摆，都不影响最优性——平票产生"不同但同样最优"的树（官方强调这一点，编码时只需保证解码用同一棵树即可）。

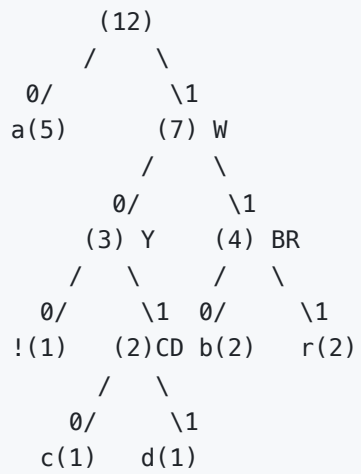
6. 完整手算小例：编码 "abracadabra!"。 文本共 12 个字符，含 6 种不同字符。先统计频率：

字符	a	b	r	c	d	!
出现次数	5	2	2	1	1	1

（1）初始森林为 6 棵单节点树：a(5) b(2) r(2) c(1) d(1) !(1)，括号内是权重。 （2）合并过程（每轮取两个最小）：

```
初始:  a(5)  b(2)  r(2)  c(1)  d(1)  !(1)
第1轮: 合并 c(1) 与 d(1) → CD(2)        森林: a5 b2 r2 CD2 !1
第2轮: 合并 !(1) 与 CD(2) → Y(3)        森林: a5 b2 r2 Y3
第3轮: 合并 b(2) 与 r(2) → BR(4)        森林: a5 Y3 BR4
第4轮: 合并 Y(3) 与 BR(4) → W(7)        森林: a5 W7
第5轮: 合并 a(5) 与 W(7) → 根(12)       森林: 只剩一棵（根权重 = 总字符数 12）
```

（3）最终编码树（左 0 右 1）：



（4）由树读出码表（路径即码字）：a=0、b=110、r=111、c=1010、d=1011、!=100。注意 c/d 被埋到第 4 层，码长 4，正因它们只出现一次；a 出现 5 次独占最短的 1 位码。 （5）编码整句（逐字符拼接码字）：

```
a b  r  a c    a d    a b  r  a !
0 110 111 0 1010 0 1011 0 110 111 0 100
= 01101110101001011101110100    (28 位)
```

（6）压缩率对比：

方案	总比特数	占 ASCII 的比例
ASCII 定长 8 位	12 × 8 = 96	100%
自制定长 3 位（6 种字符需 3 位）	12 × 3 = 36	37.5%
Huffman 变长	28	约 29.2%

Huffman 比自制定长又省了 (36−28)/36 ≈ 22% 的比特。这棵树总比特数 28 是否已是最短？是——Huffman 算法保证了构造出的树对给定频率分布是最优的（这是该算法被引用半个多世纪的原因；课堂上官方另有 13 字符示例：104 → 39 → 34 位，逻辑与本例完全一致）。

7. 展平与文件格式：解码必须拿到同一棵树。 光把比特流发给对方还不够，对方解码需要知道码表/编码树。真实做法是把编码树"展平（flatten）"成一串文本随文件一起发送：前序遍历整棵树，内部节点记为字符 'I'，叶子记为 'L' 加其字符。上面的树展平后为 `ILaIIL!ILcLdILbLr`（17 字节）。解码端先按同样的约定把展平

串"重建"成树，再用它解码比特流。**编码与解码必须使用同一棵树**（官方比喻：没有"秘密解码戒指"，就无法传小纸条）——不同的树（哪怕同样最优）会解出完全不同的乱码。

代码示例与实现详解

示例 1：完整霍夫曼工具——统计频率 → 最小堆建树 → 递归生成码表 → encode / decode

```
#include <iostream>
#include <functional>
#include <map>
#include <queue>
#include <string>
using namespace std;

// 霍夫曼树节点：叶子存字符，内部节点 ch 为 '\0'
struct HuffNode {
    char ch;
    int freq;               // 权重：叶子的频次，或两子树频次之和
    HuffNode *left, *right;
    HuffNode(char c, int f) : ch(c), freq(f), left(nullptr), right(nullptr) {}
    bool isLeaf() const { return left == nullptr && right == nullptr; }
};

// 最小堆比较器：freq 小的优先；同频时按字符比较，让结果可复现
struct Compare {
    bool operator()(HuffNode *a, HuffNode *b) const {
        if (a->freq != b->freq) return a->freq > b->freq;
        return a->ch > b->ch;   // 内部节点 '\0' 比任何字母都"小"
    }
};

void buildCodes(HuffNode *n, const string &prefix, map<char, string> &codes) {
    if (n->isLeaf()) { codes[n->ch] = prefix; return; }
    buildCodes(n->left,  prefix + "0", codes);  // 左子树走 0
    buildCodes(n->right, prefix + "1", codes);  // 右子树走 1
}

string encodeText(const string &text, const map<char, string> &codes) {
    string bits;
    for (char c : text) bits += codes.at(c);
    return bits;
}

string decodeBits(const string &bits, HuffNode *tree) {
    if (tree->isLeaf()) {               // 特例：全文只有一种字符
        return string(bits.size(), tree->ch);
    }
    string out;
    HuffNode *cur = tree;
    for (char bit : bits) {
        cur = (bit == '0') ? cur->left : cur->right;
        if (cur->isLeaf()) {            // 走到叶子：输出字符并回到根
            out += cur->ch;
            cur = tree;
        }
    }
    return out;
}

void freeTree(HuffNode *&n) {           // 后序释放所有节点
```

```
    if (n == nullptr) return;
    freeTree(n->left);
    freeTree(n->right);
    delete n;
    n = nullptr;
}

int main() {
    const string text = "abracadabra!";

    // ① 统计每个字符出现的次数
    map<char, int> freq;
    for (char c : text) ++freq[c];

    // ② 每个字符成为一棵单节点树（叶子），全部放进最小堆
    priority_queue<HuffNode *, vector<HuffNode *>, Compare> pq;
    for (const auto &p : freq) pq.push(new HuffNode(p.first, p.second));

    // ③ 贪心：反复合并两棵最小树，直到只剩一棵（霍夫曼树）
    while (pq.size() > 1) {
        HuffNode *a = pq.top(); pq.pop();
        HuffNode *b = pq.top(); pq.pop();
        HuffNode *parent = new HuffNode('\0', a->freq + b->freq);
        parent->left = a;
        parent->right = b;
        pq.push(parent);
    }
    HuffNode *tree = pq.top();          // 最后一棵树即编码树

    // ④ 递归生成码表（叶子字符 -> 0/1 串）
    map<char, string> codes;
    if (tree->isLeaf()) codes[tree->ch] = "0";   // 单字符文本特殊约定
    else buildCodes(tree, "", codes);

    // ⑤ 编码（此处以 '0'/'1' 字符模拟比特流）
    string bits = encodeText(text, codes);
    cout << "原文(" << text.size() << "字符): " << text << endl;
    cout << "码表: ";
    for (const auto &p : codes)
        cout << p.first << "=" << p.second << "  ";
    cout << endl;
    cout << "编码后比特流(" << bits.size() << " 位): " << bits << endl;

    // ⑥ 解码并校验"往返一致"
    string decoded = decodeBits(bits, tree);
    cout << "解码结果: " << decoded << endl;
    cout << "round-trip 一致? " << (decoded == text ? "是" : "否") << endl;

    // 压缩率统计
    cout << "ASCII 定长 8 位:   " << text.size() * 8 << " 位" << endl;
    cout << "Huffman 变长:      " << bits.size() << " 位"
         << " (原大小的 " << 100.0 * bits.size() / (text.size() * 8) << "%) " << endl;
```

```
    freeTree(tree);
    return 0;
}
```

【代码做什么】 main() 走完整流程：① 用 std::map 统计 "abracadabra!" 各字符频次；② 每字符建一个叶子放入 std::priority_queue（自定义比较器实现最小堆）；③ while 循环反复合并两棵最小树（新父节点权重 = 两子树之和），直到只剩一棵作为霍夫曼树；④ buildCodes 递归遍历树生成码表（左 0 右 1）；⑤ encodeText 逐字符查表拼出比特串（示例以字符 '0'/'1' 模拟比特）；⑥ decodeBits 沿树解码并打印往返校验结果与压缩率。运行输出中码表为 a=0、b=110、r=111、c=1011、d=100、!=1010（与手算例的 c/d/! 码字略有出入，见下），总比特数同为 28——最优值不因平票配对而改变。

【实现机制解说】 代码把第 5 节的算法直接翻译成了指针操作。priority_queue 默认是"大顶堆"，所以 Compare 把比较倒转（`a->freq > b->freq` 表示 freq 小的优先级高）来模拟最小堆；同频时按字符兜底比较，使输出在相同输入下可复现——若去掉这个兜底，平票时堆序未定义，结果仍是最优树但每次运行码表可能不同。合并循环执行 m−1 轮（m 为不同字符数），每轮 `pq.top()+pop()` 两次、`push` 一次，O(log m)；每棵新树都 new 出来，最后 freeTree 以后序顺序释放全部节点（先两个孩子后自己），与 BST 的 forestFire 同理——任何 new 出来的树节点都必须有对应的 delete。buildCodes 是"前序式"递归：到叶子就把当前累积的 0/1 前缀登记为码字，否则分别向左右追加 '0'/'1' 深入。decodeBits 是第 4 节流程的直接实现：cur 沿比特下行，`isLeaf()` 为真即输出并回根；循环结束时若比特流恰好停在叶子则说明数据完整，否则是"截断的码字"（文件损坏）。main 里还特别处理了单字符文本：整棵树只有一个叶子根，码字只能是空串，故约定其码为 "0"，decodeBits 也对该情形做了特判（否则沿空树解引用会崩溃）。

示例 2：把编码树展平存进文件、再重建解码（编码与解码共享同一棵树）

```
#include <iostream>
#include <map>
#include <string>
using namespace std;

struct HNode {
    char ch;
    HNode *left, *right;
    HNode(char c) : ch(c), left(nullptr), right(nullptr) {}
    bool isLeaf() const { return left == nullptr && right == nullptr; }
};

// 前序展平：内部节点记为 'I'，叶子记为 'L' + 字符
// 例：a=0 的整棵树可写成一串 "ILaIIL!ILcLdILbLr"
void flatten(HNode *n, string &out) {
    if (n->isLeaf()) { out += 'L'; out += n->ch; return; }
    out += 'I';
    flatten(n->left, out);
    flatten(n->right, out);
}

// 从展平串重建树：i 以引用方式在整串中推进
HNode *rebuild(const string &s, size_t &i) {
    if (s[i] == 'L') {
        ++i;                        // 跳过 'L'
        HNode *n = new HNode(s[i]);
        ++i;
        return n;
    }
    ++i;                            // 跳过 'I'
    HNode *n = new HNode('\0');     // 内部节点
    n->left = rebuild(s, i);
    n->right = rebuild(s, i);
    return n;
}

void collectCodes(HNode *n, const string &prefix, map<char, string> &codes) {
    if (n->isLeaf()) { codes[n->ch] = prefix; return; }
    collectCodes(n->left, prefix + "0", codes);
    collectCodes(n->right, prefix + "1", codes);
}

string encodeText(const string &text, const map<char, string> &codes) {
    string bits;
    for (char c : text) bits += codes.at(c);
    return bits;
}

string decodeBits(const string &bits, HNode *tree) {
    string out;
    HNode *cur = tree;
    for (char bit : bits) {
        cur = (bit == '0') ? cur->left : cur->right;
```

```
        if (cur->isLeaf()) { out += cur->ch; cur = tree; }
    }
    return out;
}

void destroy(HNode *&n) {
    if (n == nullptr) return;
    destroy(n->left);
    destroy(n->right);
    delete n;
    n = nullptr;
}

int main() {
    // 手工搭出"abracadabra!"的手算最优树（结构见正文图示）：
    // 根(12)= a(5) + 内部(7)；内部(7)= 内部(3) + 内部(4)；
    // 内部(3)= !(1) + 内部(2)；内部(2)= c(1)+d(1)；内部(4)= b(2)+r(2)
    HNode *a = new HNode('a');
    HNode *b = new HNode('b');
    HNode *r = new HNode('r');
    HNode *c = new HNode('c');
    HNode *d = new HNode('d');
    HNode *bang = new HNode('!');
    HNode *cd = new HNode('\0');  cd->left = c;  cd->right = d;
    HNode *br = new HNode('\0');  br->left = b;  br->right = r;
    HNode *y3 = new HNode('\0');  y3->left = bang; y3->right = cd;
    HNode *w7 = new HNode('\0');  w7->left = y3;  w7->right = br;
    HNode *root = new HNode('\0'); root->left = a; root->right = w7;

    // ① 展平：把树"写"成一行字符串（解码端需要它来重建同一棵树）
    string flat;
    flatten(root, flat);
    cout << "展平表示: " << flat << "   (共 " << flat.size() << " 字节) " << endl;

    // ② 重建：读回展平串，得到与原来结构相同的树
    size_t pos = 0;
    HNode *root2 = rebuild(flat, pos);
    cout << "重建树成功? " << (pos == flat.size() ? "是" : "否") << endl;

    // ③ 用重建的树生成码表 → 编码 → 解码（编码/解码必须用同一棵树）
    map<char, string> codes;
    collectCodes(root2, "", codes);
    string bits = encodeText("abracadabra!", codes);
    cout << "编码: " << bits << " (" << bits.size() << " 位) " << endl;
    cout << "解码: " << decodeBits(bits, root2) << endl;

    destroy(root);
    destroy(root2);
    return 0;
}
```

【代码做什么】 main() 手工重建第 6 节手算的最优树（左 0 右 1 的布局与正文图一致），然后：① flatten 前序遍历把树展平成 17 字节字符串 `ILaIIL!ILcLdILbLr`（模拟"写进文件头"）；② rebuild 从该串重建出结构相同的第二棵树，并校验 pos 恰好走到串尾；③ 用重建的树生成码表，编码 "abracadabra!" 得到 28 位比特串 `01101110101001011011011110100`（与手算第 (5) 步完全一致），解码还原原文。

【实现机制解说】 展平采用前序的理由：内部节点总是先于它的两棵子树被写出，重建时读到 'I' 就知道"后面还有两棵子树"并递归消费，读到 'L'+字符即叶子、递归在此终结——前缀式的自包含结构让重建无需额外状态，只靠一个下标 i 在字符串上前进。重建函数把 i 以引用传入，是"递归消费一个输入流"的惯用法：父调用读 'I' 后，两个递归调用会依次吃掉左右子树的全部字符，返回时 i 正好停在子树末尾。为什么必须"编码解码同一棵树"在此一目了然：若重建出的树与编码用的树结构不同（哪怕同样最优），collectCodes 产出的码表不同，decodeBits 沿错误路径行走，遇叶时机错位，输出就是乱码。真实文件格式即"文件头（展平串）+ 正文（比特流）"：打开文件先重建树、再解码正文；示例把比特表示为 '0'/'1' 字符仅为可读性，真实实现应把比特真正打包进字节（每 8 位一个 char），并额外记录末尾不足一字节时的填充位数。

复杂度分析

阶段	时间复杂度	说明
统计频次	O(n)	n = 文本长度，扫一遍即可；空间 O(m)，m = 不同字符数
构建霍夫曼树	O(m log m)	合并恰 m−1 轮；每轮两次出堆 + 一次入堆，各 O(log m)
生成码表	O(m)	递归访问每个节点一次；码字长度不超过树高
编码	O(L)	L = 输出比特数 = Σ(字符频次 × 其码长)，平均码长介于 1 与约 log m 之间
解码	O(L)	每个比特沿树走一步，常数时间
空间	O(m)	堆 + 树节点 + 码表均为 m 量级

关于最好/平均/最坏的说明。 霍夫曼各阶段的运行时间几乎与输入形态无关（合并轮数恒为 m−1，逐比特解码恒定 O(1)/位），没有像 BST 那样随插入顺序剧烈波动的"最坏退化"。真正有最好/最坏之分的是**压缩率**：若所有字符等频，树接近平衡、各码字长度接近，压缩收益最小（等频信息量最大，无从压缩）；若频率悬殊（如一段文本 90% 是同一个字符），少数高频字符拿极短码，压缩率最漂亮。换句话说，算法的时间复杂度稳定，而"省了多少"取决于文本的频率分布——这正是压缩的本质：能压多少，取决于冗余有多少。

关键要点

- 编码 = 给符号分配比特串：定长简单但浪费，变长高效但必须满足**无前缀性质**，否则解码有歧义。
- 编码树里字符只出现在叶子，码字 = 根到叶的路径（左 0 右 1）；解码 = 从根沿比特走，遇叶输出并回根。
- 构造最优树是贪心：统计频率 → 最小堆反复合并两棵最小树 → 根权重等于文本长度（Huffman, 1952）；平票合并出"不同但同样最优"的树。
- 树越"歪"越省：高频字符码字短、低频字符码字长；总比特数 = Σ(频率 × 码长)，对给定分布 Huffman 保证最小。

- 编码与解码必须共用同一棵树：文件 = 展平树（前序 'I'/'L' 串）+ 比特流，解码先重建树；丢了树，比特流就是乱码。

常见陷阱与注意事项

- **码字互为前缀**：如 e="0"、t="01"，解码边界立刻产生歧义。规避：保证字符只放在叶子节点，前缀性质由树的结构自动保证。
- **解码走到叶子不回根**：指针继续下行会解引用叶子的空孩子而崩溃或输出乱码。规避：遇 `isLeaf()` 立即输出并 `cur = tree`。
- **编码/解码用了不同的树**：平票时"另一棵同样最优"的树码表不同。规避：树的展平串必须随文件传输，解码端严格重建同一棵树。
- **priority_queue 比较器写反**：忘了倒转比较，堆变成大顶堆，合并的就不是"最小"两棵，树不再最优。规避：牢记默认是大顶堆，最小堆要 `return a->freq > b->freq`。
- **频次统计遗漏或重复**：码表缺字符会在 `codes.at(c)` 处抛异常。规避：编码前对文本里每个字符逐一验证码表存在；解码端校验最终停在叶子。
- **单字符文本特例**：整棵树只是一个叶子，码字为空串，标准流程会崩。规避：约定单字符码为 "0"（或直接按字符数编码），并让解码特判单叶树。
- **内存泄漏**：每次合并 new 出父节点，用完不释放。规避：后序 freeTree/destroy 释放全部节点，析构或程序末尾调用。
- **把 '0'/'1' 字符当比特**：示例为可读性用字符模拟比特（1 字符 = 1 字节），真实压缩需打包成字节并记录末尾填充位数，否则压缩率无从谈起。
- **展平串与字符集冲突**：若原文恰好含 'I' 或 'L'，朴素展平会与标记混淆。规避：真实实现用转义/位标记区分（本示例演示的字符集不含二者，故简化处理）。
- **以为 Huffman 只能压文本**：它适用于任何"有重复模式"的数据（图像颜色、音频采样等），只要统计出出现频率即可分配码字。

思考题（带答案）

问题 1：为什么变长编码必须满足无前缀性质？如果违反会怎样？请构造一个反例。 **答案**：解码时比特流没有固定边界，解码器只能"累积比特直到匹配某个码字"；若某字符的码是另一个码的前缀，累积过程中会在两种切分之间摇摆不定。例如 e="0"、t="01"：比特串 "01" 既可解为 e+t（0 与 1……但 1 未必是合法码），更直接的是它本身可以是 t 的码——同一串有两种读法即歧义。而若所有码互不为前缀，累积匹配成功的那一瞬间就是唯一正确的切分点。编码树中"字符只放叶子"恰好保证这一点。

问题 2：构造霍夫曼树时为什么用最小堆，而不是每次在数组里线性扫描找两个最小？合并策略"每次选最小两棵"为什么是最优的（直觉即可）？ **答案**：m 棵树的森林要合并 m−1 次，若每次线性扫描找最小两棵，总代价 $O(m^2)$；用最小堆每次出堆/入堆 $O(\log m)$，总代价 $O(m \log m)$，对大字母表差距巨大。最优性的直觉：编码树中一个字符的码长等于它在树中的深度，合并得越晚、离根越近；把低频率字符尽早合并（埋深）等于"主动让罕见字符付长码"，而高频率字符留在浅层"享受短码"——总比特数 Σ(频率×深度) 因此最小。这是贪心正确性的经典案例：每一步局部最优（最小两棵先合）累积出全局最优。

问题 3：若待压缩文本只有一种字符（如 1000 个 'a'），霍夫曼流程会发生什么？压缩率如何？这暴露了文件格式设计的什么要点？ **答案**：统计后只有一种字符，森林里只有一棵叶子树，循环一次都不执行，编码树退化为单个叶子：它的码字是空串，需要特殊约定（如示例中的 "0"）。此时正文只需 1000 位（甚至可退化为"长度 + 单字符"的零比特表示），相对 ASCII 的 8000 位是极端压缩。但要点在于：解码端仍需要知道"字符是 a"这一信息，即文件头（展平树）本身也要占字节——对超短文本，文件头开销可能超过正文节省，压缩反而得不偿失。这提醒我们：任何压缩格式都必须把"树/码表 + 数据"一起打包，而压缩是否划算要连同头部开销一起衡量。

Lecture 14: 散列与哈希表（Hashing & Hash Tables）（对应课程真实讲座 L24）

概述

本讲以“高效存储并检索 17 000 名学生记录（每名学生的学号唯一、8 位数字）”为驱动问题，逐一权衡了巨型直接索引数组、排序 + 二分、平衡树等方案后，隆重推出本季“最惊艳的数据结构”——哈希表（hash table）：一个普通数组加上一个哈希函数（hash function），就能在平均 O(1) 时间内完成插入、查找、删除，却只占用 O(n) 空间。全讲围绕两条主线展开：一是两种冲突消解策略——线性探测（linear probing）与分离链（separate chaining）——的机制、运行时间与工程细节（装填因子、聚类、墓碑）；二是好哈希函数应具备的性质，以及 std::unordered_set / unordered_map 的用法。官方对应：L24（2026 年 8 月 4 日，周二，Hashing；另附 Stanford HashSet / HashMap 文档供参考）。

核心概念与算法原理

问题定义：按学号存取学生记录。 假设要为 17 000 名在校生维护记录，学号是 00000000–99999999 之间的 8 位数字，要求“查得快、存得省”。官方课上对比了四类方案，暴露了一个贯穿全季的经典权衡——**用空间换时间，或用时间换空间**：

1. **巨型直接索引数组**：开一个长度 1 亿的数组，把学号直接当下标。插入/查找/删除都是铁打的 O(1)，但 17 000 条记录要占 1 亿个槽位，浪费到离谱。
2. **合理大小的有序数组 + 二分查找**：只存 17 000 个元素，排序后二分。空间省了，但建表要先花 O(n log n) 排序，查找退化为 O(log n)，中途插入新学生还要 O(n) 挪动。
3. **平衡二叉搜索树**：插入/查找最坏 O(log n)、空间 O(n)。比有序数组灵活（随时插入），但每个节点带指针，常数更大，且永远够不到 O(1)。
4. **哈希表**：数组 + 哈希函数。平均 O(1) 增删查、O(n) 空间——“两条方案各自最好的部分”被拼到了一起。

官方还顺带提了一句“每个节点有 10 个孩子、沿学号逐位下行”的数字树，这正是 trie（字典树）的思想；本季不展开（见 Lecture 17 延伸专题）。

哈希表是什么？ 哈希表 = 一个定长数组（称桶数组 / bucket array），外加一个哈希函数 h。插入键 key 时，先算 `h(key)` 得到一个很大的整数（哈希码 hash code），再对它取模 `% 表长` 得到合法下标，把键存进那个槽位：

```
键(key) ──哈希函数──▶ 哈希码(整数，可能很大) ──% 表长──▶ 桶下标
"apple"    h("apple")=…                          … % 8 = 3    → 存到下标 3
```

取模这一步**必须由使用者自己完成**——课堂明确提醒：哈希函数通常返回很大的数，任何拿它当数组下标的用法都要先对表长取模。不同键可能算到同一个下标，这就叫**冲突（collision）**；冲突是不可避免的（键的个数远超下标个数，鸽笼原理），于是整讲都在讨论“冲突发生后怎么办”。

策略一：线性探测（linear probing，开放寻址族）。 冲突时不去抢同一个槽，而是**沿着数组向后一个槽一个槽地找空位**；走到末尾就绕回开头（用 `(下标 + i) % 表长` 实现环形扫描）。下图以表长 7、四个“哈希值都对 7 取模等于 5”的键为例：

```
键      下标 = 键 % 7          插入过程（下标：0 1 2 3 4 5 6）
 5        5      插 5  →   [ ][ ][ ][ ][ ][5][ ]
12        5      插 12 →   [ ][ ][ ][ ][ ][5][12]    5 被占，探测到 6
19        5      插 19 →   [19][ ][ ][ ][ ][5][12]    5、6 都占，绕回 0
26        5      插 26 →   [19][26][ ][ ][ ][5][12]   5→6→0→1，最终落到 1
```

由此引出两个重要现象：

- **聚类（primary clustering）**：冲突键挤成一串连续占用的块，块越长，后面任何键探测时越容易被“路过”而放得更远，块又更大——恶性循环。缓解手段：① 把表维持得“比较空”（官方给出的经验是占用 25%—50%，宁可浪费一点空间）；② 表长取质数，减少取模后下标分布的周期性规律。
- **删除要用“墓碑”（tombstone），不能直接清空**：查找是沿着探测序列走到空位才停的。若把被删槽直接标成 EMPTY，就会把探测链拦腰截断，使**排在它后面的键再也找不到**（见下图）。正确做法是把槽标记为“已删但曾经用过”（墓碑 T），查找时**跳过墓碑继续走**，插入时则**优先复用最靠前的墓碑槽**：

```
插入 5、12、19 后（表长 7）：下标 5 放 5，6 放 12，0 放 19
删除 12 时——
  若把下标 6 置 EMPTY：找 19 从 5 出发，到 6 遇 EMPTY 即停 → 误判“不存在” ✗
  若把下标 6 置墓碑 T ：找 19 从 5 出发，跳过 6 继续 → 在 0 处找到 ✓
    [19][ ][ ][ ][ ][ 5][T]
```

策略二：分离链（separate chaining）。 让数组的每个槽不是直接放元素，而是放一条链表（桶 bucket）的头指针；冲突的键全部挂进同一条链表，**根本不需要探测**：

```
桶数组（每个槽是链表头）
 [0] → nullptr
 [1] → "apple" → nullptr
 [2] → nullptr
 [3] → "grape" → "peach" → "plum" → nullptr      ← 三个键冲突，串成一条链
 [4] → nullptr
```

新元素通常**头插**，官方给出两个理由：头插无需维护尾指针、天然 O(1)；且“最近访问的元素更可能再次被访问”，把它放链表头部对后续查询友好。分离链的性能由**装填因子（load factor）= n / b**（元素总数 ÷ 桶

数，即每条链的平均长度）决定：负载太高链就长，太低则空桶过多浪费空间。**只要负载保持小常数，期望每次查询只需看常数个元素——平均 O(1) 由此而来。**

好哈希函数四条性质（官方课末总结，设计细节超纲但性质必须懂）：

1. **确定性（deterministic）**：同一输入必须永远得到同一哈希码，否则插入时算到下标 3、查找时却算出下标 8，记录就“人间蒸发”了。
2. **输入均匀时输出也要均匀**：若 70% 的键都落进同一个桶，插入 n 个键的总代价会退化到 $O(n^2)$；反之，均匀散开时各桶都很短。
3. **输出范围要大**：若哈希函数只产出 0–9，配一个 10 000 长的表也只会用前 10 个槽，等于人为制造海量冲突。
4. **相似输入要给出差异大的哈希码**：真实数据常常成堆出现（如连续学号），把它们打散能避免在表里制造聚集。

复杂度表述要谨慎（官方特别提醒）。 不加修饰地宣称“哈希表是 O(1)”是业内常见惯例，但它隐含两个前提：好的哈希函数 + 均匀分布的输入（平均情形），以及**哈希函数本身是 O(1)**。字符串哈希通常要遍历全部 k 个字符，代价是 O(k)——所以对字符串键而言，一次“O(1)”操作的完整成本其实是 O(k)。最坏情形（所有键撞进同一桶 / 全部聚成一团）下，插入、查找、删除都是 O(n)。

标准库对应物。 std::unordered_set / std::unordered_map 正是“平均 O(1) 增删查、迭代无序”的哈希容器（标准库具体用哪种冲突策略是实现细节，常见实现仍是链式分桶）。它要求键类型可哈希（提供 `std::hash<T>` 特化或自定义哈希函数对象）且可判等（`operator==`）；作为交换，遍历时元素**不保证有序**——这与第 3 讲基于平衡树的 std::set / std::map（有序、O(log n)）形成鲜明对照。

代码示例与实现详解

示例 1：分离链 HashSet\<string>（含 rehash）。

```cpp
#include <iostream>
#include <list>
#include <string>
#include <utility>
#include <vector>
using namespace std;

class ChainedHashSet {
public:
    explicit ChainedHashSet(size_t bucketCount = 8)
        : buckets_(bucketCount), size_(0) {}

    // 插入：重复键返回 false；否则头插并视负载因子决定是否扩容
    bool insert(const string& key) {
        size_t idx = hashIndex(key);
        for (const string& s : buckets_[idx]) {
            if (s == key) return false;            // 哈希集合不允许重复
        }
        buckets_[idx].push_front(key);             // 头插：O(1)
        ++size_;
        if (size_ > buckets_.size() * 0.75) rehash();   // 负载 > 0.75 → 扩容
        return true;
    }

    bool contains(const string& key) const {
        for (const string& s : buckets_[hashIndex(key)]) {
            if (s == key) return true;             // 链很短时近似 O(1)
        }
        return false;
    }

    bool erase(const string& key) {
        auto& chain = buckets_[hashIndex(key)];
        for (auto it = chain.begin(); it != chain.end(); ++it) {
            if (*it == key) { chain.erase(it); --size_; return true; }
        }
        return false;
    }

    size_t size() const { return size_; }

    void dump() const {
        for (size_t i = 0; i < buckets_.size(); ++i) {
            cout << "桶[" << i << "]:";
            for (const string& s : buckets_[i]) cout << " -> " << s;
            cout << "\n";
        }
    }

private:
    // 经典的“乘 31 累加”字符串哈希（类似 Java String.hashCode 的思路）
    size_t hashIndex(const string& key) const {
        size_t h = 0;
```

```
        for (char c : key) h = h * 31 + static_cast<unsigned char>(c);
        return h % buckets_.size();                // 取模落桶
    }

    void rehash() {
        vector<list<string>> old = std::move(buckets_);
        buckets_.assign(old.size() * 2, list<string>());  // 桶数翻倍
        size_ = 0;                                 // 计数清零后整体重插
        for (auto& chain : old)
            for (auto& s : chain) insert(s);
    }

    vector<list<string>> buckets_;   // 桶数组：每桶一条链表
    size_t size_;                    // 元素总数
};

int main() {
    ChainedHashSet s;
    for (const string& w : {"apple", "banana", "pear", "grape",
                            "plum", "peach", "kiwi", "melon",
                            "fig", "date"}) {      // 第 9 个元素会触发 rehash
        cout << "插入 " << w << (s.insert(w) ? " 成功" : " 重复") << "\n";
    }
    cout << "再次插入 apple -> " << (s.insert("apple") ? "成功" : "拒绝重复") << "\n";
    cout << "contains(pear) = " << s.contains("pear") << "\n";
    cout << "contains(kiwi) = " << s.contains("kiwi") << "\n";
    cout << "erase(pear)    = " << s.erase("pear") << "\n";
    cout << "erase(pear)    = " << s.erase("pear") << "（第二次已删不到）\n";
    cout << "当前 size = " << s.size() << "\n\n";
    s.dump();
    return 0;
}
```

【代码做什么】 main 依次插入 10 个单词（初始 8 桶、rehash 阈值 0.75：前 6 次插入后 size=6 未超限，**第 7 次插入 kiwi 时 size=7 > 8×0.75，触发 rehash、桶数翻到 16**），随后演示查重拒绝、contains、erase（含删除不存在的键返回 false），最后 dump 出每个桶的链表内容，直观看到元素如何被“打散”进不同桶。真实输出片段：

```
插入 apple 成功
...
插入 kiwi 成功     ← 此处发生 rehash（8 桶 → 16 桶）
插入 melon 成功
...
再次插入 apple -> 拒绝重复
contains(pear) = 1
erase(pear)    = 1
当前 size = 9
桶[0]: -> kiwi
桶[1]: -> peach
桶[4]: -> fig -> plum     ← 冲突的两个键共享一条链
桶[10]: -> apple
```

【实现机制解说】 ① `hashIndex` 用“乘 31 累加”把任意长字符串压成一个 size_t，再对 `buckets_.size()` 取模——无论表多大都得到合法下标；`static_cast<unsigned char>` 保证负 char 值不捣乱。② 冲突键全部进同一条 `std::list`，链表天然支持 O(1) 头插与任意位置删除（erase 前先线性扫一遍查重，符合集合语义）。③ **rehash 是哈希表唯一的“重活”**：桶数翻倍后，原来的取模结果几乎全部失效，必须把每个元素重新哈希、重新入桶，代价 O(n)；但因为每次扩容都翻倍，摊到 n 次插入上平均仍是 O(1)（与第 1 讲 vector 的扩容如出一辙）。④ 扩容后负载约为原来一半，保证负载因子长期在阈值以下。

示例 2：线性探测版（整数集合，含墓碑处理）。

```cpp
#include <iostream>
#include <string>
#include <vector>
using namespace std;

// 槽位三种状态：EMPTY 从未用过 / USED 在用 / TOMBSTONE 已删但曾用过
enum class Slot { EMPTY, USED, TOMBSTONE };

class LinearHashSet {
public:
    explicit LinearHashSet(size_t cap = 7)
        : status_(cap, Slot::EMPTY), values_(cap, 0), size_(0) {}

    bool insert(int key) {
        if (contains(key)) return false;           // 先查重
        int tomb = -1;                             // 探测途中遇到的最靠前墓碑
        size_t start = hashIndex(key);
        for (size_t step = 0; step < status_.size(); ++step) {
            size_t pos = (start + step) % status_.size();   // 环形探测
            if (status_[pos] == Slot::TOMBSTONE && tomb < 0) tomb = (int)pos;
            if (status_[pos] == Slot::EMPTY) {     // 探测链终点
                size_t target = tomb < 0 ? pos : (size_t)tomb; // 优先复用墓碑
                status_[target] = Slot::USED; values_[target] = key;
                ++size_;
                return true;
            }
        }
        return false;                              // 表满（本例容量大于元素数）
    }

    bool contains(int key) const {
        size_t start = hashIndex(key);
        for (size_t step = 0; step < status_.size(); ++step) {
            size_t pos = (start + step) % status_.size();
            if (status_[pos] == Slot::EMPTY) return false;  // 空位 = 探测链尽头
            if (status_[pos] == Slot::USED && values_[pos] == key) return true;
            // TOMBSTONE：绝不能停，必须继续向后探测
        }
        return false;
    }

    bool erase(int key) {
        size_t start = hashIndex(key);
        for (size_t step = 0; step < status_.size(); ++step) {
            size_t pos = (start + step) % status_.size();
            if (status_[pos] == Slot::EMPTY) return false;
            if (status_[pos] == Slot::USED && values_[pos] == key) {
                status_[pos] = Slot::TOMBSTONE;    // 打墓碑，而不是置 EMPTY!
                --size_;
                return true;
            }
        }
        return false;
```

```
    }

    size_t size() const { return size_; }

    void dump() const {
        for (size_t i = 0; i < status_.size(); ++i) {
            char tag = status_[i] == Slot::USED ? 'U'
                     : status_[i] == Slot::TOMBSTONE ? 'T' : 'E';
            cout << "下标 " << i << " [" << tag << "]";
            if (status_[i] == Slot::USED) cout << "  值=" << values_[i];
            cout << "\n";
        }
    }

private:
    size_t hashIndex(int key) const {
        size_t h = static_cast<size_t>(key);  // 先转无符号再取模，杜绝负下标
        return h % status_.size();
    }
    vector<Slot> status_;
    vector<int>  values_;
    size_t size_;
};

int main() {
    LinearHashSet t(7);
    // 5、12、19、26 对 7 取模都等于 5 —— 故意制造连环冲突
    for (int k : {5, 12, 19, 26, 3, 0})
        cout << "插入 " << k << (t.insert(k) ? " 成功" : " 失败") << "\n";
    cout << "\n删除 12 后：\n";
    t.erase(12);
    t.dump();
    cout << "\ncontains(19) = " << t.contains(19)
         << "  ← 跨过墓碑 T 仍能找到\n";
    cout << "contains(12) = " << t.contains(12)
         << "  ← 已删，找不到\n";
    return 0;
}
```

【代码做什么】 表长固定 7，前四个键全“撞”在下标 5，被迫依次探到 6、绕回 0、1（与核心概念部分的图示完全对应）；随后 dump 打印每个槽的状态，接着删除 12（位于探测链中段），再验证 19 依然能被找到——直观演示“墓碑必须被跳过而非终止探测”。

【实现机制解说】 ① 用独立的 `status_` 数组与 `values_` 数组并行存储，状态与数据分离，EMPTY/TOMBSTONE 槽无需“假值”占位。② 删除只改状态不改值：置 TOMBSTONE 后，查找照常把它当“占用过的位置”跨过去，直到遇见真正的 EMPTY 才判定不存在；插入时记录探测路上第一个墓碑，把新键**复用**到那个槽，避免表里墓碑越积越多。③ 三个操作统一用 `(start + step) % 容量` 的环形下标，天然实现“绕回表头”。④ 对比示例 1 可见：分离链的删除是“从链表摘下节点、空间即刻回收”，而开放寻址的删除是“打标记、空间延迟复用”——这是两类策略最本质的工程差异。

示例 3：std::unordered_set 实战——twoSum（官方第 24 讲的“超级重要练习”）。

```cpp
#include <iostream>
#include <unordered_set>
#include <vector>
using namespace std;

// 判断数组中是否存在两个不同位置的数，其和等于 target
bool twoSum(const vector<int>& v, int target) {
    unordered_set<int> seen;             // 记录已扫描过的值
    for (int x : v) {
        if (seen.count(target - x)) return true;  // 补数已在前面出现过?
        seen.insert(x);                  // 先查后插：避免把 x 自己当配对
    }
    return false;
}

int main() {
    vector<int> v = {5, 1, 3, 1, 9};
    cout << "target=8  -> " << twoSum(v, 8) << "  (5+3)\n";
    cout << "target=2  -> " << twoSum(v, 2) << "  (两个 1)\n";
    cout << "target=9  -> " << twoSum(v, 9) << "  (不能只用单个 9)\n";
    cout << "target=18 -> " << twoSum(v, 18) << " (只有一个 9)\n";
    return 0;
}
```

【代码做什么】 一次线性扫描，边走边把见过的数存进 `unordered_set`；对当前数 x，只查“补数 target − x 是否已在集合里”。官方把它列为第 24 讲的“超级重要练习”：朴素双层循环是 $O(n^2)$，而哈希版是 O(n)——正好复习“哈希表 = 快速查重”这一核心用途。

【实现机制解说】 ① 为什么“先查后插”而不是“先插后查”？若先把 x 插进去再查补数，当 `target == 2x` 时会拿同一个元素自己配自己（如单元素 {9}、target 18 会被误判成功）。② 只关心“是否出现”时用 `unordered_set` 就够；若还要返回下标，换 `unordered_map<int,int>`（值→下标）即可。③ 若想给自定义结构体（如学生记录）当键，需特化 `std::hash<T>` 或给容器传自定义哈希函数对象，并保证 `operator==` 与哈希一致——这是 STL 哈希容器的“入场券”。

复杂度分析

设 n 为元素总数、b 为桶数（分离链）或表长（线性探测）。以下“平均”均基于好哈希函数 + 均匀输入 + 负载保持小常数的前提。

操作	分离链 平均	分离链 最坏	线性探测 平均	线性探测 最坏
插入	O(1)	O(n)*	O(1)	O(n)
查找	O(1)	O(n)	O(1)	O(n)
删除	O(1)	O(n)	O(1)	O(n)
空间	O(n)	O(n)	O(n)	O(n)

*最坏插入 O(n) 是因为集合要查重：若 n 个键全落进同一条链，插入前的查重就要扫整条链；若允许重复则最坏可降到 O(1)。线性探测的最坏情形是表几乎满、全部元素聚成一个大簇时，要探遍全表才找到空位。

原因简述：平均情形下每条链 / 每次探测的期望长度是负载因子（小常数），故平均 O(1)；最坏情形（糟糕的哈希函数 + 恶意输入）所有键挤进同一桶或同一大簇，操作退化为 O(n)。两个补充点：① 字符串键的哈希本身 O(k)（k = 串长），实际开销应写作 O(k)；② 单次 rehash 代价 O(n)，但桶数翻倍使 n 次插入的总代价仍为 O(n)，均摊 O(1)。官方还给出经验值：线性探测把表维持在 25%–50% 占用可显著压低昂贵探测的概率；分离链则靠“负载超阈值就 rehash”把链长钉在小常数上。

关键要点

- 哈希表 = 数组 + 哈希函数 + 冲突消解策略，平均 O(1) 增删查、O(n) 空间，是“用一点空间换回常数时间”的典范。
- 冲突不可避免：线性探测靠向后找空位（删除须用墓碑）、分离链靠同桶挂链表（头插 O(1)），两策各有取舍。
- 一切平均 O(1) 的承诺都建立在“好哈希函数 + 负载不过高”之上：哈希要确定性、均匀、范围大、相似输入差异大。
- 负载因子是哈希表的“健康指标”：链式结构超阈值就 rehash 翻倍（均摊 O(1)），开放寻址则宜让表保持 25%–50% 空闲。
- std::unordered_set/map 给出平均 O(1) 操作但遍历无序；要有序遍历请回到第 3 讲的 std::set/map。

常见陷阱与注意事项

- **哈希函数不确定**（例如掺入随机数）：同一键插入、查找算出的下标不同，记录“失踪”。规避：哈希必须纯函数。
- **取模得负下标**：C++ 对负数取模结果为负（Python 不会），哈希码溢出变负后 `% 表长` 仍是负值，越界访问直接崩。规避：先转成无符号类型（如 `static_cast<size_t>`）再取模；也不要迷信 `abs()`（`INT_MIN` 的绝对值仍是负数）。
- **线性探测删除后直接置空**：探测链被截断，后续键查不到。规避：一律打墓碑，且插入优先复用墓碑槽。
- **把“平均 O(1)”当成“绝对 O(1)”**：坏哈希 + 全撞一桶时是 O(n)，插 n 个键可到 $O(n^2)$；字符串哈希本身也是 O(k)。规避：理解最坏情形存在，并选均匀的哈希。
- **表塞得太满才想起扩容**：开放寻址表接近满时，单次插入要探过几乎整张表。规避：像 vector 一样提前翻倍扩容，让占用率长期低于阈值。
- **负载因子过低也不健康**：几百个元素配几百万个桶，空桶链头指针本身也是内存。规避：扩容/缩容策略兼顾空间与时间。
- **unordered 容器用于自定义类型不写哈希与相等**：编译报错或行为错误。规避：特化 `std::hash` 或传入哈希函数对象，并保证 `operator==` 与哈希逻辑自洽。

思考题（带答案）

问题 1：分离链的插入最坏为什么是 O(n)？如果允许集合里出现重复元素，最坏会变成多少？ **答案**：因为集合不允许重复，插入前必须先在目标桶的链表里查重；若 n 个键全撞进同一桶，查重就要扫整条 O(n) 的链。若允许重复、直接头插不做查重，最坏可降回 O(1)。这说明“集合语义的查重”本身是插入代价的一部分。

问题 2：线性探测查找时遇到 TOMBSTONE 为什么必须继续走、遇到 EMPTY 却可以立刻停？ **答案**：插入采用“沿探测序列找第一个空位”的规则，因此任何“曾经被占用过的位置”（USED 或 TOMBSTONE）都可能是某键探测链的中间站；只有 EMPTY 才是所有探测链的公认终点，遇到它即可断定“后面不可能再有该键”。墓碑若被当作终点，排在它后面的键就永远查不到了。

问题 3：为什么 rehash 时桶数通常翻倍而不是只加一两个？ **答案**：rehash 要把全部 n 个元素重新哈希入桶，代价 O(n)；若每次只小幅扩容，插入 n 个元素可能触发 O(n) 次 rehash，总代价退化到 $O(n^2)$。翻倍扩容使 rehash 次数只有 O(log n) 次，n 次插入总代价仍为 O(n)，均摊 O(1)——和 vector 扩容是同一个道理。

Lecture 15: 图：概念、表示、DFS/BFS 与拓扑排序（Graphs: Concepts, Representations, Traversals & Topological Sort）（对应课程真实讲座 L25）

概述

链表与树之后，本讲迎来本季最“万金油”的数据结构——图（graph）：由顶点（vertex）与连接顶点的边（edge）构成，凡是“一堆东西之间存在两两关系”的问题几乎都能建模成图，如社交网络、路网、课程先修、网页链接。全讲覆盖三块：图的基本术语与“口味”（有向/无向、带权/无权、连通性）、三种表示法（邻接矩阵、邻接表、边表）的取舍，以及两种遍历（DFS、BFS）与只适用于有向无环图的拓扑排序（topological sort）。官方对应：L25（2026 年 8 月 5 日，周三，Graphs；本讲内容被官方列为期末考试重点，而 A6 之后没有图作业，主要靠练习与讲义）。

核心概念与算法原理

什么是图？ 图是“节点式（linked）结构”：一堆顶点 + 连接它们的边。它和链表、树同宗同源——链表可以看作“每条边只指向下一个节点”的特殊图，树是“无环、有单一根、边有层级方向”的特殊图。但图挣脱了两重束缚：**图不一定只有一个入口**（链表有头节点、树有根节点，图可以有很多起点）；**图的关系不必有顺序或层级**（可以不是 next/child 关系），而且**可以有环**（cycle）——这是树严格禁止的。顶点可以带编号，边可以带权重（weight，如距离、费用、耗时），可以带方向。

术语表（官方明确：期末考试可能直接考术语，务必熟记）：

- **路径（path）**：一串顶点，相邻两个之间都有边相连；路径长度 = 经过的边数。
- **环/回路（cycle）**：起点与终点相同的路径；**自环（loop）** 是一条从某顶点指向它自己的边。
- **相邻/邻居（adjacent/neighbor）**：两个顶点之间有边。

- **可达（reachable）**：存在从 A 到 B 的路径，则称 B 从 A 可达。
- **连通（connected）**：无向图中任意两顶点互相可达；否则称不连通（disconnected）。有向图中“任意两顶点互相可达”则称**强连通（strongly connected）**。
- **完全图（complete graph）**：任意两顶点之间都有边。
- **稠密（dense）与稀疏（sparse）**：边的数量相对于可能的最大边数（n 个顶点最多约 $n^2/2$ 条无向边）而言是“很多”还是“很少”。
- **有向（directed）**：边是单行箭头；**无向（undirected）**：边双向通行。
- **带权（weighted）**：边上带数值；**无权（unweighted）**：只有“有没有边”。

三种表示法对比。设 n 为顶点数、E 为边数、deg(v) 为顶点 v 的度数（邻居个数）：

表示法	空间	查“u、v 是否相邻”	列 u 的所有邻居	适用场景
邻接矩阵 adjacency matrix（n×n 二维数组，格子存 0/1 或权重）	$O(n^2)$	O(1)（直接看格子）	O(n)（扫一整行）	稠密图；查边极频繁
邻接表 adjacency list（每顶点一条邻居列表）	O(n + E)（稀疏图近似 O(n)）	O(deg)（需在列表里找）	O(deg(v))	稀疏图；遍历邻居极频繁
边表 edge list（把所有 (u,v,w) 三元组存进一个列表）	O(E)	O(E)	O(E)	需要按边整体处理的算法（如 Kruskal 求最小生成树）

邻接矩阵还有个“对称浪费”：无向图的矩阵沿主对角线对称，每条边被存了两遍。官方的结论一句话：**稀疏图用邻接表，稠密图用邻接矩阵**；而遍历类算法（DFS/BFS/最短路径）几乎总在“列邻居”，所以邻接表是图算法的主场。官方课程自带 Stanford Graph 类（BasicGraph），本笔记一律用标准库自行实现同一思想。

最小生成树（MST）一句话带过：在带权无向图中找一棵连接全部顶点、总边权最小的树，经典算法有 Prim 与 Kruskal；官方说明本季不考，留作面试与 CS161 储备。

无权图最短路径一句话带过：BFS 天然给出无权图中“边数最少 / 换乘最少”的路径（逐层推进保证首次到达即最短）；而 DFS 只保证“若存在路径则能找到一条”，不保证最短（详细论证见 L26 之后的最短路径章节）。

深度优先搜索（DFS）。递归版逻辑极简：访问当前顶点并打上 visited 标记，然后依次对每个未访问的邻居递归深入——撞到死胡同就回溯。正是“一条道走到黑、撞墙再回头”。也可以用显式栈写成迭代版，效果等价。visited 标记不可或缺：图有环，不标记就会在环里无限打转。下图是示例图的 DFS 访问顺序（从 0 出发，按邻居顺序）：

```
DFS 访问序（数字是第几步）：
     0
    / \
   1   2        0(1) → 1(2) → 3(3) → 2(4) → 4(5) → 5(6)
   |   | \      输出：0 1 3 2 4 5
   3 – 2  4
    \     |
     \    5
```

广度优先搜索（BFS）。用队列逐层扩散：先把起点入队并标记；每次出队一个顶点 u，把 u 的所有未访问邻居标记并入队。因为严格按“层”推进，第一次访问到某个顶点时的路径就是无权图最短路径。下图是同一张图的 BFS 分层示意（从 0 出发）：

```
第 0 层：0
第 1 层：1，2          （0 的邻居）
第 2 层：3，4          （1、2 的邻居，且未访问）
第 3 层：5             （3、4 的邻居）
BFS 输出：0 1 2 3 4 5
```

拓扑排序（topological sort）。问题：有向图中，给所有顶点排一个线性顺序，要求“若有边 u→v（u 是 v 的前置/先修），u 必须排在 v 前面”。经典应用：课程表（边 = “x 是 y 的先修课”）、任务依赖（“先买面粉才能烤饼干”）、编译依赖。要点：

- 拓扑序**不必是图中的一条路径**——它只是把所有“箭头”整理成“一律朝右”的排列。
- **只有有向无环图（DAG）才有拓扑序**：一旦有环（如 A 依赖 B、B 又依赖 A），就永远排不出谁先谁后。官方期末考试口径明确：**有环 ⇔ 无拓扑序**（两者互为充要）。
- 合法拓扑序通常不止一个（只要满足所有前置约束即可），Kahn 算法只输出其中某一个。
- 实现思路有二：**Kahn 入度法**（本讲代码采用）：不断挑出“入度为 0 = 无未满足前置”的顶点输出，并抹掉它发出的边（把邻居入度减 1），周而复始；若最终输出的顶点数少于总数，说明图中存在环。另一种是**DFS 完成序倒序**：递归 DFS 记录每个顶点“访问完毕”的顺序，逆序即一个拓扑序（补充材料提及，本季不作要求）。

Kahn 过程示意（课程先修图：A 是 B、C 的先修，B、C 是 D 的先修，C 还是 E 的先修）：

```
初始入度：A:0  B:1  C:1  D:2  E:1
出队 A   → 输出 A，B、C 入度减 1 → 都变 0，入队
出队 B   → 输出 B，D 入度 2→1
出队 C   → 输出 C，D 入度 1→0、E 入度 1→0，D、E 入队
出队 D、E → 输出 D、E
拓扑序：A B C D E    （若图中存在环，队列会提前耗尽、输出不满 n 个）
```

代码示例与实现详解

示例 1：邻接表建图 + 递归 DFS + 队列 BFS。（用 `vector<vector<int>>` 直接当邻接表，顶点编号 0..n−1；如需带权或标签，可换成 `struct Edge { int to; int weight; };` 的向量。）

```cpp
#include <iostream>
#include <queue>
#include <vector>
using namespace std;

// 无权无向图：adj[u] 是 u 的所有邻居
struct Graph {
    int n;                          // 顶点数
    vector<vector<int>> adj;        // 邻接表
    explicit Graph(int vertices) : n(vertices), adj(vertices) {}

    void addUndirectedEdge(int u, int v) {
        adj[u].push_back(v);
        adj[v].push_back(u);        // 无向边 = 两条有向边
    }
};

// DFS（递归）：访问 u，再逐个深入未访问的邻居
void dfs(int u, const Graph& g, vector<bool>& visited) {
    visited[u] = true;
    cout << u << " ";
    for (int v : g.adj[u]) {
        if (!visited[v]) dfs(v, g, visited);
    }
}

// BFS（队列）：从 start 逐层扩散
void bfs(int start, const Graph& g) {
    vector<bool> visited(g.n, false);
    queue<int> q;
    visited[start] = true;          // 入队即标记，防止重复入队
    q.push(start);
    while (!q.empty()) {
        int u = q.front();
        q.pop();
        cout << u << " ";
        for (int v : g.adj[u]) {
            if (!visited[v]) {
                visited[v] = true;
                q.push(v);
            }
        }
    }
}

int main() {
    Graph g(6);
    g.addUndirectedEdge(0, 1);
    g.addUndirectedEdge(0, 2);
    g.addUndirectedEdge(1, 3);
    g.addUndirectedEdge(2, 3);
    g.addUndirectedEdge(2, 4);
    g.addUndirectedEdge(3, 5);
```

```
    g.addUndirectedEdge(4, 5);

    cout << "DFS 从 0 出发: ";
    vector<bool> visited(g.n, false);
    dfs(0, g, visited);
    cout << "\nBFS 从 0 出发: ";
    bfs(0, g);
    cout << "\n";
    return 0;
}
```

【代码做什么】 main 先搭出上节图示的 6 顶点无向图，然后分别从 0 出发跑 DFS 与 BFS。预期输出：DFS 打出 `0 1 3 2 4 5`，BFS 打出 `0 1 2 3 4 5`——同一张图、同一入口，两种策略给出截然不同的访问顺序，是体会“深度 vs 广度”的最佳实验。

【实现机制解说】 ① DFS 把“访问过”这一事实通过引用传递的 `visited` 传给所有递归分支共享，否则每个分支各持一份拷贝，标记等于白做；这是图版“传引用”的典型场景。② BFS 的标记时机是**入队时**而非出队时：若出队才标记，同一顶点可能被多个邻居重复入队，队列膨胀、甚至死循环。③ 想把 DFS 改成迭代版：把递归调用栈换成显式 `std::stack`，压栈前标记即可，访问顺序会略有不同但仍是合法 DFS。④ 若图不连通，单次 dfs/bfs 只能访问起点所在连通分量；外层再包一层“对所有未访问顶点依次启动遍历”就能覆盖全图。

示例 2：Kahn 拓扑排序（含环检测）。

```cpp
#include <iostream>
#include <queue>
#include <vector>
using namespace std;

// 对 n 个顶点（0..n-1）的有向图做 Kahn 拓扑排序。
// 成功返回 true，order 装一个合法拓扑序；发现环返回 false。
bool topologicalSort(int n, const vector<vector<int>>& adj,
                     vector<int>& order) {
    vector<int> indegree(n, 0);
    for (int u = 0; u < n; ++u)
        for (int v : adj[u]) ++indegree[v];   // 统计每个顶点的入度

    queue<int> ready;                          // 入度为 0 =“前置全部满足”
    for (int u = 0; u < n; ++u)
        if (indegree[u] == 0) ready.push(u);

    while (!ready.empty()) {
        int u = ready.front();
        ready.pop();
        order.push_back(u);
        for (int v : adj[u]) {                 // “解除”u 带来的一个前置约束
            if (--indegree[v] == 0) ready.push(v);
        }
    }
    return order.size() == n;                  // 输出不满 n 个 ⇔ 存在环
}

int main() {
    // 测试 1：DAG，边 A→B、A→C、B→D、C→D、C→E（顶点编号 A=0 … E=4）
    vector<vector<int>> g1(5);
    g1[0] = {1, 2};
    g1[1] = {3};
    g1[2] = {3, 4};
    vector<int> order1;
    cout << "测试1(DAG): "
         << (topologicalSort(5, g1, order1) ? "成功" : "失败(有环)") << " → ";
    for (int x : order1) cout << char('A' + x) << " ";
    cout << "\n";

    // 测试 2：加一条边 E→A，构成环 A→C→E→A，应检测失败
    vector<vector<int>> g2 = g1;
    g2[4].push_back(0);
    vector<int> order2;
    cout << "测试2(有环): "
         << (topologicalSort(5, g2, order2) ? "成功" : "失败(有环)") << "\n";
    return 0;
}
```

【代码做什么】 测试 1 的图正是核心概念里走查过的先修图，期望输出一个合法拓扑序（`A B C D E` 或 `A C B D E` 等，取决于队列顺序——注意这里显式展示了“合法拓扑序不唯一”）；测试 2 在 E 与 A 之间补一条

反向边造出环 A→C→E→A，Kahn 结束后队列必然提前空掉，`order.size() != n` 触发失败分支，打印“失败(有环)”。

【实现机制解说】 ① 入度统计只需一遍全图扫描（O(n+E)）；“入度为 0”的含义是“所有指向我的前置都已被处理完”，这正是拓扑序要求的精确翻译。② 用队列装“就绪顶点”而不是数组，是因为队列入/出队都是 O(1)，且天然满足“谁先就绪谁先走”；换个容器（栈/优先队列）只会改变输出的具体顺序，不改变合法性。③ **环检测零成本**：有环时环上每个顶点的入度永远减不到 0，队列耗尽后 order 缺员，比较 size 即可判定——官方 L25/L27 都强调“有环 ⇔ 无拓扑序”，此实现把判定落到了实处。④ 若把队列换成随机选点，就能随机生成一个合法拓扑序（官方练习之一）。

复杂度分析

设 n = 顶点数、E = 边数。对邻接表表示（示例代码采用）：

操作	时间复杂度	空间复杂度	原因
建图（加一条边）	O(1)（无向为两次 push_back）	O(n + E)	邻接表每顶点一条列表
DFS 遍历	O(n + E)	递归栈最深 O(n)	每个顶点标记一次、每条边看一次
BFS 遍历	O(n + E)	队列最多 O(n)	每顶点入队出队各一次
Kahn 拓扑排序	O(n + E)	O(n)（入度表 + 队列 + 结果）	每条边被扫描两次（统计入度 + 解除约束）
邻接矩阵：查相邻	O(1)	$O(n^2)$	直接读格子
邻接矩阵：列邻居	O(n)	$O(n^2)$	必须扫整行

DFS/BFS 的 O(n+E) 来自“每个顶点至多标记一次、每条边至多被它的两个端点各遍历一次”；对稀疏图（E 远小于 n^2）这比矩阵版的 $O(n^2)$ 划算得多——这也是图算法偏爱邻接表的原因。

关键要点

- 链表、树都是特殊图；图的自由在于多入口、无层级、可有环——遍历时必须用 visited 标记防死循环。
- 表示法按密度选：稀疏用邻接表（O(n+E) 空间、列邻居快），稠密用邻接矩阵（查边 O(1)），边表留给“按边处理”的算法。
- DFS 用递归（或显式栈）一探到底，适合判断连通性、找任意路径；BFS 用队列逐层扩散，无权图首次到达即最短（最少边/最少换乘）。
- 拓扑排序只对 DAG 有效，“有环 ⇔ 无拓扑序”；Kahn 法用入度表 + 队列，输出数不足 n 即判定有环。
- 术语（路径/环/连通/强连通/稠密/稀疏/带权/有向）与两种表示法的取舍是官方点名的期末考试范围。

常见陷阱与注意事项

- **遍历忘打 visited 标记**：图有环时 DFS/BFS 无限递归或死循环。规避：入栈/入队/递归前先标记，且标记须在所有分支间共享（传引用）。
- **BFS 在出队时才标记**：同一顶点被多个邻居重复入队。规避：入队瞬间就标记。
- **对无向图跑 Kahn 拓扑排序**：无向图每条边双向计数，几乎任何图都“有环”，结果无意义。规避：拓扑排序只用于有向图。
- **DFS 求无权图最短路径**：DFS 找到的只是“某一条”路径，不保证最短。规避：无权最短路径请用 BFS。
- **拓扑排序把结果直接当路径打印**：拓扑序只是一般性排列，顶点之间未必相邻，别误读成一条通路。
- **邻接矩阵忘了无向图存两遍**：只填上三角会导致查边不对称、遍历漏邻居。规避：要么存两遍，要么查询时按对称规则处理。
- **自环与平行边没想清楚**：自环（u→u）在拓扑排序里直接构成环、在 BFS 里会让自己“已访问”而跳过，建模时要想清楚它们是否合法。
- **不连通图只遍历一次**：从 0 出发的 DFS/BFS 覆盖不到另一分量。规避：需要全图遍历时外层循环启动所有未访问顶点。

思考题（带答案）

问题 1：给定任意一张无向图和一个起点，为什么 BFS 第一次“碰到”某顶点时经过的路径必然边数最少？ **答案**：BFS 严格按层扩散：第 k 层的顶点必然是“经过恰好 k 条边、且首次可达”的顶点。若存在一条更短的边数为 m < k 的路径，该顶点应出现在第 m 层，矛盾。DFS 则可能顺着一条长路先到达，故不保证最短。

问题 2：为什么“有环 ⇔ 无拓扑序”？请用环上的顶点论证。 **答案**：若图中有环 $u_1 \to u_2 \to \dots \to u_k \to u_1$，环上每个顶点都是“别人的前置”：$u_1$ 要求排在 u_k 后面、u_k 又要求排在 u_{k-1} 后面……传递下去形成 u_1 必须排在 u_1 后面的矛盾，任何线性顺序都无法满足。反之若无环（DAG），Kahn 算法总能不断找到入度为 0 的顶点并推进，最终排完所有顶点。

问题 3：邻接表与邻接矩阵在“检查 u、v 是否相邻”上的代价分别是多少？这对“边查询密集”的应用意味着什么？ **答案**：邻接矩阵 O(1)（直接看 matrix[u][v]），邻接表需在 adj[u] 里线性找 v、代价 O(deg(u))。因此若算法主体是海量“任意两点是否相邻”查询（如部分动态规划），矩阵更合适；而 DFS/BFS/Dijkstra 这类“拿到一个点就扫它所有邻居”的算法，邻接表每步只花 O(deg)，整体 O(n+E)，是更优选择——这正对应官方“稠密图用矩阵、稀疏图用邻接表”的结论。

Lecture 16: 最短路径：Dijkstra 与 A*（Shortest Paths: Dijkstra & A*）（对应课程真实讲座 L26–L27）

概述

第 15 讲教会我们用 BFS 找“无权图”里边数最少的路径；本讲把问题升级为**带权图上的最低代价路径（shortest path，代价 = 边权之和而非边数）**，主角是两个算法：Dijkstra（单源最短路径：从一个起点到图中**所有**顶点）与 A*（*单对最短路径：从起点到一个明确终点，靠启发式“抄近路”*）。*Dijkstra 的贪心直觉、堆*

优化运行时、负权边为何会“掀翻”它，以及 A 的 f = g + h 框架与可采纳启发式，是本讲核心。官方对应：L26（2026 年 8 月 6 日，周四，Dijkstra and A *Shortest Path Algorithms）与 L27（2026 年 8 月 10 日，周一，Graph Coding，课上用自建 WeightedGraph 类把拓扑排序与 Dijkstra 完整编码了一遍）。官方明确的口径：期末考试不要求手算走查 Dijkstra/A*，但要理解算法原理、三种找路算法的适用语境、以及堆优化带来的运行时差异；L27 编码课的内容（除“有环 ⇔ 无拓扑序”）不要求期末复现。

核心概念与算法原理

问题定义：单源最短路径（single-source shortest paths）。 给定带权有向/无向图与源点 source，求 source 到每个顶点的最低代价路径（“最短”在此指边权总和最小，不是边数最少）。应用：消息从一台主机最快广播到全网、货物从配送中心送往各目的地、疾病在社交网络上的扩散建模等。

Dijkstra 的直观解释（它是什么？）。 想象把每个顶点当成“会议地点”，dist[v] 是“目前已知从 source 到 v 的最低代价”。算法像一场逐级扩大的招标会：每次都把“当前已知代价最小、但尚未拍板”的顶点 u 拍板确定下来（它的 dist 从此封存不再改），然后用 u 的每条出边去“降价竞标”它的邻居——若走 u 再走一条边能比邻居现有报价更便宜，就更新报价。这个“永远先处理最便宜候选”的策略就是**贪心**。为什么确定后就可以封存？只要边权非负，任何“绕路”都必然先经过某个代价 ≥ dist[u] 的中间点，再加非负边，不可能更便宜——正是“非负权”让封存永远安全。

Dijkstra 操作/步骤分解（以邻接表 + dist/prev 数组为例）：

```
1. dist[source] = 0；其余 dist = ∞；prev[v] 记录前驱（用于回溯路径）
2. 反复执行，直到所有顶点确定（或用优先队列驱动到队空）：
   a. 挑出“未确定顶点中 dist 最小”的 u
      （朴素版：线性扫 dist 数组；堆优化版：从优先队列弹出）
   b. 标记 u 已确定
   c. 松弛（relaxation）：对 u 的每条边 (u, v, w)：
        若 v 未确定 且 dist[u] + w < dist[v]：
            dist[v] = dist[u] + w；prev[v] = u
3. 结束时 dist[v] 即 source 到 v 的最短代价；沿 prev 从 v 一路回溯到 source 得到路径
```

堆优化的关键技巧：“允许过期记录堆积”。 贪心地想把“挑最小 dist”交给优先队列，但标准的堆不支持“把已在堆里的元素改小”（decrease-key 很贵）。官方的解决方案非常直白：**每次松弛成功就把新的 (dist[v], v) 整个压进堆，旧记录留在堆底不管**——新记录更小，自然更快浮到堆顶；出堆时若发现该记录已过期（顶点早已确定，或记录的 dist 与当前 dist 不一致）就丢弃。代价是堆里会积累一些“垃圾”，但换来的是每轮“挑最小”从 O(n) 降到 O(log n)。

手算走查（5 个顶点，体会每一轮）。 图用邻接表给出（source = 0）：

```
邻接表（(邻居, 边权)）：
0: →(1,4) →(2,1)          2: →(0,1) →(1,2) →(3,5)
1: →(0,4) →(2,2) →(3,1) →(4,7)
3: →(1,1) →(2,5) →(4,3)   4: →(1,7) →(3,3)
```

轮次	取出并确定	松弛后的 dist[0..4]	本轮说明
初始	—	{0, ∞, ∞, ∞, ∞}	源点距离 0

轮次	取出并确定	松弛后的 dist[0..4]	本轮说明
1	0	{0, 4, 1, ∞, ∞}	从 0 松弛邻居 1（4）、2（1）
2	2（dist=1）	{0, 3, 1, 6, ∞}	经 2 到 1 只需 1+2=3 < 4（**绕路打败直连**）；到 3 为 6
3	1（dist=3）	{0, 3, 1, 4, 10}	经 1 到 3：3+1=4 < 6；到 4：3+7=10
4	3（dist=4）	{0, 3, 1, 4, 7}	经 3 到 4：4+3=7 < 10
5	4（dist=7）	{0, 3, 1, 4, 7}	全部确定，结束

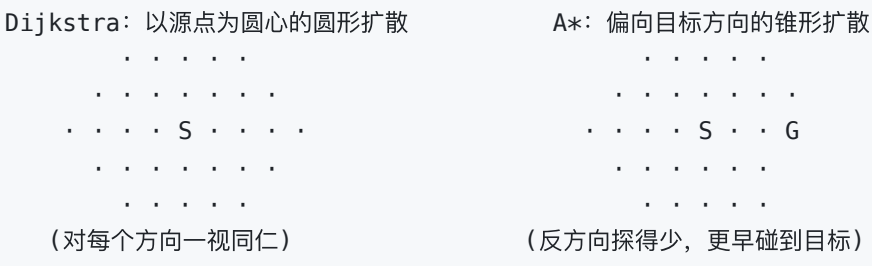
结果：dist = {0, 3, 1, 4, 7}；最短路径 0→1 是 **0–2–1**（代价 3）而非直连 0→1（代价 4），0→4 是 **0–2–1–3–4**（代价 7）。注意第 2 轮正是 Dijkstra 的“神韵”所在：代价 4 的直连边已经摆在那，算法仍先确定了代价 1 的 2，因为 2 可能带来更便宜的绕路——贪心不是“抢近路”，而是“永远先封存当前最便宜的可能”。

为什么负权边会破坏 Dijkstra？ 负权推翻“确定后即可封存”的根基：一个已经确定（甚至已经确定很久）的顶点，可能因为某条带负权的边而出现更便宜的绕路，但算法不会再回头更新它。看这个三角形（A 为源点，边权 A→B=7、A→C=6、B→C=−3）：



Dijkstra 先确定 C（dist=6，比 B 的 7 小）；等 B 被确定（dist=7）时，它的邻居 C 已“封存”，松弛被跳过——最终 C 停在 6，永远错过了正确的 4。给所有边统一加一个常数把负权抹平也没用：加常数会改变不同长度路径的“相对差价”（官方补充材料里演示过这个错误修补方案）。官方补充：能处理负权的是 **Bellman–Ford**（反复对所有边松弛 n−1 轮，O(VE)），但它对负环（能无限循环变小代价的环）也无能为力——负环上的最短路径根本不存在。这部分在官方讲义里属于可选的补充阅读，期末不考。

A* 搜索（单对最短，带“指南针”）。 Dijkstra 有个“笨”处：它从源点**朝所有方向均匀扩散**，完全不管目标在哪个方向。A *的改进是给每个节点加一个启发式 h(n)——从 n 到目标代价的“估计值”，并让优先级变成 f(n) = g(n) + h(n)*，其中 g(n) 是从起点到 n 的真实代价。f 小 = “已经花得少 + 感觉离目标近”，于是搜索被“拽”向目标方向：



A* 伪代码（与 Dijkstra 逐行对照几乎只差优先级）：

```
1. g[start] = 0；把 (f = g + h, start) 压入优先队列
2. 弹出 f 最小的节点 n（过期记录直接丢弃）
3. 若 n 就是目标 → 沿 prev 回溯路径，结束
4. 对 n 的每个可达邻居 m：
      若 g[n] + 边权(n,m) < g[m]：
          g[m] = g[n] + 边权；prev[m] = n
          以 f = g[m] + h(m) 把 m 压入队列
5. 队列空仍未碰到目标 → 不可达
```

三个关键性质：① **可采纳（admissible）**：h 永不高估到目标的真实代价。可采纳保证 A *第一次弹出目标时即最优。② 一致（consistent/单调）：h(u) ≤ w(u,v) + h(v)。一致蕴含可采纳，还保证“节点一经确定不再回头处理”，实现与 Dijkstra 完全同构。③ h ≡ 0 时 A* 退化为 Dijkstra——*所以 Dijkstra 可以看作“没有指南针的 A”。网格寻路常用曼哈顿距离、地图导航常用直线距离，二者均不高估。何时用不了 A*：目标未知（如“在图里找藏起来的宝藏”）就没有 h 可算，只能 BFS/Dijkstra。官方在课上用可视化工具演示了三者的探索范围差异，并提供了斯坦福校友 Amit Patel 的 A* 专题资源供深入阅读。

三算法适用语境速查（官方期末考试点名的“哪类场景用哪个”）：

场景	算法
无权图、求最少边/最少换乘的路径	BFS
带权（无负权）图、求单源到**所有**顶点	Dijkstra
带权图、知道明确目标、存在可估代价的启发式	A*（只保证到目标那一条最优）
带负权但无负环（超纲参考）	Bellman–Ford，O(VE)

代码示例与实现详解

示例 1：Dijkstra 完整实现（邻接表 + std::priority_queue 小顶堆，输出距离与路径）。

```cpp
#include <climits>
#include <functional>
#include <iostream>
#include <queue>
#include <utility>
#include <vector>
using namespace std;

struct Edge { int to; int weight; };

// 单源最短路径：source 到每个顶点的最短代价写入 dist，路径前驱写入 prev
void dijkstra(int source, const vector<vector<Edge>>& adj,
              vector<int>& dist, vector<int>& prev) {
    int n = (int)adj.size();
    dist.assign(n, INT_MAX);
    prev.assign(n, -1);
    vector<bool> done(n, false);          // done[v]：v 的最短距离已封存

    // 小顶堆按 (dist, 顶点) 排序；允许旧记录滞留（lazy deletion）
    using P = pair<int, int>;
    priority_queue<P, vector<P>, greater<P>> pq;
    dist[source] = 0;
    pq.push({0, source});

    while (!pq.empty()) {
        auto [d, u] = pq.top();
        pq.pop();
        if (done[u]) continue;            // 过期记录：u 早已确定
        if (d != dist[u]) continue;       // 双保险：不是最新距离也跳过
        done[u] = true;                   // 此刻 u 的 dist 正式封存
        for (const Edge& e : adj[u]) {    // 松弛 u 的所有邻居
            int v = e.to;
            if (done[v]) continue;        // 已确定的顶点不再更新
            if (dist[u] + e.weight < dist[v]) {
                dist[v] = dist[u] + e.weight;   // 松弛成功：找到更便宜的路径
                prev[v] = u;
                pq.push({dist[v], v});    // 新记录入堆，旧记录自动作废
            }
        }
    }
}

// 递归回溯打印 source → v 的完整路径
void printPath(int v, const vector<int>& prev) {
    if (prev[v] == -1) { cout << v; return; }   // 到达 source
    printPath(prev[v], prev);
    cout << " → " << v;
}

int main() {
    int n = 5;
    vector<vector<Edge>> adj(n);          // 与手算走查同一张图
    auto add = [&](int u, int v, int w) {
```

```
        adj[u].push_back({v, w});
        adj[v].push_back({u, w});          // 无向图：两条有向边
    };
    add(0, 1, 4); add(0, 2, 1); add(1, 2, 2);
    add(1, 3, 1); add(2, 3, 5); add(3, 4, 3); add(1, 4, 7);

    vector<int> dist, prev;
    dijkstra(0, adj, dist, prev);
    for (int v = 0; v < n; ++v) {
        cout << "到顶点 " << v << " 的最短代价 = "
             << (dist[v] == INT_MAX ? -1 : dist[v]) << ", 路径: ";
        printPath(v, prev);
        cout << "\n";
    }
    return 0;
}
```

【代码做什么】 用与手算走查完全一致的图跑 Dijkstra，期望输出：到 1 代价 3 路径 `0 → 2 → 1`、到 4 代价 7 路径 `0 → 2 → 1 → 3 → 4`——把上文的表格在真实代码里复现一遍，是检验理解的黄金练习。

【实现机制解说】 ① 堆里存的 `(dist, 顶点)` 在松弛成功后**不断有新版本入堆**：某顶点可能同时存在多条不同 dist 的记录，但最小的那个一定先出堆——这正是“允许过期记录堆积”的实现形态；出堆时用 `done[u]` 与 `d != dist[u]` 两道检查滤掉垃圾。② `dist[u] + e.weight` 用 INT_MAX 会溢出：好在只有被松弛成功（有限值）的顶点才会入堆并出堆，出堆顶点必有有限 dist，因此加法安全；即便如此，习惯上仍可把初始值设为 `INT_MAX/2` 更稳妥。③ `prev` 前驱链在打印时**必须先回溯到 source 再正向输出**（递归天然做到），直接顺着 prev 打印得到的是反序路径。④ 无向图 = 每条边存两遍；有向图只存一遍，代码其余部分完全不变——算法本身不关心方向，只关心邻接表内容。

示例 2：A* 网格寻路（曼哈顿距离启发式）。

```cpp
#include <algorithm>
#include <climits>
#include <cstdlib>
#include <functional>
#include <iostream>
#include <queue>
#include <tuple>
#include <vector>
using namespace std;

const int dr[4] = {-1, 1, 0, 0};      // 上、下、左、右
const int dc[4] = {0, 0, -1, 1};

int manhattan(int r, int c, int gr, int gc) {
    return abs(r - gr) + abs(c - gc); // 启发式 h：最少还要走多少步（不高估）
}

// grid 中 0 可走、1 是障碍。找到 (sr,sc)→(tr,tc) 的最短路径写入 path。
bool aStar(const vector<vector<int>>& grid, int sr, int sc, int tr, int tc,
           vector<pair<int, int>>& path) {
    int R = (int)grid.size(), C = (int)grid[0].size();
    vector<vector<int>> g(R, vector<int>(C, INT_MAX / 2));  // 真实代价
    vector<vector<int>> prev(R, vector<int>(C, -1));        // 编码的前驱

    // 状态 (f, g, 行, 列)：按 f 升序，f 相同 g 小的先出（词典序天然如此）
    using State = tuple<int, int, int, int>;
    priority_queue<State, vector<State>, greater<State>> pq;
    g[sr][sc] = 0;
    pq.push({manhattan(sr, sc, tr, tc), 0, sr, sc});

    while (!pq.empty()) {
        auto [f, cost, r, c] = pq.top();
        pq.pop();
        if (r == tr && c == tc) {       // 目标第一次出堆 ⇒ 已是最优
            int cr = tr, cc = tc;
            while (!(cr == sr && cc == sc)) {
                path.push_back({cr, cc});
                int code = prev[cr][cc];      // 解出前驱坐标
                cr = code / C;
                cc = code % C;
            }
            path.push_back({sr, sc});
            reverse(path.begin(), path.end());
            return true;
        }
        if (cost != g[r][c]) continue;  // 过期记录
        for (int d = 0; d < 4; ++d) {
            int nr = r + dr[d], nc = c + dc[d];
            if (nr < 0 || nr >= R || nc < 0 || nc >= C) continue;
            if (grid[nr][nc] == 1) continue;          // 撞墙
            int ng = cost + 1;                        // 每步代价 1
            if (ng < g[nr][nc]) {
                g[nr][nc] = ng;
```

```
                prev[nr][nc] = r * C + c;              // 压缩存储前驱
                pq.push({ng + manhattan(nr, nc, tr, tc), ng, nr, nc});
            }
        }
    }
    return false;                          // 目标不可达
}

int main() {
    vector<vector<int>> grid = {        // 1 为障碍，S=(0,0)，G=(4,4)
        {0, 0, 0, 0, 0},
        {0, 1, 0, 1, 0},
        {0, 0, 0, 1, 0},
        {0, 1, 0, 0, 0},
        {0, 0, 0, 0, 0},
    };
    vector<pair<int, int>> path;
    bool ok = aStar(grid, 0, 0, 4, 4, path);
    if (!ok) { cout << "不可达\n"; return 0; }
    cout << "找到路径，长度 = " << path.size() - 1 << "\n";
    for (auto [r, c] : path) cout << "(" << r << "," << c << ") ";
    cout << "\n";
    return 0;
}
```

【代码做什么】 在 5×5 网格（含 4 块障碍）上从左上角寻路到右下角，输出最短路径的长度与逐点坐标。启发式取曼哈顿距离：它永远 ≤ 真实剩余步数（可采纳），且满足三角不等式（一致），因此“目标第一次出堆即最优”成立。把 `manhattan` 的调用全部换成返回 0 的函数，这个程序就退化成了逐格代价 1 的 Dijkstra——读者可以亲手改一行验证“A* ⊇ Dijkstra”的说法。

【实现机制解说】 ① 与示例 1 的唯一本质差异在优先级：Dijkstra 压 `(g, 节点)`，A 压 `(g + h, 节点)` ——*`f = g + h` 不是新算法，而是“给 Dijkstra 换了个更聪明的排序键”。② h 可采纳时，任何“先出堆的目标”都不可能再被更便宜的绕路取代，因为那条绕路若存在，其 f 必然更小、会先出堆（与 Dijkstra 封存正确性同构的论证）。③ 前驱用 `r*C+c` 单个整数编码，省一个二维结构；回溯时除回去。④ 障碍迫使 A* “绕路”* ——最优路径有时会先朝远离目标的方向走几步再折返（先上高速再转弯），这恰是“偏置搜索但不禁止绕路”的设计意图，也是 h 只做估计、不做承诺的原因。

复杂度分析

设 V = 顶点数、E = 边数。dist/prev 数组空间 O(V)（邻接表另占 O(V+E)）。

算法 / 实现	最坏时间复杂度	原因简述
Dijkstra（朴素：每次线性挑最小）	$O(V^2 + E)$	每轮挑最小扫一遍数组 O(V)，共 V 轮；松弛合计 O(E)
Dijkstra（二叉堆 + 允许过期记录）	$O((V+E) \log V)$（常用口径）；官方按“堆内至多 $O(V^2)$ 条过期记录”给 $O(V^2 \log V)$	每次成功松弛压一条新记录；堆操作 O(log)

算法 / 实现	最坏时间复杂度	原因简述
Dijkstra（斐波那契堆，提级参考）	O(E + V log V)	支持 O(1) 均摊的 decrease-key
A*	理论上不劣于同实现的 Dijkstra；好启发式下实践中显著更少	探索范围取决于 h 的质量与图结构
Bellman–Ford（负权参考）	O(VE)	对所有边反复松弛 V−1 轮
BFS（无权图对照）	O(V + E)	每顶点出入队一次

易错点：堆优化的 O((V+E) log V) 是“把每次出堆/入堆算 O(log) 再乘以总次数”的直观结果；官方在讲义里专门提醒，像“连续 n 次插入堆”这类操作不能简单相乘——插入第 k 个元素的实际代价是 O(log k)，累加后才得到 O(n log n)（Stirling 近似）。同理，rehash/heapify 之类“看着像 O(n log n) 实则 O(n)”的反直觉结论，都源于对“操作对象规模在变化”的严谨求和——考试层面只需记住堆优化版把“挑最小”从 O(V) 降为 O(log V)。

关键要点

- Dijkstra 解决“单源 → 所有顶点”的带权最短路径，前提是**边权非负**；贪心“每次确定当前最小”之所以安全，正因为非负权让已确定者不可能被反超。
- 三种找路算法按场景选：无权用 BFS、带权单源用 Dijkstra、知道目标且能估代价用 A*；这是官方点名的期末考试判断题。
- 堆优化 Dijkstra 的正确姿势是“**允许过期记录堆积**”：松弛成功就把新 (dist, v) 整个入堆，出堆时丢弃已确定/过期条目——简单且有效。
- A* 只是给 Dijkstra 换了优先级 f = g + h；h 必须可采纳（永不高估）才保证最优，h ≡ 0 即退化为 Dijkstra。
- 负权边会让 Dijkstra 的“封存”失效；需要时用 Bellman–Ford（O(VE)），负环则根本无有限最短路。
- 官方期末口径：不要求手算走查 Dijkstra/A*，但原理、适用语境与堆优化运行时必须清楚。

常见陷阱与注意事项

- **在有负权边的图上跑 Dijkstra**：得到的是错误答案且不易察觉。规避：确认边权非负；有负权改 Bellman–Ford。
- **给所有边加常数“抹平”负权**：看似聪明实则无效（加常数改变长路径与短路径的相对差价，官方演示过失败例子）。规避：换算法，别修修补补。
- **堆里过期记录不丢弃**：可能把已确定的顶点重复“处理”，甚至死循环。规避：出堆时检查 `done` 标记与 dist 是否仍是最新值。
- **松弛条件写反**：写成 `dist[v] + w < dist[u]` 之类，结果全错。规避：牢记“老路 > 新路(u 经边到 v)才更新”。

- **用 INT_MAX 当 ∞ 还直接加权重**：溢出后变成负数，dist 表报废。规避：初始值用 INT_MAX/2，或仅在有限值时做加法。
- **路径打印顺序颠倒**：沿 prev 回溯得到的是反序。规避：先递归到 source 再逐层输出（或存下再 reverse）。
- **A* 用了不可采纳的启发式**：可能“感觉很近”而错过真正的最优路径。规避：先验证 h 永不高估（网格用曼哈顿、地图用直线距离都安全）。
- **无权图也硬上 Dijkstra**：能用，但 BFS 的 O(V+E) 更简单更快；反过来无权图中用 BFS 是标准答案。
- **A* 用在“目标未知”的任务上**：没有目标就没有 h。规避：目标不明的探索任务回归 BFS/Dijkstra。

思考题（带答案）

问题 1：为什么只要存在一条负权边，Dijkstra“确定后封存”的论证就崩溃？请用一句话概括其论证断点。 **答案**：Dijkstra 的封存依赖“任何绕路都要先经过一个 dist 不小于当前顶点、再加上非负边”的推理；负权边让“先绕到别处、再走负权边”可能比任何已确定的直达路径更便宜，而算法不会回头更新已封存的顶点——封存的前提被抽掉了。

问题 2：堆优化版 Dijkstra 中，为什么旧记录可以安心留在堆里不去删除？它最坏会让堆多大？ **答案**：每次松弛成功都会产生一条更小的 (dist, v)，小记录必然先于同顶点的旧大记录出堆；旧记录出堆时用“已确定或 dist 不一致”直接丢弃，不影响正确性，只浪费一点堆空间。最坏情形（每轮几乎所有顶点都被反复改进）下堆里可能积累 $O(V^2)$ 条记录，对应官方给出的 $O(V^2 \log V)$ 上界；稀疏图上实际按松弛次数计，通常是 $O((V+E) \log V)$。

问题 3：把 A *的启发式分别设为“恒 0”“真实剩余代价”“可采纳但偶尔低估（永不高估）”，各会发生什么？* *答案：恒 0 → A* 退化为 Dijkstra，向所有方向扩散；等于真实代价 → 每个被展开的节点都在最优路径上，几乎“直线”冲到目标，探索最少；可采纳 → 保证首次弹出目标即最优，只是可能比“真实代价版”多探一些节点。若 h 偶尔高估（不可采纳），搜索会更快但结果可能不是最短——这是“速度”与“最优性”的交易。

Lecture 17: 拓展专题：Trie 与并查集（Bonus: Tries & Union-Find）（延伸专题，官方 2026 夏季未设独立讲座）

概述

本章为延伸专题：官方 2026 夏季学期共 28 讲，未设独立讲座讲解 Trie 与并查集。 不过官方与它们并非毫无交集：L24（哈希那讲）权衡“按 8 位学号存取学生记录”的方案时，提到有人建议“建一棵每个节点 10 个孩子、沿学号数字逐位下行的树”——那正是 trie（字典树）的思路；Stanford 的 Lexicon（整本英语词典的查找结构）内部也与 trie 异曲同工。并查集则完全不在本季大纲内。把它们收进笔记，是因为：Trie 是自动补全、拼写检查、前缀统计的标准答案；并查集是“动态连通分量”与 Kruskal 算法判环的标配工具；二者都是技术面试与后续课程（CS161）的高频储备。本章前半实现 Trie 的插入/查找/前缀查询，后半实现带路径压缩与按秩合并的 UnionFind，并给出连通分量计数与判环演示。官方对应：无对应讲座（延伸专题；官方 L24 曾提 trie 思路）。

核心概念与算法原理

Trie（前缀树 / 字典树）

问题定义。 维护一个词典，支持三类查询：① 某单词是否存在（search）；② 是否存在以某前缀开头的单词（startsWith，自动补全的地基）；③ 以某前缀开头的单词共有几个。用二叉搜索树存词典：查找要 O(k log n)（k 为词长，每次节点比较都要逐字符比）；用哈希表存词典：整词查找 O(k)、很优秀，但它**无法回答前缀问题**——“startsWith("ca")”要枚举所有键，退化为 O(nk)。Trie 用“按字符共享前缀”的树结构同时解决两者。

直观解释（它是什么？）。 把词典想象成电话簿整理现场：凡是共享前缀的单词，就让它们**共用前缀这一段路**，只在分叉处才另开枝杈。树的每条边标一个字符，从根出发沿边下行，路径上拼出的字符串就是“走到这里为止所代表的前缀”；某个节点若是某个完整单词的结尾，就给它打个“词尾”标记（isWord）。于是：查单词 = 沿字符下行看能否走完且终点带词尾标记；查前缀 = 只看能否走完，不问词尾。

Trie 结构图示（词表：cat, car, card, cart, dog, do）：

```
                (root)
          'c' /        \ 'd'
            [ ]          [ ]
        'a' /              \ 'o'
          [ ]                [*]          ← 词尾：拼出 "do"
      'r' /   \ 't'          \ 'g'
        [*]    [*]            [*]         ← "car" / "cat" / "dog"
     'd'/ \'t'
     [*]   [*]                            ← "card" / "cart"
(* 表示该节点是一个完整单词的结尾；同一字符在不同深度可以出现多次，
 因为前缀路径不同——图中两个 't' 分别属于 cat 与 cart)
```

操作/步骤分解（孩子集合用“字符 → 子节点”的映射存储）：

```
insert("cart")：
  1. 从根开始，逐个字符 'c'→'a'→'r'：都存在则沿指针下行
  2. 到字符 't'：当前节点没有 't' 孩子 → 新建子节点并下行
  3. 走完所有字符后，把所在节点标记 isWord = true

search("car")：沿 c-a-r 下行成功；终点节点 isWord == true  → 存在
search("ca")：沿 c-a 下行成功；但终点 isWord == false       → 不是单词
startsWith("ca")：沿 c-a 下行成功（不管 isWord）             → 有此前缀
startsWith("cx")：走到 'x' 时找不到孩子                        → 无此前缀
```

孩子容器的取舍（实现时必选其一）：**定长数组**（如 `Node* children[26]`）：按下标 O(1) 直达孩子，速度最快，但每个节点都占 26 个指针槽——词典稀疏时大量空槽，仅适合“纯小写英文字母”这类小字母表；**std::map\<char, Node*>**：孩子按需分配、内存只随实际分叉走，代价是每次找孩子 O(log 字母表)——字母表通常很小（26/52/256），可视为常数，代码也更通用（可存任意字符集）；**std::unordered_map**：期望 O(1) 找孩子，但引入哈希开销与无序遍历，对字符键收益有限。本讲示例选 map 版本，重点讲清“共享前缀”的树逻辑。

应用：拼写检查与词典（Lexicon）、搜索框自动补全与联想、IP 路由的最长前缀匹配、基因序列匹配、前缀计数/词频统计。代价与收益一句话：**一次查询只与词长 k 有关，与库中单词总数 n 无关**——这正是 Trie 相对树/哈希的杀手锏。

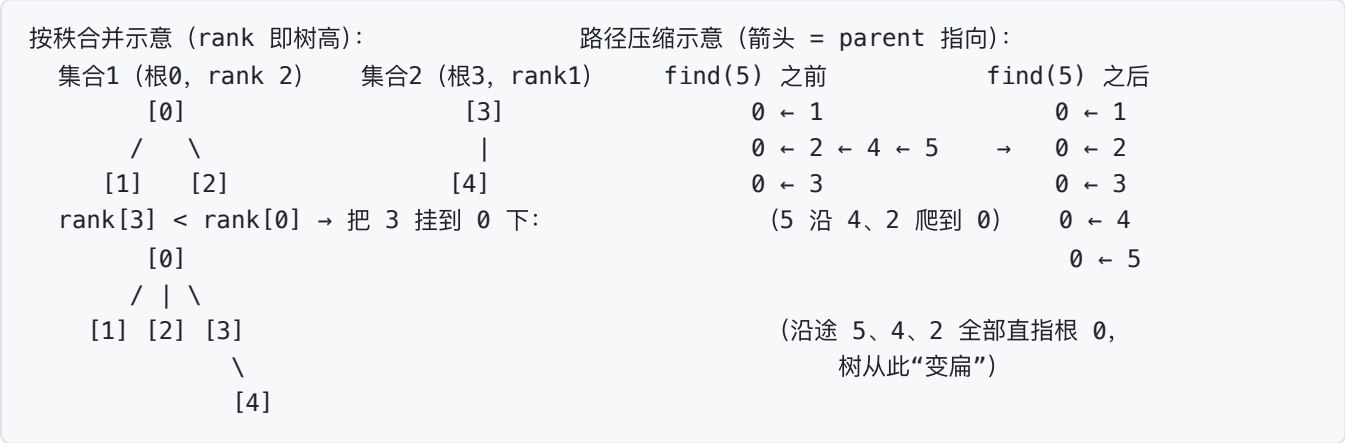
Union-Find（并查集 / 不相交集合 Disjoint Set）

问题定义。 维护 n 个元素，它们被动态地合并成若干组（集合）。支持两个操作：`find(x)` 回答“x 属于哪一组”（返回该组的代表元/根）；`union(a, b)` 把 a、b 所在的两组合并为一组。应用：社交网络“两人是否间接认识”、电网/计算机是否连通、Kruskal 求最小生成树时判断“加这条边会不会成环”、图像区域标记、网格渗透模拟等——凡是“关系不断增多、随时要问是否同组”的问题都是它的主场。

直观解释与实现思路。 把每个集合表示成一棵“只认爹”的树：每个元素记一个 parent（`parent[i]` 指向父节点），**根节点的 parent 指向自己**，根就是集合代表元。`find(x)` 沿 parent 链爬到根；`union(a,b)` 先 find 出两个根，若相同说明本就在一组（合并无意义，甚至意味着“这条边成环”），否则把一棵树的根接到另一棵根下。朴素实现最坏会退化成 O(n) 的深链（每次把一棵整树挂到另一棵下），所以必须配两个加速器：

1. **按秩合并（union by rank/size）**：永远把“矮树/小树”的根接到“高树/大树”的根下。这样树高最多 O(log n)，单次 find 最坏 O(log n)。
2. **路径压缩（path compression）**：find 爬向根的过程中，把沿途经过的所有节点直接挂到根下。下次再 find 它们就是一步直达。

两者合体后，单次操作的**摊还复杂度是 O(α(n))**——α 是增长极慢的反阿克曼函数，对任何现实规模的 n 都 ≤ 4，工程上可放心当作 O(1)。



代码示例与实现详解

示例 1：Trie 类（insert / search / startsWith / 前缀计数）。

```cpp
#include <iostream>
#include <map>
#include <string>
using namespace std;

class Trie {
public:
    Trie() : root_(new Node()) {}
    ~Trie() { delete root_; }                       // 级联释放整棵树
    Trie(const Trie&) = delete;                     // 含裸指针：禁拷贝防双重释放
    Trie& operator=(const Trie&) = delete;

    void insert(const string& word) {
        Node* cur = root_;
        for (char c : word) {
            auto it = cur->children.find(c);
            if (it == cur->children.end())          // 缺孩子就新建
                it = cur->children.emplace(c, new Node()).first;
            cur = it->second;
        }
        cur->isWord = true;                         // 词尾打标
    }

    bool search(const string& word) const {         // 整词存在?
        Node* cur = findNode(word);
        return cur != nullptr && cur->isWord;
    }

    bool startsWith(const string& prefix) const {   // 有此前缀?
        return findNode(prefix) != nullptr;
    }

    int countWordsWithPrefix(const string& prefix) const {
        Node* cur = findNode(prefix);
        return cur == nullptr ? 0 : countWords(cur);
    }

private:
    struct Node {
        map<char, Node*> children;                  // 字符 → 子节点
        bool isWord = false;
        ~Node() {                                   // 递归销毁子树
            for (auto& p : children) delete p.second;
        }
    };

    Node* findNode(const string& s) const {         // 沿字符下行，走不通返回空
        Node* cur = root_;
        for (char c : s) {
            auto it = cur->children.find(c);
            if (it == cur->children.end()) return nullptr;
            cur = it->second;
        }
```

```
        return cur;
    }

    static int countWords(const Node* cur) {        // 统计子树里的词尾数
        int total = cur->isWord ? 1 : 0;
        for (auto& p : cur->children) total += countWords(p.second);
        return total;
    }

    Node* root_;
};

int main() {
    Trie t;
    for (const string& w : {"cat", "car", "card", "cart", "dog", "do"})
        t.insert(w);
    cout << "search(cat)  = " << t.search("cat") << "  (整词)\n";
    cout << "search(ca)   = " << t.search("ca") << "  (前缀不是词!)\n";
    cout << "startsWith(ca) = " << t.startsWith("ca") << "\n";
    cout << "startsWith(do) = " << t.startsWith("do") << "\n";
    cout << "startsWith(xy) = " << t.startsWith("xy") << "\n";
    cout << "以 ca 开头的完整单词数 = " << t.countWordsWithPrefix("ca")
         << " (期望 4: cat/car/card/cart)\n";
    cout << "以 car 开头的完整单词数 = " << t.countWordsWithPrefix("car")
         << " (期望 3: car/card/cart)\n";
    return 0;
}
```

【代码做什么】 用图示词表建树，逐条验证四类查询：`search("cat")` 为真而 `search("ca")` 为假（演示“走到前缀 ≠ 单词存在”，词尾标记在此刻是关键）；`startsWith` 只问“路通不通”；前缀计数用递归统计子树里的词尾总数——输出与上节结构图完全对应。

【实现机制解说】 ① “词尾标记 + 路径可达”两个条件分别支撑 search 与 startsWith：findNode 只负责“路是否走得通”，isWord 负责“走到这里是否是一个完整的词”，两者缺一不可。② 孩子用 `std::map<char, Node*>` 按需分配，树上只有“实际分叉”才有子节点；每个节点 O(log 26) 的查孩子代价可视为常数。若换成 `Node* children[26]`：查找变 O(1) 直取，但每个节点固定 26 个指针（约 208 字节），空分叉越多浪费越大——空间换时间的老戏码，按字符集大小取舍。③ 裸指针意味着必须管好内存：Node 析构时递归 delete 所有孩子，Trie 析构 delete 根即可级联清空；同时把拷贝构造/赋值 delete 掉，防止浅拷贝导致双重释放。④ 若只问“有多少词以某前缀开头”，可给每个节点额外存一个 count（插入时沿途 +1），把递归统计降为 O(k)——自动补全里常用的优化。

示例 2：UnionFind（路径压缩 + 按秩合并，含连通分量计数与判环演示）。

```cpp
#include <iostream>
#include <numeric>
#include <utility>
#include <vector>
using namespace std;

class UnionFind {
public:
    explicit UnionFind(int n)
        : parent_(n), rank_(n, 0), count_(n) {
        iota(parent_.begin(), parent_.end(), 0);   // 初始每人自成一组，parent[i]=i
    }

    int find(int x) {                              // 带路径压缩
        if (parent_[x] != x)
            parent_[x] = find(parent_[x]);         // 沿途节点全部直挂根下
        return parent_[x];
    }

    // 合并 a、b 所在组；返回 false 表示它们本就在一组（可用于判环）
    bool unite(int a, int b) {
        int ra = find(a), rb = find(b);
        if (ra == rb) return false;                // 已同组：若这是条边，则成环
        if (rank_[ra] < rank_[rb]) swap(ra, rb);   // 矮树根挂到高树根下
        parent_[rb] = ra;
        if (rank_[ra] == rank_[rb]) ++rank_[ra];   // 两树等高手动加一
        --count_;                                  // 两个连通分量合并成一个
        return true;
    }

    bool connected(int a, int b) { return find(a) == find(b); }
    int count() const { return count_; }           // 当前连通分量个数

private:
    vector<int> parent_;
    vector<int> rank_;                             // 树高上界
    int count_;
};

int main() {
    cout << "--- 连通性演示 ---\n";
    UnionFind uf(6);                               // 顶点 0..5
    uf.unite(0, 1);
    uf.unite(1, 2);                                // 现在 {0,1,2} 一组
    uf.unite(3, 4);                                // {3,4} 一组，5 单独
    cout << "分量数 = " << uf.count() << " (期望 3)\n";
    cout << "connected(0,2) = " << uf.connected(0, 2) << "\n";
    cout << "connected(0,3) = " << uf.connected(0, 3) << "\n";
    uf.unite(2, 3);                                // 打通两大组
    cout << "合并 {0,1,2} 与 {3,4} 后分量数 = " << uf.count() << " (期望 2)\n";
    cout << "connected(0,4) = " << uf.connected(0, 4) << "\n";

    cout << "\n--- Kruskal 式判环演示（加边时若 unite 返回 false 即成环）---\n";
```

```
    UnionFind k(3);
    cout << "边 0-1: " << (k.unite(0, 1) ? "加入" : "成环!") << "\n";
    cout << "边 1-2: " << (k.unite(1, 2) ? "加入" : "成环!") << "\n";
    cout << "边 2-0: " << (k.unite(2, 0) ? "加入" : "成环!") << "  ← 0、2 已同组\n";
    return 0;
}
```

【代码做什么】 前半在 6 个顶点上做四次 union，实时打印连通分量个数（3 → 2），并验证 0 与 2 连通、0 与 3 起初不连通、打通后 0 与 4 连通——模拟“社交网络逐渐相连”。后半用 3 个顶点模拟 Kruskal 加边：前两条边正常合并，第三条边 2–0 时两端早已同组，`unite` 返回 false，报出“成环”——这正是 Kruskal 判环的完整机理。

【实现机制解说】 ① find 的递归压缩 `parent_[x] = find(parent_[x])` 一行同时完成“查根”与“把沿途节点直挂根下”：递归返回时层层改写 parent，下次查询一步到位；递归深度由按秩合并保证在 O(log n) 内，不会爆栈（压栈优化前）。② 按秩合并维护的是“树高的上界”而非精确高度：rank 小的根挂到 rank 大的根下，只有两棵等高的树合并才把新根 rank +1——这让任何一棵树的高度始终被钉在 O(log n)。若改为按大小合并（size 大的当根），效果等价、语义更直观，二者选一即可。③ `unite` 返回“是否真的合并”是刻意设计：判环（Kruskal）、统计最终连通分量都依赖这个返回值；`count_` 每次成功合并减一，比事后数根更省。④ 思考“为什么不能省略路径压缩”：只有压缩没有按秩，摊还仍接近 O(1)；只有按秩没有压缩，最坏 O(log n)——两个都做才是教科书级的 O(α(n))。

复杂度分析

设 k = 单词长度/操作涉及的词长，n = 词典词数或元素总数，S = 词典总字符数（所有单词长度之和）。

操作	Trie 时间复杂度	说明
insert / search / startsWith	O(k)	逐字符下行，与 n 无关（k 通常远小于 n）
前缀计数（带节点计数优化）	O(k)	每个节点存子树词数时
空间	O(S × 每节点开销)	最坏每词零共享；共享越多越省

操作	Union-Find 摊还复杂度	说明
find / unite	O(α(n)) ≈ O(1)	反阿克曼函数，任何现实 n 下 ≤ 4
m 次混合操作	O(m · α(n))	路径压缩 + 按秩合并合体
空间	O(n)	两个数组

对比参考：BST 版词典查找 O(k log n)；哈希表整词查找 O(k) 但无法做前缀查询——Trie 的 O(k) 前缀能力正是它不可替代之处。Union-Find 不按树高而按“接近常数”收费，是“摊还分析”的又一范例。

关键要点

- Trie 以字符为边、共享前缀：查询代价 O(词长) 与词库规模无关，且天然支持前缀匹配——BST 与哈希都做不到。

- “路径走得通”与“终点是词尾”是两回事：search 要 isWord，startsWith 只要路通——词尾标记是 Trie 设计的第一性细节。
- 孩子存储按字母表取舍：小字母表用定长数组换速度，通用/稀疏场景用 map 按需分配；裸指针树务必写析构与禁拷贝。
- Union-Find 两板斧缺一不可：按秩合并把树高钉在 O(log n)，路径压缩让 find 一步到位——合体后单次操作摊还 O(α(n))，工程上视为 O(1)。
- unite 的返回值就是判环信号：Kruskal 与“数连通分量”都建立在“合并失败 = 早已同组”这一观察上。

常见陷阱与注意事项

- **Trie 忘打 / 漏查 isWord**：`search("do")` 与 `startsWith("do")` 语义被混淆。规避：整词查询必须“走到位 + 查词尾标记”。
- **Trie 把“节点存在”当“单词存在”**：前缀节点不是词。规避：回忆 `search("ca")` 应为 false 的例子。
- **Trie 的 insert 每词都从根开始、共享前缀时重复建节点**：造成空间浪费与错误计数。规避：逐字符找现有孩子，缺了才 new。
- **Trie 内存泄漏 / 双重释放**：忘写析构会泄漏整棵树；浅拷贝会让两对象共享指针、析构两次。规避：Node 递归析构 + 类内 delete 拷贝构造/赋值（或改用 unique_ptr 树）。
- **空字符串当单词插入**：根节点即词尾，search("") 语义要定义清楚。规避：明确约定并让根节点 isWord 可被置位。
- **Union-Find 的 find 忘了压缩**：只做按秩合并最坏 O(log n) 虽可用，但失去接近 O(1) 的威力。规避：find 里顺手改写 parent。
- **union 时把 parent 方向搞反**（`parent[ra] = rb` 写成 `parent[rb] = ra` 与 rank 判断不配套），或忘了 rank 相等时 +1：树高失控。规避：先比 rank 再定谁当父；等高合并必须给新根加秩。
- **把 count_ 忘减 / 在 union 失败时也减**：分量数失真。规避：只在 `unite` 成功返回 true 时 `--count_`（本实现已内置）。
- **find 写迭代版却忘记第二次循环压缩**：迭代实现要再走一遍路径改写 parent。规避：递归版一行完成，初学者最不易错。

思考题（带答案）

问题 1：为什么 Trie 的 search 必须检查词尾标记，而 startsWith 不需要？请以词表 {“do”} 为例说明。 **答案**：查 “d” 时路径是通的（它是 “do” 的前缀节点），但 “d” 并不是词典中的词——search 若不查 isWord 就会误报存在；startsWith 只问“有没有以 d 开头的词”，路径通即可回答“有”。所以 search(“d”) = false、startsWith(“d”) = true 的差别全部由词尾标记承载。

问题 2：若要从 Trie 中删除一个单词（而不只是查询），步骤是什么？删除 “cart” 而保留 “car” 时能删掉哪些节点？ **答案**：先沿路径走到词尾节点、把 isWord 置 false；然后从该节点**自底向上**回收“不再被任何单词使用”的节点——即既非词尾、又没有孩子的节点（“cart” 的尾字符 t 节点符合，可删；其父节点 r 是 “car” 的词尾，必须保留）。实现常配合引用计数或“children 为空才删”的递归回收。

问题 3：只用 `unite` 的返回值，如何在完全不改动并查集的情况下数出“加完所有边后还剩几个连通分量”，并指出哪条边是“多余”的？ **答案**：初始分量数 = n；每调用一次 unite 且返回 true 就减一，返回 false 的那条边两端早已连通——它要么成环（判环），要么是冗余边。最终剩余计数就是连通分量个数。这正是 Kruskal 求最小生成树时“按边权从小到大加边、成环就跳过”的判环内核。

附录 A：课程网站调研记录（阶段一成果摘要）

调研对象：https://web.stanford.edu/class/cs106b/ （自动指向归档 `https://web.stanford.edu/class/archive/cs/cs106b/cs106b.1268/`，2026 夏季学期）。完整原始记录见同目录 `00_research_inventory.md`；原始 HTML/清洗文本见 `cs106b_pages/`。

A.1 页面可访问性速览

资源	状态	说明
课程主页 / Syllabus / Honor Code / 教职员页 / 资源大页	✅ 公开	课程定位、学习目标、先修、评分、教科书、工具
About Lectures / Flat Lecture Index	✅ 公开	讲座组织与小测规则；28 讲索引
28 个讲座页（`/lectures/xx-slug/`）	✅ 公开	每页含**当日完整文字讲义/纪要**（Contents 提纲 + 讲解）+ 部分附件（PDF/zip）
About Sections + 7 个 Section 页	✅ 公开	小班习题（答案周五公布）
About Assignments + 8 个作业页	✅ 公开	制度说明；题面公开
Exams 说明页 + 备考建议 + 期末参考表 refsheet.pdf	✅ 公开	含往年真题（期末页）
讲座小测（Canvas）	❌ 需斯坦福账号	每周发布
Ed 讨论区 / Gradescope / Paperless / LaIR	❌ 需课程注册	—
讲座录像	❌ 仅注册学生	教室录制，供 CGOE 远程生观看
教科书	❌ 需购买/图书馆	Roberts《Programming Abstractions in C++》

A.2 真实 28 讲一览（官方标题与日期）

讲	日期	官方标题	讲	日期	官方标题
L01	6/22	Welcome!	L15	7/20	Object-Oriented Programming

讲	日期	官方标题	讲	日期	官方标题
L02	6/23	C++ Fundamentals	L16	7/21	Pointers and Arrays
L03	6/24	C++ Strings	L17	7/22	Dynamic Memory Management
L04	6/25	Testing, Vectors, and Grids	L18	7/23	Priority Queues and Binary Heaps
L05	6/29	Stacks and Queues	L19	7/27	Introduction to Linked Lists
L06	6/30	Sets and Maps	L20	7/28	More Linked Lists
L07	7/1	Big-O and Algorithmic Analysis	L21	7/29	Binary Trees, BSTs, and Tree Traversals
L08	7/6	Introduction to Recursion	L22	7/30	More on Binary Trees
L09	7/7	More Recursion	L23	8/3	Huffman Coding
L10	7/8	Recursive Problem Solving	L24	8/4	Hashing
L11	7/9	Recursive Backtracking and Enumeration	L25	8/5	Graphs
L12	7/13	More Recursive Backtracking	L26	8/6	Dijkstra and A* Shortest Path Algorithms
L13	7/14	Sorting Algorithms	L27	8/10	Graph Coding
L14	7/15	Problem Solving Day（复习答疑）	L28	8/11	Wrap（期末回顾）

A.3 各章内容如何“落地”到公开资料

- **每章概述与概念清单**以官方讲座页的公开文字讲义（Contents 提纲 + 讲解）为事实依据，正文为原创中文讲解。
- **公开附件样例**（讲座页直接链接）：L05 StackViz.zip/QueueViz.zip；L13 sorting-stuff.zip；L15 oop-geocities-quokkas.zip；L16 pointers-worksheet.pdf（+解答）；L18 minheaps-written-notes.pdf；L21 tree-notes.pdf；L22 bst-code.zip、traversal-puzzle.pdf；L26 dijkstra-slides.pdf；L27 graph-algorithms.zip；期末 refsheet.pdf。Stanford C++ 库文档（Vector/Grid/Stack/Queue/Set/Map/HashMap/HashSet/strlib 等）在 web.stanford.edu/dept/cs_edu/resources/cslib_docs/ 公开。
- **受限项**：Canvas 讲座小测与录像、Ed/Gradescope/Paperless/LaIR、A5 等作业的 starter 工程文件需课程身份；正文已标注哪些内容“官方未公开/受限”。

附录 B：各章数据记录（lecture_data_records.json 摘要）

下表为阶段一为每“讲（章节）”生成的结构化数据记录概要；完整 JSON（含 key_concepts_raw 全量与 available_public_info/not_public 字段）见同目录 `lecture_data_records.json`。

章	主题（lecture_topic）	对应官方讲座	公开信息可用性（一句话）
1	C++ 基础回顾与 STL 容器入门（C++ Fundamentals & ADT Containers Intro）	L01, L02, L03, L04	官方公开 4 个讲座页：每页含当日完整文字讲义（要点提纲+讲解），公开附件 BlankProject.zip、strlib.h/Vector/Grid 文档链接；对应作业 0/1 题面页公开
2	栈与队列（Stacks and Queues: LIFO/FIFO ADTs）	L05	官方讲座页完整公开：Stack/Queue 操作表、应用（含后缀表达式）、StackViz.zip/QueueViz.zip 可视化程序公开可下载
3	集合与映射（Sets and Maps: ordered, tree-based containers）	L06	官方讲座页完整公开：Set/Map 关键操作与运算符、有序性讨论、去重/词频应用（德古拉全文词频）、Set/Map vs HashSet/HashMap 概念对比
4	算法分析：大 O 记号与运行时间估算（Big-O & Algorithmic Analysis）	L07	官方讲座页完整公开：大 O 术语、常见函数族、求和恒等式推导、add/insert 幕后分析、线性/二次/指数/对数增长与运行时间估算示例
5	递归：原理、策略与递归式思维（Recursion: Principles & Strategies）	L08, L09, L10	官方 3 个讲座页完整公开：递归入门（阶乘/回文/打印/包装函数/常见陷阱）、二分查找与枚举生成（硬币序列、排列）、递归解题（分形、骰子序列）；对应作业 3（Recursion Etudes）题面页公开
6	递归回溯与枚举（Recursive Backtracking & Enumeration）	L11, L12	官方 2 个讲座页完整公开：回溯范式与函数骨架、子集生成/计数、划分问题 isPartitionable、0-1 背包三种递归写法及最好/最坏复杂度讨论；对应作业 4（Recursive Backtracking）题面页
7	排序算法（Sorting: Selection, Insertion, Merge, Quicksort）	L13	官方讲座页完整公开：选择/插入/归并排序要点与运行时间对比数据、slides 与代码（sorting-stuff.zip 公开）
8	面向对象编程：类、对象与封装（OOP: Classes, Objects & Encapsulation）	L15	官方讲座页完整公开：OOP 范式转变、.h/.cpp 分离、Quokka 课堂示例全代码（oop-geocities-quokkas.zip 公开）；对应作业 5（Tone Matrix）题面页公开
9	指针、数组与动态内存管理（Pointers, Arrays & Dynamic Memory）	L16, L17	官方 2 个讲座页完整公开：地址/指针语法、&与*的双重语义、数组与指针关系、动态内存与内存图、new/delete 法则、ArrayBasedStack 讲解与练习；L16 附 pointers-worksheet.p
10	优先队列与二叉堆（Priority Queues & Binary Heaps）	L18	官方讲座页完整公开：树术语、最小堆性质与操作、percolation、最好情形删除、堆排序、数组表示、补充 heapify/maxheap 讨论；手写讲义 minheaps-written-notes.pdf 公开
11	链表（Linked Lists: singly/doubly, tail pointers）	L19, L20	官方 2 个讲座页完整公开：链表解剖与内存图、从笨拙到优雅的多版实现（头插/尾插/删除）、尾指针维护、双向链表、用链表实现栈/队列、数组 vs 链表权衡；对应作业 6（Listy Things）题面页公开
12	二叉树、二叉搜索树与树遍历（Binary Trees, BSTs & Traversals）	L21, L22	官方 2 个讲座页完整公开：树术语、TreeNode 初探、BST 运行时、遍历算法；L22 覆盖删除三情形、自平衡 BST、遍历应用与代码（tree-notes.pdf、bst-code.zip、traversal-p

章	主题（lecture_topic）	对应官方讲座	公开信息可用性（一句话）
13	霍夫曼编码：前缀码与编码树（Huffman Coding）	L23	官方讲座页完整公开：编码概览、ASCII、紧凑定长/变长编码、编码树构建与解码、最优树构造（讲义源自 A7 handout，作者 Julie Zelenski 等）；对应作业 7（Huffman Coding）题面页公开
14	散列与哈希表（Hashing & Hash Tables）	L24	官方讲座页完整公开：学生记录检索问题的多种方案权衡、线性探测（含聚类与表大小）、分离链、运行时分析、好哈希函数性质、斯坦福 HashSet/HashMap 与复杂度表达注意事项；L28 亦再次回顾 HashSet/Has
15	图：概念、表示、DFS/BFS 与拓扑排序（Graphs）	L25	官方讲座页完整公开：图术语、种类与性质、三种表示法、斯坦福 Graph 类、MST/拓扑排序概念、DFS/BFS 遍历与路径动画（页内含 Prezi 动画链接）
16	最短路径：Dijkstra 与 A*（Shortest Paths）	L26, L27	官方 2 个讲座页完整公开：Dijkstra 原理与最小 dist 选取的运行时考量、负权问题、Bellman–Ford 补充、A* 与启发式（附外部资源链接）、WeightedGraph 类编码与输入文件（dijkst
17	拓展专题：Trie 与并查集（Bonus: Tries & Union–Find）	（延伸内容）官方	官方公开信息有限：L24 文字讲义中明确提及 trie（按学号数字逐位走 10 叉树）并建议课外深究；本章主体为面向求职/后续课程的原创扩展笔记

C++ STL 容器与算法速查表

> 课程官方使用 Stanford C++ Library（`Vector`、`Grid`、`Stack`、`Queue`、`Set`、`Map`、`HashSet`、`HashMap`、`Lexicon`、`PriorityQueue`...）；本表统一给**标准库（STL）等价物**与常用操作，便于你在任何现代 C++（C++17）环境里动手。A.5 节给出两者对照。

高频“基础设施”备忘

```
#include <iostream>      // cin / cout / endl
#include <string>        // std::string
#include <vector>        // std::vector 等容器
#include <algorithm>     // sort / find / reverse ...
#include <numeric>       // accumulate / iota
#include <cctype>        // isalpha / isdigit / tolower ...
#include <sstream>       // 字符串流（按词切分等）
using namespace std;     // 教学代码简化写法（工程上避免）

// 常用写法速记
auto    v = vector<int>{1,2,3};       // 类型推导 + 列表初始化
for (const auto& x : v) { /* 只读遍历，零拷贝 */ }
for (auto& x : v)       { /* 可修改遍历 */ }
const string& s2 = s;   // 引用：不拷贝、不修改（传参首选）
nullptr                  // 空指针字面量（替代 NULL/0）
```

传参铁律：**对象默认按值拷贝**。只读传 `const T&`，要改传 `T&`；小类型（`int`、`char`、`bool`、指针）按值即可。

std::string（字符串）

C++ 字符串是**可变**的（区别于 Python/Java 的不可变）字符序列。

操作	示例	说明
长度	`s.size()` / `s.length()`	O(1)
判空	`s.empty()`	
访问	`s[i]`、`s.at(i)`	`at` 越界抛异常；`[]` 不检查
追加	`s += c; s.append(t); s.push_back('x')`	
插入/删除	`s.insert(pos, t)`、`s.erase(pos, len)`	下标版本 O(n)
子串	`s.substr(pos, len)`	len 缺省到末尾
查找	`s.find(t)`、`s.rfind(t)`	返回 `size_t`；找不到返回 `string::npos`
比较	`s == t`、`s < t`	字典序
C 风格	`s.c_str()`	得到 `const char*`
读一行	`getline(cin, s)`	含空格整行

数值 ↔ 字符串：`to_string(42)`、`stoi(s)`（`stol/stod/...`）。

字符处理（`<cctype>`，参数按 `unsigned char` 转）：`isalpha/isalnum/isdigit/isspace/isupper/islower`、`toupper/tolower`。字符即小整数：`'a'` 是 97，`c - 'a'` 得 0—25。

```
// 词频演示：统计一段文本中每个单词出现次数
#include <iostream>
#include <string>
#include <sstream>
#include <map>
using namespace std;
int main() {
    string text = "to be or not to be";
    map<string,int> freq;                  // 键自动按字典序排好
    istringstream iss(text);               // 按空白切词
    string w;
    while (iss >> w) ++freq[w];            // 首次访问自动插入并置 0，再自增
    for (const auto& [word, cnt] : freq)   // 结构化绑定 (C++17)
        cout << word << ": " << cnt << "\n";
}
// 输出：be: 2  not: 1  or: 1  to: 2
```

顺序容器

容器	底层	特点	常用操作
`vector<T>`	连续数组（自动扩容）	尾部 O(1)（均摊）；随机访问 O(1)；中间插删 O(n)	`push_back/pop_back`、`back/front`、`size/empty`、`v[i]`、`insert/erase`、`reserve/resize`、`clear`
`deque<T>`	分段连续	头尾都 O(1)（均摊）	`push_back/push_front/pop_back/pop_front` + vector 全部
`list<T>`	双向链表	任意位置插删 O(1)（已知位置）；**无随机访问**	`push_back/push_front`、`insert/erase`、`splice`
`array<T,N>`	定长数组	编译期定长	同 vector（无 push_back）

vector 扩容机制（面试高频）：满时按倍数（常见 2×）申请新块→拷贝/移动旧元素→释放旧块。单次 push_back 最坏 O(n)，但 n 次 push_back 总代价 O(n)，**均摊 O(1)**——这正是 Lecture 4 里“`add` 均摊快、`insert(0,·)` 每次都要搬动所有元素 O(n)”的原因。

```
#include <vector>
vector<int> v{3,1,4,1,5};
v.push_back(9);                 // 3 1 4 1 5 9
v.insert(v.begin() + 2, 100);   // 在下标 2 前插入 → 3 1 100 4 1 5 9
v.erase(v.begin());             // 删第一个 → 1 100 4 1 5 9
int x = v.back(); v.pop_back(); // 取并弹尾部
```

迭代器：`begin()/end()`（`cbegin/cend` 只读），支持 `++/--/*` 与 `it + n`（随机访问容器）。**注意**：对 vector 做插入/删除会使指向其后元素的迭代器失效。

容器适配器（栈 / 队列 / 优先队列）

适配器 = 在底层容器上“限量”操作，只暴露一种出入口语义。

适配器	语义	入	出/看	常用
`stack<T>`	LIFO 后进先出（默认底层 deque）	`push`	`pop`（无返回值）、`top`	`empty/size`
`queue<T>`	FIFO 先进先出	`push`（入队）	`pop`（出队）、`front/back`	
`priority_queue<T>`	最大堆（默认）；`greater<T>` 变最小堆	`push`	`pop`、`top`	底层 vector + make_heap

```
#include <stack>   #include <queue>
stack<int> st;  st.push(1); st.push(2);      // top=2
int t = st.top(); st.pop();                  // 弹出 2

queue<int> q;   q.push(1); q.push(2);        // front=1
int f = q.front(); q.pop();                  // 出 1

priority_queue<int> mx;                      // 默认最大堆
mx.push(3); mx.push(5);                      // top()==5
priority_queue<int, vector<int>, greater<int>> mn; // 最小堆 top()==最小
// 自定义类型：给 operator<，或传比较器/仿函数
```

关联容器（有序树 vs 哈希桶）

容器	底层	键序	查找/插删复杂度
`set<T>` / `map<K,V>`	红黑树（平衡 BST）	有序（可遍历出升序）	O(log n)
`multiset` / `multimap`	红黑树	有序，允许重复键	O(log n)
`unordered_set<T>` / `unordered_map<K,V>`	哈希表（桶）	无序	平均 O(1)，最坏 O(n)
`unordered_multiset/multimap`	哈希表	无序，允许重复	平均 O(1)

```
#include <set>    #include <map>
#include <unordered_set>   #include <unordered_map>
set<int> s{3,1,4}; s.insert(2);          // {1,2,3,4} 自动有序
if (s.count(4)) { /* 存在 */ }           // count∈{0,1}
s.erase(3);

map<string,int> m;
m["alice"] = 1;                          // [] 会“探键”：不存在则插入默认值
m.at("alice");                           // at() 不存在时抛异常（安全版）
auto it = m.find("bob");                 // 找不到 == m.end()
for (const auto& [k, v] : m) { /* 键升序 */ }

unordered_map<string,int> u;             // 无序版：平均更快，无序遍历
```

取舍：需要“有序遍历/前驱后继/范围查询”→ `set/map`（BST）；只做查存、不关心顺序、量大 → `unordered_*`（哈希）。这正是 Lecture 3（树版有序）与 Lecture 14（哈希版）的对照。

常用算法（`<algorithm>` / `<numeric>`）

算法	示例	说明
排序	`sort(v.begin(), v.end())`	升序；`sort(b,e,greater<int>())` 降序
稳定排序	`stable_sort(...)`	相等元素保持原相对顺序
反转	`reverse(v.begin(), v.end())`	
线性查找	`find(v.begin(), v.end(), x)`	返回迭代器；`==end()` 未找到
计数	`count(...)`	
最值	`min/max(a,b)`；`min_element/max_element(b,e)`	返回迭代器
二分查找	`binary_search(vb,ve,x)`；`lower_bound/upper_bound`	**前提：已排序**，O(log n)
累加	`accumulate(vb,ve,0)`；`accumulate(vb,ve,0.0)`	整数/浮点初值要写对
填值	`fill(vb,ve,0)`；`iota(vb,ve,0)`（填 0,1,2,…）	
去重	`sort` 后 `unique` + `erase`	见下
全排列	`next_permutation(vb,ve)`	返回 bool，配合 do-while 枚举
变换/遍历	`transform`、`for_each`	配合 lambda

```
#include <algorithm>
vector<int> v{5,1,4,1,3};
sort(v.begin(), v.end());                        // 1 1 3 4 5
v.erase(unique(v.begin(), v.end()), v.end());    // 去重 → 1 3 4 5
bool ok = binary_search(v.begin(), v.end(), 4); // true（已排序）
auto it = lower_bound(v.begin(), v.end(), 3);    // 第一个 >=3 的位置
sort(v.begin(), v.end(), [](int a,int b){ return a > b; }); // lambda 自定义
```

其他实用件

- `pair<K,V>`：`make_pair` / `{k,v}`；`tie` /结构化绑定解包。`tuple` 同理。
- 计时（`<chrono>`，Lecture 4 可用来实测运行时间）：`cpp auto t0 = chrono::steady_clock::now(); /* ...被测代码... */ auto ms = chrono::duration_cast<chrono::milliseconds>( chrono::steady_clock::now() - t0).count(); cout << ms << " ms\n";`
- 随机数（`<random>`）：`mt19937 gen(rd()); uniform_int_distribution<int> d(1,6); d(gen);`
- 输入输出流 `cin/cout`：读整数 `cin >> x`；读整行 `getline(cin,s)`；`<iomanip>` 的 `setw/setprecision` 排版。

课程 Stanford 库 ↔ STL 对照

Stanford C++ 库	语义	本笔记/STL 等价
`Vector<T>`	动态数组	`std::vector<T>`
`Grid<T>`	二维网格	`std::vector<std::vector<T>>`
`Stack<T>`	LIFO	`std::stack<T>`
`Queue<T>`	FIFO	`std::queue<T>`
`Set<T>`	有序集合（平衡 BST）	`std::set<T>`
`Map<K,V>`	有序映射（平衡 BST）	`std::map<K,V>`
`HashSet<T>`	无序集合（哈希）	`std::unordered_set<T>`
`HashMap<K,V>`	无序映射（哈希）	`std::unordered_map<K,V>`
`PriorityQueue<T>`	优先队列	`std::priority_queue<T>`（注意默认是最大堆，课程常用最小堆语义）
`Lexicon`	词典（trie/哈希实现）	`std::unordered_set<std::string>` 或 Trie（Lecture 17）

Stanford C++ 库	语义	本笔记/STL 等价
`strlib`（toLowerCase/…）	字符串工具	`std::tolower` / `<algorithm>` 手写小工具
`SimpleTest`	单元测试	自写 `assert`/简单测试函数
`randomInteger(a,b)`	随机整数	`<random>` 的 uniform_int_distribution

> 一句话总结：**学 ADT 先学“语义”（栈/队列/映射…），再学“实现”（数组/链表/树/哈希）**——CS106B 的精髓就是把“用什么”和“怎么造”分开想清楚，本笔记各章正是按这条主线展开的。

笔记完。祝学习愉快——画图、动手、多问为什么。